

API generic splitting

decorators

decorators module

Decorators for splitting.

cv_split function

```
cv_split(
    *args,
    parameterized_kwargs=None,
    selection='max',
    return_grid=False,
    skip_errored=False,
    raise_no_results=True,
    template_context=None,
    **split_kwargs
)
```

Decorator that combines [split](#) and [parameterized](#) for cross-validation.

Creates a new apply function that is going to be decorated with [split](#) and thus applied at each single range using [Splitter.apply](#). Inside this apply function, there is a test whether the current range belongs to the first (training) set. If yes, parameterizes the underlying function and runs it on the entire grid of parameters. The returned results are then stored in a global list. These results are then read by the other (testing) sets in the same split. If `selection` is a template, it can evaluate the grid results (available as `grid_results`) and return the best parameter combination. This parameter combination is then executed by each set (including training).

Argument `selection` also accepts "min" for `np.argmin` and "max" for `np.argmax`.

Keyword arguments `parameterized_kwargs` will be passed to [parameterized](#) and will have their templates substituted with a context that will also include the split-related context (including `split_idx`, `set_idx`, etc., see [Splitter.apply](#)).

If `return_grid` is True or 'first', returns both the grid and the selection. If `return_grid` is 'all', executes the grid on each set and returns along with the selection. Otherwise, returns only the selection.

If [NoResultsException](#) is raised or `skip_errored` is True and any exception is raised, will skip the current iteration and remove it from the final index.

Usage

- Permutate a series and pick the first value. Make the seed parameterizable. Cross-validate based on the highest picked value:

```
>>> from vectorbtpro import *

>>> @vbt.cv_split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=3, split=0.5),
...     takeable_args=["sr"],
...     merge_func="concat",
... )
... def f(sr, seed):
...     np.random.seed(seed)
...     return np.random.permutation(sr)[0]

>>> index = pd.date_range("2020-01-01", "2020-02-01")
>>> np.random.seed(0)
>>> sr = pd.Series(np.random.permutation(np.arange(len(index))), index=index)
>>> f(sr, vbt.Param([41, 42, 43]))
split set seed
0     set_0 41    22
      set_1 41    28
1     set_0 43     8
      set_1 43    31
2     set_0 43    19
      set_1 43     0
dtype: int64
```

- Extend the example above to also return the grid results of each set:

```
>>> f(sr, vbt.Param([41, 42, 43]), _return_grid="all")
(split set seed
0     set_0 41    22
      42    22
      43     2
      set_1 41    28
      42    28
      43    20
1     set_0 41     5
      42     5
      43     8
      set_1 41    23
```

		42	23
		43	31
2	set_0	41	18
		42	18
		43	19
	set_1	41	27
		42	27
		43	0
dtype: int64,			
split	set	seed	
0	set_0	41	22
	set_1	41	28
1	set_0	43	8
	set_1	43	31
2	set_0	43	19
	set_1	43	0
dtype: int64)			

split function

```
split(
    *args,
    splitter=None,
    splitter_cls=None,
    splitter_kwargs=None,
    index=None,
    index_from=None,
    takeable_args=None,
    template_context=None,
    forward_kwargs_as=None,
    return_splitter=False,
    apply_kwargs=None,
    **var_kwargs
)
```

Decorator that splits the inputs of a function.

Does the following:

1. Resolves the splitter of the type [Splitter](#) using the argument `splitter`. It can be either an already provided splitter instance, the name of a factory method (such as "from_n_rolling"), or the factory method itself. If `splitter` is None, the right method will be guessed based on the supplied arguments using [Splitter.guess_method](#). To construct a splitter, it will pass `index` and `**splitter_kwargs`. Index is getting resolved either using an already provided `index`, by parsing the argument under a name/position provided in `index_from`, or by parsing the first argument from `takeable_args` (in this order).
2. Wraps arguments in `takeable_args` with [Takeable](#)
3. Runs [Splitter.apply](#) with arguments passed to the function as `args` and `kwargs`, but also `apply_kwargs` (the ones passed to the decorator)

Keyword arguments `splitter_kwargs` are passed to the factory method. Keyword arguments `apply_kwargs` are passed to [Splitter.apply](#). If variable keyword arguments are provided, they will be used as `splitter_kwargs` if `apply_kwargs` is already set, and vice versa. If `splitter_kwargs` and `apply_kwargs` aren't set, they will be used as `splitter_kwargs` if a splitter instance hasn't been built yet, otherwise as `apply_kwargs`. If both arguments are set, will raise an error.

Usage

- Split a Series and return its sum:

```
>>> from vectorbtpro import *

>>> @vbt.split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=2),
...     takeable_args=["sr"]
... )
... def f(sr):
...     return sr.sum()

>>> index = pd.date_range("2020-01-01", "2020-01-06")
>>> sr = pd.Series(np.arange(len(index)), index=index)
>>> f(sr)
split
0      3
1     12
dtype: int64
```

- Perform a split manually:

```
>>> @vbt.split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=2),
...     takeable_args=["index"]
... )
... def f(index, sr):
...     return sr[index].sum()

>>> f(index, sr)
split
0      3
1     12
```

```
dtype: int64
```

- Construct splitter and mark arguments as "takeable" manually:

```
>>> splitter = vbt.Splitter.from_n_rolling(index, n=2)
>>> @vbt.split(splitter=splitter)
... def f(sr):
...     return sr.sum()

>>> f(vbt.Takeable(sr))
split
0      3
1     12
dtype: int64
```

- Split multiple timeframes using a custom index:

```
>>> @vbt.split(
...     splitter="from_n_rolling",
...     splitter_kwargs=dict(n=2),
...     index=index,
...     takeable_args=["h12_sr", "d2_sr"]
... )
... def f(h12_sr, d2_sr):
...     return h12_sr.sum() + d2_sr.sum()

>>> h12_index = pd.date_range("2020-01-01", "2020-01-06", freq="12H")
>>> d2_index = pd.date_range("2020-01-01", "2020-01-06", freq="2D")
>>> h12_sr = pd.Series(np.arange(len(h12_index)), index=h12_index)
>>> d2_sr = pd.Series(np.arange(len(d2_index)), index=d2_index)
>>> f(h12_sr, d2_sr)
split
0      15
1     42
dtype: int64
```

API portfolio

orders

records

orders module

Base class for working with order records.

Order records capture information on filled orders. Orders are mainly populated when simulating a portfolio and can be accessed as [Portfolio.orders](#).

```
>>> from vectorbtpro import *

>>> price = vbt.RandomData.pull(
...     ['a', 'b'],
...     start=datetime(2020, 1, 1),
...     end=datetime(2020, 3, 1),
...     seed=vbt.key_dict(a=42, b=43)
... ).get()
```

100%

```
>>> size = pd.DataFrame({
...     'a': np.random.randint(-1, 2, size=len(price.index)),
...     'b': np.random.randint(-1, 2, size=len(price.index)),
... }, index=price.index, columns=price.columns)
>>> pf = vbt.Portfolio.from_orders(price, size, fees=0.01, freq='d')

>>> pf.orders.side_buy.count()
symbol
a    17
b    15
Name: count, dtype: int64

>>> pf.orders.side_sell.count()
symbol
a    24
b    26
Name: count, dtype: int64
```

Stats

Hint

See [StatsBuilderMixin.stats](#) and [Orders.metrics](#).

```
>>> pf.orders['a'].stats()
Start                2019-12-31 22:00:00+00:00
End                  2020-02-29 22:00:00+00:00
Period                61 days 00:00:00
Total Records                41
Side Counts: Buy                17
Side Counts: Sell                24
Size: Min                    0 days 19:33:05.006182372
Size: Median                  1 days 00:00:00
Size: Max                    1 days 00:00:00
Fees: Min                    0 days 20:26:25.905776572
Fees: Median                0 days 22:46:22.693324744
Fees: Max                    1 days 01:04:25.541681491
Weighted Buy Price                94.69917
Weighted Sell Price            95.742148
Name: a, dtype: object
```

[StatsBuilderMixin.stats](#) also supports (re-)grouping:

```
>>> pf.orders.stats(group_by=True)
Start                2019-12-31 22:00:00+00:00
End                  2020-02-29 22:00:00+00:00
Period                61 days 00:00:00
Total Records                82
Side Counts: Buy                32
Side Counts: Sell                50
Size: Min                    0 days 19:33:05.006182372
Size: Median                  1 days 00:00:00
Size: Max                    1 days 00:00:00
Fees: Min                    0 days 20:26:25.905776572
Fees: Median                0 days 23:58:29.773897679
Fees: Max                    1 days 02:29:08.904770159
Weighted Buy Price                98.804452
Weighted Sell Price            99.969934
Name: group, dtype: object
```

Plots

Hint

See `PlotsBuilderMixin.plots` and `Orders.subplots`.

`Orders` class has a single subplot based on `Orders.plot`:

```
>>> pf.orders['a'].plots().show()
```



fs_orders_attach_field_config ReadOnlyConfig

Config of fields to be attached to `FSOrders`.

```
ReadOnlyConfig(  
    type=dict(  
        attach_filters=True  
    ),  
    stop_type=dict(  
        attach_filters=True  
    )  
)
```

fs_orders_field_config ReadOnlyConfig

Field config for `FSOrders`.

```
ReadOnlyConfig(  
    dtype=np.dtype([  
        ('id', 'int64'),  
        ('col', 'int64'),  
        ('signal_idx', 'int64'),  
        ('creation_idx', 'int64'),  
        ('idx', 'int64'),  
        ('size', 'float64'),  
        ('price', 'float64'),  
        ('fees', 'float64'),  
        ('side', 'int64'),  
        ('type', 'int64'),  
        ('stop_type', 'int64')  
    ]),  
    settings=dict(  
        idx=dict(  
            title='Fill Index'  
        ),  
        signal_idx=dict(  
            title='Signal Index',  
            mapping='index',  
            noindex=True  
        ),  
        creation_idx=dict(  
            title='Creation Index',  
            mapping='index',  
            noindex=True  
        ),  
        type=dict(  
            title='Type',  
            mapping=OrderTypeT(  
                Market=0,  
                Limit=1  
            )  
        ),  
        stop_type=dict(  
            title='Stop Type',  
            mapping=StopTypeT(  
                SL=0,  
                TSL=1,  

```

```

        TTP=2,
        TP=3,
        TD=4,
        DT=5
    )
)
)
)

```

fs_orders_shortcut_config ReadOnlyConfig

Config of shortcut properties to be attached to **FSOrders**.

```

ReadOnlyConfig(
    stop_orders=dict(),
    ranges=dict(),
    creation_ranges=dict(),
    fill_ranges=dict(),
    signal_to_creation_duration=dict(
        obj_type='mapped_array'
    ),
    creation_to_fill_duration=dict(
        obj_type='mapped_array'
    ),
    signal_to_fill_duration=dict(
        obj_type='mapped_array'
    )
)

```

orders_attach_field_config ReadOnlyConfig

Config of fields to be attached to **Orders**.

```

ReadOnlyConfig(
    side=dict(
        attach_filters=True
    )
)

```

orders_field_config ReadOnlyConfig

Field config for **Orders**.

```

ReadOnlyConfig(
    dtype=np.dtype([
        ('id', 'int64'),
        ('col', 'int64'),
        ('idx', 'int64'),
        ('size', 'float64'),
        ('price', 'float64'),
        ('fees', 'float64'),
        ('side', 'int64')
    ]),
    settings=dict(
        id=dict(
            title='Order Id'
        ),
        idx=dict(),
        size=dict(
            title='Size'
        ),
        price=dict(
            title='Price'
        ),
        fees=dict(
            title='Fees'
        ),
        side=dict(
            title='Side',
            mapping=OrderSideT(
                Buy=0,
                Sell=1
            ),
            as_customdata=False
        )
    )
)

```

orders_shortcut_config ReadOnlyConfig

Config of shortcut properties to be attached to [Orders](#).

```
ReadonlyConfig(
    long_view=dict(),
    short_view=dict(),
    signed_size=dict(
        obj_type='mapped'
    ),
    value=dict(
        obj_type='mapped'
    ),
    weighted_price=dict(
        obj_type='red_array'
    ),
    price_status=dict(
        obj_type='mapped'
    )
)
```

FSOrders class

```
FSOrders(
    wrapper,
    records_arr,
    open=None,
    high=None,
    low=None,
    close=None,
    **kwargs
)
```

Extends [Orders](#) for working with order records generated from signals.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [Orders](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [PriceRecords](#)
- [Records](#)
- [RecordsWithFields](#)
- [StatsBuilderMixin](#)
- [Wrapping](#)

Inherited members

- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)
- [AttrResolverMixin.resolve_attr](#)
- [AttrResolverMixin.resolve_shortcut_attr](#)
- [Base.chat](#)
- [Base.find_api](#)

- `Base.find_assets`
- `Base.find_docs`
- `Base.find_examples`
- `Base.find_messages`
- `Cacheable.get_ca_setup`
- `Chainable.chain`
- `Chainable.pipe`
- `Configured.copy`
- `Configured.equals`
- `Configured.get_writable_attrs`
- `Configured.prettify`
- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `HasSettings.get_path_setting`
- `HasSettings.get_path_settings`
- `HasSettings.get_setting`
- `HasSettings.get_settings`
- `HasSettings.has_path_setting`
- `HasSettings.has_path_settings`
- `HasSettings.has_setting`
- `HasSettings.has_settings`
- `HasSettings.reset_settings`
- `HasSettings.resolve_setting`
- `HasSettings.resolve_settings_paths`
- `HasSettings.set_settings`
- `IndexApplier.add_levels`
- `IndexApplier.drop_duplicate_levels`
- `IndexApplier.drop_levels`
- `IndexApplier.drop_redundant_levels`
- `IndexApplier.rename_levels`
- `IndexApplier.select_levels`
- `IndexingBase.indexing_setter_func`
- `Orders.bar_close`
- `Orders.bar_close_time`
- `Orders.bar_high`
- `Orders.bar_low`
- `Orders.bar_open`
- `Orders.bar_open_time`
- `Orders.close`
- `Orders.cls_dir`
- `Orders.col`
- `Orders.col_arr`
- `Orders.col_mapper`
- `Orders.column_only_select`
- `Orders.config`
- `Orders.fees`
- `Orders.field_names`
- `Orders.get_long_view`
- `Orders.get_price_status`
- `Orders.get_short_view`
- `Orders.get_signed_size`
- `Orders.get_value`
- `Orders.get_weighted_price`
- `Orders.group_select`

- `Orders.high`
- `Orders.id`
- `Orders.id_arr`
- `Orders.idx`
- `Orders.idx_arr`
- `Orders.iloc`
- `Orders.indexing_kwargs`
- `Orders.loc`
- `Orders.long_view`
- `Orders.low`
- `Orders.open`
- `Orders.pd_mask`
- `Orders.plot`
- `Orders.plots_defaults`
- `Orders.price`
- `Orders.price_status`
- `Orders.range_only_select`
- `Orders.readable`
- `Orders.rec_state`
- `Orders.reccarray`
- `Orders.records`
- `Orders.records_arr`
- `Orders.records_readable`
- `Orders.self_aliases`
- `Orders.short_view`
- `Orders.side`
- `Orders.side_buy`
- `Orders.side_sell`
- `Orders.signed_size`
- `Orders.size`
- `Orders.stats_defaults`
- `Orders.value`
- `Orders.values`
- `Orders.weighted_price`
- `Orders.wrapper`
- `Orders.xloc`
- `PandasIndexer.xs`
- `Pickleable.decode_config`
- `Pickleable.decode_config_node`
- `Pickleable.dumps`
- `Pickleable.encode_config`
- `Pickleable.encode_config_node`
- `Pickleable.file_exists`
- `Pickleable.getsize`
- `Pickleable.load`
- `Pickleable.loads`
- `Pickleable.modify_state`
- `Pickleable.resolve_file_path`
- `Pickleable.save`
- `PlotsBuilderMixin.build_subplots_doc`
- `PlotsBuilderMixin.override_subplots_doc`
- `PlotsBuilderMixin.plots`
- `PlotsBuilderMixin.resolve_plots_setting`
- `Prettified.pprint`

- [PriceRecords.from_records](#)
- [PriceRecords.get_bar_close](#)
- [PriceRecords.get_bar_close_time](#)
- [PriceRecords.get_bar_high](#)
- [PriceRecords.get_bar_low](#)
- [PriceRecords.get_bar_open](#)
- [PriceRecords.get_bar_open_time](#)
- [PriceRecords.indexing_func](#)
- [PriceRecords.indexing_func_meta](#)
- [PriceRecords.resample](#)
- [PriceRecords.resolve_column_stack_kwargs](#)
- [PriceRecords.resolve_row_stack_kwargs](#)
- [Records.apply](#)
- [Records.apply_mask](#)
- [Records.build_field_config_doc](#)
- [Records.column_stack](#)
- [Records.column_stack_records_arrs](#)
- [Records.count](#)
- [Records.coverage_map](#)
- [Records.first_n](#)
- [Records.get_apply_mapping_arr](#)
- [Records.get_apply_mapping_str_arr](#)
- [Records.get_column_stack_record_indices](#)
- [Records.get_field_arr](#)
- [Records.get_field_mapping](#)
- [Records.get_field_name](#)
- [Records.get_field_setting](#)
- [Records.get_field_title](#)
- [Records.get_map_field](#)
- [Records.get_map_field_to_columns](#)
- [Records.get_map_field_to_index](#)
- [Records.get_pd_mask](#)
- [Records.get_row_stack_record_indices](#)
- [Records.has_conflicts](#)
- [Records.is_sorted](#)
- [Records.last_n](#)
- [Records.map](#)
- [Records.map_array](#)
- [Records.map_field](#)
- [Records.override_field_config_doc](#)
- [Records.prepare_customdata](#)
- [Records.random_n](#)
- [Records.replace](#)
- [Records.resample_meta](#)
- [Records.resample_records_arr](#)
- [Records.row_stack](#)
- [Records.row_stack_records_arrs](#)
- [Records.select_cols](#)
- [Records.sort](#)
- [Records.to_readable](#)
- [StatsBuilderMixin.build_metrics_doc](#)
- [StatsBuilderMixin.override_metrics_doc](#)
- [StatsBuilderMixin.resolve_stats_setting](#)
- [StatsBuilderMixin.stats](#)

- [Wrapping.apply_to_index](#)
- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping.regroup](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

creation_idx cached_property 

Mapped array of the field `creation_idx`.

creation_ranges cached_property 

[FSOrders.get_creation_ranges](#) with default arguments.

creation_to_fill_duration cacheable_property 

[FSOrders.get_creation_to_fill_duration](#) with default arguments.

field_config property 

Field config of [FSOrders](#).

```
HybridConfig(
    dtype=np.dtype([
        ('id', 'int64'),
        ('col', 'int64'),
        ('signal_idx', 'int64'),
        ('creation_idx', 'int64'),
        ('idx', 'int64'),
        ('size', 'float64'),
        ('price', 'float64'),
        ('fees', 'float64'),
        ('side', 'int64'),
        ('type', 'int64'),
        ('stop_type', 'int64')
    ]),
    settings=dict(
        id=dict(
            name='id',
            title='Order Id',
            mapping='ids'
        ),
        col=dict(
            name='col',
            title='Column',
            mapping='columns',
            as_customdata=False
        ),
        idx=dict(
            name='idx',
            title='Fill Index',
            mapping='index'
        ),
        size=dict(
            title='Size'
        ),
        price=dict(
            title='Price'
        ),
        fees=dict(
            title='Fees'
        ),
        side=dict(
            title='Side',
            mapping=OrderSideT(
                Buy=0,
                Sell=1
            ),
            as_customdata=False
        ),
        signal_idx=dict(

```

```

        title='Signal Index',
        mapping='index',
        noindex=True
    ),
    creation_idx=dict(
        title='Creation Index',
        mapping='index',
        noindex=True
    ),
    type=dict(
        title='Type',
        mapping=OrderTypeT(
            Market=0,
            Limit=1
        )
    ),
    stop_type=dict(
        title='Stop Type',
        mapping=StopTypeT(
            SL=0,
            TSL=1,
            TTP=2,
            TP=3,
            TD=4,
            DT=5
        )
    )
)
)
)

```

Returns `FSOrders._field_config`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change fields, you can either change the config in-place, override this property, or overwrite the instance variable `FSOrders._field_config`.

fill_ranges cached_property

[FSOrders.get_fill_ranges](#) with default arguments.

get_creation_ranges method

```

FSOrders.get_creation_ranges(
    **kwargs
)

```

Get records of type [Ranges](#) for signal-to-creation ranges.

get_creation_to_fill_duration method

```

FSOrders.get_creation_to_fill_duration(
    **kwargs
)

```

Get duration between creation and fill.

get_fill_ranges method

```

FSOrders.get_fill_ranges(
    **kwargs
)

```

Get records of type [Ranges](#) for creation-to-fill ranges.

get_ranges method

```

FSOrders.get_ranges(
    **kwargs
)

```

Get records of type [Ranges](#) for signal-to-fill ranges.

get_signal_to_creation_duration method




```
FSOrders.get_signal_to_creation_duration(
    **kwargs
)
```

Get duration between signal and creation.

get_signal_to_fill_duration method

```
FSOrders.get_signal_to_fill_duration(
    **kwargs
)
```

Get duration between signal and fill.

get_stop_orders method

```
FSOrders.get_stop_orders(
    **kwargs
)
```

Get stop orders.

metrics property

Metrics supported by [FSOrders](#).

```
HybridConfig(
    start_index=dict(
        title='Start Index',
        calc_func=<function Orders.<lambda> at 0x15fc36e80>,
        agg_func=None,
        tags='wrapper'
    ),
    end_index=dict(
        title='End Index',
        calc_func=<function Orders.<lambda> at 0x15fc36f20>,
        agg_func=None,
        tags='wrapper'
    ),
    total_duration=dict(
        title='Total Duration',
        calc_func=<function Orders.<lambda> at 0x15fc36fc0>,
        apply_to_timedelta=True,
        agg_func=None,
        tags='wrapper'
    ),
    total_records=dict(
        title='Total Records',
        calc_func='count',
        tags='records'
    ),
    side_counts=dict(
        title='Side Counts',
        calc_func='side.value_counts',
        incl_all_keys=True,
        post_calc_func=<function Orders.<lambda> at 0x15fc37060>,
        tags=[
            'orders',
            'side'
        ]
    ),
    type_counts=dict(
        title='Type Counts',
        calc_func='type.value_counts',
        incl_all_keys=True,
        post_calc_func=<function FSOrders.<lambda> at 0x15fc5c360>,
        tags=[
            'orders',
            'type'
        ]
    ),
    stop_type_counts=dict(
        title='Stop Type Counts',
        calc_func='stop_type.value_counts',
        incl_all_keys=True,
        post_calc_func=<function FSOrders.<lambda> at 0x15fc5c400>,
        tags=[
            'orders',
            'stop_type'
        ]
    ),
    size=dict(
        title='Size',
        calc_func='size.describe',
        post_calc_func=<function Orders.<lambda> at 0x15fc37100>,

```

```
tags=[
    'orders',
    'size'
],
fees=dict(
    title='Fees',
    calc_func='fees.describe',
    post_calc_func=<function Orders.<lambda> at 0x15fc371a0>,
    tags=[
        'orders',
        'fees'
    ]
),
weighted_buy_price=dict(
    title='Weighted Buy Price',
    calc_func='side_buy.get_weighted_price',
    tags=[
        'orders',
        'buy',
        'price'
    ]
),
weighted_sell_price=dict(
    title='Weighted Sell Price',
    calc_func='side_sell.get_weighted_price',
    tags=[
        'orders',
        'sell',
        'price'
    ]
),
avg_signal_to_creation_duration=dict(
    title='Avg Signal-Creation Duration',
    calc_func='signal_to_creation_duration.mean',
    apply_to_timedelta=True,
    tags=[
        'orders',
        'duration'
    ]
),
avg_creation_to_fill_duration=dict(
    title='Avg Creation-Fill Duration',
    calc_func='creation_to_fill_duration.mean',
    apply_to_timedelta=True,
    tags=[
        'orders',
        'duration'
    ]
),
avg_signal_to_fill_duration=dict(
    title='Avg Signal-Fill Duration',
    calc_func='signal_to_fill_duration.mean',
    apply_to_timedelta=True,
    tags=[
        'orders',
        'duration'
    ]
)
)
```

Returns `FSOrders._metrics`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change metrics, you can either change the config in-place, override this property, or overwrite the instance variable `FSOrders._metrics`.

ranges cached_property 

[FSOrders.get_ranges](#) with default arguments.

signal_idx cached_property 

Mapped array of the field `signal_idx`.

signal_to_creation_duration cacheable_property 

[FSOrders.get_signal_to_creation_duration](#) with default arguments.

signal_to_fill_duration cacheable_property 

[FSOrders.get_signal_to_fill_duration](#) with default arguments.

stop_orders cached_property 

[FSOrders.get_stop_orders](#) with default arguments.

stop_type cached_property 

Mapped array of the field `stop_type`.

stop_type_dt cached_property 

Records filtered by `stop_type == 5`.

stop_type_sl cached_property 

Records filtered by `stop_type == 0`.

stop_type_td cached_property 

Records filtered by `stop_type == 4`.

stop_type_tp cached_property 

Records filtered by `stop_type == 3`.

stop_type_tsl cached_property 

Records filtered by `stop_type == 1`.

stop_type_ttp cached_property 

Records filtered by `stop_type == 2`.

type cached_property 

Mapped array of the field `type`.

type_limit cached_property 

Records filtered by `type == 1`.

type_market cached_property 

Records filtered by `type == 0`.

Orders class 

```
Orders(
    wrapper,
    records_arr,
    open=None,
    high=None,
    low=None,
    close=None,
    **kwargs
)
```

Extends [PriceRecords](#) for working with order records.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)

- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [PriceRecords](#)
- [Records](#)
- [RecordsWithFields](#)
- [StatsBuilderMixin](#)
- [Wrapping](#)

Inherited members

- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)
- [AttrResolverMixin.resolve_attr](#)
- [AttrResolverMixin.resolve_shortcut_attr](#)
- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)
- [Configured.resolve_merge_kwargs](#)
- [Configured.update_config](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)
- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [IndexApplier.add_levels](#)
- [IndexApplier.drop_duplicate_levels](#)

- `IndexApplier.drop_levels`
- `IndexApplier.drop_redundant_levels`
- `IndexApplier.rename_levels`
- `IndexApplier.select_levels`
- `IndexingBase.indexing_setter_func`
- `PandasIndexer.xs`
- `Pickleable.decode_config`
- `Pickleable.decode_config_node`
- `Pickleable.dumps`
- `Pickleable.encode_config`
- `Pickleable.encode_config_node`
- `Pickleable.file_exists`
- `Pickleable.getsize`
- `Pickleable.load`
- `Pickleable.loads`
- `Pickleable.modify_state`
- `Pickleable.resolve_file_path`
- `Pickleable.save`
- `PlotsBuilderMixin.build_subplots_doc`
- `PlotsBuilderMixin.override_subplots_doc`
- `PlotsBuilderMixin.plots`
- `PlotsBuilderMixin.resolve_plots_setting`
- `Prettified.pprint`
- `PriceRecords.bar_close`
- `PriceRecords.bar_close_time`
- `PriceRecords.bar_high`
- `PriceRecords.bar_low`
- `PriceRecords.bar_open`
- `PriceRecords.bar_open_time`
- `PriceRecords.close`
- `PriceRecords.cls_dir`
- `PriceRecords.col_arr`
- `PriceRecords.col_mapper`
- `PriceRecords.column_only_select`
- `PriceRecords.config`
- `PriceRecords.field_names`
- `PriceRecords.from_records`
- `PriceRecords.get_bar_close`
- `PriceRecords.get_bar_close_time`
- `PriceRecords.get_bar_high`
- `PriceRecords.get_bar_low`
- `PriceRecords.get_bar_open`
- `PriceRecords.get_bar_open_time`
- `PriceRecords.group_select`
- `PriceRecords.high`
- `PriceRecords.id_arr`
- `PriceRecords.idx_arr`
- `PriceRecords.iloc`
- `PriceRecords.indexing_func`
- `PriceRecords.indexing_func_meta`
- `PriceRecords.indexing_kwargs`
- `PriceRecords.loc`
- `PriceRecords.low`
- `PriceRecords.open`

- [PriceRecords.pd_mask](#)
- [PriceRecords.range_only_select](#)
- [PriceRecords.readable](#)
- [PriceRecords.rec_state](#)
- [PriceRecords.reccarray](#)
- [PriceRecords.records](#)
- [PriceRecords.records_arr](#)
- [PriceRecords.records_readable](#)
- [PriceRecords.resample](#)
- [PriceRecords.resolve_column_stack_kwargs](#)
- [PriceRecords.resolve_row_stack_kwargs](#)
- [PriceRecords.self_aliases](#)
- [PriceRecords.values](#)
- [PriceRecords.wrapper](#)
- [PriceRecords.xloc](#)
- [Records.apply](#)
- [Records.apply_mask](#)
- [Records.build_field_config_doc](#)
- [Records.column_stack](#)
- [Records.column_stack_records_arrs](#)
- [Records.count](#)
- [Records.coverage_map](#)
- [Records.first_n](#)
- [Records.get_apply_mapping_arr](#)
- [Records.get_apply_mapping_str_arr](#)
- [Records.get_column_stack_record_indices](#)
- [Records.get_field_arr](#)
- [Records.get_field_mapping](#)
- [Records.get_field_name](#)
- [Records.get_field_setting](#)
- [Records.get_field_title](#)
- [Records.get_map_field](#)
- [Records.get_map_field_to_columns](#)
- [Records.get_map_field_to_index](#)
- [Records.get_pd_mask](#)
- [Records.get_row_stack_record_indices](#)
- [Records.has_conflicts](#)
- [Records.is_sorted](#)
- [Records.last_n](#)
- [Records.map](#)
- [Records.map_array](#)
- [Records.map_field](#)
- [Records.override_field_config_doc](#)
- [Records.prepare_customdata](#)
- [Records.random_n](#)
- [Records.replace](#)
- [Records.resample_meta](#)
- [Records.resample_records_arr](#)
- [Records.row_stack](#)
- [Records.row_stack_records_arrs](#)
- [Records.select_cols](#)
- [Records.sort](#)
- [Records.to_readable](#)
- [StatsBuilderMixin.build_metrics_doc](#)

- [StatsBuilderMixin.override_metrics_doc](#)
- [StatsBuilderMixin.resolve_stats_setting](#)
- [StatsBuilderMixin.stats](#)
- [Wrapping.apply_to_index](#)
- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping regroup](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

Subclasses

- [FSOrders](#)

col cached_property ⓘ

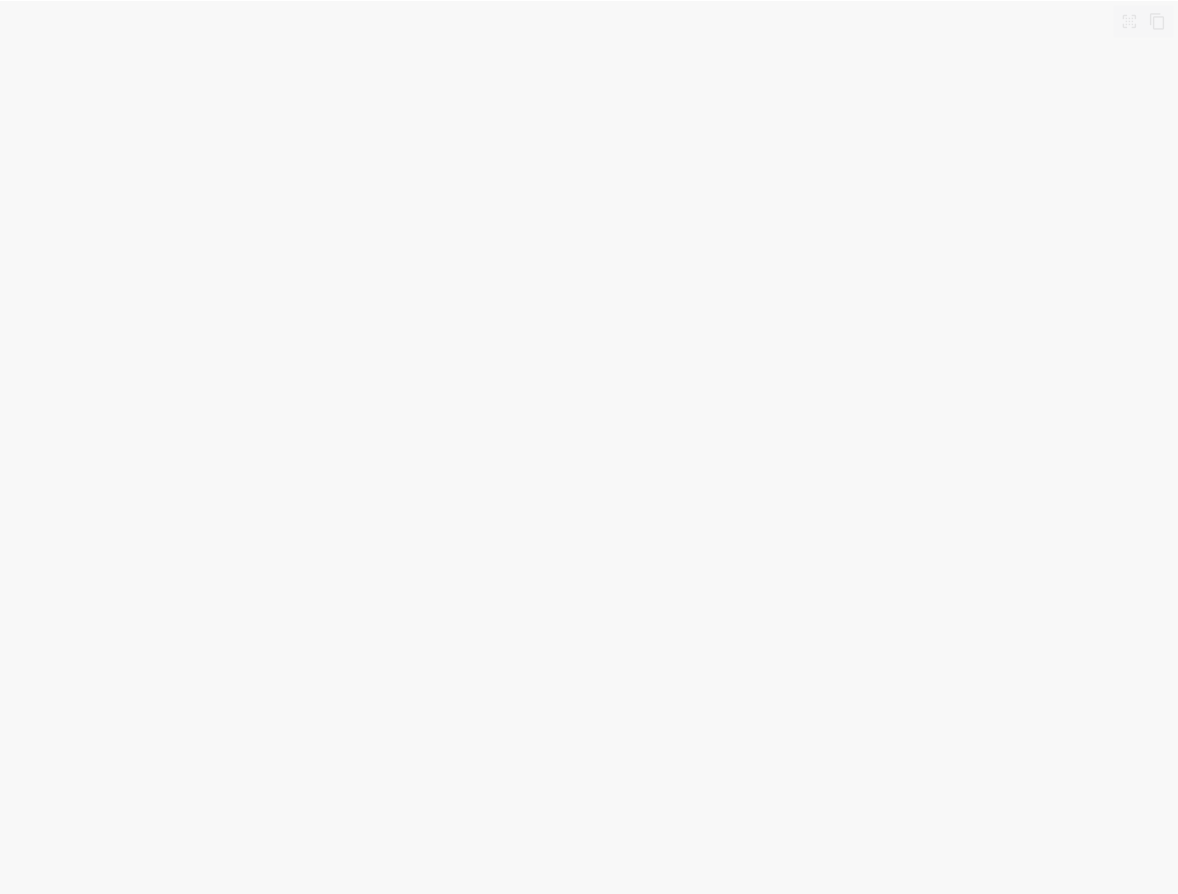
Mapped array of the field `col`.

fees cached_property ⓘ

Mapped array of the field `fees`.

field_config property ⓘ

Field config of [Orders](#).



```

HybridConfig(
  dtype=np.dtype([
    ('id', 'int64'),
    ('col', 'int64'),
    ('idx', 'int64'),
    ('size', 'float64'),
    ('price', 'float64'),
    ('fees', 'float64'),
    ('side', 'int64')
  ]),
  settings=dict(
    id=dict(
      name='id',
      title='Order Id',
      mapping='ids'
    ),
    col=dict(
      name='col',
      title='Column',
      mapping='columns',
      as_customdata=False
    ),
    idx=dict(
      name='idx',
      title='Index',
      mapping='index'
    ),
    size=dict(
      title='Size'
    ),
    price=dict(
      title='Price'
    ),
    fees=dict(
      title='Fees'
    ),
    side=dict(
      title='Side',
      mapping=OrderSideT(
        Buy=0,
        Sell=1
      ),
      as_customdata=False
    )
  )
)

```

Returns `Orders._field_config`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change fields, you can either change the config in-place, override this property, or overwrite the instance variable `Orders._field_config`.

get_long_view method

```

Orders.get_long_view(
    init_position=0.0,
    init_price=nan,
    jitted=None,
    chunked=None
)

```

See [get_long_view_orders_nb](#).

get_price_status method

```

Orders.get_price_status(
    **kwargs
)

```

See [price_status_nb](#).

get_short_view method

```

Orders.get_short_view(
    init_position=0.0,
    init_price=nan,
    jitted=None,
    chunked=None
)

```

See [get_short_view_orders_nb](#).

get_signed_size method


```
Orders.get_signed_size(  
    **kwargs  
)
```

Get signed size.

get_value method

```
Orders.get_value(  
    **kwargs  
)
```

Get value.

get_weighted_price method

```
Orders.get_weighted_price(  
    group_by=None,  
    jitted=None,  
    chunked=None,  
    wrap_kwargs=None,  
    **kwargs  
)
```

Get size-weighted price average.

id cached_property

Mapped array of the field `id`.

idx cached_property

Mapped array of the field `idx`.

long_view cached_property

[Orders.get_long_view](#) with default arguments.

metrics property

Metrics supported by [Orders](#).

```
HybridConfig(  
    start_index=dict(  
        title='Start Index',  
        calc_func=<function Orders.<lambda> at 0x15fc36e80>,  
        agg_func=None,  
        tags='wrapper'  
    ),  
    end_index=dict(  
        title='End Index',  
        calc_func=<function Orders.<lambda> at 0x15fc36f20>,  
        agg_func=None,  
        tags='wrapper'  
    ),  
    total_duration=dict(  
        title='Total Duration',  
        calc_func=<function Orders.<lambda> at 0x15fc36fc0>,  
        apply_to_timedelta=True,  
        agg_func=None,  
        tags='wrapper'  
    ),  
    total_records=dict(  
        title='Total Records',  
        calc_func='count',  
        tags='records'  
    ),  
    side_counts=dict(  
        title='Side Counts',  
        calc_func='side.value_counts',  
        incl_all_keys=True,  
        post_calc_func=<function Orders.<lambda> at 0x15fc37060>,  
        tags=[  
            'orders',  
            'side'  
        ]  
    )  
)
```

```

    ),
    size=dict(
        title='Size',
        calc_func='size.describe',
        post_calc_func=<function Orders.<lambda> at 0x15fc37100>,
        tags=[
            'orders',
            'size'
        ]
    ),
    fees=dict(
        title='Fees',
        calc_func='fees.describe',
        post_calc_func=<function Orders.<lambda> at 0x15fc371a0>,
        tags=[
            'orders',
            'fees'
        ]
    ),
    weighted_buy_price=dict(
        title='Weighted Buy Price',
        calc_func='side_buy.get_weighted_price',
        tags=[
            'orders',
            'buy',
            'price'
        ]
    ),
    weighted_sell_price=dict(
        title='Weighted Sell Price',
        calc_func='side_sell.get_weighted_price',
        tags=[
            'orders',
            'sell',
            'price'
        ]
    )
)
)

```

Returns `Orders._metrics`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change metrics, you can either change the config in-place, override this property, or overwrite the instance variable `Orders._metrics`.

plot method

```

Orders.plot(
    column=None,
    plot_ohlc=True,
    plot_close=True,
    ohlc_type=None,
    ohlc_trace_kwargs=None,
    close_trace_kwargs=None,
    buy_trace_kwargs=None,
    sell_trace_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    **layout_kwargs
)

```

Plot orders.

Args

column : str

Name of the column to plot.

plot_ohlc : bool

Whether to plot OHLC.

plot_close : bool

Whether to plot close.

ohlc_type

Either 'OHLC', 'Candlestick' or Plotly trace.

Pass None to use the default.

ohlc_trace_kwargs : dict

Keyword arguments passed to `ohlc_type`.

close_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for **Orders.close**.

`buy_trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Buy" markers.

`sell_trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Sell" markers.

`add_trace_kwargs` : dict

Keyword arguments passed to `add_trace`.

`fig` : Figure or FigureWidget

Figure to add traces to.

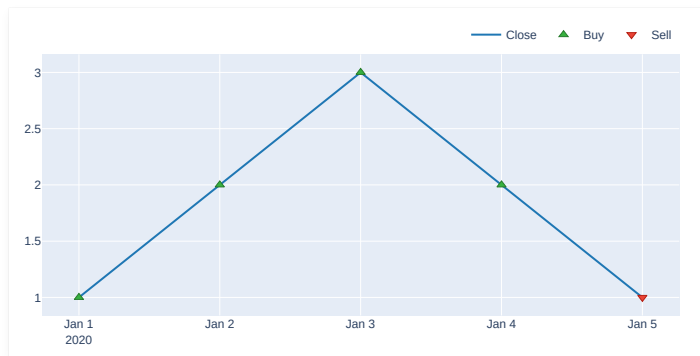
`**layout_kwargs`

Keyword arguments for layout.

Usage

```
>>> index = pd.date_range("2020", periods=5)
>>> price = pd.Series([1., 2., 3., 2., 1.], index=index)
>>> size = pd.Series([1., 1., 1., 1., -1.], index=index)
>>> orders = vbt.Portfolio.from_orders(price, size).orders

>>> orders.plot().show()
```



`plots_defaults` class property

Defaults for `PlotsBuilderMixin.plots`.

Merges `PriceRecords.plots_defaults` and `plots` from `orders`.

`price` cached_property

Mapped array of the field `price`.

`price_status` cacheable_property

`Orders.get_price_status` with default arguments.

`short_view` cached_property

`Orders.get_short_view` with default arguments.

`side` cached_property

Mapped array of the field `side`.

`side_buy` cached_property

Records filtered by `side == 0`.

`side_sell` cached_property

Records filtered by `side == 1`.

`signed_size` cacheable_property

[Orders.get_signed_size](#) with default arguments.

`size` cached_property

Mapped array of the field `size`.

`stats_defaults` class property

Defaults for [StatsBuilderMixin.stats](#).

Merges [PriceRecords.stats_defaults](#) and `stats` from [orders](#).

`subplots` property

Subplots supported by [Orders](#).

```
HybridConfig(
    plot=dict(
        title='Orders',
        yaxis_kwargs=dict(
            title='Price'
        ),
        check_is_not_grouped=True,
        plot_func='plot',
        tags='orders'
    )
)
```

Returns `Orders._subplots`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change subplots, you can either change the config in-place, override this property, or overwrite the instance variable `Orders._subplots`.

`value` cacheable_property

[Orders.get_value](#) with default arguments.

`weighted_price` cached_property

[Orders.get_weighted_price](#) with default arguments.

API portfolio

base portfolio

base module

Base class for simulating a portfolio and measuring its performance.

returns_acc_config ReadonlyConfig

Config of returns accessor methods to be attached to [Portfolio](#).

```
ReadonlyConfig(
    daily_returns=dict(
        source_name='daily'
    ),
    annual_returns=dict(
        source_name='annual'
    ),
    cumulative_returns=dict(
        source_name='cumulative'
    ),
    annualized_return=dict(
        source_name='annualized'
    ),
    annualized_volatility=dict(),
    calmar_ratio=dict(),
    omega_ratio=dict(),
    sharpe_ratio=dict(),
    sharpe_ratio_std=dict(),
    prob_sharpe_ratio=dict(),
    deflated_sharpe_ratio=dict(),
    downside_risk=dict(),
    sortino_ratio=dict(),
    information_ratio=dict(),
    beta=dict(),
    alpha=dict(),
    tail_ratio=dict(),
    value_at_risk=dict(),
    cond_value_at_risk=dict(),
    capture_ratio=dict(),
    up_capture_ratio=dict(),
    down_capture_ratio=dict(),
    drawdown=dict(),
    max_drawdown=dict()
)
```

shortcut_config ReadonlyConfig

Config of shortcut properties to be attached to [Portfolio](#).

```
ReadonlyConfig(
    filled_close=dict(
        group_by_aware=False,
        decorator=<class 'vectorbtpro.utils.decorators.cached_property'>
    ),
    filled_bm_close=dict(
        group_by_aware=False,
        decorator=<class 'vectorbtpro.utils.decorators.cached_property'>
    ),
    weights=dict(
        group_by_aware=False,
        decorator=<class 'vectorbtpro.utils.decorators.cached_property'>,
        obj_type='red_array'
    ),
    long_view=dict(
        obj_type='portfolio'
    ),
    short_view=dict(
        obj_type='portfolio'
    ),
    orders=dict(
        obj_type='records',
        field_aliases=(
            'order_records'
        ),
        wrap_func=<function <lambda> at 0x16a986e80>,
        indexing_func=functools.partial(<function records_indexing_func at 0x16a925b20>, cls='orders_cls'),
        resample_func=functools.partial(<function records_resample_func at 0x16a925c60>, cls='orders_cls')
    ),
    logs=dict(
        obj_type='records',
        field_aliases=(

```

```

        'log_records'
    ),
    wrap_func=<function <lambda> at 0x16a986f20>,
    indexing_func=functools.partial(<function records_indexing_func at 0x16a925b20>, cls='logs_cls'),
    resample_func=functools.partial(<function records_resample_func at 0x16a925c60>, cls='logs_cls')
),
entry_trades=dict(
    obj_type='records',
    field_aliases=(
        'entry_trade_records'
    ),
    wrap_func=<function <lambda> at 0x16a986fc0>,
    indexing_func=functools.partial(<function records_indexing_func at 0x16a925b20>, cls='entry_trades_cls'),
    resample_func=functools.partial(<function records_resample_func at 0x16a925c60>, cls='entry_trades_cls')
),
exit_trades=dict(
    obj_type='records',
    field_aliases=(
        'exit_trade_records'
    ),
    wrap_func=<function <lambda> at 0x16a987060>,
    indexing_func=functools.partial(<function records_indexing_func at 0x16a925b20>, cls='exit_trades_cls'),
    resample_func=functools.partial(<function records_resample_func at 0x16a925c60>, cls='exit_trades_cls')
),
trades=dict(
    obj_type='records',
    field_aliases=(
        'trade_records'
    ),
    wrap_func=<function <lambda> at 0x16a987100>,
    indexing_func=functools.partial(<function records_indexing_func at 0x16a925b20>, cls='trades_cls'),
    resample_func=functools.partial(<function records_resample_func at 0x16a925c60>, cls='trades_cls')
),
trade_history=dict(),
signals=dict(),
positions=dict(
    obj_type='records',
    field_aliases=(
        'position_records'
    ),
    wrap_func=<function <lambda> at 0x16a9871a0>,
    indexing_func=functools.partial(<function records_indexing_func at 0x16a925b20>, cls='positions_cls'),
    resample_func=functools.partial(<function records_resample_func at 0x16a925c60>, cls='positions_cls')
),
drawdowns=dict(
    obj_type='records',
    field_aliases=(
        'drawdown_records'
    ),
    wrap_func=<function <lambda> at 0x16a987240>,
    indexing_func=functools.partial(<function records_indexing_func at 0x16a925b20>, cls='drawdowns_cls', groups_only=True),
    resample_func=functools.partial(<function records_resample_func at 0x16a925c60>, cls='drawdowns_cls')
),
init_position=dict(
    obj_type='red_array',
    group_by_aware=False
),
asset_flow=dict(
    group_by_aware=False,
    resample_func='sum',
    resample_kwargs=dict(
        wrap_kwargs=dict(
            fillna=0.0
        )
    )
),
long_asset_flow=dict(
    method_name='get_asset_flow',
    group_by_aware=False,
    method_kwargs=dict(
        direction='longonly'
    ),
    resample_func='sum',
    resample_kwargs=dict(
        wrap_kwargs=dict(
            fillna=0.0
        )
    )
),
short_asset_flow=dict(
    method_name='get_asset_flow',
    group_by_aware=False,
    method_kwargs=dict(
        direction='shortonly'
    ),
    resample_func='sum',
    resample_kwargs=dict(
        wrap_kwargs=dict(
            fillna=0.0
        )
    )
),
assets=dict(
    group_by_aware=False
),
long_assets=dict(
    method_name='get_assets',
    group_by_aware=False,

```

```

        method_kwargs=dict(
            direction='longonly'
        )
    ),
    short_assets=dict(
        method_name='get_assets',
        group_by_aware=False,
        method_kwargs=dict(
            direction='shortonly'
        )
    ),
    position_mask=dict(),
    long_position_mask=dict(
        method_name='get_position_mask',
        method_kwargs=dict(
            direction='longonly'
        )
    ),
    short_position_mask=dict(
        method_name='get_position_mask',
        method_kwargs=dict(
            direction='shortonly'
        )
    ),
    position_coverage=dict(
        obj_type='red_array'
    ),
    long_position_coverage=dict(
        method_name='get_position_coverage',
        obj_type='red_array',
        method_kwargs=dict(
            direction='longonly'
        )
    ),
    short_position_coverage=dict(
        method_name='get_position_coverage',
        obj_type='red_array',
        method_kwargs=dict(
            direction='shortonly'
        )
    ),
    position_entry_price=dict(
        group_by_aware=False
    ),
    position_exit_price=dict(
        group_by_aware=False
    ),
    init_cash=dict(
        obj_type='red_array'
    ),
    cash_deposits=dict(
        resample_func='sum',
        resample_kwargs=dict(
            wrap_kwargs=dict(
                fillna=0.0
            )
        )
    ),
    total_cash_deposits=dict(
        obj_type='red_array'
    ),
    cash_earnings=dict(
        resample_func='sum',
        resample_kwargs=dict(
            wrap_kwargs=dict(
                fillna=0.0
            )
        )
    ),
    total_cash_earnings=dict(
        obj_type='red_array'
    ),
    cash_flow=dict(
        resample_func='sum',
        resample_kwargs=dict(
            wrap_kwargs=dict(
                fillna=0.0
            )
        )
    ),
    free_cash_flow=dict(
        method_name='get_cash_flow',
        method_kwargs=dict(
            free=True
        ),
        resample_func='sum',
        resample_kwargs=dict(
            wrap_kwargs=dict(
                fillna=0.0
            )
        )
    ),
    cash=dict(),
    position=dict(
        method_name='get_assets',
        group_by_aware=False
    ),
    debt=dict(

```

```

        method_name=None,
        group_by_aware=False
    ),
    locked_cash=dict(
        method_name=None,
        group_by_aware=False
    ),
    free_cash=dict(
        method_name='get_cash',
        method_kwargs=dict(
            free=True
        )
    ),
    init_price=dict(
        obj_type='red_array',
        group_by_aware=False
    ),
    init_position_value=dict(
        obj_type='red_array'
    ),
    init_value=dict(
        obj_type='red_array'
    ),
    input_value=dict(
        obj_type='red_array'
    ),
    asset_value=dict(),
    long_asset_value=dict(
        method_name='get_asset_value',
        method_kwargs=dict(
            direction='longonly'
        )
    ),
    short_asset_value=dict(
        method_name='get_asset_value',
        method_kwargs=dict(
            direction='shortonly'
        )
    ),
    gross_exposure=dict(),
    long_gross_exposure=dict(
        method_name='get_gross_exposure',
        method_kwargs=dict(
            direction='longonly'
        )
    ),
    short_gross_exposure=dict(
        method_name='get_gross_exposure',
        method_kwargs=dict(
            direction='shortonly'
        )
    ),
    net_exposure=dict(),
    value=dict(),
    allocations=dict(
        group_by_aware=False
    ),
    long_allocations=dict(
        method_name='get_allocations',
        method_kwargs=dict(
            direction='longonly'
        )
    ),
    group_by_aware=False
    ),
    short_allocations=dict(
        method_name='get_allocations',
        method_kwargs=dict(
            direction='shortonly'
        )
    ),
    group_by_aware=False
    ),
    total_profit=dict(
        obj_type='red_array'
    ),
    final_value=dict(
        obj_type='red_array'
    ),
    total_return=dict(
        obj_type='red_array'
    ),
    returns=dict(
        resample_func=<function returns_resample_func at 0x16a986a20>
    ),
    log_returns=dict(
        method_name='get_returns',
        method_kwargs=dict(
            log_returns=True
        )
    ),
    resample_func=functools.partial(<function returns_resample_func at 0x16a986a20>, log_returns=True)
    ),
    daily_log_returns=dict(
        method_name='get_returns',
        method_kwargs=dict(
            daily_returns=True,
            log_returns=True
        )
    ),
    resample_func=functools.partial(<function returns_resample_func at 0x16a986a20>, log_returns=True)
    ),

```



```

asset_pnl=dict(
    resample_func='sum',
    resample_kwargs=dict(
        wrap_kwargs=dict(
            fillna=0.0
        )
    )
),
asset_returns=dict(
    resample_func=<function returns_resample_func at 0x16a986a20>
),
market_value=dict(),
market_returns=dict(
    resample_func=<function returns_resample_func at 0x16a986a20>
),
bm_value=dict(),
bm_returns=dict(
    resample_func=<function returns_resample_func at 0x16a986a20>
),
total_market_return=dict(
    obj_type='red_array'
),
daily_returns=dict(
    resample_func=<function returns_resample_func at 0x16a986a20>
),
annual_returns=dict(
    resample_func=<function returns_resample_func at 0x16a986a20>
),
cumulative_returns=dict(),
annualized_return=dict(
    obj_type='red_array'
),
annualized_volatility=dict(
    obj_type='red_array'
),
calmar_ratio=dict(
    obj_type='red_array'
),
omega_ratio=dict(
    obj_type='red_array'
),
sharpe_ratio=dict(
    obj_type='red_array'
),
sharpe_ratio_std=dict(
    obj_type='red_array'
),
prob_sharpe_ratio=dict(
    obj_type='red_array'
),
deflated_sharpe_ratio=dict(
    obj_type='red_array'
),
downside_risk=dict(
    obj_type='red_array'
),
sortino_ratio=dict(
    obj_type='red_array'
),
information_ratio=dict(
    obj_type='red_array'
),
beta=dict(
    obj_type='red_array'
),
alpha=dict(
    obj_type='red_array'
),
tail_ratio=dict(
    obj_type='red_array'
),
value_at_risk=dict(
    obj_type='red_array'
),
cond_value_at_risk=dict(
    obj_type='red_array'
),
capture_ratio=dict(
    obj_type='red_array'
),
up_capture_ratio=dict(
    obj_type='red_array'
),
down_capture_ratio=dict(
    obj_type='red_array'
),
drawdown=dict(),
max_drawdown=dict(
    obj_type='red_array'
)
)
)

```

fix_wrapper_for_records function 

```
fix_wrapper_for_records(  
    pf  
)
```

Allow flags for records that were restricted for portfolio.

records_indexing_func function

```
records_indexing_func(  
    obj,  
    wrapper_meta,  
    cls,  
    groups_only=False,  
    **kwargs  
)
```

Apply indexing function on records.

records_resample_func function

```
records_resample_func(  
    obj,  
    resampler,  
    wrapper,  
    cls,  
    **kwargs  
)
```

Apply resampling function on records.

returns_resample_func function

```
returns_resample_func(  
    obj,  
    resampler,  
    wrapper,  
    fill_with_zero=True,  
    log_returns=False,  
    **kwargs  
)
```

Apply resampling function on returns.

MetaInOutputs class

```
MetaInOutputs(  
    *args,  
    **kwargs  
)
```

Meta class that exposes a read-only class property `MetaFields.in_output_config`.

Superclasses

- `builtins.type`

Subclasses

- `MetaPortfolio`

in_output_config class property

In-output config.

MetaPortfolio class

```
MetaPortfolio(  
    *args,  
    **kwargs  
)
```

Meta class that exposes a read-only class property `StatsBuilderMixin.metrics`.

Superclasses

- [MetaAnalyzable](#)
- [MetaInOutputs](#)
- [MetaPlotsBuilderMixin](#)
- [MetaStatsBuilderMixin](#)
- `builtins.type`

Inherited members

- [MetaAnalyzable.metrics](#)
- [MetaAnalyzable.subplots](#)
- [MetaInOutputs.in_output_config](#)

Portfolio class

```
Portfolio(
    wrapper,
    order_records,
    *,
    close,
    open=None,
    high=None,
    low=None,
    log_records=None,
    cash_sharing=False,
    init_cash='auto',
    init_position=0.0,
    init_price=nan,
    cash_deposits=0.0,
    cash_deposits_as_input=None,
    cash_earnings=0.0,
    sim_start=None,
    sim_end=None,
    call_seq=None,
    in_outputs=None,
    use_in_outputs=None,
    bm_close=None,
    fillna_close=None,
    year_freq=None,
    returns_acc_defaults=None,
    trades_type=None,
    orders_cls=None,
    logs_cls=None,
    trades_cls=None,
    entry_trades_cls=None,
    exit_trades_cls=None,
    positions_cls=None,
    drawdowns_cls=None,
    weights=None,
    **kwargs
)
```

Class for simulating a portfolio and measuring its performance.

Args

wrapper : `ArrayWrapper`

Array wrapper.

See [ArrayWrapper](#).

close : `array_like`

Last asset price at each time step.

order_records : `array_like`

A structured NumPy array of order records.

open : `array_like`

Open price of each bar.

high : `array_like`

High price of each bar.

low : `array_like`

Low price of each bar.

`log_records` : `array_like`

A structured NumPy array of log records.

`cash_sharing` : `bool`

Whether to share cash within the same group.

`init_cash` : `InitCashMode` or `array_like` of `float`

Initial capital.

Can be provided in a format suitable for flexible indexing.

`init_position` : `array_like` of `float`

Initial position.

Can be provided in a format suitable for flexible indexing.

`init_price` : `array_like` of `float`

Initial position price.

Can be provided in a format suitable for flexible indexing.

`cash_deposits` : `array_like` of `float`

Cash deposited/withdrawn at each timestamp.

Can be provided in a format suitable for flexible indexing.

`cash_earnings` : `array_like` of `float`

Earnings added at each timestamp.

Can be provided in a format suitable for flexible indexing.

`sim_start` : `int`, `datetime_like`, or `array_like`

Simulation start per column. Defaults to `None`.

`sim_end` : `int`, `datetime_like`, or `array_like`

Simulation end per column. Defaults to `None`.

`call_seq` : `array_like` of `int`

Sequence of calls per row and group. Defaults to `None`.

`in_outputs` : `namedtuple`

Named tuple with in-output objects.

To substitute [Portfolio](#) attributes, provide already broadcasted and grouped objects. Also see [Portfolio.in_outputs_indexing_func](#) on how in-output objects are indexed.

`use_in_outputs` : `bool`

Whether to return in-output objects when calling properties.

`bm_close` : `array_like`

Last benchmark asset price at each time step.

`fillna_close` : `bool`

Whether to forward and backward fill NaN values in `close`.

Applied after the simulation to avoid NaNs in asset value.

See [Portfolio.get_filled_close](#).

`weights` : `array_like`

Asset weights.

Applied to the initial position, initial cash, cash deposits, cash earnings, and orders.

`trades_type : str or int`

Default [Trades](#) to use across [Portfolio](#).

See [TradesType](#).

`orders_cls : type`

Class for wrapping order records.

`logs_cls : type`

Class for wrapping log records.

`trades_cls : type`

Class for wrapping trade records.

`entry_trades_cls : type`

Class for wrapping entry trade records.

`exit_trades_cls : type`

Class for wrapping exit trade records.

`positions_cls : type`

Class for wrapping position records.

`drawdowns_cls : type`

Class for wrapping drawdown records.

For defaults, see [portfolio](#).

 **Note**

Use class methods with `from_` prefix to build a portfolio. The `__init__` method is reserved for indexing purposes.

 **Note**

This class is meant to be immutable. To change any attribute, use [Configured.replace](#).

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [PortfolioWithInOutputs](#)
- [Prettified](#)
- [SimRangeMixin](#)
- [StatsBuilderMixin](#)
- [Wrapping](#)

Inherited members

- `Analyzable.cls_dir`
- `Analyzable.column_only_select`
- `Analyzable.config`
- `Analyzable.group_select`
- `Analyzable.iloc`
- `Analyzable.indexing_kwargs`
- `Analyzable.loc`
- `Analyzable.range_only_select`
- `Analyzable.rec_state`
- `Analyzable.self_aliases`
- `Analyzable.wrapper`
- `Analyzable.xloc`
- `AttrResolverMixin.deep_getattr`
- `AttrResolverMixin.resolve_attr`
- `Base.chat`
- `Base.find_api`
- `Base.find_assets`
- `Base.find_docs`
- `Base.find_examples`
- `Base.find_messages`
- `Cacheable.get_ca_setup`
- `Chainable.chain`
- `Chainable.pipe`
- `Configured.copy`
- `Configured.equals`
- `Configured.get_writeable_attrs`
- `Configured.prettify`
- `Configured.replace`
- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `HasSettings.get_path_setting`
- `HasSettings.get_path_settings`
- `HasSettings.get_setting`
- `HasSettings.get_settings`
- `HasSettings.has_path_setting`
- `HasSettings.has_path_settings`
- `HasSettings.has_setting`
- `HasSettings.has_settings`
- `HasSettings.reset_settings`
- `HasSettings.resolve_setting`
- `HasSettings.resolve_settings_paths`
- `HasSettings.set_settings`
- `IndexApplier.add_levels`
- `IndexApplier.drop_duplicate_levels`
- `IndexApplier.drop_levels`
- `IndexApplier.drop_redundant_levels`
- `IndexApplier.rename_levels`
- `IndexApplier.select_levels`
- `IndexingBase.indexing_setter_func`
- `PandasIndexer.xs`
- `Pickleable.decode_config`
- `Pickleable.decode_config_node`
- `Pickleable.dumps`
- `Pickleable.encode_config`

- `Pickleable.encode_config_node`
 - `Pickleable.file_exists`
 - `Pickleable.getsize`
 - `Pickleable.load`
 - `Pickleable.loads`
 - `Pickleable.modify_state`
 - `Pickleable.resolve_file_path`
 - `Pickleable.save`
 - `PlotsBuilderMixin.build_subplots_doc`
 - `PlotsBuilderMixin.override_subplots_doc`
 - `PlotsBuilderMixin.plots`
 - `PlotsBuilderMixin.resolve_plots_setting`
 - `Prettified.pprint`
 - `SimRangeMixin.column_stack_sim_end`
 - `SimRangeMixin.column_stack_sim_start`
 - `SimRangeMixin.fit_fig_to_sim_range`
 - `SimRangeMixin.get_sim_duration`
 - `SimRangeMixin.get_sim_end`
 - `SimRangeMixin.get_sim_end_index`
 - `SimRangeMixin.get_sim_start`
 - `SimRangeMixin.get_sim_start_index`
 - `SimRangeMixin.resample_sim_end`
 - `SimRangeMixin.resample_sim_start`
 - `SimRangeMixin.resolve_sim_end`
 - `SimRangeMixin.resolve_sim_end_value`
 - `SimRangeMixin.resolve_sim_start`
 - `SimRangeMixin.resolve_sim_start_value`
 - `SimRangeMixin.row_stack_sim_end`
 - `SimRangeMixin.row_stack_sim_start`
 - `SimRangeMixin.sim_duration`
 - `SimRangeMixin.sim_end`
 - `SimRangeMixin.sim_end_index`
 - `SimRangeMixin.sim_end_indexing_func`
 - `SimRangeMixin.sim_start`
 - `SimRangeMixin.sim_start_index`
 - `SimRangeMixin.sim_start_indexing_func`
 - `StatsBuilderMixin.build_metrics_doc`
 - `StatsBuilderMixin.override_metrics_doc`
 - `StatsBuilderMixin.resolve_stats_setting`
 - `StatsBuilderMixin.stats`
 - `Wrapping.apply_to_index`
 - `Wrapping.as_param`
 - `Wrapping.get_item_keys`
 - `Wrapping.items`
 - `Wrapping.resolve_column_stack_kwargs`
 - `Wrapping.resolve_row_stack_kwargs`
 - `Wrapping.resolve_self`
 - `Wrapping.resolve_stack_kwargs`
 - `Wrapping.select_col`
 - `Wrapping.select_col_from_obj`
 - `Wrapping.split`
 - `Wrapping.split_apply`
-

allocations cacheable_property 

[Portfolio.get_allocations](#) with default arguments.

alpha cached_property 

[Portfolio.get_alpha](#) with default arguments.

annual_returns cacheable_property 

[Portfolio.get_annual_returns](#) with default arguments.

annualized_return cached_property 

[Portfolio.get_annualized_return](#) with default arguments.

annualized_volatility cached_property 

[Portfolio.get_annualized_volatility](#) with default arguments.

apply_weights method 

```
Portfolio.apply_weights(  
    weights=None,  
    rescale=False,  
    group_by=None,  
    apply_group_by=False,  
    **kwargs  
)
```

Get view of portfolio with asset weights applied and optionally rescaled.

If `rescale` is True, weights are rescaled in respect to other weights in the same group. For example, weights 0.5 and 0.5 are rescaled to 1.0 and 1.0 respectively, while weights 0.7 and 0.3 are rescaled to 1.4 ($1.4 * 0.5 = 0.7$) and 0.6 ($0.6 * 0.5 = 0.3$) respectively.

asset_flow cacheable_property 

[Portfolio.get_asset_flow](#) with default arguments.

asset_pnl cacheable_property 

[Portfolio.get_asset_pnl](#) with default arguments.

asset_returns cacheable_property 

[Portfolio.get_asset_returns](#) with default arguments.

asset_value cacheable_property 

[Portfolio.get_asset_value](#) with default arguments.

assets cacheable_property 

[Portfolio.get_assets](#) with default arguments.

beta cached_property 

[Portfolio.get_beta](#) with default arguments.

bm_close custom_property 

Benchmark price per unit series.

bm_returns cacheable_property

[Portfolio.get_bm_returns](#) with default arguments.

bm_value cacheable_property

[Portfolio.get_bm_value](#) with default arguments.

build_in_output_config_doc class method

```
Portfolio.build_in_output_config_doc(
    source_cls=None
)
```

Build in-output config documentation.

call_seq custom_property

Sequence of calls per row and group.

calmar_ratio cached_property

[Portfolio.get_calmar_ratio](#) with default arguments.

capture_ratio cached_property

[Portfolio.get_capture_ratio](#) with default arguments.

cash cacheable_property

[Portfolio.get_cash](#) with default arguments.

cash_deposits cacheable_property

[Portfolio.get_cash_deposits](#) with default arguments.

cash_deposits_as_input class property

Whether to add cash deposits to the input value when calculating returns.

Otherwise, will subtract them from the output value.

cash_earnings cacheable_property

[Portfolio.get_cash_earnings](#) with default arguments.

cash_flow cacheable_property

[Portfolio.get_cash_flow](#) with default arguments.

cash_sharing class property

Whether to share cash within the same group.

close custom_property

Last asset price at each time step.

`close_flex` class property 

[Portfolio.close](#) in a format suitable for flexible indexing.

`column_stack` class method 

```
Portfolio.column_stack(
    *objs,
    wrapper_kwargs=None,
    group_by=None,
    ffill_close=False,
    fbfill_close=False,
    **kwargs
)
```

Stack multiple [Portfolio](#) instances along columns.

Uses [ArrayWrapper.column_stack](#) to stack the wrappers.

Cash sharing must be the same among all objects.

Two-dimensional arrays are stacked using [ArrayWrapper.column_stack_arrs](#) while one-dimensional arrays are stacked using [ArrayWrapper.concat_arrs](#). In-outputs are stacked using [Portfolio.column_stack_in_outputs](#). Records are stacked using [Records.column_stack_records_arrs](#).

`column_stack_in_outputs` class method 

```
Portfolio.column_stack_in_outputs(
    *objs,
    **kwargs
)
```

Stack [Portfolio.in_outputs](#) along columns.

All in-output tuples must be either None or have the same fields.

If the field can be found in the attributes of this [Portfolio](#) instance, reads the attribute's options to get requirements for the type and layout of the in-output object.

For each field in [Portfolio.in_outputs](#), resolves the field's options by parsing its name with [Portfolio.parse_field_options](#) and also looks for options in [Portfolio.in_output_config](#). Performs stacking on the in-output objects of the same field using [Portfolio.column_stack_objs](#).

`column_stack_objs` class method 

```
Portfolio.column_stack_objs(
    objs,
    wrappers,
    grouping='columns_or_groups',
    obj_name=None,
    obj_type=None,
    wrapper=None,
    cash_sharing=False,
    column_stack_func=None,
    **kwargs
)
```

Stack (one and two-dimensional) objects along column.

`column_stack_func` must take the portfolio class, and all the arguments passed to this method. If you don't need any of the arguments, make `column_stack_func` accept them as `**kwargs`.

If all the objects are None, boolean, or empty, returns the first one.

`cond_value_at_risk` cached_property 

[Portfolio.get_cond_value_at_risk](#) with default arguments.

`cumulative_returns` cacheable_property 

[Portfolio.get_cumulative_returns](#) with default arguments.

daily_log_returns cacheable_property 

[Portfolio.get_returns](#) with arguments `{'daily_returns': True, 'log_returns': True}`.

daily_returns cacheable_property 

[Portfolio.get_daily_returns](#) with default arguments.

debt cacheable_property 

`Portfolio.None` with default arguments.

deflated_sharpe_ratio cached_property 

[Portfolio.get_deflated_sharpe_ratio](#) with default arguments.

disable_weights method 

```
Portfolio.disable_weights(  
    **kwargs  
)
```



Get view of portfolio with asset weights disabled.

down_capture_ratio cached_property 

[Portfolio.get_down_capture_ratio](#) with default arguments.

downside_risk cached_property 

[Portfolio.get_downside_risk](#) with default arguments.

drawdown cacheable_property 

[Portfolio.get_drawdown](#) with default arguments.

drawdowns cached_property 

[Portfolio.get_drawdowns](#) with default arguments.

drawdowns_cls class property 

Class for wrapping drawdown records.

entry_trades cached_property 

[Portfolio.get_entry_trades](#) with default arguments.

entry_trades_cls class property 

Class for wrapping entry trade records.

exit_trades cached_property 

[Portfolio.get_exit_trades](#) with default arguments.

exit_trades_cls class property 

Class for wrapping exit trade records.

`filled_bm_close` cached_property 

[Portfolio.get_filled_bm_close](#) with default arguments.

`filled_close` cached_property 

[Portfolio.get_filled_close](#) with default arguments.

`fillna_close` class property 

Whether to forward-backward fill NaN values in [Portfolio.close](#).

`final_value` cached_property 

[Portfolio.get_final_value](#) with default arguments.

`free_cash` cacheable_property 

[Portfolio.get_cash](#) with arguments `{'free': True}`.

`free_cash_flow` cacheable_property 

[Portfolio.get_cash_flow](#) with arguments `{'free': True}`.

`from_def_order_func` class method 

```
Portfolio.from_def_order_func(
    close,
    size=None,
    size_type=None,
    direction=None,
    price=None,
    fees=None,
    fixed_fees=None,
    slippage=None,
    min_size=None,
    max_size=None,
    size_granularity=None,
    leverage=None,
    leverage_mode=None,
    reject_prob=None,
    price_area_vio_mode=None,
    allow_partial=None,
    raise_reject=None,
    log=None,
    pre_segment_func_nb=None,
    order_func_nb=None,
    flex_order_func_nb=None,
    val_price=None,
    sim_start=None,
    sim_end=None,
    call_seq=None,
    flexible=False,
    broadcast_named_args=None,
    broadcast_kwargs=None,
    chunked=None,
    return_preparer=False,
    return_prep_result=False,
    return_sim_out=False,
    **kwargs
)
```

Build portfolio from the default order function.

Default order function takes size, price, fees, and other available information, and issues an order at each column and time step. Additionally, it uses a segment preprocessing function that overrides the valuation price and sorts the call sequence. This way, it behaves similarly to [Portfolio.from_orders](#), but allows injecting pre- and postprocessing functions to have more control over the execution. It also knows how to chunk each argument. The only disadvantage is that [Portfolio.from_orders](#) is more optimized towards performance (up to 5x).

If `flexible` is True:

- `pre_segment_func_nb` is [def_flex_pre_segment_func_nb](#)

- `flex_order_func_nb` is `def_flex_order_func_nb`

If `flexible` is `False`:

- `pre_segment_func_nb` is `def_pre_segment_func_nb`
- `order_func_nb` is `def_order_func_nb`

Prepared by [FDOFPreparer](#).

For details on other arguments, see [Portfolio.from_orders](#) and [Portfolio.from_order_func](#).

Usage

- Working with [Portfolio.from_def_order_func](#) is a similar experience as working with [Portfolio.from_orders](#):

```
>>> close = pd.Series([1, 2, 3, 4, 5])
>>> pf = vbt.Portfolio.from_def_order_func(close, 10)

>>> pf.assets
0    10.0
1    20.0
2    30.0
3    40.0
4    40.0
dtype: float64
>>> pf.cash
0    90.0
1    70.0
2    40.0
3     0.0
4     0.0
dtype: float64
```

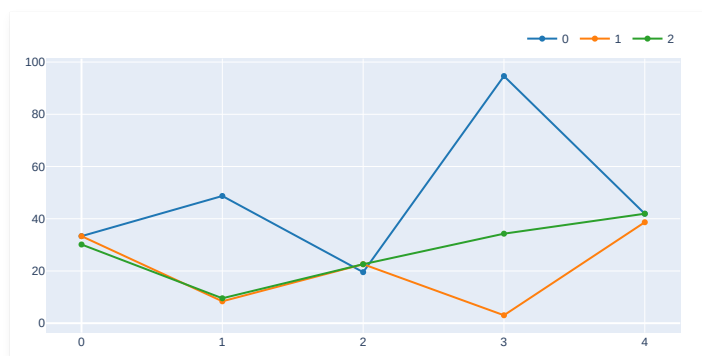
- Equal-weighted portfolio as in the example under [Portfolio.from_order_func](#) but much less verbose and with asset value pre-computed during the simulation (= faster):

```
>>> np.random.seed(42)
>>> close = np.random.uniform(1, 10, size=(5, 3))

>>> @njit
... def post_segment_func_nb(c):
...     for col in range(c.from_col, c.to_col):
...         c.in_outputs.asset_value_pc[c.i, col] = c.last_position[col] * c.last_val_price[col]

>>> pf = vbt.Portfolio.from_def_order_func(
...     close,
...     size=1/3,
...     size_type='targetpercent',
...     direction='longonly',
...     fees=0.001,
...     fixed_fees=1.,
...     slippage=0.001,
...     segment_mask=2,
...     cash_sharing=True,
...     group_by=True,
...     call_seq='auto',
...     post_segment_func_nb=post_segment_func_nb,
...     call_post_segment=True,
...     in_outputs=dict(asset_value_pc=vbt.RepEval('np.empty_like(close)'))
... )

>>> asset_value = pf.wrapper.wrap(pf.in_outputs.asset_value_pc, group_by=False)
>>> asset_value.vbt.plot().show()
```



from_holding class method

```
Portfolio.from_holding(
    close,
    direction=None,
    at_first_valid_in='close',
```

```

        close_at_end=None,
        dynamic_mode=False,
        **kwargs
    )

```

Simulate portfolio from plain holding using signals.

If `close_at_end` is True, will place an opposite signal at the very end.

`**kwargs` are passed to the class method [Portfolio.from_signals](#).

from_optimizer class method

```

Portfolio.from_optimizer(
    close,
    optimizer,
    pf_method='from_orders',
    squeeze_groups=True,
    dropna=None,
    fill_value=nan,
    size_type='targetpercent',
    direction=None,
    cash_sharing=True,
    call_seq='auto',
    group_by=None,
    **kwargs
)

```

Build portfolio from an optimizer of type [PortfolioOptimizer](#).

Uses [Portfolio.from_orders](#) as the base simulation method.

The size type is 'targetpercent'. If there are positive and negative values, the direction is automatically set to 'both', otherwise to 'longonly' for positive-only and 'shortonly' for negative-only values. Also, the cash sharing is set to True, the call sequence is set to 'auto', and the grouper is set to the grouper of the optimizer by default.

Usage

```

>>> close = pd.DataFrame({
...     "MSFT": [1, 2, 3, 4, 5],
...     "GOOG": [5, 4, 3, 2, 1],
...     "AAPL": [1, 2, 3, 2, 1]
... }, index=pd.date_range(start="2020-01-01", periods=5))

>>> pfo = vbt.PortfolioOptimizer.from_random(
...     close.vbt.wrapper,
...     every="2D",
...     seed=42
... )
>>> pfo.fill_allocations()

```

	MSFT	GOOG	AAPL
2020-01-01	0.182059	0.462129	0.355812
2020-01-02	NaN	NaN	NaN
2020-01-03	0.657381	0.171323	0.171296
2020-01-04	NaN	NaN	NaN
2020-01-05	0.038078	0.567845	0.394077

```

>>> pf = vbt.Portfolio.from_optimizer(close, pfo)
>>> pf.get_asset_value(group_by=False).vbt / pf.value

```

alloc_group	MSFT	GOOG	AAPL	group
2020-01-01	0.182059	0.462129	0.355812	<< rebalanced
2020-01-02	0.251907	0.255771	0.492322	
2020-01-03	0.657381	0.171323	0.171296	<< rebalanced
2020-01-04	0.793277	0.103369	0.103353	
2020-01-05	0.038078	0.567845	0.394077	<< rebalanced

from_order_func class method

```

Portfolio.from_order_func(
    close,
    *,
    init_cash=None,
    init_position=None,
    init_price=None,
    cash_deposits=None,
    cash_earnings=None,
    cash_sharing=None,
    sim_start=None,
    sim_end=None,
    call_seq=None,
    attach_call_seq=None,
    segment_mask=None,
    call_pre_segment=None,
    call_post_segment=None,
    pre_sim_func_nb=None,
    pre_sim_args=(),
)

```

```

    post_sim_func_nb=None,
    post_sim_args=(),
    pre_group_func_nb=None,
    pre_group_args=(),
    post_group_func_nb=None,
    post_group_args=(),
    pre_row_func_nb=None,
    pre_row_args=(),
    post_row_func_nb=None,
    post_row_args=(),
    pre_segment_func_nb=None,
    pre_segment_args=(),
    post_segment_func_nb=None,
    post_segment_args=(),
    order_func_nb=None,
    order_args=(),
    flex_order_func_nb=None,
    flex_order_args=(),
    post_order_func_nb=None,
    post_order_args=(),
    open=None,
    high=None,
    low=None,
    ffill_val_price=None,
    update_value=None,
    fill_pos_info=None,
    track_value=None,
    row_wise=None,
    max_order_records=None,
    max_log_records=None,
    in_outputs=None,
    seed=None,
    group_by=None,
    broadcast_named_args=None,
    broadcast_kwargs=None,
    template_context=None,
    keep_inout_flex=None,
    jitted=None,
    chunked=None,
    staticized=None,
    bm_close=None,
    records=None,
    return_preparer=False,
    return_prep_result=False,
    return_sim_out=False,
    **kwargs
)

```

Build portfolio from a custom order function.

Hint

See [from_order_func_nb](#) for illustrations and argument definitions.

For more details on individual simulation functions:

- `order_func_nb`: See [from_order_func_nb](#)
- `order_func_nb` and `row_wise`: See [from_order_func_rw_nb](#)
- `flex_order_func_nb`: See [from_flex_order_func_nb](#)
- `flex_order_func_nb` and `row_wise`: See [from_flex_order_func_rw_nb](#)

Prepared by [FOFPreparer](#).

Args

`close` : array_like, OHLCDataMixin, FOFPreparer, or PFPPrepResult

Latest asset price at each time step. Will broadcast.

If an instance of [OHLCDataMixin](#), will extract the open, high, low, and close price.

Used for calculating unrealized PnL and portfolio value.

`init_cash` : InitCashMode, float or array_like

See [Portfolio.from_orders](#).

`init_position` : float or array_like

See [Portfolio.from_orders](#).

`init_price` : float or array_like

See [Portfolio.from_orders](#).

`cash_deposits` : float or array_like

See [Portfolio.from_orders](#).

`cash_earnings` : float Or array_like

See [Portfolio.from_orders](#).

`cash_sharing` : bool

Whether to share cash within the same group.

If `group_by` is None, `group_by` becomes True to form a single group with cash sharing.

`sim_start` : int, datetime_like, or array_like

Simulation start row or index (inclusive).

`sim_end` : int, datetime_like, or array_like

Simulation end row or index (exclusive).

`call_seq` : CallSeqType or array_like

Default sequence of calls per row and group.

- Use [CallSeqType](#) to select a sequence type.
- Set to array to specify custom sequence. Will not broadcast.



Note

CallSeqType.Auto must be implemented manually. Use [sort_call_seq_1d_nb](#) or [sort_call_seq_out_1d_nb](#) in `pre_segment_func_nb`.

`attach_call_seq` : bool

See [Portfolio.from_orders](#).

`segment_mask` : int or array_like of bool

Mask of whether a particular segment should be executed.

Supplying an integer will activate every n-th row. Supplying a boolean or an array of boolean will broadcast to the number of rows and groups.

Does not broadcast together with `close` and `broadcast_named_args`, only against the final shape.

`call_pre_segment` : bool

Whether to call `pre_segment_func_nb` regardless of `segment_mask`.

`call_post_segment` : bool

Whether to call `post_segment_func_nb` regardless of `segment_mask`.

`pre_sim_func_nb` : callable

Function called before simulation. Defaults to [no_pre_func_nb](#).

`pre_sim_args` : tuple

Packed arguments passed to `pre_sim_func_nb`.

`post_sim_func_nb` : callable

Function called after simulation. Defaults to [no_post_func_nb](#).

`post_sim_args` : tuple

Packed arguments passed to `post_sim_func_nb`.

`pre_group_func_nb` : callable

Function called before each group. Defaults to [no_pre_func_nb](#).

Called only if `row_wise` is False.

`pre_group_args` : tuple

Packed arguments passed to `pre_group_func_nb`.

`post_group_func_nb` : callable

Function called after each group. Defaults to [no_post_func_nb](#).

Called only if `row_wise` is False.

`post_group_args` : tuple

Packed arguments passed to `post_group_func_nb`.

`pre_row_func_nb` : callable

Function called before each row. Defaults to [no_pre_func_nb](#).

Called only if `row_wise` is True.

`pre_row_args` : tuple

Packed arguments passed to `pre_row_func_nb`.

`post_row_func_nb` : callable

Function called after each row. Defaults to [no_post_func_nb](#).

Called only if `row_wise` is True.

`post_row_args` : tuple

Packed arguments passed to `post_row_func_nb`.

`pre_segment_func_nb` : callable

Function called before each segment. Defaults to [no_pre_func_nb](#).

`pre_segment_args` : tuple

Packed arguments passed to `pre_segment_func_nb`.

`post_segment_func_nb` : callable

Function called after each segment. Defaults to [no_post_func_nb](#).

`post_segment_args` : tuple

Packed arguments passed to `post_segment_func_nb`.

`order_func_nb` : callable

Order generation function.

`order_args`

Packed arguments passed to `order_func_nb`.

`flex_order_func_nb` : callable

Flexible order generation function.

`flex_order_args`

Packed arguments passed to `flex_order_func_nb`.

`post_order_func_nb` : callable

Callback that is called after the order has been processed.

`post_order_args` : tuple

Packed arguments passed to `post_order_func_nb`.

`open` : array_like of float

See [Portfolio.from_orders](#).

`high` : array_like of float

See [Portfolio.from_orders](#).

`low` : array_like of float

See [Portfolio.from_orders](#).

`ffill_val_price : bool`

Whether to track valuation price only if it's known.

Otherwise, unknown `close` will lead to NaN in valuation price at the next timestamp.

`update_value : bool`

Whether to update group value after each filled order.

`fill_pos_info : bool`

Whether to fill position record.

Disable this to make simulation faster for simple use cases.

`track_value : bool`

Whether to track value metrics such as the current valuation price, value, and return.

Disable this to make simulation faster for simple use cases.

`row_wise : bool`

Whether to iterate over rows rather than columns/groups.

`max_order_records : int`

The max number of order records expected to be filled at each column. Defaults to the number of rows in the broadcasted shape.

Set to a lower number if you run out of memory, to 0 to not fill, and to a higher number if there are more than one order expected at each timestamp.

`max_log_records : int`

The max number of log records expected to be filled at each column. Defaults to the number of rows in the broadcasted shape.

Set to a lower number if you run out of memory, to 0 to not fill, and to a higher number if there are more than one order expected at each timestamp.

`in_outputs : mapping_like`

Mapping with in-output objects.

Will be available via [Portfolio.in_outputs](#) as a named tuple.

To substitute [Portfolio](#) attributes, provide already broadcasted and grouped objects, for example, by using `broadcast_named_args` and templates. Also see [Portfolio.in_outputs_indexing_func](#) on how in-output objects are indexed.

When chunking, make sure to provide the chunk taking specification and the merging function. See [merge_sim_outs](#).

**Note**

When using Numba below 0.54, `in_outputs` cannot be a mapping, but must be a named tuple defined globally so Numba can introspect its attributes for pickling.

`seed : int`

See [Portfolio.from_orders](#).

`group_by : any`

See [Portfolio.from_orders](#).

`broadcast_named_args : dict`

See [Portfolio.from_signals](#).

`broadcast_kwargs : dict`

See [Portfolio.from_orders](#).

`template_context : mapping`

See [Portfolio.from_signals](#).

`keep_inout_flex : bool`

Whether to keep arrays that can be edited in-place raw when broadcasting.

Disable this to be able to edit `segment_mask`, `cash_deposits`, and `cash_earnings` during the simulation.

`jitted : any`

See [Portfolio.from_orders](#).

Note

Disabling jitting will not disable jitter (such as Numba) on other functions, only on the main (simulation) function. If necessary, you should ensure that every other function is not compiled as well. For example, when working with Numba, you can do this by using the `py_func` attribute of that function. Or, you can disable Numba entirely by running `os.environ['NUMBA_DISABLE_JIT'] = '1'` before importing vectorbtpro.

Warning

Parallelization assumes that groups are independent and there is no data flowing between them.

`chunked` : any

See [resolve_chunked_option](#).

`staticized` : bool, dict, hashable, or callable

Keyword arguments or task id for staticizing.

If True or dictionary, will be passed as keyword arguments to [cut_and_save_func](#) to save a cacheable version of the simulator to a file. If a hashable or callable, will be used as a task id of an already registered jittable and chunkable simulator. Dictionary allows additional options `override` and `reload` to override and reload an already existing module respectively.

`bm_close` : array_like

See [Portfolio.from_orders](#).

`records` : array_like

See [Portfolio.from_orders](#).

`return_preparer` : bool

See [Portfolio.from_orders](#).

`return_prep_result` : bool

See [Portfolio.from_orders](#).

`return_sim_out` : bool

See [Portfolio.from_orders](#).

****kwargs**

Keyword arguments passed to the [Portfolio](#) constructor.

For defaults, see [portfolio](#). Those defaults are not used to fill NaN values after reindexing: vectorbt uses its own sensible defaults, which are usually NaN for floating arrays and default flags for integer arrays. Use [BCO](#) with `fill_value` to override.

Note

All passed functions must be Numba-compiled if Numba is enabled.

Also see notes on [Portfolio.from_orders](#).

Note

In contrast to other methods, the valuation price is previous `close` instead of the order price since the price of an order is unknown before the call (which is more realistic by the way). You can still override the valuation price in `pre_segment_func_nb`.

Usage

- Buy 10 units each tick using closing price:

```
>>> @njit
... def order_func_nb(c, size):
...     return vbt.pf_nb.order_nb(size=size)

>>> close = pd.Series([1, 2, 3, 4, 5])
>>> pf = vbt.Portfolio.from_order_func(
...     close,
...     order_func_nb=order_func_nb,
...     order_args=(10,)
... )

>>> pf.assets
0    10.0
1    20.0
2    30.0
```

```

3    40.0
4    40.0
dtype: float64
>>> pf.cash
0    90.0
1    70.0
2    40.0
3     0.0
4     0.0
dtype: float64

```

- Reverse each position by first closing it. Keep state of last position to determine which position to open next (just as an example, there are easier ways to do this):

```

>>> @njit
... def pre_group_func_nb(c):
...     last_pos_state = np.array([-1])
...     return (last_pos_state,)

>>> @njit
... def order_func_nb(c, last_pos_state):
...     if c.position_now != 0:
...         return vbt.pf_nb.close_position_nb()
...
...     if last_pos_state[0] == 1:
...         size = -np.inf # open short
...         last_pos_state[0] = -1
...     else:
...         size = np.inf # open long
...         last_pos_state[0] = 1
...     return vbt.pf_nb.order_nb(size=size)

>>> pf = vbt.Portfolio.from_order_func(
...     close,
...     order_func_nb=order_func_nb,
...     pre_group_func_nb=pre_group_func_nb
... )

>>> pf.assets
0    100.000000
1     0.000000
2   -66.666667
3     0.000000
4    26.666667
dtype: float64
>>> pf.cash
0     0.000000
1    200.000000
2    400.000000
3    133.333333
4     0.000000
dtype: float64

```

- Equal-weighted portfolio as in the example under [from_order_func_nb](#):

```

>>> @njit
... def pre_group_func_nb(c):
...     order_value_out = np.empty(c.group_len, dtype=float_)
...     return (order_value_out,)

>>> @njit
... def pre_segment_func_nb(c, order_value_out, size, price, size_type, direction):
...     for col in range(c.from_col, c.to_col):
...         c.last_val_price[col] = vbt.pf_nb.select_from_col_nb(c, col, price)
...     vbt.pf_nb.sort_call_seq_nb(c, size, size_type, direction, order_value_out)
...     return ()

>>> @njit
... def order_func_nb(c, size, price, size_type, direction, fees, fixed_fees, slippage):
...     return vbt.pf_nb.order_nb(
...         size=vbt.pf_nb.select_nb(c, size),
...         price=vbt.pf_nb.select_nb(c, price),
...         size_type=vbt.pf_nb.select_nb(c, size_type),
...         direction=vbt.pf_nb.select_nb(c, direction),
...         fees=vbt.pf_nb.select_nb(c, fees),
...         fixed_fees=vbt.pf_nb.select_nb(c, fixed_fees),
...         slippage=vbt.pf_nb.select_nb(c, slippage)
...     )

>>> np.random.seed(42)
>>> close = np.random.uniform(1, 10, size=(5, 3))
>>> size_template = vbt.RepEval('np.array([[1 / group_lens[0]]])')

>>> pf = vbt.Portfolio.from_order_func(
...     close,
...     order_func_nb=order_func_nb,
...     order_args=(
...         size_template,
...         vbt.Rep('price'),
...         vbt.Rep('size_type'),
...         vbt.Rep('direction'),
...         vbt.Rep('fees'),
...         vbt.Rep('fixed_fees'),
...         vbt.Rep('slippage'),
...     ),
... )

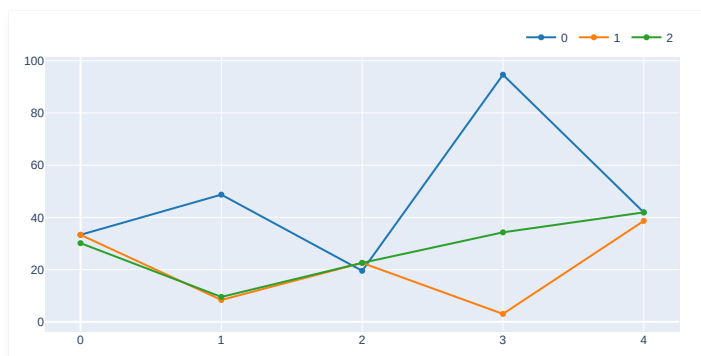
```

```

...     segment_mask=2, # rebalance every second tick
...     pre_group_func_nb=pre_group_func_nb,
...     pre_segment_func_nb=pre_segment_func_nb,
...     pre_segment_args=(
...         size_template,
...         vbt.Rep('price'),
...         vbt.Rep('size_type'),
...         vbt.Rep('direction')
...     ),
...     broadcast_named_args=dict( # broadcast against each other
...         price=close,
...         size_type=vbt.pf_enums.SizeType.TargetPercent,
...         direction=vbt.pf_enums.Direction.LongOnly,
...         fees=0.001,
...         fixed_fees=1.,
...         slippage=0.001
...     ),
...     template_context=dict(np=np), # required by size_template
...     cash_sharing=True, group_by=True, # one group with cash sharing
... )

>>> pf.get_asset_value(group_by=False).vbt.plot().show()

```



Templates are a very powerful tool to prepare any custom arguments after they are broadcast and before they are passed to the simulation function. In the example above, we use `broadcast_named_args` to broadcast some arguments against each other and templates to pass those objects to callbacks. Additionally, we used an evaluation template to compute the size based on the number of assets in each group.

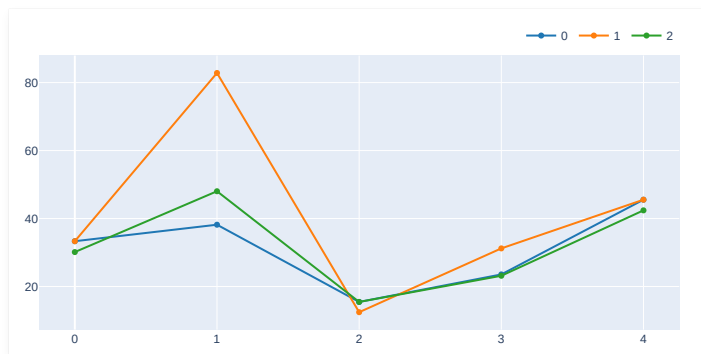
You may ask: why should we bother using broadcasting and templates if we could just pass `size=1/3`? Because of flexibility those features provide: we can now pass whatever parameter combinations we want and it will work flawlessly. For example, to create two groups of equally-allocated positions, we need to change only two parameters:

```

>>> close = np.random.uniform(1, 10, size=(5, 6)) # 6 columns instead of 3
>>> group_by = ['g1', 'g1', 'g1', 'g2', 'g2', 'g2'] # 2 groups instead of 1
>>> # Replace close and group_by in the example above

>>> pf['g1'].get_asset_value(group_by=False).vbt.plot().show()

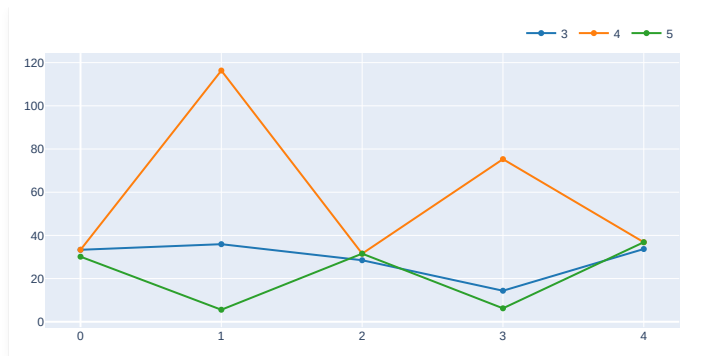
```



```

>>> pf['g2'].get_asset_value(group_by=False).vbt.plot().show()

```



- Combine multiple exit conditions. Exit early if the price hits some threshold before an actual exit:

```
>>> @njit
... def pre_sim_func_nb(c):
...     # We need to define stop price per column once
...     stop_price = np.full(c.target_shape[1], np.nan, dtype=float_)
...     return (stop_price,)

>>> @njit
... def order_func_nb(c, stop_price, entries, exits, size):
...     # Select info related to this order
...     entry_now = vbt.pf_nb.select_nb(c, entries)
...     exit_now = vbt.pf_nb.select_nb(c, exits)
...     size_now = vbt.pf_nb.select_nb(c, size)
...     price_now = vbt.pf_nb.select_nb(c, c.close)
...     stop_price_now = stop_price[c.col]
...
...     # Our logic
...     if entry_now:
...         if c.position_now == 0:
...             return vbt.pf_nb.order_nb(
...                 size=size_now,
...                 price=price_now,
...                 direction=vbt.pf_enums.Direction.LongOnly)
...         elif exit_now or price_now >= stop_price_now:
...             if c.position_now > 0:
...                 return vbt.pf_nb.order_nb(
...                     size=-size_now,
...                     price=price_now,
...                     direction=vbt.pf_enums.Direction.LongOnly)
...             return vbt.pf_enums.NoOrder
...
...     if c.order_result.status == vbt.pf_enums.OrderStatus.Filled:
...         if c.order_result.side == vbt.pf_enums.OrderSide.Buy:
...             # Position entered: Set stop condition
...             stop_price[c.col] = (1 + stop_now) * c.order_result.price
...         else:
...             # Position exited: Remove stop condition
...             stop_price[c.col] = np.nan

>>> @njit
... def post_order_func_nb(c, stop_price, stop):
...     # Same broadcasting as for size
...     stop_now = vbt.pf_nb.select_nb(c, stop)
...
...     if c.order_result.status == vbt.pf_enums.OrderStatus.Filled:
...         if c.order_result.side == vbt.pf_enums.OrderSide.Buy:
...             # Position entered: Set stop condition
...             stop_price[c.col] = (1 + stop_now) * c.order_result.price
...         else:
...             # Position exited: Remove stop condition
...             stop_price[c.col] = np.nan

>>> def simulate(close, entries, exits, size, stop):
...     return vbt.Portfolio.from_order_func(
...         close,
...         order_func_nb=order_func_nb,
...         order_args=(vbt.Rep('entries'), vbt.Rep('exits'), vbt.Rep('size')),
...         pre_sim_func_nb=pre_sim_func_nb,
...         post_order_func_nb=post_order_func_nb,
...         post_order_args=(vbt.Rep('stop'),),
...         broadcast_named_args=dict( # broadcast against each other
...             entries=entries,
...             exits=exits,
...             size=size,
...             stop=stop
...         )
...     )

>>> close = pd.Series([10, 11, 12, 13, 14])
>>> entries = pd.Series([True, True, False, False, False])
>>> exits = pd.Series([False, False, False, True, True])
>>> simulate(close, entries, exits, np.inf, 0.1).asset_flow
0    10.0
1     0.0
2   -10.0
3     0.0
4     0.0
dtype: float64

>>> simulate(close, entries, exits, np.inf, 0.2).asset_flow
0    10.0
1     0.0
2   -10.0
3     0.0
```

```

4      0.0
dtype: float64

>>> simulate(close, entries, exits, np.inf, np.nan).asset_flow
0      10.0
1       0.0
2       0.0
3     -10.0
4       0.0
dtype: float64

```

The reason why stop of 10% does not result in an order at the second time step is because it comes at the same time as entry, so it must wait until no entry is present. This can be changed by replacing the statement "elif" with "if", which would execute an exit regardless if an entry is present (similar to using `ConflictMode.Opposite` in [Portfolio.from_signals](#)).

We can also test the parameter combinations above all at once (thanks to broadcasting using [broadcast](#)):

```

>>> stop = pd.DataFrame([[0.1, 0.2, np.nan]])
>>> simulate(close, entries, exits, np.inf, stop).asset_flow
      0      1      2
0  10.0  10.0  10.0
1   0.0   0.0   0.0
2 -10.0 -10.0   0.0
3   0.0   0.0 -10.0
4   0.0   0.0   0.0

```

Or much simpler using Cartesian product:

```

>>> stop = vbt.Param([0.1, 0.2, np.nan])
>>> simulate(close, entries, exits, np.inf, stop).asset_flow
threshold  0.1  0.2  NaN
0          10.0  10.0  10.0
1           0.0   0.0   0.0
2        -10.0 -10.0   0.0
3           0.0   0.0 -10.0
4           0.0   0.0   0.0

```

This works because `pd.Index` automatically translates into `BCO` with `product` set to `True`.

- Let's illustrate how to generate multiple orders per symbol and bar. For each bar, buy at open and sell at close:

```

>>> @njit
... def flex_order_func_nb(c, size):
...     if c.call_idx == 0:
...         return c.from_col, vbt.pf_nb.order_nb(size=size, price=c.open[c.i, c.from_col])
...     if c.call_idx == 1:
...         return c.from_col, vbt.pf_nb.close_position_nb(price=c.close[c.i, c.from_col])
...     return -1, vbt.pf_enums.NoOrder

>>> open = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})
>>> close = pd.DataFrame({'a': [2, 3, 4], 'b': [3, 4, 5]})
>>> size = 1
>>> pf = vbt.Portfolio.from_order_func(
...     close,
...     flex_order_func_nb=flex_order_func_nb,
...     flex_order_args=(size,),
...     open=open,
...     max_order_records=close.shape[0] * 2
... )

>>> pf.orders.readable
      Order Id Column Timestamp  Size Price Fees Side
0          0      a          0    1.0   1.0  0.0  Buy
1          1      a          0    1.0   2.0  0.0  Sell
2          2      a          1    1.0   2.0  0.0  Buy
3          3      a          1    1.0   3.0  0.0  Sell
4          4      a          2    1.0   3.0  0.0  Buy
5          5      a          2    1.0   4.0  0.0  Sell
6          0      b          0    1.0   4.0  0.0  Buy
7          1      b          0    1.0   3.0  0.0  Sell
8          2      b          1    1.0   5.0  0.0  Buy
9          3      b          1    1.0   4.0  0.0  Sell
10         4      b          2    1.0   6.0  0.0  Buy
11         5      b          2    1.0   5.0  0.0  Sell

```

Warning

Each bar is effectively a black box - we don't know how the price moves in-between. Since trades should come in an order that closely replicates that of the real world, the only pieces of information that always remain in the correct order are the opening and closing price.

`from_orders` class method 

```

Portfolio.from_orders(
    close,
    size=None,
    size_type=None,
    direction=None,

```

```

price=None,
fees=None,
fixed_fees=None,
slippage=None,
min_size=None,
max_size=None,
size_granularity=None,
leverage=None,
leverage_mode=None,
reject_prob=None,
price_area_vio_mode=None,
allow_partial=None,
raise_reject=None,
log=None,
val_price=None,
from_ago=None,
open=None,
high=None,
low=None,
init_cash=None,
init_position=None,
init_price=None,
cash_deposits=None,
cash_earnings=None,
cash_dividends=None,
cash_sharing=None,
sim_start=None,
sim_end=None,
call_seq=None,
attach_call_seq=None,
ffill_val_price=None,
update_value=None,
save_state=None,
save_value=None,
save_returns=None,
skip_empty=None,
max_order_records=None,
max_log_records=None,
seed=None,
group_by=None,
broadcast_kwargs=None,
jitted=None,
chunked=None,
bm_close=None,
records=None,
return_preparer=False,
return_prep_result=False,
return_sim_out=False,
**kwargs
)

```

Simulate portfolio from orders - size, price, fees, and other information.

See [from_orders_nb](#).

Prepared by [FOPreparer](#).

Args

close : array_like, OHLCDataMixin, FOPreparer, or PFPrepResult

Latest asset price at each time step. Will broadcast.

Used for calculating unrealized PnL and portfolio value.

If an instance of [OHLCDataMixin](#), will extract the open, high, low, and close price. If an instance of [FOPreparer](#), will use it as a preparer. If an instance of [PFPrepResult](#), will use it as a preparer result.

size : float or array_like

Size to order. See [Order.size](#). Will broadcast.

size_type : SizeType or array_like

See [SizeType](#) and [Order.size_type](#). Will broadcast.

direction : Direction or array_like

See [Direction](#) and [Order.direction](#). Will broadcast.

price : array_like of float

Order price. Will broadcast.

See [Order.price](#). Can be also provided as [PriceType](#). Options `PriceType.NextOpen` and `PriceType.NextClose` are only applicable per column, that is, they cannot be used inside full arrays. In addition, they require the argument `from_ago` to be None.

fees : float or array_like

Fees in percentage of the order value. See [Order.fees](#). Will broadcast.

`fixed_fees` : float or array_like

Fixed amount of fees to pay per order. See [Order.fixed_fees](#). Will broadcast.

`slippage` : float or array_like

Slippage in percentage of price. See [Order.slippage](#). Will broadcast.

`min_size` : float or array_like

Minimum size for an order to be accepted. See [Order.min_size](#). Will broadcast.

`max_size` : float or array_like

Maximum size for an order. See [Order.max_size](#). Will broadcast.

Will be partially filled if exceeded.

`size_granularity` : float or array_like

Granularity of the size. See [Order.size_granularity](#). Will broadcast.

`leverage` : float or array_like

Leverage. See [Order.leverage](#). Will broadcast.

`leverage_mode` : LeverageMode or array_like

Leverage mode. See [Order.leverage_mode](#). Will broadcast.

`reject_prob` : float or array_like

Order rejection probability. See [Order.reject_prob](#). Will broadcast.

`price_area_vio_mode` : PriceAreaVioMode or array_like

See [PriceAreaVioMode](#). Will broadcast.

`allow_partial` : bool or array_like

Whether to allow partial fills. See [Order.allow_partial](#). Will broadcast.

Does not apply when size is `np.inf`.

`raise_reject` : bool or array_like

Whether to raise an exception if order gets rejected. See [Order.raise_reject](#). Will broadcast.

`log` : bool or array_like

Whether to log orders. See [Order.log](#). Will broadcast.

`val_price` : array_like of float

Asset valuation price. Will broadcast.

Can be also provided as [ValPriceType](#).

- Any `-np.inf` element is replaced by the latest valuation price (`open` or the latest known valuation price if `ffill_val_price`).
- Any `np.inf` element is replaced by the current order price.

Used at the time of decision making to calculate value of each asset in the group, for example, to convert target value into target amount.

Note

In contrast to [Portfolio.from_order_func](#), order price is known beforehand (kind of), thus `val_price` is set to the current order price (using `np.inf`) by default. To valueate using previous close, set it in the settings to `-np.inf`.

Note

Make sure to use timestamp for `val_price` that comes before timestamps of all orders in the group with cash sharing (previous `close` for example), otherwise you're cheating yourself.

`open` : array_like of float

First asset price at each time step. Defaults to `np.nan`. Will broadcast.

Used as a price boundary (see [PriceArea](#)).

`high` : array_like of float

Highest asset price at each time step. Defaults to `np.nan`. Will broadcast.

Used as a price boundary (see [PriceArea](#)).

`low` : array_like of float

Lowest asset price at each time step. Defaults to `np.nan`. Will broadcast.

Used as a price boundary (see [PriceArea](#)).

`init_cash` : InitCashMode, float or array_like

Initial capital.

By default, will broadcast to the final number of columns. But if cash sharing is enabled, will broadcast to the number of groups. See [InitCashMode](#) to find optimal initial cash.



Note

Mode `InitCashMode.AutoAlign` is applied after the portfolio is initialized to set the same initial cash for all columns/groups. Changing grouping will change the initial cash, so be aware when indexing.

`init_position` : float or array_like

Initial position.

By default, will broadcast to the final number of columns.

`init_price` : float or array_like

Initial position price.

By default, will broadcast to the final number of columns.

`cash_deposits` : float or array_like

Cash to be deposited/withdrawn at each timestamp. Will broadcast to the final shape. Must have the same number of columns as `init_cash`.

Applied at the beginning of each timestamp.

`cash_earnings` : float or array_like

Earnings in cash to be added at each timestamp. Will broadcast to the final shape.

Applied at the end of each timestamp.

`cash_dividends` : float or array_like

Dividends in cash to be added at each timestamp. Will broadcast to the final shape.

Gets multiplied by the position and saved into `cash_earnings`.

Applied at the end of each timestamp.

`cash_sharing` : bool

Whether to share cash within the same group.

If `group_by` is None and `cash_sharing` is True, `group_by` becomes True to form a single group with cash sharing.



Warning

Introduces cross-asset dependencies.

This method presumes that in a group of assets that share the same capital all orders will be executed within the same tick and retain their price regardless of their position in the queue, even though they depend upon each other and thus cannot be executed in parallel.

`from_ago` : int or array_like

Take order information from a number of bars ago. Will broadcast.

Negative numbers will be cast to positive to avoid the look-ahead bias. Defaults to 0. Remember to account of it if you're using a custom signal function!

`sim_start` : int, datetime_like, or array_like

Simulation start row or index (inclusive).

Can be "auto", which will be substituted by the index of the first non-NA size value.

`sim_end` : int, datetime_like, or array_like

Simulation end row or index (exclusive).

Can be "auto", which will be substituted by the index of the first non-NA size value.

`call_seq` : `CallSeqType` or `array_like`

Default sequence of calls per row and group.

Each value in this sequence must indicate the position of column in the group to call next. Processing of `call_seq` goes always from left to right. For example, `[2, 0, 1]` would first call column 'c', then 'a', and finally 'b'.

Supported are multiple options:

- Set to None to generate the default call sequence on the fly. Will create a full array only if `attach_call_seq` is True.
- Use `CallSeqType` to create a full array of a specific type.
- Set to array to specify a custom call sequence.

If `CallSeqType.Auto` selected, rearranges calls dynamically based on order value. Calculates value of all orders per row and group, and sorts them by this value. Sell orders will be executed first to release funds for buy orders.

Warning

`CallSeqType.Auto` should be used with caution:

- It not only presumes that order prices are known beforehand, but also that orders can be executed in arbitrary order and still retain their price. In reality, this is hardly the case: after processing one asset, some time has passed and the price for other assets might have already changed.
- Even if you're able to specify a slippage large enough to compensate for this behavior, slippage itself should depend upon execution order. This method doesn't let you do that.
- Orders in the same queue are executed regardless of whether previous orders have been filled, which can leave them without required funds.

For more control, use `Portfolio.from_order_func`.

`attach_call_seq` : `bool`

Whether to attach `call_seq` to the instance.

Makes sense if you want to analyze the simulation order. Otherwise, just takes memory.

`ffill_val_price` : `bool`

Whether to track valuation price only if it's known.

Otherwise, unknown `close` will lead to NaN in valuation price at the next timestamp.

`update_value` : `bool`

Whether to update group value after each filled order.

`save_state` : `bool`

Whether to save the state.

The arrays will be available as `cash`, `position`, `debt`, `locked_cash`, and `free_cash` in in-outputs.

`save_value` : `bool`

Whether to save the value.

The array will be available as `value` in in-outputs.

`save_returns` : `bool`

Whether to save the returns.

The array will be available as `returns` in in-outputs.

`skip_empty` : `bool`

Whether to skip rows with no order.

`max_order_records` : `int`

The max number of order records expected to be filled at each column. Defaults to the maximum number of non-NaN values across all columns of the size array.

Set to a lower number if you run out of memory, and to 0 to not fill.

`max_log_records` : `int`

The max number of log records expected to be filled at each column. Defaults to the maximum number of True values across all columns of the log array.

Set to a lower number if you run out of memory, and to 0 to not fill.

`seed : int`

Seed to be set for both `call_seq` and at the beginning of the simulation.

`group_by : any`

Group columns. See [Grouper](#).

`broadcast_kwargs : dict`

Keyword arguments passed to [broadcast](#).

`jitted : any`

See [resolve_jitted_option](#).

`chunked : any`

See [resolve_chunked_option](#).

`bm_close : array_like`

Latest benchmark price at each time step. Will broadcast.

If not provided, will use `close`. If False, will not use any benchmark.

`records : array_like`

Records to construct arrays from.

See [IdxRecords](#).

`return_preparer : bool`

Whether to return the preparer of the type [FOPPreparer](#).

Note

Seed won't be set in this case, you need to explicitly call `preparer.set_seed()`.

`return_prep_result : bool`

Whether to return the preparer result of the type [PFPrepResult](#).

`return_sim_out : bool`

Whether to return the simulation output of the type [SimulationOutput](#).

****kwargs**

Keyword arguments passed to the [Portfolio](#) constructor.

All broadcastable arguments will broadcast using [broadcast](#) but keep original shape to utilize flexible indexing and to save memory.

For defaults, see [portfolio](#). Those defaults are not used to fill NaN values after reindexing: vectorbt uses its own sensible defaults, which are usually NaN for floating arrays and default flags for integer arrays. Use [BCO](#) with `fill_value` to override.

Note

When `call_seq` is not `CallSeqType.Auto`, at each timestamp, processing of the assets in a group goes strictly in order defined in `call_seq`. This order can't be changed dynamically.

This has one big implication for this particular method: the last asset in the call stack cannot be processed until other assets are processed. This is the reason why rebalancing cannot work properly in this setting: one has to specify percentages for all assets beforehand and then tweak the processing order to sell to-be-sold assets first in order to release funds for to-be-bought assets. This can be automatically done by using `CallSeqType.Auto`.

Hint

All broadcastable arguments can be set per frame, series, row, column, or element.

Usage

- Buy 10 units each tick:

```
>>> close = pd.Series([1, 2, 3, 4, 5])
>>> pf = vbt.Portfolio.from_orders(close, 10)

>>> pf.assets
0    10.0
```

```

1    20.0
2    30.0
3    40.0
4    40.0
dtype: float64
>>> pf.cash
0    90.0
1    70.0
2    40.0
3     0.0
4     0.0
dtype: float64

```

- Reverse each position by first closing it:

```

>>> size = [1, 0, -1, 0, 1]
>>> pf = vbt.Portfolio.from_orders(close, size, size_type='targetpercent')

>>> pf.assets
0    100.000000
1     0.000000
2   -66.666667
3     0.000000
4    26.666667
dtype: float64
>>> pf.cash
0     0.000000
1    200.000000
2    400.000000
3    133.333333
4     0.000000
dtype: float64

```

- Regularly deposit cash at open and invest it within the same bar at close:

```

>>> close = pd.Series([1, 2, 3, 4, 5])
>>> cash_deposits = pd.Series([10., 0., 10., 0., 10.])
>>> pf = vbt.Portfolio.from_orders(
...     close,
...     size=cash_deposits, # invest the amount deposited
...     size_type='value',
...     cash_deposits=cash_deposits
... )

>>> pf.cash
0    100.0
1    100.0
2    100.0
3    100.0
4    100.0
dtype: float64

>>> pf.asset_flow
0    10.000000
1     0.000000
2    3.333333
3     0.000000
4    2.000000
dtype: float64

```

- Equal-weighted portfolio as in [from_order_func_nb](#) example (it's more compact but has less control over execution):

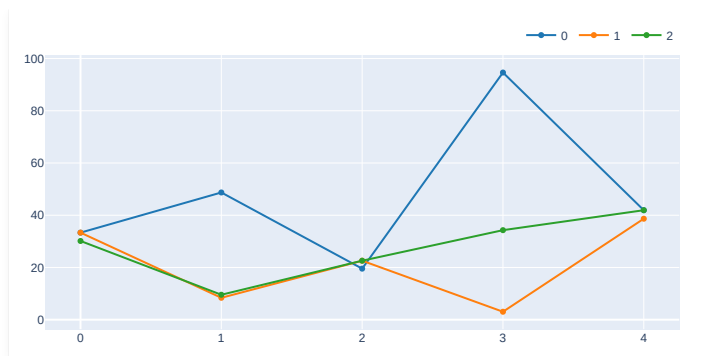
```

>>> np.random.seed(42)
>>> close = pd.DataFrame(np.random.uniform(1, 10, size=(5, 3)))
>>> size = pd.Series(np.full(5, 1/3)) # each column 33.3%
>>> size[1::2] = np.nan # skip every second tick

>>> pf = vbt.Portfolio.from_orders(
...     close, # acts both as reference and order price here
...     size,
...     size_type='targetpercent',
...     direction='longonly',
...     call_seq='auto', # first sell then buy
...     group_by=True, # one group
...     cash_sharing=True, # assets share the same cash
...     fees=0.001, fixed_fees=1., slippage=0.001 # costs
... )

>>> pf.get_asset_value(group_by=False).vbt.plot().show()

```



- Test 10 random weight combinations:

```
>>> np.random.seed(42)
>>> close = pd.DataFrame(
...     np.random.uniform(1, 10, size=(5, 3)),
...     columns=pd.Index(['a', 'b', 'c'], name='asset'))

>>> # Generate random weight combinations
>>> rand_weights = []
>>> for i in range(10):
...     rand_weights.append(np.random.dirichlet(np.ones(close.shape[1]), size=1)[0])
>>> rand_weights
[array([0.15474873, 0.27706078, 0.5681905 ]),
 array([0.30468598, 0.18545189, 0.50986213]),
 array([0.15780486, 0.36292607, 0.47926907]),
 array([0.25697713, 0.64902589, 0.09399698]),
 array([0.43310548, 0.53836359, 0.02853093]),
 array([0.78628605, 0.15716865, 0.0565453 ]),
 array([0.37186671, 0.42150531, 0.20662798]),
 array([0.22441579, 0.06348919, 0.71209502]),
 array([0.41619664, 0.09338007, 0.49042329]),
 array([0.01279537, 0.87770864, 0.10949599])]

>>> # Bring close and rand_weights to the same shape
>>> rand_weights = np.concatenate(rand_weights)
>>> close = close.vbt.tile(10, keys=pd.Index(np.arange(10), name='weights_vector'))
>>> size = vbt.broadcast_to(weights, close).copy()
>>> size[1::2] = np.nan
>>> size
weights_vector
asset      a      b      c      ...      a      b      c
0      0.154749  0.277061  0.56819  ...      0.012795  0.877709  0.109496
1      NaN      NaN      NaN      ...      NaN      NaN      NaN
2      0.154749  0.277061  0.56819  ...      0.012795  0.877709  0.109496
3      NaN      NaN      NaN      ...      NaN      NaN      NaN
4      0.154749  0.277061  0.56819  ...      0.012795  0.877709  0.109496

[5 rows x 30 columns]

>>> pf = vbt.Portfolio.from_orders(
...     close,
...     size,
...     size_type='targetpercent',
...     direction='longonly',
...     call_seq='auto',
...     group_by='weights_vector', # group by column level
...     cash_sharing=True,
...     fees=0.001, fixed_fees=1., slippage=0.001
... )

>>> pf.total_return
weights_vector
0      -0.294372
1      0.139207
2      -0.281739
3      0.041242
4      0.467566
5      0.829925
6      0.320672
7      -0.087452
8      0.376681
9      -0.702773
Name: total_return, dtype: float64
```

from_random_signals class method

```
Portfolio.from_random_signals(
    close,
    n=None,
    prob=None,
    entry_prob=None,
    exit_prob=None,
    param_product=False,
```

```
seed=None,
run_kwargs=None,
**kwargs
)
```

Simulate portfolio from random entry and exit signals.

Generates signals based either on the number of signals `n` or the probability of encountering a signal `prob`.

- If `n` is set, see [RANDNX](#).
- If `prob` is set, see [RPROBNX](#).

Based on [Portfolio.from_signals](#).

Note

To generate random signals, the shape of `close` is used. Broadcasting with other arrays happens after the generation.

Usage

- Test multiple combinations of random entries and exits:

```
>>> close = pd.Series([1, 2, 3, 4, 5])
>>> pf = vbt.Portfolio.from_random_signals(close, n=[2, 1, 0], seed=42)
>>> pf.orders.count()
randnx_n
2      4
1      2
0      0
Name: count, dtype: int64
```

- Test the Cartesian product of entry and exit encounter probabilities:

```
>>> pf = vbt.Portfolio.from_random_signals(
...     close,
...     entry_prob=[0, 0.5, 1],
...     exit_prob=[0, 0.5, 1],
...     param_product=True,
...     seed=42)
>>> pf.orders.count()
rprobnx_entry_prob  rprobnx_exit_prob
0.0                 0.0                 0
                  0.5                 0
                  1.0                 0
0.5                 0.0                 1
                  0.5                 4
                  1.0                 3
1.0                 0.0                 1
                  0.5                 4
                  1.0                 5
Name: count, dtype: int64
```

from_signals class method

```
Portfolio.from_signals(
    close,
    entries=None,
    exits=None,
    *,
    direction=None,
    long_entries=None,
    long_exits=None,
    short_entries=None,
    short_exits=None,
    adjust_func_nb=None,
    adjust_args=(),
    signal_func_nb=None,
    signal_args=(),
    post_segment_func_nb=None,
    post_segment_args=(),
    order_mode=False,
    size=None,
    size_type=None,
    price=None,
    fees=None,
    fixed_fees=None,
    slippage=None,
    min_size=None,
    max_size=None,
    size_granularity=None,
    leverage=None,
    leverage_mode=None,
    reject_prob=None,
    price_area_vio_mode=None,
    allow_partial=None,
    raise_reject=None,
    log=None,
    val_price=None,
```

```

        accumulate=None,
        upon_long_conflict=None,
        upon_short_conflict=None,
        upon_dir_conflict=None,
        upon_opposite_entry=None,
        order_type=None,
        limit_delta=None,
        limit_tif=None,
        limit_expiry=None,
        limit_reverse=None,
        limit_order_price=None,
        upon_adj_limit_conflict=None,
        upon_opp_limit_conflict=None,
        use_stops=None,
        stop_ladder=None,
        sl_stop=None,
        tsl_stop=None,
        tsl_th=None,
        tp_stop=None,
        td_stop=None,
        dt_stop=None,
        stop_entry_price=None,
        stop_exit_price=None,
        stop_exit_type=None,
        stop_order_type=None,
        stop_limit_delta=None,
        upon_stop_update=None,
        upon_adj_stop_conflict=None,
        upon_opp_stop_conflict=None,
        delta_format=None,
        time_delta_format=None,
        open=None,
        high=None,
        low=None,
        init_cash=None,
        init_position=None,
        init_price=None,
        cash_deposits=None,
        cash_earnings=None,
        cash_dividends=None,
        cash_sharing=None,
        from_ago=None,
        sim_start=None,
        sim_end=None,
        call_seq=None,
        attach_call_seq=None,
        ffill_val_price=None,
        update_value=None,
        fill_pos_info=None,
        save_state=None,
        save_value=None,
        save_returns=None,
        skip_empty=None,
        max_order_records=None,
        max_log_records=None,
        in_outputs=None,
        seed=None,
        group_by=None,
        broadcast_named_args=None,
        broadcast_kwargs=None,
        template_context=None,
        jitted=None,
        chunked=None,
        staticized=None,
        bm_close=None,
        records=None,
        return_preparer=False,
        return_prep_result=False,
        return_sim_out=False,
        **kwargs
    )

```

Simulate portfolio from entry and exit signals.

Supports the following modes:

1. `entries` and `exits`: Uses `dir_signal_func_nb` as `signal_func_nb` if an adjustment function is provided (not cacheable), otherwise translates signals using `dir_to_ls_signals_nb` then simulates statically (cacheable)
2. `entries` (acting as long), `exits` (acting as long), `short_entries`, and `short_exits`: Uses `ls_signal_func_nb` as `signal_func_nb` if an adjustment function is provided (not cacheable), otherwise simulates statically (cacheable)
3. `order_mode=True` without signals: Uses `order_signal_func_nb` as `signal_func_nb` (not cacheable)
4. `signal_func_nb` and `signal_args`: Custom signal function (not cacheable)

Prepared by [FSPreparer](#).

Args

`close` : array_like, OHLCDataMixin, FSPreparer, or PFPrepResult

See [Portfolio.from_orders](#).

`entries` : array_like of bool

Boolean array of entry signals. Defaults to True if all other signal arrays are not set, otherwise False. Will broadcast.

- If `short_entries` and `short_exits` are not set: Acts as a long signal if `direction` is 'all' or 'longonly', otherwise short.
- If `short_entries` or `short_exits` are set: Acts as `long_entries`.

`exits` : array_like of bool

Boolean array of exit signals. Defaults to False. Will broadcast.

- If `short_entries` and `short_exits` are not set: Acts as a short signal if `direction` is 'all' or 'longonly', otherwise long.
- If `short_entries` or `short_exits` are set: Acts as `long_exits`.

`direction` : Direction or array_like

See [Portfolio.from_orders](#).

Takes only effect if `short_entries` and `short_exits` are not set.

`long_entries` : array_like of bool

Boolean array of long entry signals. Defaults to False. Will broadcast.

`long_exits` : array_like of bool

Boolean array of long exit signals. Defaults to False. Will broadcast.

`short_entries` : array_like of bool

Boolean array of short entry signals. Defaults to False. Will broadcast.

`short_exits` : array_like of bool

Boolean array of short exit signals. Defaults to False. Will broadcast.

`adjust_func_nb` : path_like or callable

User-defined function to adjust the current simulation state. Defaults to [no_adjust_func_nb](#).

Passed as argument to [dir_signal_func_nb](#), [ls_signal_func_nb](#), and [order_signal_func_nb](#). Has no effect when using other signal functions.

Can be a path to a module when using staticizing.

`adjust_args` : tuple

Packed arguments passed to `adjust_func_nb`.

`signal_func_nb` : path_like or callable

Function called to generate signals.

See [from_signal_func_nb](#).

Can be a path to a module when using staticizing.

`signal_args` : tuple

Packed arguments passed to `signal_func_nb`.

`post_segment_func_nb` : path_like or callable

Post-segment function.

See [from_signal_func_nb](#).

Can be a path to a module when using staticizing.

`post_segment_args` : tuple

Packed arguments passed to `post_segment_func_nb`.

`order_mode` : bool

Whether to simulate as orders without signals.

`size` : float or array_like

See [Portfolio.from_orders](#).

**Note**

Negative size is not allowed. You must express direction using signals.

`size_type` : `SizeType` or `array_like`

See [Portfolio.from_orders](#).

Only `SizeType.Amount`, `SizeType.Value`, `SizeType.Percent(100)`, and `SizeType.ValuePercent(100)` are supported. Other modes such as target percentage are not compatible with signals since their logic may contradict the direction of the signal.

**Note**

`SizeType.Percent(100)` does not support position reversal. Switch to a single direction or use `OppositeEntryMode.Close` to close the position first.

See warning in [Portfolio.from_orders](#).

`price` : `array_like` of `float`

See [Portfolio.from_orders](#).

`fees` : `float` or `array_like`

See [Portfolio.from_orders](#).

`fixed_fees` : `float` or `array_like`

See [Portfolio.from_orders](#).

`slippage` : `float` or `array_like`

See [Portfolio.from_orders](#).

`min_size` : `float` or `array_like`

See [Portfolio.from_orders](#).

`max_size` : `float` or `array_like`

See [Portfolio.from_orders](#).

Will be partially filled if exceeded. You might not be able to properly close the position if accumulation is enabled and `max_size` is too low.

`size_granularity` : `float` or `array_like`

See [Portfolio.from_orders](#).

`leverage` : `float` or `array_like`

See [Portfolio.from_orders](#).

`leverage_mode` : `LeverageMode` or `array_like`

See [Portfolio.from_orders](#).

`reject_prob` : `float` or `array_like`

See [Portfolio.from_orders](#).

`price_area_vio_mode` : `PriceAreaVioMode` or `array_like`

See [Portfolio.from_orders](#).

`allow_partial` : `bool` or `array_like`

See [Portfolio.from_orders](#).

`raise_reject` : `bool` or `array_like`

See [Portfolio.from_orders](#).

`log` : `bool` or `array_like`

See [Portfolio.from_orders](#).

`val_price` : `array_like` of `float`

See [Portfolio.from_orders](#).

`accumulate` : `bool`, `AccumulationMode` or `array_like`

See [AccumulationMode](#). If True, becomes 'both'. If False, becomes 'disabled'. Will broadcast.

When enabled, [Portfolio.from_signals](#) behaves similarly to [Portfolio.from_orders](#).

`upon_long_conflict` : `ConflictMode` or `array_like`

Conflict mode for long signals. See [ConflictMode](#). Will broadcast.

`upon_short_conflict` : `ConflictMode` or `array_like`

Conflict mode for short signals. See [ConflictMode](#). Will broadcast.

`upon_dir_conflict` : `DirectionConflictMode` or `array_like`

See [DirectionConflictMode](#). Will broadcast.

`upon_opposite_entry` : `OppositeEntryMode` or `array_like`

See [OppositeEntryMode](#). Will broadcast.

`order_type` : `OrderType` or `array_like`

See [OrderType](#).

Only one active limit order is allowed at a time.

`limit_delta` : `float` or `array_like`

Delta from `price` to build the limit price. Will broadcast.

If NaN, `price` becomes the limit price. Otherwise, applied on top of `price` depending on the current direction: if the direction-aware size is positive (= buying), a positive delta will decrease the limit price; if the direction-aware size is negative (= selling), a positive delta will increase the limit price. Delta can be negative.

Set an element to `np.nan` to disable. Use `delta_format` to specify the format.

`limit_tif` : `frequency_like` or `array_like`

Time in force for limit signals. Will broadcast.

Any frequency-like object is converted using [to_timedelta64](#). Any array must either contain timedeltas or integers, and will be cast into integer format after broadcasting. If the object provided is of data type `object`, will be converted to timedelta automatically.

Measured in the distance after the open time of the signal bar. If the expiration time happens in the middle of the current bar, we pessimistically assume that the order has been expired. The check is performed at the beginning of the bar, and the first check is performed at the next bar after the signal. For example, if the format is `TimeDeltaFormat.Rows`, 0 or 1 means the order must execute at the same bar or not at all; 2 means the order must execute at the same or next bar or not at all.

Set an element to `-1` to disable. Use `time_delta_format` to specify the format.

`limit_expiry` : `frequency_like`, `datetime_like`, or `array_like`

Expiration time. Will broadcast.

Any frequency-like object is used to build a period index, such that each timestamp in the original index is pointing to the timestamp where the period ends. For example, providing "d" will make any limit order expire on the next day. Any array must either contain timestamps or integers (not timedeltas!), and will be cast into integer format after broadcasting. If the object provided is of data type `object`, will be converted to datetime and its timezone will be removed automatically (as done on the index).

Behaves in a similar way as `limit_tif`.

Set an element to `-1` or `pd.Timestamp.max` to disable. Use `time_delta_format` to specify the format.

`limit_reverse` : `bool` or `array_like`

Whether to reverse the price hit detection. Will broadcast.

If True, a buy/sell limit price will be checked against high/low (not low/high). Also, the limit delta will be applied above/below (not below/above) the initial price.

`limit_order_price` : `LimitOrderPrice` or `array_like`

See [LimitOrderPrice](#). Will broadcast.

If provided on per-element basis, gets applied upon order creation. If a positive value is provided, used directly as a price, otherwise used as an enumerated value.

`upon_adj_limit_conflict` : `PendingConflictMode` or `array_like`

Conflict mode for limit and user-defined signals of adjacent sign. See [PendingConflictMode](#). Will broadcast.

`upon_opp_limit_conflict` : `PendingConflictMode` or `array_like`

Conflict mode for limit and user-defined signals of opposite sign. See [PendingConflictMode](#). Will broadcast.

`use_stops` : `bool`

Whether to use stops. Defaults to `None`, which becomes `True` if any of the stops are not `NaN` or the adjustment function is not the default one.

Disable this to make simulation a bit faster for simple use cases.

`stop_ladder` : `bool` or `StopLadderMode`

Whether and which kind of stop laddering to use. See [StopLadderMode](#).

If so, rows in the supplied arrays will become ladder steps. Make sure that they are increasing. If one column should have less steps, pad it with `NaN` for price-based stops and `-1` for time-based stops.

Rows in each array can be of an arbitrary length but columns must broadcast against the number of columns in the data. Applied on all stop types.

`sl_stop` : `array_like` of `float`

Stop loss. Will broadcast.

Set an element to `np.nan` to disable. Use `delta_format` to specify the format.

`tsl_stop` : `array_like` of `float`

Trailing stop loss. Will broadcast.

Set an element to `np.nan` to disable. Use `delta_format` to specify the format.

`tsl_th` : `array_like` of `float`

Take profit threshold for the trailing stop loss. Will broadcast.

Set an element to `np.nan` to disable. Use `delta_format` to specify the format.

`tp_stop` : `array_like` of `float`

Take profit. Will broadcast.

Set an element to `np.nan` to disable. Use `delta_format` to specify the format.

`td_stop` : `frequency_like` or `array_like`

Timedelta-stop. Will broadcast.

Set an element to `-1` to disable. Use `time_delta_format` to specify the format.

`dt_stop` : `frequency_like`, `datetime_like`, or `array_like`

Datetime-stop. Will broadcast.

Set an element to `-1` to disable. Use `time_delta_format` to specify the format.

`stop_entry_price` : `StopEntryPrice` or `array_like`

See [StopEntryPrice](#). Will broadcast.

If provided on per-element basis, gets applied upon entry. If a positive value is provided, used directly as a price, otherwise used as an enumerated value.

`stop_exit_price` : `StopExitPrice` or `array_like`

See [StopExitPrice](#). Will broadcast.

If provided on per-element basis, gets applied upon entry. If a positive value is provided, used directly as a price, otherwise used as an enumerated value.

`stop_exit_type` : `StopExitType` or `array_like`

See [StopExitType](#). Will broadcast.

If provided on per-element basis, gets applied upon entry.

`stop_order_type` : `OrderType` or `array_like`

Similar to `order_type` but for stop orders. Will broadcast.

If provided on per-element basis, gets applied upon entry.

`stop_limit_delta` : float or array_like

Similar to `limit_delta` but for stop orders. Will broadcast.

`upon_stop_update` : StopUpdateMode or array_like

See [StopUpdateMode](#). Will broadcast.

Only has effect if accumulation is enabled.

If provided on per-element basis, gets applied upon repeated entry.

`upon_adj_stop_conflict` : PendingConflictMode or array_like

Conflict mode for stop and user-defined signals of adjacent sign. See [PendingConflictMode](#). Will broadcast.

`upon_opp_stop_conflict` : PendingConflictMode or array_like

Conflict mode for stop and user-defined signals of opposite sign. See [PendingConflictMode](#). Will broadcast.

`delta_format` : DeltaFormat or array_like

See [DeltaFormat](#). Will broadcast.

`time_delta_format` : TimeDeltaFormat or array_like

See [TimeDeltaFormat](#). Will broadcast.

`open` : array_like of float

See [Portfolio.from_orders](#).

For stop signals, `np.nan` gets replaced by `close`.

`high` : array_like of float

See [Portfolio.from_orders](#).

For stop signals, `np.nan` replaced by the maximum out of `open` and `close`.

`low` : array_like of float

See [Portfolio.from_orders](#).

For stop signals, `np.nan` replaced by the minimum out of `open` and `close`.

`init_cash` : InitCashMode, float or array_like

See [Portfolio.from_orders](#).

`init_position` : float or array_like

See [Portfolio.from_orders](#).

`init_price` : float or array_like

See [Portfolio.from_orders](#).

`cash_deposits` : float or array_like

See [Portfolio.from_orders](#).

`cash_earnings` : float or array_like

See [Portfolio.from_orders](#).

`cash_dividends` : float or array_like

See [Portfolio.from_orders](#).

`cash_sharing` : bool

See [Portfolio.from_orders](#).

`from_ago` : int or array_like

See [Portfolio.from_orders](#).

Take effect only for user-defined signals, not for stop signals.

`sim_start` : int, datetime_like, or array_like

Simulation start row or index (inclusive).

Can be "auto", which will be substituted by the index of the first signal across long and short entries and long and short exits.

`sim_end` : int, datetime_like, or array_like

Simulation end row or index (exclusive).

Can be "auto", which will be substituted by the index of the last signal across long and short entries and long and short exits.

`call_seq` : CallSeqType or array_like

See [Portfolio.from_orders](#).

`attach_call_seq` : bool

See [Portfolio.from_orders](#).

`ffill_val_price` : bool

See [Portfolio.from_orders](#).

`update_value` : bool

See [Portfolio.from_orders](#).

`fill_pos_info` : bool

fill_pos_info (bool): Whether to fill position record.

Disable this to make simulation faster for simple use cases.

`save_state` : bool

See [Portfolio.from_orders](#).

`save_value` : bool

See [Portfolio.from_orders](#).

`save_returns` : bool

See [Portfolio.from_orders](#).

`skip_empty` : bool

See [Portfolio.from_orders](#).

`max_order_records` : int

See [Portfolio.from_orders](#).

`max_log_records` : int

See [Portfolio.from_orders](#).

`in_outputs` : mapping_like

Mapping with in-output objects. Only for flexible mode.

Will be available via [Portfolio.in_outputs](#) as a named tuple.

To substitute [Portfolio](#) attributes, provide already broadcasted and grouped objects, for example, by using `broadcast_named_args` and templates. Also see [Portfolio.in_outputs_indexing_func](#) on how in-output objects are indexed.

When chunking, make sure to provide the chunk taking specification and the merging function. See [merge_sim_outs](#).

Note

When using Numba below 0.54, `in_outputs` cannot be a mapping, but must be a named tuple defined globally so Numba can introspect its attributes for pickling.

`seed` : int

See [Portfolio.from_orders](#).

`group_by` : any

See [Portfolio.from_orders](#).

`broadcast_named_args` : dict

Dictionary with named arguments to broadcast.

You can then pass argument names wrapped with [Rep](#) and this method will substitute them by their corresponding broadcasted objects.

`broadcast_kwargs` : dict

See [Portfolio.from_orders](#).

`template_context` : mapping

Context used to substitute templates in arguments.

`jitted` : any

See [Portfolio.from_orders](#).

`chunked` : any

See [Portfolio.from_orders](#).

`staticized` : bool, dict, hashable, or callable

Keyword arguments or task id for staticizing.

If True or dictionary, will be passed as keyword arguments to [cut_and_save_func](#) to save a cacheable version of the simulator to a file. If a hashable or callable, will be used as a task id of an already registered jittable and chunkable simulator. Dictionary allows additional options `override` and `reload` to override and reload an already existing module respectively.

`bm_close` : array_like

See [Portfolio.from_orders](#).

`records` : array_like

See [Portfolio.from_orders](#).

`return_preparer` : bool

See [Portfolio.from_orders](#).

`return_prep_result` : bool

See [Portfolio.from_orders](#).

`return_sim_out` : bool

See [Portfolio.from_orders](#).

****kwargs**

Keyword arguments passed to the [Portfolio](#) constructor.

All broadcastable arguments will broadcast using [broadcast](#) but keep original shape to utilize flexible indexing and to save memory.

For defaults, see [portfolio](#). Those defaults are not used to fill NaN values after reindexing: vectorbt uses its own sensible defaults, which are usually NaN for floating arrays and default flags for integer arrays. Use [BCO](#) with `fill_value` to override.

Also see notes and hints for [Portfolio.from_orders](#).

Usage

- By default, if all signal arrays are None, `entries` becomes True, which opens a position at the very first tick and does nothing else:

```
>>> close = pd.Series([1, 2, 3, 4, 5])
>>> pf = vbt.Portfolio.from_signals(close, size=1)
>>> pf.asset_flow
0    1.0
1    0.0
2    0.0
3    0.0
4    0.0
dtype: float64
```

- Entry opens long, exit closes long:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1,
...     direction='longonly')
```

```

... )
>>> pf.asset_flow
0    1.0
1    0.0
2    0.0
3   -1.0
4    0.0
dtype: float64

>>> # Using direction-aware arrays instead of `direction`
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]), # long_entries
...     exits=pd.Series([False, False, True, True, True]), # long_exits
...     short_entries=False,
...     short_exits=False,
...     size=1
... )
>>> pf.asset_flow
0    1.0
1    0.0
2    0.0
3   -1.0
4    0.0
dtype: float64

```

Notice how both `short_entries` and `short_exits` are provided as constants - as any other broadcastable argument, they are treated as arrays where each element is False.

- Entry opens short, exit closes short:

```

>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1,
...     direction='shortonly'
... )
>>> pf.asset_flow
0   -1.0
1    0.0
2    0.0
3    1.0
4    0.0
dtype: float64

>>> # Using direction-aware arrays instead of `direction`
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=False, # long_entries
...     exits=False, # long_exits
...     short_entries=pd.Series([True, True, True, False, False]),
...     short_exits=pd.Series([False, False, True, True, True]),
...     size=1
... )
>>> pf.asset_flow
0   -1.0
1    0.0
2    0.0
3    1.0
4    0.0
dtype: float64

```

- Entry opens long and closes short, exit closes long and opens short:

```

>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1,
...     direction='both'
... )
>>> pf.asset_flow
0    1.0
1    0.0
2    0.0
3   -2.0
4    0.0
dtype: float64

>>> # Using direction-aware arrays instead of `direction`
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]), # long_entries
...     exits=False, # long_exits
...     short_entries=pd.Series([False, False, True, True, True]),
...     short_exits=False,
...     size=1
... )
>>> pf.asset_flow
0    1.0
1    0.0
2    0.0
3   -2.0
4    0.0

```



```
dtype: float64
```

- More complex signal combinations are best expressed using direction-aware arrays. For example, ignore opposite signals as long as the current position is open:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, False, False, False, False]), # long_entries
...     exits=pd.Series([False, False, True, False, False]), # long_exits
...     short_entries=pd.Series([False, True, False, True, False]),
...     short_exits=pd.Series([False, False, False, False, True]),
...     size=1,
...     upon_opposite_entry='ignore'
... )
>>> pf.asset_flow
0    1.0
1    0.0
2   -1.0
3   -1.0
4    1.0
dtype: float64
```

- First opposite signal closes the position, second one opens a new position:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1,
...     direction='both',
...     upon_opposite_entry='close'
... )
>>> pf.asset_flow
0    1.0
1    0.0
2    0.0
3   -1.0
4   -1.0
dtype: float64
```

- If both long entry and exit signals are True (a signal conflict), choose exit:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1,
...     direction='longonly',
...     upon_long_conflict='exit')
>>> pf.asset_flow
0    1.0
1    0.0
2   -1.0
3    0.0
4    0.0
dtype: float64
```

- If both long entry and short entry signal are True (a direction conflict), choose short:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1,
...     direction='both',
...     upon_dir_conflict='short')
>>> pf.asset_flow
0    1.0
1    0.0
2   -2.0
3    0.0
4    0.0
dtype: float64
```

Note

Remember that when direction is set to 'both', entries become `long_entries` and exits become `short_entries`, so this becomes a conflict of directions rather than signals.

- If there are both signal and direction conflicts:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=True, # long_entries
...     exits=True, # long_exits
...     short_entries=True,
...     short_exits=True,
...     size=1,
...     upon_long_conflict='entry',
...     upon_short_conflict='entry',
...     upon_dir_conflict='short')
```

```
... )
>>> pf.asset_flow
0    -1.0
1     0.0
2     0.0
3     0.0
4     0.0
dtype: float64
```

- Turn on accumulation of signals. Entry means long order, exit means short order (acts similar to `from_orders`):

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1.,
...     direction='both',
...     accumulate=True)
>>> pf.asset_flow
0     1.0
1     1.0
2     0.0
3    -1.0
4    -1.0
dtype: float64
```

- Allow increasing a position (of any direction), deny decreasing a position:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     size=1.,
...     direction='both',
...     accumulate='addonly')
>>> pf.asset_flow
0     1.0 << open a long position
1     1.0 << add to the position
2     0.0
3    -3.0 << close and open a short position
4    -1.0 << add to the position
dtype: float64
```

- Test multiple parameters via regular broadcasting:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     direction=[list(Direction)],
...     broadcast_kwargs=dict(columns_from=pd.Index(vbt.pf_enums.Direction._fields, name='direction')))
>>> pf.asset_flow
direction LongOnly ShortOnly Both
0          100.0    -100.0  100.0
1           0.0         0.0   0.0
2           0.0         0.0   0.0
3        -100.0         50.0 -200.0
4           0.0         0.0   0.0
```

- Test multiple parameters via **BCO**:

```
>>> pf = vbt.Portfolio.from_signals(
...     close,
...     entries=pd.Series([True, True, True, False, False]),
...     exits=pd.Series([False, False, True, True, True]),
...     direction=vbt.Param(Direction))
>>> pf.asset_flow
direction LongOnly ShortOnly Both
0          100.0    -100.0  100.0
1           0.0         0.0   0.0
2           0.0         0.0   0.0
3        -100.0         50.0 -200.0
4           0.0         0.0   0.0
```

- Set risk/reward ratio by passing trailing stop loss and take profit thresholds:

```
>>> close = pd.Series([10, 11, 12, 11, 10, 9])
>>> entries = pd.Series([True, False, False, False, False, False])
>>> exits = pd.Series([False, False, False, False, False, True])
>>> pf = vbt.Portfolio.from_signals(
...     close, entries, exits,
...     tsl_stop=0.1, tp_stop=0.2) # take profit hit
>>> pf.asset_flow
0     10.0
1     0.0
2    -10.0
3     0.0
4     0.0
5     0.0
dtype: float64

>>> pf = vbt.Portfolio.from_signals(
```

```

...     close, entries, exits,
...     tsl_stop=0.1, tp_stop=0.3) # trailing stop loss hit
>>> pf.asset_flow
0    10.0
1     0.0
2     0.0
3     0.0
4    -10.0
5     0.0
dtype: float64

>>> pf = vbt.Portfolio.from_signals(
...     close, entries, exits,
...     tsl_stop=np.inf, tp_stop=np.inf) # nothing hit, exit as usual
>>> pf.asset_flow
0    10.0
1     0.0
2     0.0
3     0.0
4     0.0
5    -10.0
dtype: float64

```

- Test different stop combinations:

```

>>> pf = vbt.Portfolio.from_signals(
...     close, entries, exits,
...     tsl_stop=vbt.Param([0.1, 0.2]),
...     tp_stop=vbt.Param([0.2, 0.3])
... )
>>> pf.asset_flow
tsl_stop  0.1      0.2
tp_stop   0.2  0.3  0.2  0.3
0    10.0  10.0  10.0  10.0
1     0.0   0.0   0.0   0.0
2    -10.0   0.0 -10.0   0.0
3     0.0   0.0   0.0   0.0
4     0.0 -10.0   0.0   0.0
5     0.0   0.0   0.0 -10.0

```

This works because `pd.Index` automatically translates into `BCO` with `product` set to `True`.

- We can implement our own stop loss or take profit, or adjust the existing one at each time step. Let's implement **stepped stop-loss**:

```

>>> @njit
... def adjust_func_nb(c):
...     val_price_now = c.last_val_price[c.col]
...     tsl_init_price = c.last_tsl_info["init_price"][c.col]
...     current_profit = (val_price_now - tsl_init_price) / tsl_init_price
...     if current_profit >= 0.40:
...         c.last_tsl_info["stop"][c.col] = 0.25
...     elif current_profit >= 0.25:
...         c.last_tsl_info["stop"][c.col] = 0.15
...     elif current_profit >= 0.20:
...         c.last_tsl_info["stop"][c.col] = 0.07

>>> close = pd.Series([10, 11, 12, 11, 10])
>>> pf = vbt.Portfolio.from_signals(close, adjust_func_nb=adjust_func_nb)
>>> pf.asset_flow
0    10.0
1     0.0
2     0.0
3    -10.0 # 7% from 12 hit
4    11.16
dtype: float64

```

- Sometimes there is a need to provide or transform signals dynamically. For this, we can implement a custom signal function `signal_func_nb`. For example, let's implement a signal function that takes two numerical arrays - long and short one - and transforms them into 4 direction-aware boolean arrays that vectorbt understands:

```

>>> @njit
... def signal_func_nb(c, long_num_arr, short_num_arr):
...     long_num = vbt.pf_nb.select_nb(c, long_num_arr)
...     short_num = vbt.pf_nb.select_nb(c, short_num_arr)
...     is_long_entry = long_num > 0
...     is_long_exit = long_num < 0
...     is_short_entry = short_num > 0
...     is_short_exit = short_num < 0
...     return is_long_entry, is_long_exit, is_short_entry, is_short_exit

>>> pf = vbt.Portfolio.from_signals(
...     pd.Series([1, 2, 3, 4, 5]),
...     signal_func_nb=signal_func_nb,
...     signal_args=(vbt.Rep('long_num_arr'), vbt.Rep('short_num_arr')),
...     broadcast_named_args=dict(
...         long_num_arr=pd.Series([1, 0, -1, 0, 0]),
...         short_num_arr=pd.Series([0, 1, 0, 1, -1])
...     ),
...     size=1,
...     upon_opposite_entry='ignore'
... )
>>> pf.asset_flow
0    1.0

```

```

1  0.0
2 -1.0
3 -1.0
4  1.0
dtype: float64

```

Passing both arrays as `broadcast_named_args` broadcasts them internally as any other array, so we don't have to worry about their dimensions every time we change our data.

get_allocations class method

```

Portfolio.get_allocations(
    direction='both',
    asset_value=None,
    value=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)

```

Get portfolio allocation series per column.

get_alpha method

```

Portfolio.get_alpha(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

See [ReturnsAccessor.alpha](#).

get_annual_returns method

```

Portfolio.get_annual_returns(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

See [ReturnsAccessor.annual](#).

get_annualized_return method

```

Portfolio.get_annualized_return(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,

```

```

log_returns=False,
daily_returns=False,
sim_start=None,
sim_end=None,
rec_sim_range=False,
freq=None,
year_freq=None,
defaults=None,
jitted=None,
chunked=None,
wrapper=None,
group_by=None,
**kwargs
)

```

See [ReturnsAccessor.annualized](#).

get_annualized_volatility method

```

Portfolio.get_annualized_volatility(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

See [ReturnsAccessor.annualized_volatility](#).

get_asset_flow class method

```

Portfolio.get_asset_flow(
    direction='both',
    orders=None,
    init_position=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    wrap_kwargs=None
)

```

Get asset flow series per column.

Returns the total transacted amount of assets at each time step.

get_asset_pnl class method

```

Portfolio.get_asset_pnl(
    init_position_value=None,
    asset_value=None,
    cash_flow=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)

```

Get asset (realized and unrealized) PnL series per column or group.

get_asset_returns class method

```

Portfolio.get_asset_returns(
    init_position_value=None,

```

```

        asset_value=None,
        cash_flow=None,
        log_returns=False,
        daily_returns=False,
        sim_start=None,
        sim_end=None,
        rec_sim_range=False,
        jitted=None,
        chunked=None,
        wrapper=None,
        group_by=None,
        wrap_kwargs=None
    )

```

Get asset return series per column or group.

This type of returns is based solely on cash flows and asset value rather than portfolio value. It ignores passive cash and thus it will return the same numbers irrespective of the amount of cash currently available, even `np.inf`. The scale of returns is comparable to that of going all in and keeping available cash at zero.

get_asset_value class method

```

Portfolio.get_asset_value(
    direction='both',
    close=None,
    assets=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)

```

Get asset value series per column or group.

get_assets class method

```

Portfolio.get_assets(
    direction='both',
    asset_flow=None,
    init_position=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    wrap_kwargs=None
)

```

Get asset series per column.

Returns the position at each time step.

get_beta method

```

Portfolio.get_beta(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

See [ReturnsAccessor.beta](#).

get_bm_returns class method

```
Portfolio.get_bm_returns(
    init_value=None,
    cash_deposits=None,
    bm_value=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get benchmark return series per column or group.

Based on [Portfolio.bm_close](#) and [Portfolio.get_market_returns](#).

get_bm_value class method

```
Portfolio.get_bm_value(
    bm_close=None,
    init_value=None,
    cash_deposits=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get benchmark value series per column or group.

Based on [Portfolio.bm_close](#) and [Portfolio.get_market_value](#).

get_calmar_ratio method

```
Portfolio.get_calmar_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.calmar_ratio](#).

get_capture_ratio method

```
Portfolio.get_capture_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.capture_ratio](#).

get_cash class method

```
Portfolio.get_cash(
    free=False,
    init_cash=None,
    cash_deposits=None,
    cash_flow=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get cash balance series per column or group.

For `free`, see [Portfolio.get_cash_flow](#).

get_cash_deposits class method

```
Portfolio.get_cash_deposits(
    cash_deposits_raw=None,
    cash_sharing=None,
    split_shared=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    weights=None,
    keep_flex=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get cash deposit series per column or group.

Set `keep_flex` to True to keep format suitable for flexible indexing. This consumes less memory.

get_cash_earnings class method

```
Portfolio.get_cash_earnings(
    cash_earnings_raw=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    weights=None,
    keep_flex=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get earnings in cash series per column or group.

Set `keep_flex` to True to keep format suitable for flexible indexing. This consumes less memory.

get_cash_flow class method

```
Portfolio.get_cash_flow(
    free=False,
    orders=None,
    cash_earnings=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    weights=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get cash flow series per column or group.

Use `free` to return the flow of the free cash, which never goes above the initial level, because an operation always costs money.

 Note

Does not include cash deposits, but includes earnings.

Using `free` yields the same result as during the simulation only when `Leverage=1`. For anything else, prefill the state instead of reconstructing it.

get_cond_value_at_risk method

```
Portfolio.get_cond_value_at_risk(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.cond_value_at_risk](#).

get_cumulative_returns method

```
Portfolio.get_cumulative_returns(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.cumulative](#).

get_daily_returns method

```
Portfolio.get_daily_returns(
    *,
    returns=None,
    use_asset_returns=False,
```

```
        bm_returns=None,  
        log_returns=False,  
        daily_returns=False,  
        sim_start=None,  
        sim_end=None,  
        rec_sim_range=False,  
        freq=None,  
        year_freq=None,  
        defaults=None,  
        jitted=None,  
        chunked=None,  
        wrapper=None,  
        group_by=None,  
        **kwargs  
    )
```

See [ReturnsAccessor.daily](#).

get_deflated_sharpe_ratio method

```
Portfolio.get_deflated_sharpe_ratio(  
    *,  
    returns=None,  
    use_asset_returns=False,  
    bm_returns=None,  
    log_returns=False,  
    daily_returns=False,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    freq=None,  
    year_freq=None,  
    defaults=None,  
    jitted=None,  
    chunked=None,  
    wrapper=None,  
    group_by=None,  
    **kwargs  
)
```

See [ReturnsAccessor.deflated_sharpe_ratio](#).

get_down_capture_ratio method

```
Portfolio.get_down_capture_ratio(  
    *,  
    returns=None,  
    use_asset_returns=False,  
    bm_returns=None,  
    log_returns=False,  
    daily_returns=False,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    freq=None,  
    year_freq=None,  
    defaults=None,  
    jitted=None,  
    chunked=None,  
    wrapper=None,  
    group_by=None,  
    **kwargs  
)
```

See [ReturnsAccessor.down_capture_ratio](#).

get_downside_risk method

```
Portfolio.get_downside_risk(  
    *,  
    returns=None,  
    use_asset_returns=False,  
    bm_returns=None,  
    log_returns=False,  
    daily_returns=False,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    freq=None,  
    year_freq=None,  
    defaults=None,  
    jitted=None,  
    chunked=None,  
    wrapper=None,  
    group_by=None,  
    **kwargs  
)
```

)

See [ReturnsAccessor.downside_risk](#).**get_drawdown** method

```
Portfolio.get_drawdown(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.drawdown](#).**get_drawdowns** class method

```
Portfolio.get_drawdowns(
    value=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    drawdowns_cls=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

Get drawdown records from [Portfolio.get_value](#).See [Drawdowns](#).**get_entry_trades** class method

```
Portfolio.get_entry_trades(
    orders=None,
    init_position=None,
    init_price=None,
    entry_trades_cls=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

Get entry trade records.

See [EntryTrades](#).**get_exit_trades** class method

```
Portfolio.get_exit_trades(
    orders=None,
    init_position=None,
    init_price=None,
    exit_trades_cls=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

Get exit trade records.

See [ExitTrades](#).

get_filled_bm_close class method

```
Portfolio.get_filled_bm_close(
    bm_close=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    wrap_kwargs=None
)
```

Get forward and backward filled benchmark closing price.

See [fbfill_nb](#).

get_filled_close class method

```
Portfolio.get_filled_close(
    close=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    wrap_kwargs=None
)
```

Get forward and backward filled closing price.

See [fbfill_nb](#).

get_final_value class method

```
Portfolio.get_final_value(
    input_value=None,
    total_profit=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get total profit per column or group.

get_gross_exposure class method

```
Portfolio.get_gross_exposure(
    direction='both',
    asset_value=None,
    value=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get gross exposure.

Note

When both directions, `asset_value` must include the addition of the absolute long-only and short-only asset values.

get_in_output method

```
Portfolio.get_in_output(
    field,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

)

Find and wrap an in-output object matching the field.

If the field can be found in the attributes of this [Portfolio](#) instance, reads the attribute's options to get requirements for the type and layout of the in-output object.

For each field in [Portfolio.in_outputs](#), resolves the field's options by parsing its name with [Portfolio.parse_field_options](#) and also looks for options in [Portfolio.in_output_config](#). If `field` is not in [Portfolio.in_outputs](#), searches for the field in aliases and options. In such case, to narrow down the number of candidates, options are additionally matched against the requirements using [Portfolio.matches_field_options](#). Finally, the matched in-output object is wrapped using [Portfolio.wrap_obj](#).

get_information_ratio method

```
Portfolio.get_information_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.information_ratio](#).

get_init_cash class method

```
Portfolio.get_init_cash(
    init_cash_raw=None,
    cash_deposits=None,
    free_cash_flow=None,
    cash_sharing=None,
    split_shared=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    weights=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get initial amount of cash per column or group.

get_init_position class method

```
Portfolio.get_init_position(
    init_position_raw=None,
    weights=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None,
    keep_flex=False
)
```

Get initial position per column.

get_init_position_value class method

```
Portfolio.get_init_position_value(
    init_position=None,
    init_price=None,
    jitted=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get initial position value per column.

get_init_price class method 

```
Portfolio.get_init_price(  
    init_price_raw=None,  
    wrapper=None,  
    wrap_kwargs=None,  
    keep_flex=False  
)
```

Get initial price per column.

get_init_value class method 

```
Portfolio.get_init_value(  
    init_position_value=None,  
    init_cash=None,  
    split_shared=False,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    jitted=None,  
    chunked=None,  
    wrapper=None,  
    group_by=None,  
    wrap_kwargs=None  
)
```

Get initial value per column or group.

Includes initial cash and the value of initial position.

get_input_value class method 

```
Portfolio.get_input_value(  
    total_cash_deposits=None,  
    init_value=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    jitted=None,  
    chunked=None,  
    wrapper=None,  
    group_by=None,  
    wrap_kwargs=None  
)
```

Get total input value per column or group.

Includes initial value and any cash deposited at any point in time.

get_logs class method 

```
Portfolio.get_logs(  
    log_records=None,  
    open=None,  
    high=None,  
    low=None,  
    close=None,  
    logs_cls=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    jitted=None,  
    wrapper=None,  
    group_by=None,  
    **kwargs  
)
```

Get log records.

See [Logs](#).

get_long_view method 

```
Portfolio.get_long_view(  
    orders=None,  
    ...  
)
```

```

init_position=None,
init_price=None,
sim_start=None,
sim_end=None,
rec_sim_range=False,
jitted=None,
chunked=None,
**kwargs
)

```

Get view of portfolio with long positions only.

get_market_returns class method

```

Portfolio.get_market_returns(
    init_value=None,
    cash_deposits=None,
    cash_deposits_as_input=None,
    market_value=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)

```

Get market return series per column or group.

get_market_value class method

```

Portfolio.get_market_value(
    close=None,
    init_value=None,
    cash_deposits=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)

```

Get market value series per column or group.

If grouped, evenly distributes the initial cash among assets in the group.

Note

Does not take into account fees and slippage. For this, create a separate portfolio.

get_max_drawdown method

```

Portfolio.get_max_drawdown(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

See [ReturnsAccessor.max_drawdown](#).

get_net_exposure class method

```
Portfolio.get_net_exposure(
    long_exposure=None,
    short_exposure=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get net exposure.

get_omega_ratio method

```
Portfolio.get_omega_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.omega_ratio](#).

get_orders class method

```
Portfolio.get_orders(
    order_records=None,
    open=None,
    high=None,
    low=None,
    close=None,
    orders_cls=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    weights=None,
    jitted=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

Get order records.

See [Orders](#).

get_position_coverage class method

```
Portfolio.get_position_coverage(
    direction='both',
    assets=None,
    granular_groups=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get position coverage per column or group.

Position coverage is the number of time steps in the market divided by the total number of time steps.

get_position_entry_price class method

```
Portfolio.get_position_entry_price(
    orders=None,
    init_position=None,
    init_price=None,
    fill_closed_position=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    wrap_kwargs=None
)
```

Get the position's entry price at each time step.

get_position_exit_price class method

```
Portfolio.get_position_exit_price(
    orders=None,
    init_position=None,
    init_price=None,
    fill_closed_position=False,
    fill_exit_price=True,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    wrap_kwargs=None
)
```

Get the position's exit price at each time step.

get_position_mask class method

```
Portfolio.get_position_mask(
    direction='both',
    assets=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get position mask per column or group.

An element is True if there is a position at the given time step.

get_positions class method

```
Portfolio.get_positions(
    trades=None,
    positions_cls=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

Get position records.

See [Positions](#).

get_prob_sharpe_ratio method

```
Portfolio.get_prob_sharpe_ratio(
    *,
    returns=None,
    use_asset_returns=False,
```

```

        bm_returns=None,
        log_returns=False,
        daily_returns=False,
        sim_start=None,
        sim_end=None,
        rec_sim_range=False,
        freq=None,
        year_freq=None,
        defaults=None,
        jitted=None,
        chunked=None,
        wrapper=None,
        group_by=None,
        **kwargs
    )

```

See [ReturnsAccessor.prob_sharpe_ratio](#).

get_qs class method

```

Portfolio.get_qs(
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

Get quantstats adapter of type [QSAdapter](#).

****kwargs** are passed to the adapter constructor.

get_returns class method

```

Portfolio.get_returns(
    init_value=None,
    cash_deposits=None,
    cash_deposits_as_input=None,
    value=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)

```

Get return series per column or group based on portfolio value.

get_returns_acc class method

```

Portfolio.get_returns_acc(
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

Get returns accessor of type [ReturnsAccessor](#).

Hint

You can find most methods of this accessor as (cacheable) attributes of this portfolio.

get_sharpe_ratio method

```
Portfolio.get_sharpe_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.sharpe_ratio](#).

get_sharpe_ratio_std method

```
Portfolio.get_sharpe_ratio_std(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.sharpe_ratio_std](#).

get_short_view method

```
Portfolio.get_short_view(
    orders=None,
    init_position=None,
    init_price=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    **kwargs
)
```

Get view of portfolio with short positions only.

get_signals class method

```
Portfolio.get_signals(
    orders=None,
    entry_trades=None,
    exit_trades=None,
    idx_arr=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    wrapper=None,
    **kwargs
)
```

```

    group_by=None
)

```

Get long entries, long exits, short entries, and short exits.

Returns per group is grouping is enabled. Pass `group_by=False` to disable.

get_sortino_ratio method

```

Portfolio.get_sortino_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

See [ReturnsAccessor.sortino_ratio](#).

get_tail_ratio method

```

Portfolio.get_tail_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)

```

See [ReturnsAccessor.tail_ratio](#).

get_total_cash_deposits class method

```

Portfolio.get_total_cash_deposits(
    cash_deposits_raw=None,
    cash_sharing=None,
    split_shared=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    weights=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)

```

Get total cash deposit series per column or group.

get_total_cash_earnings class method

```

Portfolio.get_total_cash_earnings(
    cash_earnings_raw=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    weights=None,
)

```

```
jitted=None,
chunked=None,
wrapper=None,
group_by=None,
wrap_kwargs=None
)
```

Get total cash earning series per column or group.

get_total_market_return class method

```
Portfolio.get_total_market_return(
    input_value=None,
    market_value=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get total market return.

get_total_profit class method

```
Portfolio.get_total_profit(
    close=None,
    orders=None,
    init_position=None,
    init_price=None,
    cash_earnings=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get total profit per column or group.

Calculated directly from order records (fast).

get_total_return class method

```
Portfolio.get_total_return(
    input_value=None,
    total_profit=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get total return per column or group.

get_trade_history class method

```
Portfolio.get_trade_history(
    orders=None,
    entry_trades=None,
    exit_trades=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    wrapper=None,
    group_by=None
)
```

Get order history merged with entry and exit trades as a readable DataFrame.

Note

The P&L and return aggregated across the DataFrame may not match the actual total P&L and return, as this DataFrame annotates entry and exit orders with the performance relative to their respective trade types. To obtain accurate total statistics, aggregate only the statistics of either trade type. Additionally, entry orders include open statistics, whereas exit orders do not.

get_trades method

```
Portfolio.get_trades(
    trades_type=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

Get trade/position records depending upon [Portfolio.trades_type](#).

get_up_capture_ratio method

```
Portfolio.get_up_capture_ratio(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    **kwargs
)
```

See [ReturnsAccessor.up_capture_ratio](#).

get_value class method

```
Portfolio.get_value(
    cash=None,
    asset_value=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    jitted=None,
    chunked=None,
    wrapper=None,
    group_by=None,
    wrap_kwargs=None
)
```

Get portfolio value series per column or group.

By default, will generate portfolio value for each asset based on cash flows and thus independent of other assets, with the initial cash balance and position being that of the entire group. Useful for generating returns and comparing assets within the same group.

get_value_at_risk method

```
Portfolio.get_value_at_risk(
    *,
    returns=None,
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    freq=None,
    year_freq=None,
    defaults=None,
    jitted=None,
    chunked=None,
    wrapper=None,

```

```

        group_by=None,
        **kwargs
    )

```

See [ReturnsAccessor.value_at_risk](#).

get_weights class method 

```

Portfolio.get_weights(
    weights=None,
    wrapper=None
)

```

Get asset weights.

gross_exposure cacheable_property 

[Portfolio.get_gross_exposure](#) with default arguments.

high custom_property 

High price of each bar.

high_flex class property 

[Portfolio.high](#) in a format suitable for flexible indexing.

in_output_config property 

In-output config of [Portfolio](#).

```

HybridConfig(
    cash=dict(
        grouping='cash_sharing'
    ),
    position=dict(
        grouping='columns'
    ),
    debt=dict(
        grouping='columns'
    ),
    locked_cash=dict(
        grouping='columns'
    ),
    free_cash=dict(
        grouping='cash_sharing'
    ),
    returns=dict(
        grouping='cash_sharing'
    )
)

```

Returns `Portfolio._in_output_config`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change in_outputs, you can either change the config in-place, override this property, or overwrite the instance variable `Portfolio._in_output_config`.

in_outputs class property 

Named tuple with in-output objects.

in_outputs_indexing_func method 

```

Portfolio.in_outputs_indexing_func(
    wrapper_meta,
    **kwargs
)

```

Perform indexing on [Portfolio.in_outputs](#).

If the field can be found in the attributes of this [Portfolio](#) instance, reads the attribute's options to get requirements for the type and layout of the in-output object.

For each field in [Portfolio.in_outputs](#), resolves the field's options by parsing its name with [Portfolio.parse_field_options](#) and also looks for options in [Portfolio.in_output_config](#). Performs indexing on the in-output object using [Portfolio.index_obj](#).

`index_obj` method

```
Portfolio.index_obj(
    obj,
    wrapper_meta,
    obj_name=None,
    grouping='columns_or_groups',
    obj_type=None,
    wrapper=None,
    group_by=None,
    indexing_func=None,
    force_indexing=False,
    silence_warnings=False,
    **kwargs
)
```

Perform indexing on an object.

`indexing_func` must take the portfolio, all the arguments passed to this method, and `**kwargs`. If you don't need any of the arguments, make `indexing_func` accept them as `**kwargs`.

If the object is None, boolean, or empty, returns as-is.

`indexing_func` method

```
Portfolio.indexing_func(
    *args,
    in_output_kwargs=None,
    wrapper_meta=None,
    **kwargs
)
```

Perform indexing on [Portfolio](#).

In-outputs are indexed using [Portfolio.in_outputs_indexing_func](#).

`information_ratio` cached_property

[Portfolio.get_information_ratio](#) with default arguments.

`init_cash` cached_property

[Portfolio.get_init_cash](#) with default arguments.

`init_position` cached_property

[Portfolio.get_init_position](#) with default arguments.

`init_position_value` cached_property

[Portfolio.get_init_position_value](#) with default arguments.

`init_price` cached_property

[Portfolio.get_init_price](#) with default arguments.

`init_value` cached_property

[Portfolio.get_init_value](#) with default arguments.

`input_value` cached_property

[Portfolio.get_input_value](#) with default arguments.

locked_cash	cacheable_property	
Portfolio.None with default arguments.		
log_records	class property	
A structured NumPy array of log records.		
log_returns	cacheable_property	
Portfolio.get_returns with arguments {'log_returns': True}.		
logs	cached_property	
Portfolio.get_logs with default arguments.		
logs_cls	class property	
Class for wrapping log records.		
long_allocations	cacheable_property	
Portfolio.get_allocations with arguments {'direction': 'longonly'}.		
long_asset_flow	cacheable_property	
Portfolio.get_asset_flow with arguments {'direction': 'longonly'}.		
long_asset_value	cacheable_property	
Portfolio.get_asset_value with arguments {'direction': 'longonly'}.		
long_assets	cacheable_property	
Portfolio.get_assets with arguments {'direction': 'longonly'}.		
long_gross_exposure	cacheable_property	
Portfolio.get_gross_exposure with arguments {'direction': 'longonly'}.		
long_position_coverage	cached_property	
Portfolio.get_position_coverage with arguments {'direction': 'longonly'}.		
long_position_mask	cacheable_property	
Portfolio.get_position_mask with arguments {'direction': 'longonly'}.		
long_view	cacheable_property	
Portfolio.get_long_view with default arguments.		
low	custom_property	
Low price of each bar.		

low_flex class property 

[Portfolio.low](#) in a format suitable for flexible indexing.

market_returns cacheable_property 

[Portfolio.get_market_returns](#) with default arguments.

market_value cacheable_property 

[Portfolio.get_market_value](#) with default arguments.

matches_field_options method 

```
Portfolio.matches_field_options(  
    options,  
    obj_type=None,  
    group_by_aware=True,  
    wrapper=None,  
    group_by=None  
)
```

Return whether options of a field match the requirements.

Requirements include the type of the object (array, reduced array, records), the grouping of the object (1/2 dimensions, group/column-wise layout). The current grouping and cash sharing of this portfolio object are also taken into account.

When an option is not in `options`, it's automatically marked as matching.

max_drawdown cached_property 

[Portfolio.get_max_drawdown](#) with default arguments.

metrics property 

Metrics supported by [Portfolio](#).

```
HybridConfig(  
    start_index=dict(  
        title='Start Index',  
        calc_func='sim_start_index',  
        tags='wrapper'  
    ),  
    end_index=dict(  
        title='End Index',  
        calc_func='sim_end_index',  
        tags='wrapper'  
    ),  
    total_duration=dict(  
        title='Total Duration',  
        calc_func='sim_duration',  
        apply_to_timedelta=True,  
        tags='wrapper'  
    ),  
    start_value=dict(  
        title='Start Value',  
        calc_func='init_value',  
        tags='portfolio'  
    ),  
    min_value=dict(  
        title='Min Value',  
        calc_func='value.vbt.min',  
        tags='portfolio'  
    ),  
    max_value=dict(  
        title='Max Value',  
        calc_func='value.vbt.max',  
        tags='portfolio'  
    ),  
    end_value=dict(  
        title='End Value',  
        calc_func='final_value',  
        tags='portfolio'  
    ),  
    cash_deposits=dict(  
        title='Total Cash Deposits',  
        calc_func='total_cash_deposits',  
        check_has_cash_deposits=True,  
        tags='portfolio'  
    ),  
)
```

```

cash_earnings=dict(
    title='Total Cash Earnings',
    calc_func='total_cash_earnings',
    check_has_cash_earnings=True,
    tags='portfolio'
),
total_return=dict(
    title='Total Return [%]',
    calc_func='total_return',
    post_calc_func=<function Portfolio.<lambda> at 0x16a9580e0>,
    tags='portfolio'
),
bm_return=dict(
    title='Benchmark Return [%]',
    calc_func='bm_returns.vbt.returns.total',
    post_calc_func=<function Portfolio.<lambda> at 0x16a958180>,
    check_has_bm_returns=True,
    tags='portfolio'
),
total_time_exposure=dict(
    title='Position Coverage [%]',
    calc_func='position_coverage',
    post_calc_func=<function Portfolio.<lambda> at 0x16a958220>,
    tags='portfolio'
),
max_gross_exposure=dict(
    title='Max Gross Exposure [%]',
    calc_func='gross_exposure.vbt.max',
    post_calc_func=<function Portfolio.<lambda> at 0x16a9582c0>,
    tags='portfolio'
),
max_dd=dict(
    title='Max Drawdown [%]',
    calc_func='drawdowns.max_drawdown',
    post_calc_func=<function Portfolio.<lambda> at 0x16a958360>,
    tags=[
        'portfolio',
        'drawdowns'
    ]
),
max_dd_duration=dict(
    title='Max Drawdown Duration',
    calc_func='drawdowns.max_duration',
    fill_wrap_kwargs=True,
    tags=[
        'portfolio',
        'drawdowns',
        'duration'
    ]
),
total_orders=dict(
    title='Total Orders',
    calc_func='orders.count',
    tags=[
        'portfolio',
        'orders'
    ]
),
total_fees_paid=dict(
    title='Total Fees Paid',
    calc_func='orders.fees.sum',
    tags=[
        'portfolio',
        'orders'
    ]
),
total_trades=dict(
    title='Total Trades',
    calc_func='trades.count',
    incl_open=True,
    tags=[
        'portfolio',
        'trades'
    ]
),
win_rate=dict(
    title='Win Rate [%]',
    calc_func='trades.win_rate',
    post_calc_func=<function Portfolio.<lambda> at 0x16a958400>,
    tags=RepEval(
        template="['portfolio', 'trades', *incl_open_tags]",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
best_trade=dict(
    title='Best Trade [%]',
    calc_func='trades.returns.max',
    post_calc_func=<function Portfolio.<lambda> at 0x16a9584a0>,
    tags=RepEval(
        template="['portfolio', 'trades', *incl_open_tags]",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
)

```

```

    ),
    worst_trade=dict(
        title='Worst Trade [%]',
        calc_func='trades.returns.min',
        post_calc_func=<function Portfolio.<lambda> at 0x16a958540>,
        tags=RepEval(
            template="['portfolio', 'trades', *incl_open_tags]",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    avg_winning_trade=dict(
        title='Avg Winning Trade [%]',
        calc_func='trades.winning.returns.mean',
        post_calc_func=<function Portfolio.<lambda> at 0x16a9585e0>,
        tags=RepEval(
            template="['portfolio', 'trades', *incl_open_tags, 'winning']",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    avg_losing_trade=dict(
        title='Avg Losing Trade [%]',
        calc_func='trades.losing.returns.mean',
        post_calc_func=<function Portfolio.<lambda> at 0x16a958600>,
        tags=RepEval(
            template="['portfolio', 'trades', *incl_open_tags, 'losing']",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    avg_winning_trade_duration=dict(
        title='Avg Winning Trade Duration',
        calc_func='trades.winning.duration.mean',
        apply_to_timedelta=True,
        tags=RepEval(
            template="['portfolio', 'trades', *incl_open_tags, 'winning', 'duration']",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    avg_losing_trade_duration=dict(
        title='Avg Losing Trade Duration',
        calc_func='trades.losing.duration.mean',
        apply_to_timedelta=True,
        tags=RepEval(
            template="['portfolio', 'trades', *incl_open_tags, 'losing', 'duration']",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    profit_factor=dict(
        title='Profit Factor',
        calc_func='trades.profit_factor',
        tags=RepEval(
            template="['portfolio', 'trades', *incl_open_tags]",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    expectancy=dict(
        title='Expectancy',
        calc_func='trades.expectancy',
        tags=RepEval(
            template="['portfolio', 'trades', *incl_open_tags]",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    sharpe_ratio=dict(
        title='Sharpe Ratio',
        calc_func='returns_acc.sharpe_ratio',
        check_has_freq=True,
        check_has_year_freq=True,
        tags=[
            'portfolio',
            'returns'
        ]
    ),
    calmar_ratio=dict(
        title='Calmar Ratio',
        calc_func='returns_acc.calmar_ratio',
        check_has_freq=True,
        check_has_year_freq=True,

```

```
        tags=[
            'portfolio',
            'returns'
        ]
    ),
    omega_ratio=dict(
        title='Omega Ratio',
        calc_func='returns_acc.omega_ratio',
        check_has_freq=True,
        check_has_year_freq=True,
        tags=[
            'portfolio',
            'returns'
        ]
    ),
    sortino_ratio=dict(
        title='Sortino Ratio',
        calc_func='returns_acc.sortino_ratio',
        check_has_freq=True,
        check_has_year_freq=True,
        tags=[
            'portfolio',
            'returns'
        ]
    )
)
```

Returns `Portfolio._metrics`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change metrics, you can either change the config in-place, override this property, or overwrite the instance variable `Portfolio._metrics`.

net_exposure cacheable_property

[Portfolio.get_net_exposure](#) with default arguments.

omega_ratio cached_property

[Portfolio.get_omega_ratio](#) with default arguments.

open custom_property

Open price of each bar.

open_flex class property

[Portfolio.open](#) in a format suitable for flexible indexing.

order_records class property

A structured NumPy array of order records.

orders cached_property

[Portfolio.get_orders](#) with default arguments.

orders_cls class property

Class for wrapping order records.

override_in_output_config_doc class method

```
Portfolio.override_in_output_config_doc(
    __pdoc__,
    source_cls=None
)
```

Call this method on each subclass that overrides [Portfolio.in_output_config](#).

parse_field_options class method

```
Portfolio.parse_field_options(
    field
)
```

Parse options based on the name of a field.

Returns a dictionary with the parsed grouping, object type, and cleaned field name.

Grouping is parsed by looking for the following suffixes:

- '_cs': per group if grouped with cash sharing, otherwise per column
- '_pcg': per group if grouped, otherwise per column
- '_pg': per group
- '_pc': per column
- '_records': records

Object type is parsed by looking for the following suffixes:

- '_2d': element per timestamp and column or group (time series)
- '_1d': element per column or group (reduced time series)

Those substrings are then removed to produce a clean field name.

plot method

```
PlotsBuilderMixin.plots(
    subplots=None,
    tags=None,
    column=None,
    group_by=None,
    per_column=None,
    split_columns=None,
    silence_warnings=None,
    template_context=None,
    settings=None,
    filters=None,
    subplot_settings=None,
    show_titles=None,
    show_legend=None,
    show_column_label=None,
    hide_id_labels=None,
    group_id_labels=None,
    make_subplots_kwargs=None,
    fig=None,
    **layout_kwargs
)
```

See [PlotsBuilderMixin.plots](#).

plot_allocations class method

```
Portfolio.plot_allocations(
    column=None,
    allocations=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    group_by=None,
    line_shape='hv',
    line_visible=True,
    colorway='Vivid',
    xref=None,
    yref=None,
    **kwargs
)
```

Plot one group of allocations.

plot_asset_flow class method

```
Portfolio.plot_asset_flow(
    column=None,
    direction='both',
    asset_flow=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,

```

```

        wrapper=None,
        line_shape='hv',
        xref=None,
        yref=None,
        hline_shape_kwargs=None,
        **kwargs
    )

```

Plot one column of asset flow.

****kwargs** are passed to [GenericAccessor.plot](#).

plot_asset_value class method

```

Portfolio.plot_asset_value(
    column=None,
    direction='both',
    asset_value=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    group_by=None,
    line_shape='hv',
    xref=None,
    yref=None,
    hline_shape_kwargs=None,
    **kwargs
)

```

Plot one column or group of asset value.

****kwargs** are passed to [GenericAccessor.plot_against](#).

plot_assets class method

```

Portfolio.plot_assets(
    column=None,
    direction='both',
    assets=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    line_shape='hv',
    xref=None,
    yref=None,
    hline_shape_kwargs=None,
    **kwargs
)

```

Plot one column of assets.

****kwargs** are passed to [GenericAccessor.plot_against](#).

plot_cash class method

```

Portfolio.plot_cash(
    column=None,
    free=False,
    init_cash=None,
    cash=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    group_by=None,
    line_shape='hv',
    xref=None,
    yref=None,
    hline_shape_kwargs=None,
    **kwargs
)

```

Plot one column or group of cash balance.

****kwargs** are passed to [GenericAccessor.plot_against](#).

plot_cash_flow class method

```
Portfolio.plot_cash_flow(  
    column=None,  
    free=False,  
    cash_flow=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    fit_sim_range=True,  
    wrapper=None,  
    group_by=None,  
    line_shape='hv',  
    xref=None,  
    yref=None,  
    hline_shape_kwarg=None,  
    **kwargs  
)
```

Plot one column or group of cash flow.

kwargs are passed to [GenericAccessor.plot](#).

plot_cumulative_returns class method

```
Portfolio.plot_cumulative_returns(  
    column=None,  
    returns_acc=None,  
    use_asset_returns=False,  
    bm_returns=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    fit_sim_range=True,  
    wrapper=None,  
    group_by=None,  
    pct_scale=False,  
    **kwargs  
)
```

Plot one column or group of cumulative returns.

If **bm_returns** is None, will use [Portfolio.get_market_returns](#).

kwargs are passed to [ReturnsAccessor.plot_cumulative](#).

plot_drawdowns class method

```
Portfolio.plot_drawdowns(  
    column=None,  
    drawdowns=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    fit_sim_range=True,  
    wrapper=None,  
    group_by=None,  
    xref='x',  
    yref='y',  
    **kwargs  
)
```

Plot one column or group of drawdowns.

kwargs are passed to [Drawdowns.plot](#).

plot_gross_exposure class method  


```
Portfolio.plot_gross_exposure(  
    column=None,  
    direction='both',  
    gross_exposure=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    fit_sim_range=True,  
    wrapper=None,  
    group_by=None,  
    line_shape='hv',  
    xref=None,  
    yref=None,  
    hline_shape_kwarg=None,  
    **kwargs  
)
```

Plot one column or group of gross exposure.

kwargs are passed to [GenericAccessor.plot_against](#).

plot_net_exposure class method

```
Portfolio.plot_net_exposure(  
    column=None,  
    net_exposure=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    fit_sim_range=True,  
    wrapper=None,  
    group_by=None,  
    line_shape='hv',  
    xref=None,  
    yref=None,  
    hline_shape_kwarg=None,  
    **kwargs  
)
```

Plot one column or group of net exposure.

kwargs are passed to [GenericAccessor.plot_against](#).

plot_orders class method

```
Portfolio.plot_orders(  
    column=None,  
    orders=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    fit_sim_range=True,  
    wrapper=None,  
    xref=None,  
    yref=None,  
    **kwargs  
)
```

Plot one column of orders.

kwargs are passed to [Orders.plot](#).

plot_trade_pnl class method

```
Portfolio.plot_trade_pnl(  
    column=None,  
    trades=None,  
    trades_type=None,  
    sim_start=None,  
    sim_end=None,  
    rec_sim_range=False,  
    fit_sim_range=True,  
    wrapper=None,  
    pct_scale=False,  
    xref='x',  
    yref='y',  
    **kwargs  
)
```

Plot one column of trade P&L.

kwargs are passed to [Trades.plot_pnl](#).

plot_trade_signals class method

```
Portfolio.plot_trade_signals(
    column=None,
    entry_trades=None,
    exit_trades=None,
    positions=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    plot_positions='zones',
    long_entry_trace_kwargs=None,
    short_entry_trace_kwargs=None,
    long_exit_trace_kwargs=None,
    short_exit_trace_kwargs=None,
    long_shape_kwargs=None,
    short_shape_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    xref=None,
    yref=None,
    **kwargs
)
```

Plot one column or group of trade signals.

Markers and shapes are colored by trade direction (green = long, red = short).

plot_trades class method

```
Portfolio.plot_trades(
    column=None,
    trades=None,
    trades_type=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    xref='x',
    yref='y',
    **kwargs
)
```

Plot one column of trades.

****kwargs** are passed to [Trades.plot](#).

plot_underwater class method

```
Portfolio.plot_underwater(
    column=None,
    init_value=None,
    returns_acc=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    group_by=None,
    pct_scale=True,
    xref='x',
    yref='y',
    hline_shape_kwargs=None,
    **kwargs
)
```

Plot one column or group of underwater.

****kwargs** are passed to [GenericAccessor.plot](#).

plot_value class method

```
Portfolio.plot_value(
    column=None,
    init_value=None,
    value=None,
    sim_start=None,
    sim_end=None,
    rec_sim_range=False,
    fit_sim_range=True,
    wrapper=None,
    **kwargs
)
```

```
group_by=None,  
xref=None,  
yref=None,  
hline_shape_kwargs=None,  
**kwargs  
)
```

Plot one column or group of value.

`**kwargs` are passed to [GenericAccessor.plot_against](#).

plots_defaults class property 

Defaults for [PlotsBuilderMixin.plots](#).

Merges [PlotsBuilderMixin.plots_defaults](#) and `plots` from [portfolio](#).

position cacheable_property 

[Portfolio.get_assets](#) with default arguments.

position_coverage cached_property 

[Portfolio.get_position_coverage](#) with default arguments.

position_entry_price cacheable_property 

[Portfolio.get_position_entry_price](#) with default arguments.

position_exit_price cacheable_property 

[Portfolio.get_position_exit_price](#) with default arguments.

position_mask cacheable_property 

[Portfolio.get_position_mask](#) with default arguments.

positions cached_property 

[Portfolio.get_positions](#) with default arguments.

positions_cls class property 

Class for wrapping position records.

post_resolve_attr method 

```
Portfolio.post_resolve_attr(  
    attr,  
    out,  
    final_kwargs=None  
)
```

Post-process an object after resolution.

Uses the following keys:

- `incl_open`: Whether to include open trades/positions when resolving an argument that is an instance of [Trades](#).

pre_resolve_attr method 

```
Portfolio.pre_resolve_attr(
    attr,
    final_kwargs=None
)
```

Pre-process an attribute before resolution.

Uses the following keys:

- `use_asset_returns`: Whether to use [Portfolio.get_asset_returns](#) when resolving `returns` argument.
- `trades_type`: Which trade type to use when resolving `trades` argument.

prob_sharpe_ratio cached_property 

[Portfolio.get_prob_sharpe_ratio](#) with default arguments.

qs class property 

[Portfolio.get_qs](#) with default arguments.

regroup method 

```
Portfolio.regroup(
    group_by,
    **kwargs
)
```

Regroup this object.

See [Wrapping.regroup](#).

 Note

All cached objects will be lost.

resample method 

```
Portfolio.resample(
    *args,
    ffill_close=False,
    bfill_close=False,
    in_output_kwargs=None,
    wrapper_meta=None,
    **kwargs
)
```

Resample the [Portfolio](#) instance.

 Warning

Downsampling is associated with information loss:

- Cash deposits and earnings are assumed to be added/removed at the beginning of each time step. Imagine depositing \$100 and using them up in the same bar, and then depositing another \$100 and using them up. Downsampling both bars into a single bar will aggregate cash deposits and earnings, and put both of them at the beginning of the new bar, even though the second deposit was added later in time.
- Market/benchmark returns are computed by applying the initial value on the close price of the first bar and by tracking the price change to simulate holding. Moving the close price of the first bar further into the future will affect this computation and almost certainly produce a different market value and returns. To mitigate this, make sure to downsample to an index with the first bar containing only the first bar from the origin timeframe.

resample_in_outputs method 

```
Portfolio.resample_in_outputs(
    resampler,
    **kwargs
)
```

Resample [Portfolio.in_outputs](#).

If the field can be found in the attributes of this [Portfolio](#) instance, reads the attribute's options to get requirements for the type and layout of the in-output object.

For each field in [Portfolio.in_outputs](#), resolves the field's options by parsing its name with [Portfolio.parse_field_options](#) and also looks for options in [Portfolio.in_output_config](#). Performs indexing on the in-output object using [Portfolio.resample_obj](#).

resample_obj method

```
Portfolio.resample_obj(
    obj,
    resampler,
    obj_name=None,
    obj_type=None,
    wrapper=None,
    group_by=None,
    resample_func=None,
    resample_kwargs=None,
    force_resampling=False,
    silence_warnings=False,
    **kwargs
)
```

Resample an object.

`resample_func` must take the portfolio, `obj`, `resampler`, all the arguments passed to this method, and `**kwargs`. If you don't need any of the arguments, make `resample_func` accept them as `**kwargs`. If `resample_func` is a string, will use it as `reduce_func_nb` in [GenericAccessor.resample_apply](#). Default is 'last'.

If the object is None, boolean, or empty, returns as-is.

resolve_shortcut_attr method

```
Portfolio.resolve_shortcut_attr(
    attr_name,
    *args,
    **kwargs
)
```

Resolve an attribute that may have shortcut properties.

If `attr_name` has a prefix `get_`, checks whether the respective shortcut property can be called. This way, complex call hierarchies can utilize cacheable properties.

returns cacheable_property

[Portfolio.get_returns](#) with default arguments.

returns_acc class property

[Portfolio.get_returns_acc](#) with default arguments.

returns_acc_defaults class property

Defaults for [ReturnsAccessor](#).

returns_stats method

```
Portfolio.returns_stats(
    use_asset_returns=False,
    bm_returns=None,
    log_returns=False,
    daily_returns=False,
    freq=None,
    year_freq=None,
    defaults=None,
    chunked=None,
    group_by=None,
    **kwargs
)
```

Compute various statistics on returns of this portfolio.

See [Portfolio.returns_acc](#) and [ReturnsAccessor.metrics](#).

`kwargs` will be passed to [StatsBuilderMixin.stats](#) method. If `bm_returns` is not set, uses [Portfolio.get_market_returns](#).

row_stack class method

```
Portfolio.row_stack(
```

```

*objs,
wrapper_kwargs=None,
group_by=None,
combine_init_cash=False,
combine_init_position=False,
combine_init_price=False,
**kwargs
)

```

Stack multiple **Portfolio** instances along rows.

Uses **ArrayWrapper.row_stack** to stack the wrappers.

Cash sharing must be the same among all objects.

Close, benchmark close, cash deposits, cash earnings, call sequence, and other two-dimensional arrays are stacked using **ArrayWrapper.row_stack_arrs**. In-outputs are stacked using **Portfolio.row_stack_in_outputs**. Records are stacked using **Records.row_stack_records_arrs**.

If the initial cash of each object is one of the options in **InitCashMode**, it will be retained for the resulting object. Once any of the objects has the initial cash listed as an absolute amount or an array, the initial cash of the first object will be copied over to the final object, while the initial cash of all other objects will be resolved and used as cash deposits, unless they all are zero. Set `combine_init_cash` to True to simply sum all initial cash arrays.

If only the first object has an initial position greater than zero, it will be copied over to the final object. Otherwise, an error will be thrown, unless `combine_init_position` is enabled to sum all initial position arrays. The same goes for the initial price, which becomes a candidate for stacking only if any of the arrays are not NaN.

Note

When possible, avoid using initial position and price in objects to be stacked: there is currently no way of injecting them in the correct order, while simply taking the sum or weighted average may distort the reality since they weren't available prior to the actual simulation.

row_stack_in_outputs class method

```

Portfolio.row_stack_in_outputs(
    *objs,
    **kwargs
)

```

Stack **Portfolio.in_outputs** along rows.

All in-output tuples must be either None or have the same fields.

If the field can be found in the attributes of this **Portfolio** instance, reads the attribute's options to get requirements for the type and layout of the in-output object.

For each field in **Portfolio.in_outputs**, resolves the field's options by parsing its name with **Portfolio.parse_field_options** and also looks for options in **Portfolio.in_output_config**. Performs stacking on the in-output objects of the same field using **Portfolio.row_stack_objs**.

row_stack_objs class method

```

Portfolio.row_stack_objs(
    objs,
    wrappers,
    grouping='columns_or_groups',
    obj_name=None,
    obj_type=None,
    wrapper=None,
    cash_sharing=False,
    row_stack_func=None,
    **kwargs
)

```

Stack (two-dimensional) objects along rows.

`row_stack_func` must take the portfolio class, and all the arguments passed to this method. If you don't need any of the arguments, make `row_stack_func` accept them as `**kwargs`.

If all the objects are None, boolean, or empty, returns the first one.

sharpe_ratio cached_property

Portfolio.get_sharpe_ratio with default arguments.

sharpe_ratio_std cached_property

Portfolio.get_sharpe_ratio_std with default arguments.

short_allocations cacheable_property

[Portfolio.get_allocations](#) with arguments {'direction': 'shortonly'}.

short_asset_flow cacheable_property

[Portfolio.get_asset_flow](#) with arguments {'direction': 'shortonly'}.

short_asset_value cacheable_property

[Portfolio.get_asset_value](#) with arguments {'direction': 'shortonly'}.

short_assets cacheable_property

[Portfolio.get_assets](#) with arguments {'direction': 'shortonly'}.

short_gross_exposure cacheable_property

[Portfolio.get_gross_exposure](#) with arguments {'direction': 'shortonly'}.

short_position_coverage cached_property

[Portfolio.get_position_coverage](#) with arguments {'direction': 'shortonly'}.

short_position_mask cacheable_property

[Portfolio.get_position_mask](#) with arguments {'direction': 'shortonly'}.

short_view cacheable_property

[Portfolio.get_short_view](#) with default arguments.

signals cacheable_property

[Portfolio.get_signals](#) with default arguments.

sortino_ratio cached_property

[Portfolio.get_sortino_ratio](#) with default arguments.

stats_defaults class property

Defaults for [StatsBuilderMixin.stats](#).

Merges [StatsBuilderMixin.stats_defaults](#) and `stats` from [portfolio](#).

subplots property

Subplots supported by [Portfolio](#).

```
HybridConfig(
    orders=dict(
        title='Orders',
        yaxis_kwargs=dict(
            title='Price'
        ),
        check_is_not_grouped=True,
        plot_func='plot_orders',
        pass_add_trace_kwargs=True,
        tags=[
            'portfolio',
            'orders'
        ]
    )
)
```

```

    ),
    trades=dict(
        title='Trades',
        yaxis_kwargs=dict(
            title='Price'
        ),
        check_is_not_grouped=True,
        plot_func='plot_trades',
        pass_add_trace_kwargs=True,
        tags=[
            'portfolio',
            'trades'
        ]
    ),
    trade_pnl=dict(
        title='Trade PnL',
        yaxis_kwargs=dict(
            title='PnL'
        ),
        check_is_not_grouped=True,
        plot_func='plot_trade_pnl',
        pct_scale=True,
        pass_add_trace_kwargs=True,
        tags=[
            'portfolio',
            'trades'
        ]
    ),
    trade_signals=dict(
        title='Trade Signals',
        yaxis_kwargs=dict(
            title='Price'
        ),
        check_is_not_grouped=True,
        plot_func='plot_trade_signals',
        tags=[
            'portfolio',
            'trades'
        ]
    ),
    cash_flow=dict(
        title='Cash Flow',
        yaxis_kwargs=dict(
            title='Amount'
        ),
        plot_func='plot_cash_flow',
        pass_add_trace_kwargs=True,
        tags=[
            'portfolio',
            'cash'
        ]
    ),
    cash=dict(
        title='Cash',
        yaxis_kwargs=dict(
            title='Amount'
        ),
        plot_func='plot_cash',
        pass_add_trace_kwargs=True,
        tags=[
            'portfolio',
            'cash'
        ]
    ),
    asset_flow=dict(
        title='Asset Flow',
        yaxis_kwargs=dict(
            title='Amount'
        ),
        check_is_not_grouped=True,
        plot_func='plot_asset_flow',
        pass_add_trace_kwargs=True,
        tags=[
            'portfolio',
            'assets'
        ]
    ),
    assets=dict(
        title='Assets',
        yaxis_kwargs=dict(
            title='Amount'
        ),
        check_is_not_grouped=True,
        plot_func='plot_assets',
        pass_add_trace_kwargs=True,
        tags=[
            'portfolio',
            'assets'
        ]
    ),
    asset_value=dict(
        title='Asset Value',
        yaxis_kwargs=dict(
            title='Value'
        ),
        plot_func='plot_asset_value',
        pass_add_trace_kwargs=True,
        tags=[

```



```

        'portfolio',
        'assets',
        'value'
    ]
),
value=dict(
    title='Value',
    yaxis_kwargs=dict(
        title='Value'
    ),
    plot_func='plot_value',
    pass_add_trace_kwargs=True,
    tags=[
        'portfolio',
        'value'
    ]
),
cumulative_returns=dict(
    title='Cumulative Returns',
    yaxis_kwargs=dict(
        title='Cumulative return'
    ),
    plot_func='plot_cumulative_returns',
    pass_hline_shape_kwargs=True,
    pass_add_trace_kwargs=True,
    pass_xref=True,
    pass_yref=True,
    tags=[
        'portfolio',
        'returns'
    ]
),
drawdowns=dict(
    title='Drawdowns',
    yaxis_kwargs=dict(
        title='Value'
    ),
    plot_func='plot_drawdowns',
    pass_add_trace_kwargs=True,
    pass_xref=True,
    pass_yref=True,
    tags=[
        'portfolio',
        'value',
        'drawdowns'
    ]
),
underwater=dict(
    title='Underwater',
    yaxis_kwargs=dict(
        title='Drawdown'
    ),
    plot_func='plot_underwater',
    pass_add_trace_kwargs=True,
    tags=[
        'portfolio',
        'value',
        'drawdowns'
    ]
),
gross_exposure=dict(
    title='Gross Exposure',
    yaxis_kwargs=dict(
        title='Exposure'
    ),
    plot_func='plot_gross_exposure',
    pass_add_trace_kwargs=True,
    tags=[
        'portfolio',
        'exposure'
    ]
),
net_exposure=dict(
    title='Net Exposure',
    yaxis_kwargs=dict(
        title='Exposure'
    ),
    plot_func='plot_net_exposure',
    pass_add_trace_kwargs=True,
    tags=[
        'portfolio',
        'exposure'
    ]
),
allocations=dict(
    title='Allocations',
    yaxis_kwargs=dict(
        title='Allocation'
    ),
    plot_func='plot_allocations',
    pass_add_trace_kwargs=True,
    tags=[
        'portfolio',
        'allocations'
    ]
)
)

```

Returns `Portfolio._subplots`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change subplots, you can either change the config in-place, override this property, or overwrite the instance variable `Portfolio._subplots`.

tail_ratio cached_property 

[Portfolio.get_tail_ratio](#) with default arguments.

total_cash_deposits cached_property 

[Portfolio.get_total_cash_deposits](#) with default arguments.

total_cash_earnings cached_property 

[Portfolio.get_total_cash_earnings](#) with default arguments.

total_market_return cached_property 

[Portfolio.get_total_market_return](#) with default arguments.

total_profit cached_property 

[Portfolio.get_total_profit](#) with default arguments.

total_return cached_property 

[Portfolio.get_total_return](#) with default arguments.

trade_history cacheable_property 

[Portfolio.get_trade_history](#) with default arguments.

trades cached_property 

[Portfolio.get_trades](#) with default arguments.

trades_cls class property 

Class for wrapping trade records.

trades_type class property 

Default [Trades](#) to use across [Portfolio](#).

up_capture_ratio cached_property 

[Portfolio.get_up_capture_ratio](#) with default arguments.

use_in_outputs class property 

Whether to return in-output objects when calling properties.

value cacheable_property 

[Portfolio.get_value](#) with default arguments.

value_at_risk cached_property

[Portfolio.get_value_at_risk](#) with default arguments.

weights cached_property

[Portfolio.get_weights](#) with default arguments.

wrap_obj method

```
Portfolio.wrap_obj(
    obj,
    obj_name=None,
    grouping='columns_or_groups',
    obj_type=None,
    wrapper=None,
    group_by=None,
    wrap_func=None,
    wrap_kwargs=None,
    force_wrapping=False,
    silence_warnings=False,
    **kwargs
)
```

Wrap an object.

`wrap_func` must take the portfolio, `obj`, all the arguments passed to this method, and `**kwargs`. If you don't need any of the arguments, make `indexing_func` accept them as `**kwargs`.

If the object is None or boolean, returns as-is.

year_freq class property

Year frequency.

PortfolioWithInOutputs class

```
PortfolioWithInOutputs()
```

Class exposes a read-only class property `RecordsWithFields.field_config`.

Superclasses

- [Base](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)

Subclasses

- [Portfolio](#)

in_output_config class property

In-output config of `{cls_name}`.

```
{in_output_config}
```

API portfolio

preparing

preparing module

Classes for preparing portfolio simulations.

base_arg_config ReadonlyConfig

Argument config for BasePFPreparer.

```

ReadonlyConfig(
    data=dict(),
    open=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    high=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    low=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    close=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    bm_close=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    cash_earnings=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0.0
            )
        )
    ),
    init_cash=dict(
        map_enum_kwargs=dict(
            enum=InitCashModeT(
                Auto=-1,
                AutoAlign=-2
            ),
            look_for_type=<class 'str'>
        )
    ),
    init_position=dict(),
    init_price=dict(),
    cash_deposits=dict(),
    group_by=dict(),
    cash_sharing=dict(),
    freq=dict(),
    sim_start=dict(),
    sim_end=dict(),
    call_seq=dict(
        map_enum_kwargs=dict(
            enum=CallSeqTypeT(

```

```

        Default=0,
        Reversed=1,
        Random=2,
        Auto=3
    ),
    look_for_type=<class 'str'>
)
),
attach_call_seq=dict(),
keep_inout_flex=dict(),
in_outputs=dict(
    has_default=False
)
)
)

```

fdof_arg_config ReadOnlyConfig

Argument config for [FDOFPreparer](#).

```

ReadOnlyConfig(
    val_price=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=ValPriceTypeT(
                Latest=-np.inf,
                Price=np.inf
            ),
            ignore_type=(
                <class 'int'>,
                <class 'float'>
            )
        ),
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    flexible=dict()
)

```

fo_arg_config ReadOnlyConfig

Argument config for [FOPreparer](#).

```

ReadOnlyConfig(
    cash_dividends=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0.0
            )
        )
    ),
    val_price=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=ValPriceTypeT(
                Latest=-np.inf,
                Price=np.inf
            ),
            ignore_type=(
                <class 'int'>,
                <class 'float'>
            )
        ),
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    from_ago=dict(
        broadcast=True,
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0
            )
        )
    ),
    ffill_val_price=dict(),
    update_value=dict(),
    save_state=dict(),
    save_value=dict(),
)

```

```

        save_returns=dict(),
        skip_empty=dict(),
        max_order_records=dict(),
        max_log_records=dict()
    )

```

fof_arg_config ReadOnlyConfig

Argument config for **FOFPreparer**.

```

ReadOnlyConfig(
    segment_mask=dict(),
    call_pre_segment=dict(),
    call_post_segment=dict(),
    pre_sim_func_nb=dict(),
    pre_sim_args=dict(
        type='args',
        substitute_templates=True
    ),
    post_sim_func_nb=dict(),
    post_sim_args=dict(
        type='args',
        substitute_templates=True
    ),
    pre_group_func_nb=dict(),
    pre_group_args=dict(
        type='args',
        substitute_templates=True
    ),
    post_group_func_nb=dict(),
    post_group_args=dict(
        type='args',
        substitute_templates=True
    ),
    pre_row_func_nb=dict(),
    pre_row_args=dict(
        type='args',
        substitute_templates=True
    ),
    post_row_func_nb=dict(),
    post_row_args=dict(
        type='args',
        substitute_templates=True
    ),
    pre_segment_func_nb=dict(),
    pre_segment_args=dict(
        type='args',
        substitute_templates=True
    ),
    post_segment_func_nb=dict(),
    post_segment_args=dict(
        type='args',
        substitute_templates=True
    ),
    order_func_nb=dict(),
    order_args=dict(
        type='args',
        substitute_templates=True
    ),
    flex_order_func_nb=dict(),
    flex_order_args=dict(
        type='args',
        substitute_templates=True
    ),
    post_order_func_nb=dict(),
    post_order_args=dict(
        type='args',
        substitute_templates=True
    ),
    ffill_val_price=dict(),
    update_value=dict(),
    fill_pos_info=dict(),
    track_value=dict(),
    row_wise=dict(),
    max_order_records=dict(),
    max_log_records=dict()
)

```

fs_arg_config ReadOnlyConfig

Argument config for **FSPreparer**.

```

ReadOnlyConfig(
    size=dict(
        fill_default=True
    ),
    cash_dividends=dict(
        broadcast=True,
    )
)

```

```

        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0.0
            )
        ),
    ),
    entries=dict(
        has_default=False,
        broadcast=True,
        subtype=<class 'humpy.bool'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    exits=dict(
        has_default=False,
        broadcast=True,
        subtype=<class 'humpy.bool'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    long_entries=dict(
        has_default=False,
        broadcast=True,
        subtype=<class 'humpy.bool'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    long_exits=dict(
        has_default=False,
        broadcast=True,
        subtype=<class 'humpy.bool'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    short_entries=dict(
        has_default=False,
        broadcast=True,
        subtype=<class 'humpy.bool'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    short_exits=dict(
        has_default=False,
        broadcast=True,
        subtype=<class 'humpy.bool'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    adjust_func_nb=dict(),
    adjust_args=dict(
        type='args',
        substitute_templates=True
    ),
    signal_func_nb=dict(),
    signal_args=dict(
        type='args',
        substitute_templates=True
    ),
    post_signal_func_nb=dict(),
    post_signal_args=dict(
        type='args',
        substitute_templates=True
    ),
    post_segment_func_nb=dict(),
    post_segment_args=dict(
        type='args',
        substitute_templates=True
    ),
    order_mode=dict(),
    val_price=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=ValPriceTypeT(
                Latest=-np.inf,
                Price=np.inf
            )
        ),
        ignore_type=(
            <class 'int'>,
            <class 'float'>
        )
    )

```

```

    ),
    subtype=<class 'humpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=np.nan
        )
    ),
),
accumulate=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=AccumulationModeT(
            Disabled=0,
            Both=1,
            AddOnly=2,
            RemoveOnly=3
        ),
        ignore_type=(
            <class 'int'>,
            <class 'bool'>
        )
    ),
    subtype=(
        <class 'humpy.integer'>,
        <class 'humpy.bool'>
    ),
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=0
        )
    )
),
upon_long_conflict=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=ConflictModeT(
            Ignore=0,
            Entry=1,
            Exit=2,
            Adjacent=3,
            Opposite=4
        )
    ),
    subtype=<class 'humpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=0
        )
    )
),
upon_short_conflict=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=ConflictModeT(
            Ignore=0,
            Entry=1,
            Exit=2,
            Adjacent=3,
            Opposite=4
        )
    ),
    subtype=<class 'humpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=0
        )
    )
),
upon_dir_conflict=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=DirectionConflictModeT(
            Ignore=0,
            Long=1,
            Short=2,
            Adjacent=3,
            Opposite=4
        )
    ),
    subtype=<class 'humpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=0
        )
    )
),
upon_opposite_entry=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=OppositeEntryModeT(
            Ignore=0,
            Close=1,
            CloseReduce=2,
            Reverse=3,
            ReverseReduce=4
        )
    )
),

```



```

        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=4
            )
        ),
    ),
    order_type=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=OrderTypeT(
                Market=0,
                Limit=1
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0
            )
        )
    ),
    limit_delta=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    limit_tif=dict(
        broadcast=True,
        is_td=True,
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=-1
            )
        )
    ),
    limit_expiry=dict(
        broadcast=True,
        is_dt=True,
        last_before=False,
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=-1
            )
        )
    ),
    limit_reverse=dict(
        broadcast=True,
        subtype=<class 'humpy.bool_'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    limit_order_price=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=LimitOrderPriceT(
                Limit=-1,
                HardLimit=-2,
                Close=-3
            )
        ),
        ignore_type=(
            <class 'int'>,
            <class 'float'>
        )
    ),
    subtype=<class 'humpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=-1
        )
    )
),
upon_adj_limit_conflict=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=PendingConflictModeT(
            KeepIgnore=0,
            KeepExecute=1,
            CancelIgnore=2,
            CancelExecute=3
        )
    ),
    subtype=<class 'humpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=0
        )
    )
),

```

```

upon_opp_limit_conflict=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=PendingConflictModeT(
            KeepIgnore=0,
            KeepExecute=1,
            CancelIgnore=2,
            CancelExecute=3
        )
    ),
    subtype=<class 'numpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=3
        )
    )
),
use_stops=dict(),
stop_ladder=dict(
    map_enum_kwargs=dict(
        enum=StopLadderModeT(
            Disabled=0,
            Uniform=1,
            Weighted=2,
            AdaptUniform=3,
            AdaptWeighted=4,
            Dynamic=5
        )
    ),
    look_for_type=<class 'str'>
),
sl_stop=dict(
    broadcast=True,
    subtype=<class 'numpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=np.nan
        )
    )
),
tsl_stop=dict(
    broadcast=True,
    subtype=<class 'numpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=np.nan
        )
    )
),
tsl_th=dict(
    broadcast=True,
    subtype=<class 'numpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=np.nan
        )
    )
),
tp_stop=dict(
    broadcast=True,
    subtype=<class 'numpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=np.nan
        )
    )
),
td_stop=dict(
    broadcast=True,
    is_td=True,
    subtype=<class 'numpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=-1
        )
    )
),
dt_stop=dict(
    broadcast=True,
    is_dt=True,
    subtype=<class 'numpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=-1
        )
    )
),
stop_entry_price=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=StopEntryPriceT(
            ValPrice=-1,
            Open=-2,
            Price=-3,
            FillPrice=-4,
            Close=-5
        )
    ),
    ignore_type=(

```

```

        <class 'int'>,
        <class 'float'>
    ),
    subtype=<class 'humpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=-5
        )
    ),
),
stop_exit_price=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=StopExitPriceT(
            Stop=1,
            HardStop=-2,
            Close=-3
        ),
        ignore_type=(
            <class 'int'>,
            <class 'float'>
        )
    ),
    subtype=<class 'humpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=-1
        )
    )
),
stop_exit_type=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=StopExitTypeT(
            Close=0,
            CloseReduce=1,
            Reverse=2,
            ReverseReduce=3
        )
    ),
    subtype=<class 'humpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=0
        )
    )
),
stop_order_type=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=OrderTypeT(
            Market=0,
            Limit=1
        )
    ),
    subtype=<class 'humpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=0
        )
    )
),
stop_limit_delta=dict(
    broadcast=True,
    subtype=<class 'humpy.number'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=np.nan
        )
    )
),
upon_stop_update=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=StopUpdateModeT(
            Keep=0,
            Override=1,
            OverrideNaN=2
        )
    ),
    subtype=<class 'humpy.integer'>,
    broadcast_kwargs=dict(
        reindex_kwargs=dict(
            fill_value=1
        )
    )
),
upon_adj_stop_conflict=dict(
    broadcast=True,
    map_enum_kwargs=dict(
        enum=PendingConflictModeT(
            KeepIgnore=0,
            KeepExecute=1,
            CancelIgnore=2,
            CancelExecute=3
        )
    )
),

```

```

        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=1
            )
        ),
    ),
    upon_opp_stop_conflict=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=PendingConflictModeT(
                KeepIgnore=0,
                KeepExecute=1,
                CancelIgnore=2,
                CancelExecute=3
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=1
            )
        )
    ),
    delta_format=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=DeltaFormatT(
                Absolute=0,
                Percent=1,
                Percent100=2,
                Target=3
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=1
            )
        )
    ),
    time_delta_format=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=TimeDeltaFormatT(
                Rows=0,
                Index=1
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=1
            )
        )
    ),
    from_ago=dict(
        broadcast=True,
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0
            )
        )
    ),
    ffill_val_price=dict(),
    update_value=dict(),
    fill_pos_info=dict(),
    save_state=dict(),
    save_value=dict(),
    save_returns=dict(),
    skip_empty=dict(),
    max_order_records=dict(),
    max_log_records=dict(),
    records=dict(
        rename_fields=dict(
            entry='entries',
            exit='exits',
            long_entry='long_entries',
            long_exit='long_exits',
            short_entry='short_entries',
            short_exit='short_exits'
        )
    )
)

```

order_arg_config ReadonlyConfig 

Argument config for order-related information.

```

ReadonlyConfig(
    size=dict(
        broadcast=True,

```

```

        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        ),
        fill_default=False
    ),
    price=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=PriceTypeT(
                Open=-np.inf,
                Close=np.inf,
                NextOpen=-1,
                NextClose=-2,
                NextValidOpen=-3,
                NextValidClose=-4
            ),
            ignore_type=(
                <class 'int'>,
                <class 'float'>
            )
        ),
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.inf
            )
        )
    ),
    size_type=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=SizeTypeT(
                Amount=0,
                Value=1,
                Percent=2,
                Percent100=3,
                ValuePercent=4,
                ValuePercent100=5,
                TargetAmount=6,
                TargetValue=7,
                TargetPercent=8,
                TargetPercent100=9
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0
            )
        )
    ),
    direction=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=DirectionT(
                LongOnly=0,
                ShortOnly=1,
                Both=2
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=2
            )
        )
    ),
    fees=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0.0
            )
        )
    ),
    fixed_fees=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0.0
            )
        )
    ),
    slippage=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0.0
            )
        )
    ),
    min_size=dict(

```

```

        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        ),
    ),
    max_size=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    size_granularity=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=np.nan
            )
        )
    ),
    leverage=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=1.0
            )
        )
    ),
    leverage_mode=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=LeverageModeT(
                Lazy=0,
                Eager=1
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0
            )
        )
    ),
    reject_prob=dict(
        broadcast=True,
        subtype=<class 'humpy.number'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0.0
            )
        )
    ),
    price_area_vio_mode=dict(
        broadcast=True,
        map_enum_kwargs=dict(
            enum=PriceAreaVioModeT(
                Ignore=0,
                Cap=1,
                Error=2
            )
        ),
        subtype=<class 'humpy.integer'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=0
            )
        )
    ),
    allow_partial=dict(
        broadcast=True,
        subtype=<class 'humpy.bool_'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=True
            )
        )
    ),
    raise_reject=dict(
        broadcast=True,
        subtype=<class 'humpy.bool_'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(
                fill_value=False
            )
        )
    ),
    log=dict(
        broadcast=True,
        subtype=<class 'humpy.bool_'>,
        broadcast_kwargs=dict(
            reindex_kwargs=dict(

```

```

        fill_value=False
    )
    )
)

```

valid_price_from_ago_1d_nb [function](#)

```

valid_price_from_ago_1d_nb(
    price
)

```

Parse from_ago from a valid price.

BasePFPreparer [class](#)

```

BasePFPreparer(
    arg_config=None,
    **kwargs
)

```

Base class for preparing portfolio simulations.

Superclasses

- [Base](#)
- [BasePreparer](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [HasSettings](#)
- [Pickleable](#)
- [Prettified](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [BasePreparer.adapt_staticized_to_udf](#)
- [BasePreparer.args_to_broadcast](#)
- [BasePreparer.broadcast_kwargs](#)
- [BasePreparer.broadcast_named_args](#)
- [BasePreparer.broadcast_result](#)
- [BasePreparer.build_arg_config_doc](#)
- [BasePreparer.chunked](#)
- [BasePreparer.config](#)
- [BasePreparer.def_broadcast_kwargs](#)
- [BasePreparer.dt_arr_to_ns](#)
- [BasePreparer.find_target_func](#)
- [BasePreparer.freq](#)
- [BasePreparer.get_arg](#)
- [BasePreparer.get_arg_default](#)
- [BasePreparer.get_raw_arg](#)
- [BasePreparer.get_raw_arg_default](#)
- [BasePreparer.idx_setters](#)
- [BasePreparer.index](#)

- `BasePreparer.jitted`
- `BasePreparer.map_enum_value`
- `BasePreparer.override_arg_config_doc`
- `BasePreparer.post_args`
- `BasePreparer.post_broadcast_named_args`
- `BasePreparer.pre_args`
- `BasePreparer.prepare_dt_arr`
- `BasePreparer.prepare_dt_obj`
- `BasePreparer.prepare_post_arg`
- `BasePreparer.prepare_td_arr`
- `BasePreparer.prepare_td_obj`
- `BasePreparer.rec_state`
- `BasePreparer.records`
- `BasePreparer.resolve_dynamic_target_func`
- `BasePreparer.seed`
- `BasePreparer.set_seed`
- `BasePreparer.staticized`
- `BasePreparer.target_arg_map`
- `BasePreparer.target_args`
- `BasePreparer.target_func`
- `BasePreparer.target_shape`
- `BasePreparer.td_arr_to_ns`
- `BasePreparer.template_context`
- `BasePreparer.wrapper`
- `Cacheable.get_ca_setup`
- `Chainable.chain`
- `Chainable.pipe`
- `Configured.copy`
- `Configured.equals`
- `Configured.get_writeable_attrs`
- `Configured.prettify`
- `Configured.replace`
- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `HasSettings.get_path_setting`
- `HasSettings.get_path_settings`
- `HasSettings.get_setting`
- `HasSettings.get_settings`
- `HasSettings.has_path_setting`
- `HasSettings.has_path_settings`
- `HasSettings.has_setting`
- `HasSettings.has_settings`
- `HasSettings.reset_settings`
- `HasSettings.resolve_setting`
- `HasSettings.resolve_settings_paths`
- `HasSettings.set_settings`
- `Pickleable.decode_config`
- `Pickleable.decode_config_node`
- `Pickleable.dumps`
- `Pickleable.encode_config`
- `Pickleable.encode_config_node`
- `Pickleable.file_exists`
- `Pickleable.getsize`
- `Pickleable.load`

- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

Subclasses

- [FOFPreparer](#)
- [FOPreparer](#)
- [FSPreparer](#)

align_pc_arr method

```
BasePFPreparer.align_pc_arr(  
    arr,  
    group_lens=None,  
    check_dtype=None,  
    cast_to_dtype=None,  
    reduce_func=None,  
    arg_name=None  
)
```

Align a per-column array.

attach_call_seq cached_property

Argument `attach_call_seq`.

auto_call_seq cached_property

Whether automatic call sequence is enabled.

auto_sim_end cached_property

Get automatic `sim_end`

auto_sim_start cached_property

Get automatic `sim_start`

bm_close cached_property

Argument `bm_close`.

call_seq cached_property

Argument `call_seq`.

cash_deposits cached_property

Argument `cash_deposits`.

cash_earnings cached_property

Argument `cash_earnings`.

cash_sharing cached_property

Argument `cash_sharing`.

close cached_property 

Argument `close`.

cs_group_lens cached_property 

Cash sharing aware group lengths.

data cached_property 

Argument `data`.

group_by cached_property 

Argument `group_by`.

group_lens cached_property 

Group lengths.

high cached_property 

Argument `high`.

in_outputs cached_property 

Argument `in_outputs`.

init_cash cached_property 

Argument `init_cash`.

init_cash_mode cached_property 

Initial cash mode.

init_position cached_property 

Argument `init_position`.

init_price cached_property 

Argument `init_price`.

keep_inout_flex cached_property 

Argument `keep_inout_flex`.

low cached_property 

Argument `low`.

open cached_property 

Argument `open`.

parse_data class method

```
BasePFPreparer.parse_data(  
    data,  
    all_ohlcv=False  
)
```

Parse an instance with OHLC features.

pf_args cached_property

Arguments to be passed to the portfolio.

result cached_property

Result as an instance of **PFPRepResult**.

sim_end cached_property

Argument `sim_end`.

sim_group_lens cached_property

Simulation group lengths.

sim_start cached_property

Argument `sim_start`.

FDOFPreparer class

```
FDOFPreparer(  
    arg_config=None,  
    **kwargs  
)
```

Class for preparing **Portfolio.from_def_order_func**.

Superclasses

- **Base**
- **BasePFPreparer**
- **BasePreparer**
- **Cacheable**
- **Chainable**
- **Comparable**
- **Configured**
- **FOFPreparer**
- **HasSettings**
- **Pickleable**
- **Prettified**

Inherited members

- **Base.chat**
- **Base.find_api**
- **Base.find_assets**
- **Base.find_docs**
- **Base.find_examples**
- **Base.find_messages**
- **BasePFPreparer.align_pc_arr**
- **BasePFPreparer.find_target_func**

- `BasePFPreparer.parse_data`
- `BasePFPreparer.sim_end`
- `BasePFPreparer.sim_start`
- `BasePFPreparer.target_arg_map`
- `BasePFPreparer.target_func`
- `BasePreparer.adapt_staticized_to_udf`
- `BasePreparer.build_arg_config_doc`
- `BasePreparer.dt_arr_to_ns`
- `BasePreparer.get_arg`
- `BasePreparer.get_arg_default`
- `BasePreparer.get_raw_arg`
- `BasePreparer.get_raw_arg_default`
- `BasePreparer.map_enum_value`
- `BasePreparer.override_arg_config_doc`
- `BasePreparer.prepare_dt_arr`
- `BasePreparer.prepare_dt_obj`
- `BasePreparer.prepare_post_arg`
- `BasePreparer.prepare_td_arr`
- `BasePreparer.prepare_td_obj`
- `BasePreparer.resolve_dynamic_target_func`
- `BasePreparer.set_seed`
- `BasePreparer.td_arr_to_ns`
- `BasePreparer.template_context`
- `Cacheable.get_ca_setup`
- `Chainable.chain`
- `Chainable.pipe`
- `Configured.copy`
- `Configured.equals`
- `Configured.get_writeable_attrs`
- `Configured.prettify`
- `Configured.replace`
- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `FOFPreparer.args_to_broadcast`
- `FOFPreparer.attach_call_seq`
- `FOFPreparer.auto_call_seq`
- `FOFPreparer.auto_sim_end`
- `FOFPreparer.auto_sim_start`
- `FOFPreparer.bm_close`
- `FOFPreparer.broadcast_kwargs`
- `FOFPreparer.broadcast_named_args`
- `FOFPreparer.broadcast_result`
- `FOFPreparer.call_post_segment`
- `FOFPreparer.call_pre_segment`
- `FOFPreparer.call_seq`
- `FOFPreparer.cash_deposits`
- `FOFPreparer.cash_earnings`
- `FOFPreparer.cash_sharing`
- `FOFPreparer.chunked`
- `FOFPreparer.close`
- `FOFPreparer.config`
- `FOFPreparer.cs_group_lens`
- `FOFPreparer.data`
- `FOFPreparer.def_broadcast_kwargs`

- FOFPPreparer.ffill_val_price
- FOFPPreparer.fill_pos_info
- FOFPPreparer.flex_order_args
- FOFPPreparer.flex_order_func_nb
- FOFPPreparer.flexible
- FOFPPreparer.freq
- FOFPPreparer.group_by
- FOFPPreparer.group_lens
- FOFPPreparer.high
- FOFPPreparer.idx_setters
- FOFPPreparer.in_outputs
- FOFPPreparer.index
- FOFPPreparer.init_cash
- FOFPPreparer.init_cash_mode
- FOFPPreparer.init_position
- FOFPPreparer.init_price
- FOFPPreparer.jitted
- FOFPPreparer.keep_inout_flex
- FOFPPreparer.low
- FOFPPreparer.max_log_records
- FOFPPreparer.max_order_records
- FOFPPreparer.open
- FOFPPreparer.order_args
- FOFPPreparer.order_func_nb
- FOFPPreparer.pf_args
- FOFPPreparer.post_args
- FOFPPreparer.post_broadcast_named_args
- FOFPPreparer.post_group_args
- FOFPPreparer.post_group_func_nb
- FOFPPreparer.post_order_args
- FOFPPreparer.post_order_func_nb
- FOFPPreparer.post_row_args
- FOFPPreparer.post_row_func_nb
- FOFPPreparer.post_segment_args
- FOFPPreparer.post_segment_func_nb
- FOFPPreparer.post_sim_args
- FOFPPreparer.post_sim_func_nb
- FOFPPreparer.pre_args
- FOFPPreparer.pre_group_args
- FOFPPreparer.pre_group_func_nb
- FOFPPreparer.pre_row_args
- FOFPPreparer.pre_row_func_nb
- FOFPPreparer.pre_segment_args
- FOFPPreparer.pre_segment_func_nb
- FOFPPreparer.pre_sim_args
- FOFPPreparer.pre_sim_func_nb
- FOFPPreparer.rec_state
- FOFPPreparer.records
- FOFPPreparer.result
- FOFPPreparer.row_wise
- FOFPPreparer.seed
- FOFPPreparer.segment_mask
- FOFPPreparer.sim_group_lens
- FOFPPreparer.staticized

- [FOFPreparer.target_args](#)
- [FOFPreparer.target_shape](#)
- [FOFPreparer.track_value](#)
- [FOFPreparer.update_value](#)
- [FOFPreparer.wrapper](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)
- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

allow_partial cached_property 

Argument `allow_partial`.

direction cached_property 

Argument `direction`.

fees cached_property 

Argument `fees`.

fixed_fees cached_property 

Argument `fixed_fees`.

leverage cached_property 

Argument `leverage`.

leverage_mode cached_property 

Argument `leverage_mode`.

log cached_property ⓘ

Argument `log`.

max_size cached_property ⓘ

Argument `max_size`.

min_size cached_property ⓘ

Argument `min_size`.

price cached_property ⓘ

Argument `price`.

price_area_vio_mode cached_property ⓘ

Argument `price_area_vio_mode`.

raise_reject cached_property ⓘ

Argument `raise_reject`.

reject_prob cached_property ⓘ

Argument `reject_prob`.

size cached_property ⓘ

Argument `size`.

size_granularity cached_property ⓘ

Argument `size_granularity`.

size_type cached_property ⓘ

Argument `size_type`.

slippage cached_property ⓘ

Argument `slippage`.

val_price cached_property ⓘ

Argument `val_price`.

FOFPreparer class ⓘ

```
FOFPreparer(
    arg_config=None,
    **kwargs
)
```

Class for preparing `Portfolio.from_order_func`.

Superclasses

- [Base](#)
- [BasePFPreparer](#)
- [BasePreparer](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [HasSettings](#)
- [Pickleable](#)
- [Prettified](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [BasePFPreparer.align_pc_arr](#)
- [BasePFPreparer.args_to_broadcast](#)
- [BasePFPreparer.attach_call_seq](#)
- [BasePFPreparer.auto_call_seq](#)
- [BasePFPreparer.auto_sim_end](#)
- [BasePFPreparer.auto_sim_start](#)
- [BasePFPreparer.bm_close](#)
- [BasePFPreparer.broadcast_kwargs](#)
- [BasePFPreparer.broadcast_named_args](#)
- [BasePFPreparer.broadcast_result](#)
- [BasePFPreparer.call_seq](#)
- [BasePFPreparer.cash_deposits](#)
- [BasePFPreparer.cash_earnings](#)
- [BasePFPreparer.cash_sharing](#)
- [BasePFPreparer.chunked](#)
- [BasePFPreparer.close](#)
- [BasePFPreparer.config](#)
- [BasePFPreparer.cs_group_lens](#)
- [BasePFPreparer.data](#)
- [BasePFPreparer.def_broadcast_kwargs](#)
- [BasePFPreparer.find_target_func](#)
- [BasePFPreparer.freq](#)
- [BasePFPreparer.group_by](#)
- [BasePFPreparer.group_lens](#)
- [BasePFPreparer.high](#)
- [BasePFPreparer.idx_setters](#)
- [BasePFPreparer.in_outputs](#)
- [BasePFPreparer.index](#)
- [BasePFPreparer.init_cash](#)
- [BasePFPreparer.init_cash_mode](#)
- [BasePFPreparer.init_position](#)
- [BasePFPreparer.init_price](#)
- [BasePFPreparer.jitted](#)
- [BasePFPreparer.keep_inout_flex](#)
- [BasePFPreparer.low](#)
- [BasePFPreparer.open](#)
- [BasePFPreparer.parse_data](#)

- `BasePFPreparer.pf_args`
- `BasePFPreparer.post_args`
- `BasePFPreparer.post_broadcast_named_args`
- `BasePFPreparer.pre_args`
- `BasePFPreparer.rec_state`
- `BasePFPreparer.records`
- `BasePFPreparer.result`
- `BasePFPreparer.seed`
- `BasePFPreparer.sim_end`
- `BasePFPreparer.sim_group_lens`
- `BasePFPreparer.sim_start`
- `BasePFPreparer.staticized`
- `BasePFPreparer.target_arg_map`
- `BasePFPreparer.target_args`
- `BasePFPreparer.target_func`
- `BasePFPreparer.target_shape`
- `BasePFPreparer.wrapper`
- `BasePreparer.adapt_staticized_to_udf`
- `BasePreparer.build_arg_config_doc`
- `BasePreparer.dt_arr_to_ns`
- `BasePreparer.get_arg`
- `BasePreparer.get_arg_default`
- `BasePreparer.get_raw_arg`
- `BasePreparer.get_raw_arg_default`
- `BasePreparer.map_enum_value`
- `BasePreparer.override_arg_config_doc`
- `BasePreparer.prepare_dt_arr`
- `BasePreparer.prepare_dt_obj`
- `BasePreparer.prepare_post_arg`
- `BasePreparer.prepare_td_arr`
- `BasePreparer.prepare_td_obj`
- `BasePreparer.resolve_dynamic_target_func`
- `BasePreparer.set_seed`
- `BasePreparer.td_arr_to_ns`
- `BasePreparer.template_context`
- `Cacheable.get_ca_setup`
- `Chainable.chain`
- `Chainable.pipe`
- `Configured.copy`
- `Configured.equals`
- `Configured.get_writeable_attrs`
- `Configured.prettify`
- `Configured.replace`
- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `HasSettings.get_path_setting`
- `HasSettings.get_path_settings`
- `HasSettings.get_setting`
- `HasSettings.get_settings`
- `HasSettings.has_path_setting`
- `HasSettings.has_path_settings`
- `HasSettings.has_setting`
- `HasSettings.has_settings`
- `HasSettings.reset_settings`

- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

Subclasses

- [FDOFPreparer](#)

`call_post_segment` cached_property ⓘ

Argument `call_post_segment`.

`call_pre_segment` cached_property ⓘ

Argument `call_pre_segment`.

`ffill_val_price` cached_property ⓘ

Argument `ffill_val_price`.

`fill_pos_info` cached_property ⓘ

Argument `fill_pos_info`.

`flex_order_args` cached_property ⓘ

Argument `flex_order_args`.

`flex_order_func_nb` cached_property ⓘ

Argument `flex_order_func_nb`.

`flexible` cached_property ⓘ

Whether the flexible mode is enabled.

`max_log_records` cached_property ⓘ

Argument `max_log_records`.

`max_order_records` cached_property ⓘ

Argument `max_order_records`.

order_args cached_property 

Argument `order_args`.

order_func_nb cached_property 

Argument `order_func_nb`.

post_group_args cached_property 

Argument `post_group_args`.

post_group_func_nb cached_property 

Argument `post_group_func_nb`.

post_order_args cached_property 

Argument `post_order_args`.

post_order_func_nb cached_property 

Argument `post_order_func_nb`.

post_row_args cached_property 

Argument `post_row_args`.

post_row_func_nb cached_property 

Argument `post_row_func_nb`.

post_segment_args cached_property 

Argument `post_segment_args`.

post_segment_func_nb cached_property 

Argument `post_segment_func_nb`.

post_sim_args cached_property 

Argument `post_sim_args`.

post_sim_func_nb cached_property 

Argument `post_sim_func_nb`.

pre_group_args cached_property 

Argument `pre_group_args`.

pre_group_func_nb cached_property 

Argument `pre_group_func_nb`.

pre_row_args cached_property ⓘ

Argument pre_row_args.

pre_row_func_nb cached_property ⓘ

Argument pre_row_func_nb.

pre_segment_args cached_property ⓘ

Argument pre_segment_args.

pre_segment_func_nb cached_property ⓘ

Argument pre_segment_func_nb.

pre_sim_args cached_property ⓘ

Argument pre_sim_args.

pre_sim_func_nb cached_property ⓘ

Argument pre_sim_func_nb.

row_wise cached_property ⓘ

Argument row_wise.

segment_mask cached_property ⓘ

Argument segment_mask.

track_value cached_property ⓘ

Argument track_value.

update_value cached_property ⓘ

Argument update_value.

FOPreparer class ⓘ

```
FOPreparer(
    arg_config=None,
    **kwargs
)
```

Class for preparing [Portfolio.from_orders](#).

Superclasses

- [Base](#)
- [BasePFPreparer](#)
- [BasePreparer](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [HasSettings](#)

- [Pickleable](#)
- [Prettified](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [BasePFPreparer.align_pc_arr](#)
- [BasePFPreparer.args_to_broadcast](#)
- [BasePFPreparer.attach_call_seq](#)
- [BasePFPreparer.auto_call_seq](#)
- [BasePFPreparer.auto_sim_end](#)
- [BasePFPreparer.auto_sim_start](#)
- [BasePFPreparer.bm_close](#)
- [BasePFPreparer.broadcast_kwargs](#)
- [BasePFPreparer.broadcast_named_args](#)
- [BasePFPreparer.broadcast_result](#)
- [BasePFPreparer.call_seq](#)
- [BasePFPreparer.cash_deposits](#)
- [BasePFPreparer.cash_earnings](#)
- [BasePFPreparer.cash_sharing](#)
- [BasePFPreparer.chunked](#)
- [BasePFPreparer.close](#)
- [BasePFPreparer.config](#)
- [BasePFPreparer.cs_group_lens](#)
- [BasePFPreparer.data](#)
- [BasePFPreparer.def_broadcast_kwargs](#)
- [BasePFPreparer.find_target_func](#)
- [BasePFPreparer.freq](#)
- [BasePFPreparer.group_by](#)
- [BasePFPreparer.group_lens](#)
- [BasePFPreparer.high](#)
- [BasePFPreparer.idx_setters](#)
- [BasePFPreparer.in_outputs](#)
- [BasePFPreparer.index](#)
- [BasePFPreparer.init_cash](#)
- [BasePFPreparer.init_cash_mode](#)
- [BasePFPreparer.init_position](#)
- [BasePFPreparer.init_price](#)
- [BasePFPreparer.jitted](#)
- [BasePFPreparer.keep_inout_flex](#)
- [BasePFPreparer.low](#)
- [BasePFPreparer.open](#)
- [BasePFPreparer.parse_data](#)
- [BasePFPreparer.pf_args](#)
- [BasePFPreparer.post_args](#)
- [BasePFPreparer.post_broadcast_named_args](#)
- [BasePFPreparer.pre_args](#)
- [BasePFPreparer.rec_state](#)
- [BasePFPreparer.records](#)
- [BasePFPreparer.result](#)
- [BasePFPreparer.seed](#)

- `BasePFPreparer.sim_end`
- `BasePFPreparer.sim_group_lens`
- `BasePFPreparer.sim_start`
- `BasePFPreparer.staticized`
- `BasePFPreparer.target_arg_map`
- `BasePFPreparer.target_args`
- `BasePFPreparer.target_func`
- `BasePFPreparer.target_shape`
- `BasePFPreparer.wrapper`
- `BasePreparer.adapt_staticized_to_udf`
- `BasePreparer.build_arg_config_doc`
- `BasePreparer.dt_arr_to_ns`
- `BasePreparer.get_arg`
- `BasePreparer.get_arg_default`
- `BasePreparer.get_raw_arg`
- `BasePreparer.get_raw_arg_default`
- `BasePreparer.map_enum_value`
- `BasePreparer.override_arg_config_doc`
- `BasePreparer.prepare_dt_arr`
- `BasePreparer.prepare_dt_obj`
- `BasePreparer.prepare_post_arg`
- `BasePreparer.prepare_td_arr`
- `BasePreparer.prepare_td_obj`
- `BasePreparer.resolve_dynamic_target_func`
- `BasePreparer.set_seed`
- `BasePreparer.td_arr_to_ns`
- `BasePreparer.template_context`
- `Cacheable.get_ca_setup`
- `Chainable.chain`
- `Chainable.pipe`
- `Configured.copy`
- `Configured.equals`
- `Configured.get_writeable_attrs`
- `Configured.prettify`
- `Configured.replace`
- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `HasSettings.get_path_setting`
- `HasSettings.get_path_settings`
- `HasSettings.get_setting`
- `HasSettings.get_settings`
- `HasSettings.has_path_setting`
- `HasSettings.has_path_settings`
- `HasSettings.has_setting`
- `HasSettings.has_settings`
- `HasSettings.reset_settings`
- `HasSettings.resolve_setting`
- `HasSettings.resolve_settings_paths`
- `HasSettings.set_settings`
- `Pickleable.decode_config`
- `Pickleable.decode_config_node`
- `Pickleable.dumps`
- `Pickleable.encode_config`
- `Pickleable.encode_config_node`

- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

allow_partial cached_property ⓘ

Argument `allow_partial`.

cash_dividends cached_property ⓘ

Argument `cash_dividends`.

direction cached_property ⓘ

Argument `direction`.

fees cached_property ⓘ

Argument `fees`.

ffill_val_price cached_property ⓘ

Argument `ffill_val_price`.

fixed_fees cached_property ⓘ

Argument `fixed_fees`.

from_ago cached_property ⓘ

Argument `from_ago`.

leverage cached_property ⓘ

Argument `leverage`.

leverage_mode cached_property ⓘ

Argument `leverage_mode`.

log cached_property ⓘ

Argument `log`.

max_log_records cached_property ⓘ

Argument `max_log_records`.

max_order_records cached_property ⓘ

Argument `max_order_records`.

max_size cached_property ⓘ Argument max_size.
min_size cached_property ⓘ Argument min_size.
price cached_property ⓘ Argument price.
price_and_from_ago cached_property ⓘ Arguments price and from_ago after broadcasting.
price_area_vio_mode cached_property ⓘ Argument price_area_vio_mode.
raise_reject cached_property ⓘ Argument raise_reject.
reject_prob cached_property ⓘ Argument reject_prob.
save_returns cached_property ⓘ Argument save_returns.
save_state cached_property ⓘ Argument save_state.
save_value cached_property ⓘ Argument save_value.
size cached_property ⓘ Argument size.
size_granularity cached_property ⓘ Argument size_granularity.
size_type cached_property ⓘ Argument size_type.
skip_empty cached_property ⓘ Argument skip_empty.

slippage cached_property 

Argument `slippage`.

update_value cached_property 

Argument `update_value`.

val_price cached_property 

Argument `val_price`.

FSPreparer class 

```
FSPreparer(
    arg_config=None,
    **kwargs
)
```

Class for preparing [Portfolio.from_signals](#).

Superclasses

- [Base](#)
- [BasePFPreparer](#)
- [BasePreparer](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [HasSettings](#)
- [Pickleable](#)
- [Prettified](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [BasePFPreparer.align_pc_arr](#)
- [BasePFPreparer.args_to_broadcast](#)
- [BasePFPreparer.attach_call_seq](#)
- [BasePFPreparer.auto_call_seq](#)
- [BasePFPreparer.auto_sim_end](#)
- [BasePFPreparer.auto_sim_start](#)
- [BasePFPreparer.bm_close](#)
- [BasePFPreparer.broadcast_kwargs](#)
- [BasePFPreparer.broadcast_named_args](#)
- [BasePFPreparer.broadcast_result](#)
- [BasePFPreparer.call_seq](#)
- [BasePFPreparer.cash_deposits](#)
- [BasePFPreparer.cash_earnings](#)
- [BasePFPreparer.cash_sharing](#)
- [BasePFPreparer.chunked](#)
- [BasePFPreparer.close](#)
- [BasePFPreparer.config](#)

- `BasePFPreparer.cs_group_lens`
- `BasePFPreparer.data`
- `BasePFPreparer.def_broadcast_kwargs`
- `BasePFPreparer.find_target_func`
- `BasePFPreparer.freq`
- `BasePFPreparer.group_by`
- `BasePFPreparer.group_lens`
- `BasePFPreparer.high`
- `BasePFPreparer.idx_setters`
- `BasePFPreparer.in_outputs`
- `BasePFPreparer.index`
- `BasePFPreparer.init_cash`
- `BasePFPreparer.init_cash_mode`
- `BasePFPreparer.init_position`
- `BasePFPreparer.init_price`
- `BasePFPreparer.jitted`
- `BasePFPreparer.keep_inout_flex`
- `BasePFPreparer.low`
- `BasePFPreparer.open`
- `BasePFPreparer.parse_data`
- `BasePFPreparer.pf_args`
- `BasePFPreparer.post_args`
- `BasePFPreparer.post_broadcast_named_args`
- `BasePFPreparer.pre_args`
- `BasePFPreparer.rec_state`
- `BasePFPreparer.records`
- `BasePFPreparer.result`
- `BasePFPreparer.seed`
- `BasePFPreparer.sim_end`
- `BasePFPreparer.sim_group_lens`
- `BasePFPreparer.sim_start`
- `BasePFPreparer.staticized`
- `BasePFPreparer.target_arg_map`
- `BasePFPreparer.target_args`
- `BasePFPreparer.target_func`
- `BasePFPreparer.target_shape`
- `BasePFPreparer.wrapper`
- `BasePreparer.adapt_staticized_to_udf`
- `BasePreparer.build_arg_config_doc`
- `BasePreparer.dt_arr_to_ns`
- `BasePreparer.get_arg`
- `BasePreparer.get_arg_default`
- `BasePreparer.get_raw_arg`
- `BasePreparer.get_raw_arg_default`
- `BasePreparer.map_enum_value`
- `BasePreparer.override_arg_config_doc`
- `BasePreparer.prepare_dt_arr`
- `BasePreparer.prepare_dt_obj`
- `BasePreparer.prepare_post_arg`
- `BasePreparer.prepare_td_arr`
- `BasePreparer.prepare_td_obj`
- `BasePreparer.resolve_dynamic_target_func`
- `BasePreparer.set_seed`
- `BasePreparer.td_arr_to_ns`

- [BasePreparer.template_context](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)
- [Configured.replace](#)
- [Configured.resolve_merge_kwargs](#)
- [Configured.update_config](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)
- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

accumulate cached_property 

Argument `accumulate`.

adjust_args cached_property 

Argument `adjust_args`.

adjust_func_nb cached_property 

Argument `adjust_func_nb`.

allow_partial cached_property 

Argument `allow_partial`.

basic_mode cached_property 

Whether the basic mode is enabled.

cash_dividends cached_property

Argument `cash_dividends`.

combined_mask cached_property

Signals combined using the OR rule into a mask.

delta_format cached_property

Argument `delta_format`.

direction cached_property

Argument `direction`.

dt_stop cached_property

Argument `dt_stop`.

dynamic_mode cached_property

Whether the dynamic mode is enabled.

entries cached_property

Argument `entries`.

exits cached_property

Argument `exits`.

explicit_mode cached_property

Whether the explicit mode is enabled.

fees cached_property

Argument `fees`.

ffill_val_price cached_property

Argument `ffill_val_price`.

fill_pos_info cached_property

Argument `fill_pos_info`.

fixed_fees cached_property

Argument `fixed_fees`.

from_ago cached_property

Argument `from_ago`.

implicit_mode cached_property 

Whether the explicit mode is enabled.

init_in_outputs class method 

```
FSPreparer.init_in_outputs(
    wrapper,
    group_lens=None,
    cash_sharing=False,
    save_state=True,
    save_value=True,
    save_returns=True
)
```

Initialize **F\$InOutputs**.

leverage cached_property 

Argument `leverage`.

leverage_mode cached_property 

Argument `leverage_mode`.

limit_delta cached_property 

Argument `limit_delta`.

limit_expiry cached_property 

Argument `limit_expiry`.

limit_order_price cached_property 

Argument `limit_order_price`.

limit_reverse cached_property 

Argument `limit_reverse`.

limit_tif cached_property 

Argument `limit_tif`.

log cached_property 

Argument `log`.

long_entries cached_property 

Argument `long_entries`.

long_exits cached_property 

Argument `long_exits`.

`ls_mode` cached_property ⓘ

Whether direction-aware mode is enabled.

`max_log_records` cached_property ⓘ

Argument `max_log_records`.

`max_order_records` cached_property ⓘ

Argument `max_order_records`.

`max_size` cached_property ⓘ

Argument `max_size`.

`min_size` cached_property ⓘ

Argument `min_size`.

`order_mode` cached_property ⓘ

Argument `order_mode`.

`order_type` cached_property ⓘ

Argument `order_type`.

`post_segment_args` cached_property ⓘ

Argument `post_segment_args`.

`post_segment_func_nb` cached_property ⓘ

Argument `post_segment_func_nb`.

`post_signal_args` cached_property ⓘ

Argument `post_signal_args`.

`post_signal_func_nb` cached_property ⓘ

Argument `post_signal_func_nb`.

`price` cached_property ⓘ

Argument `price`.

`price_and_from_ago` cached_property ⓘ

Arguments `price` and `from_ago` after broadcasting.

`price_area_vio_mode` cached_property ⓘ

Argument `price_area_vio_mode`.

raise_reject cached_property ⓘ

Argument `raise_reject`.

reject_prob cached_property ⓘ

Argument `reject_prob`.

save_returns cached_property ⓘ

Argument `save_returns`.

save_state cached_property ⓘ

Argument `save_state`.

save_value cached_property ⓘ

Argument `save_value`.

short_entries cached_property ⓘ

Argument `short_entries`.

short_exits cached_property ⓘ

Argument `short_exits`.

signal_args cached_property ⓘ

Argument `signal_args`.

signal_func_mode cached_property ⓘ

Whether signal function mode is enabled.

signal_func_nb cached_property ⓘ

Argument `signal_func_nb`.

signals cached_property ⓘ

Arguments `entries`, `exits`, `short_entries`, and `short_exits` after broadcasting.

signals_mode cached_property ⓘ

Whether signals mode is enabled.

size cached_property ⓘ

Argument `size`.

size_granularity cached_property ⓘ

Argument `size_granularity`.

size_type cached_property ⓘ

Argument `size_type`.

skip_empty cached_property ⓘ

Argument `skip_empty`.

sl_stop cached_property ⓘ

Argument `sl_stop`.

slippage cached_property ⓘ

Argument `slippage`.

stop_entry_price cached_property ⓘ

Argument `stop_entry_price`.

stop_exit_price cached_property ⓘ

Argument `stop_exit_price`.

stop_exit_type cached_property ⓘ

Argument `stop_exit_type`.

stop_ladder cached_property ⓘ

Argument `stop_ladder`.

stop_limit_delta cached_property ⓘ

Argument `stop_limit_delta`.

stop_order_type cached_property ⓘ

Argument `stop_order_type`.

td_stop cached_property ⓘ

Argument `td_stop`.

time_delta_format cached_property ⓘ

Argument `time_delta_format`.

tp_stop cached_property ⓘ

Argument `tp_stop`.

tsl_stop cached_property ⓘ

Argument `tsl_stop`.

tsl_th cached_property ⓘ

Argument `tsl_th`.

update_value cached_property ⓘ

Argument `update_value`.

upon_adj_limit_conflict cached_property ⓘ

Argument `upon_adj_limit_conflict`.

upon_adj_stop_conflict cached_property ⓘ

Argument `upon_adj_stop_conflict`.

upon_dir_conflict cached_property ⓘ

Argument `upon_dir_conflict`.

upon_long_conflict cached_property ⓘ

Argument `upon_long_conflict`.

upon_opp_limit_conflict cached_property ⓘ

Argument `upon_opp_limit_conflict`.

upon_opp_stop_conflict cached_property ⓘ

Argument `upon_opp_stop_conflict`.

upon_opposite_entry cached_property ⓘ

Argument `upon_opposite_entry`.

upon_short_conflict cached_property ⓘ

Argument `upon_short_conflict`.

upon_stop_update cached_property ⓘ

Argument `upon_stop_update`.

use_limit_orders cached_property ⓘ

Whether to use limit orders.

use_stops cached_property ⓘ

Argument `use_stops`.

val_price cached_property ⓘ

Argument `val_price`.

PFPrepResult class

```
PFPrepResult(  
    target_func=None,  
    target_args=None,  
    pf_args=None  
)
```

Result of preparation.

Superclasses

- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [HasSettings](#)
- [Pickleable](#)
- [Prettified](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.config](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)
- [Configured.rec_state](#)
- [Configured.replace](#)
- [Configured.resolve_merge_kwargs](#)
- [Configured.update_config](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)
- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)

- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

pf_args cached_property 

Portfolio arguments.

target_args cached_property 

Target arguments.

target_func cached_property 

Target function.

settings

 _settings module

Global settings of vectorbtpro.

settings config is also accessible via `vectorbtpro.settings`.

 Note

All places in vectorbt import **settings**, not `vectorbtpro.settings`. Overwriting `vectorbtpro.settings` only overwrites the reference created for the user. Consider updating the settings config instead of replacing it.

Here are the main properties of the **settings** config:

- It's a nested config, that is, a config that consists of multiple sub-configs. one per sub-package (e.g., 'data'), module (e.g., 'wrapping'), or even class (e.g., 'configured'). Each sub-config may consist of other sub-configs.
- It has frozen keys - you cannot add other sub-configs or remove the existing ones, but you can modify them.
- Each sub-config can be **frozen_cfg** or **flex_cfg**. The main reason for defining a flexible config is to allow adding new keys (e.g., 'plotting.layout').

For example, you can change default width and height of each plot:

```
>>> from vectorbtpro import *

>>> vbt.settings['plotting']['layout']['width'] = 800
>>> vbt.settings['plotting']['layout']['height'] = 400
```

The main sub-configs such as for plotting can be also accessed/modified using the dot notation:

```
>>> vbt.settings.plotting['layout']['width'] = 800
```

Some sub-configs allow the dot notation too but this depends on whether they are an instance of **frozen_cfg**:

```
>>> type(vbt.settings)
vectorbtpro._settings.frozen_cfg
>>> vbt.settings.data # ok

>>> type(vbt.settings.data)
vectorbtpro._settings.frozen_cfg
>>> vbt.settings.data.silence_warnings # ok

>>> type(vbt.settings.data.custom)
vectorbtpro._settings.flex_cfg
>>> vbt.settings.data.custom.binance # error
>>> vbt.settings.data.custom["binance"] # ok
```

Since this is only visible when looking at the source code, the advice is to always use the bracket notation.

 Note

Whether the change takes effect immediately depends upon the place that accesses the settings. For example, changing 'wrapping.freq' has an immediate effect because the value is resolved every time **ArrayWrapper.freq** is called. On the other hand, changing 'portfolio.fillna_close' has only effect on **Portfolio** instances created in the future, not the existing ones, because the value is resolved upon the object's construction. Moreover, some settings are only accessed when importing the package for the first time, such as 'jitting.jit_decorator'. In any case, make sure to check whether the update actually took place.

Saving and loading

Like any other class subclassing **Config**, we can persist settings to the disk, load it back, and replace in-place. There are several ways of how to update the settings.

Binary file

Pickling will dump the entire settings object into a byte stream and save as a binary file. Supported file extensions are "pickle" (default) and "pkl".

```
>>> vbt.settings.save('my_settings')
>>> vbt.settings['caching']['disable'] = True
>>> vbt.settings['caching']['disable']
True

>>> vbt.settings.load_update('my_settings', clear=True) # replace in-place
>>> vbt.settings['caching']['disable']
False
```

Note

Argument `clear=True` will replace the entire settings object. Disable it to apply only a subset of settings (default).

Config file

We can also encode the settings object into a config and save as a text file that can be edited easily. Supported file extensions are "config" (default), "cfg", and "ini".

```
>>> vbt.settings.save('my_settings', file_format="config")
>>> vbt.settings['caching']['disable'] = True
>>> vbt.settings['caching']['disable']
True

>>> vbt.settings.load_update('my_settings', file_format="config", clear=True) # replace in-place
>>> vbt.settings['caching']['disable']
False
```

On import

Some settings (such as Numba-related ones) are applied only on import, so changing them during the runtime will have no effect. In this case, change the settings, save them to the disk, and then either rename the file to "vbt" (with extension) and place it in the working directory for it to be recognized automatically, or create an environment variable "VBT_SETTINGS_PATH" that holds the full path to the file - vectorbt will load it before any other module. You can also change the recognized file name using an environment variable "VBT_SETTINGS_NAME", which defaults to "vbt".

Note

Environment variables must be set before importing vectorbtpro.

For example, to set the default theme to dark, create the following "vbt.ini" file:

```
[plotting]
default_theme = dark
```

broadcasting frozen_cfg

Sub-config with settings applied to broadcasting functions across [vectorbtpro.base](#).

```
frozen_cfg(
    align_index=True,
    align_columns=True,
    index_from='strict',
    columns_from='stack',
    ignore_sr_names=True,
    check_index_names=True,
    drop_duplicates=True,
    keep='last',
    drop_redundant=True,
    ignore_ranges=True,
    keep_wrap_default=False,
    keep_flex=False,
    min_ndim=None,
    expand_axis=1,
    index_to_param=True
)
```

caching frozen_cfg

Sub-config with settings applied across [vectorbtpro.registries.ca_registry](#), [vectorbtpro.utils.caching](#), and cacheable decorators in [vectorbtpro.utils.decorators](#).

Hint

Apply setting `register_lazily` on startup to register all unbound cacheables.

Setting `use_cached_accessors` is applied only on import.

```
frozen_cfg(
    disable=False,
    disable_whitelist=False,
    disable_machinery=False,
    silence_warnings=False,
    register_lazily=True,
    ignore_args=[
        'jitted',
        'chunked'
    ],
    use_cached_accessors=True
)
```

chunking frozen_cfg

Sub-config with settings applied across [vectorbtpro.registries.ch_registry](#) and [vectorbtpro.utils.chunking](#).

Note

Options (with `_options` suffix) and setting `disable_wrapping` are applied only on import.

```
frozen_cfg(  
    disable=False,  
    disable_wrapping=False,  
    option=False,  
    chunker=None,  
    size=None,  
    min_size=None,  
    n_chunks=None,  
    chunk_len=None,  
    chunk_meta=None,  
    prepend_chunk_meta=None,  
    skip_single_chunk=True,  
    arg_take_spec=None,  
    template_context=flex_cfg(),  
    merge_func=None,  
    merge_kwargs=flex_cfg(),  
    return_raw_chunks=False,  
    silence_warnings=False,  
    forward_kwargs_as=flex_cfg(),  
    execute_kwargs=flex_cfg(),  
    replace_chunker=False,  
    merge_to_execute_kwargs=True,  
    options=flex_cfg(),  
    override_setup_options=flex_cfg(),  
    override_options=flex_cfg()  
)
```

config frozen_cfg

Sub-config with settings applied to [Config](#).

```
frozen_cfg(  
    options=flex_cfg()  
)
```

configured frozen_cfg

Sub-config with settings applied to [Configured](#).

```
frozen_cfg(  
    check_expected_keys=True,  
    config=frozen_cfg(  
        options=flex_cfg(  
            readonly=True,  
            nested=False  
        )  
    )  
)
```

data frozen_cfg

Sub-config with settings applied across [vectorbtpro.data](#).

```
frozen_cfg(  
    keys_are_features=False,  
    wrapper_kwargs=flex_cfg(),  
    skip_on_error=False,  
    silence_warnings=False,  
    execute_kwargs=flex_cfg(),  
    tz_localize='utc',  
    tz_convert='utc',  
    missing_index='nan',  
    missing_columns='raise',  
    custom=flex_cfg(  
        synthetic=flex_cfg(  
            start=None,  
            end=None,  
            timeframe=None,  
            tz=None,  
            normalize=False,  
        )  
    )  
)
```

```

        inclusive='left'
    ),
    random=flex_cfg(
        start_value=100.0,
        mean=0.0,
        std=0.01,
        symmetric=False,
        seed=None
    ),
    random_ohlc=flex_cfg(
        n_ticks=50,
        start_value=100.0,
        mean=0.0,
        std=0.001,
        symmetric=False,
        seed=None
    ),
    gbm=flex_cfg(
        start_value=100.0,
        mean=0.0,
        std=0.01,
        dt=1.0,
        seed=None
    ),
    gbm_ohlc=flex_cfg(
        n_ticks=50,
        start_value=100.0,
        mean=0.0,
        std=0.001,
        dt=1.0,
        seed=None
    ),
    local=flex_cfg(),
    file=flex_cfg(
        match_paths=True,
        match_regex=None,
        sort_paths=True
    ),
    csv=flex_cfg(
        start=None,
        end=None,
        tz=None,
        start_row=None,
        end_row=None,
        header=0,
        index_col=0,
        parse_dates=True,
        chunk_func=None,
        squeeze=True,
        read_kwargs=flex_cfg()
    ),
    hdf=flex_cfg(
        start=None,
        end=None,
        tz=None,
        start_row=None,
        end_row=None,
        read_kwargs=flex_cfg()
    ),
    feather=flex_cfg(
        tz=None,
        index_col=0,
        squeeze=True,
        read_kwargs=flex_cfg()
    ),
    parquet=flex_cfg(
        tz=None,
        squeeze=True,
        keep_partition_cols=None,
        engine='auto',
        read_kwargs=flex_cfg()
    ),
    db=flex_cfg(),
    sql=flex_cfg(
        engine=None,
        engine_name=None,
        engine_config=flex_cfg(),
        schema=None,
        start=None,
        end=None,
        align_dates=True,
        parse_dates=True,
        to_utc=True,
        tz=None,
        start_row=None,
        end_row=None,
        keep_row_number=True,
        row_number_column='row_number',
        index_col=0,
        columns=None,
        dtype=None,
        chunksize=None,
        chunk_func=None,
        squeeze=True,
        read_sql_kwargs=flex_cfg(),
        engines=flex_cfg()
    ),
    duckdb=flex_cfg(

```

```

        connection=None,
        connection_config=flex_cfg(),
        schema=None,
        catalog=None,
        start=None,
        end=None,
        align_dates=True,
        parse_dates=True,
        to_utc=True,
        tz=None,
        index_col=0,
        squeeze=True,
        df_kwargs=flex_cfg(),
        sql_kwargs=flex_cfg()
    ),
    remote=flex_cfg(),
    yf=flex_cfg(
        period='max',
        start=None,
        end=None,
        timeframe='1d',
        tz=None,
        history_kwargs=flex_cfg()
    ),
    binance=flex_cfg(
        client=None,
        client_config=flex_cfg(
            api_key=None,
            api_secret=None
        ),
        start=0,
        end='now',
        timeframe='1d',
        tz='utc',
        klines_type='spot',
        limit=1000,
        delay=0.5,
        show_progress=True,
        pbar_kwargs=flex_cfg(),
        silence_warnings=False,
        get_klines_kwargs=flex_cfg()
    ),
    ccxt=flex_cfg(
        exchange=None,
        exchange_config=flex_cfg(
            enableRateLimit=True
        ),
        start=None,
        end=None,
        timeframe='1d',
        tz='utc',
        find_earliest_date=False,
        limit=1000,
        delay=None,
        retries=3,
        fetch_params=flex_cfg(),
        show_progress=True,
        pbar_kwargs=flex_cfg(),
        silence_warnings=False,
        exchanges=flex_cfg()
    ),
    alpaca=flex_cfg(
        client=None,
        client_type='stocks',
        client_config=flex_cfg(
            api_key=None,
            secret_key=None,
            oauth_token=None,
            paper=False
        ),
        start=0,
        end='now',
        timeframe='1d',
        tz='utc',
        adjustment='raw',
        feed=None,
        limit=None
    ),
    polygon=flex_cfg(
        client=None,
        client_config=flex_cfg(
            api_key=None
        ),
        start=0,
        end='now',
        timeframe='1d',
        tz='utc',
        adjusted=True,
        limit=50000,
        params=flex_cfg(),
        delay=0.5,
        retries=3,
        show_progress=True,
        pbar_kwargs=flex_cfg(),
        silence_warnings=False
    ),
    av=flex_cfg(
        use_parser=None,

```



```

        apikey=None,
        api_meta=None,
        category=None,
        function=None,
        timeframe=None,
        tz=None,
        adjusted=False,
        extended=False,
        slice='year1month1',
        series_type='close',
        time_period=10,
        outputsize='full',
        read_csv_kwargs=flex_cfg(
            index_col=0,
            parse_dates=True
        ),
        match_params=True,
        params=flex_cfg(),
        silence_warnings=False
    ),
    ndl=flex_cfg(
        api_key=None,
        data_format='dataset',
        start=None,
        end=None,
        tz='utc',
        column_indices=None,
        params=flex_cfg()
    ),
    tv=flex_cfg(
        client=None,
        client_config=flex_cfg(
            username=None,
            password=None,
            auth_token=None
        ),
        exchange=None,
        timeframe='D',
        tz='utc',
        fut_contract=None,
        adjustment='splits',
        extended_session=False,
        pro_data=True,
        limit=20000,
        delay=0.5,
        retries=3,
        search=flex_cfg(
            pages=None,
            delay=0.5,
            retries=3,
            show_progress=True,
            pbar_kwargs=flex_cfg()
        ),
        scanner=flex_cfg(
            markets=None,
            fields=None,
            filter_by=None,
            groups=None,
            template_context=flex_cfg(),
            scanner_kwargs=flex_cfg()
        )
    ),
    bento=flex_cfg(
        client=None,
        client_config=flex_cfg(
            key=None
        ),
        start=None,
        end=None,
        resolve_dates=True,
        timeframe=None,
        tz='utc',
        dataset=None,
        schema=None,
        df_kwargs=flex_cfg(),
        params=flex_cfg()
    ),
    finpy=flex_cfg(
        market=None,
        market_config=flex_cfg(),
        config_manager=None,
        config_manager_config=flex_cfg(),
        start='one year ago',
        end='now',
        timeframe='daily',
        tz='utc',
        request_kwargs=flex_cfg()
    ),
    stats=flex_cfg(
        filters=flex_cfg(
            is_feature_oriented=flex_cfg(
                filter_func=<function <lambda> at 0x118dcc720>
            ),
            is_symbol_oriented=flex_cfg(
                filter_func=<function <lambda> at 0x118dcc7c0>
            )
        )
    )
)

```

```
),
plots=flex_cfg()
)
```

Binance

See `binance.client.Client`.

CCXT

See [Configuring API Keys](#). Keys can be defined per exchange. If a key is defined at the root, it applies to all exchanges.

Alpaca

Sign up for Alpaca API keys under <https://app.alpaca.markets/signup>.

datetime frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.datetime_](#).

```
frozen_cfg(
    naive_tz='tzlocal()',
    to_fixed_offset=None,
    parse_with_dateparser=True,
    index=frozen_cfg(
        parse_index=True,
        parse_with_dateparser=False
    ),
    dateparser_kwargs=flex_cfg(),
    freq_from_n=20,
    tz_naive_ns=True,
    readable=frozen_cfg(
        drop_tz=True
    )
)
```

drawdowns frozen_cfg

Sub-config with settings applied to [Drawdowns](#).

```
frozen_cfg(
    stats=flex_cfg(
        settings=flex_cfg(
            incl_active=False
        )
    ),
    plots=flex_cfg()
)
```

execution frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.execution](#).

```
frozen_cfg(
    executor=None,
    engine='SerialEngine',
    engine_config=flex_cfg(),
    min_size=None,
    n_chunks=None,
    chunk_len=None,
    chunk_meta=None,
    distribute='tasks',
    warmup=False,
    in_chunk_order=False,
    cache_chunks=False,
    chunk_cache_dir=None,
    chunk_cache_save_kwargs=flex_cfg(
        mkdir_kwargs=dict(
            mkdir=True
        )
    ),
    chunk_cache_load_kwargs=flex_cfg(),
    pre_clear_chunk_cache=False,
    post_clear_chunk_cache=True,
    release_chunk_cache=False,
    chunk_clear_cache=False,
    chunk_collect_garbage=False,
    chunk_delay=None,
    pre_execute_func=None,
    pre_execute_kwargs=flex_cfg(),
    pre_chunk_func=None,
    pre_chunk_kwargs=flex_cfg(),
)
```

```

post_chunk_func=None,
post_chunk_kwargs=flex_cfg(),
post_execute_func=None,
post_execute_kwargs=flex_cfg(),
post_execute_on_sorted=False,
filter_results=False,
raise_no_results=True,
merge_func=None,
merge_kwargs=flex_cfg(),
template_context=flex_cfg(),
show_progress=True,
pbar_kwargs=flex_cfg(),
replace_executor=False,
merge_to_engine_config=True,
engines=flex_cfg(
    serial=flex_cfg(
        cls='SerialEngine',
        show_progress=True,
        pbar_kwargs=flex_cfg(),
        clear_cache=False,
        collect_garbage=False,
        delay=None
    ),
    threadpool=flex_cfg(
        cls='ThreadPoolEngine',
        init_kwargs=flex_cfg(),
        timeout=None
    ),
    processpool=flex_cfg(
        cls='ProcessPoolEngine',
        init_kwargs=flex_cfg(),
        timeout=None
    ),
    pathos=flex_cfg(
        cls='PathosEngine',
        pool_type='process',
        init_kwargs=flex_cfg(),
        timeout=None,
        check_delay=0.001,
        show_progress=False,
        pbar_kwargs=flex_cfg(),
        join_pool=False
    ),
    mpire=flex_cfg(
        cls='MpireEngine',
        init_kwargs=flex_cfg(
            use_dill=True
        ),
        apply_kwargs=flex_cfg()
    ),
    dask=flex_cfg(
        cls='DaskEngine',
        compute_kwargs=flex_cfg()
    ),
    ray=flex_cfg(
        cls='RayEngine',
        restart=False,
        reuse_refs=True,
        del_refs=True,
        shutdown=False,
        init_kwargs=flex_cfg(),
        remote_kwargs=flex_cfg()
    )
)
)
)

```

generic frozen_cfg 

Sub-config with settings applied to [GenericAccessor](#).

```

frozen_cfg(
    use_jitted=False,
    stats=flex_cfg(
        filters=flex_cfg(
            has_mapping=flex_cfg(
                filter_func=<function <lambda> at 0x118af8680>
            )
        ),
        settings=flex_cfg(
            incl_all_keys=False
        )
    ),
    plots=flex_cfg()
)

```

importing frozen_cfg 

Sub-config with settings applied on importing.

Disabling these options will make vectorbt load faster, but will limit the flexibility of accessing various features of the package.

Note

If `auto_import` is False, you won't be able to access most important modules and objects such as via `vbt.Portfolio`, only by explicitly importing them such as via `from vectorbtpro.portfolio.base import Portfolio`.

```
frozen_cfg(
    clear_pycache=False,
    auto_import=True,
    star_import='minimal',
    plotly=True,
    telegram=True,
    quantstats=True,
    sklearn=True
)
```

indexing frozen_cfg

Sub-config with settings applied to indexing functions across [vectorbtpro.base](#).

Note

Options `rotate_rows` and `rotate_cols` are applied only on import.

```
frozen_cfg(
    rotate_rows=False,
    rotate_cols=False
)
```

jitting frozen_cfg

Sub-config with settings applied across [vectorbtpro.registries.jit_registry](#) and [vectorbtpro.utils.jitting](#).

Note

Options (with `_options` suffix) are applied only on import.

Keyword arguments (with `_kwargs` suffix) are applied right away.

```
frozen_cfg(
    disable=False,
    disable_wrapping=False,
    disable_resolution=False,
    option=True,
    allow_new=False,
    register_new=False,
    jitters=flex_cfg(
        nb=frozen_cfg(
            cls='NumbaJitter',
            aliases={
                'numba'
            },
            options=flex_cfg(),
            override_options=flex_cfg(),
            resolve_kwargs=flex_cfg(),
            tasks=flex_cfg()
        ),
        np=frozen_cfg(
            cls='NumPyJitter',
            aliases={
                'numpy'
            },
            options=flex_cfg(),
            override_options=flex_cfg(),
            resolve_kwargs=flex_cfg(),
            tasks=flex_cfg()
        )
    ),
    template_context=flex_cfg()
)
```

knowledge frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.knowledge](#).

```
frozen_cfg(
    cache=True,
```

```

cache_dir='./knowledge',
cache_mkdir_kwargs=dict(
    mkdir=True
),
per_path=True,
find_all=False,
keep_path=False,
skip_missing=False,
make_copy=True,
query_engine=None,
return_type='item',
return_path=False,
merge_matches=True,
merge_fields=True,
unique_matches=True,
unique_fields=True,
changed_only=False,
code=flex_cfg(
    require_language=False,
    in_blocks=True
),
dump_all=False,
dump_engine='yaml',
dump_engine_kwargs=flex_cfg(
    nestedtext=flex_cfg(
        indent=2
    ),
    pyyaml=flex_cfg(
        sort_keys=False,
        default_flow_style=False,
        allow_unicode=True
    ),
    ruamel=flex_cfg(
        default_flow_style=False,
        allow_unicode=True,
        width=4096,
        preserve_quotes=True,
        indent=dict(
            mapping=2,
            sequence=4,
            offset=2
        )
    ),
    json=flex_cfg(
        ensure_ascii=False,
        indent=4
    )
),
in_dumps=False,
dump_kwargs=flex_cfg(),
sort_keys=False,
ignore_empty=True,
describe_kwargs=flex_cfg(
    percentiles=[]
),
uniform_groups=False,
prepend_index=False,
template_context=flex_cfg(),
silence_warnings=False,
show_progress=None,
pbar_kwargs=flex_cfg(),
execute_kwargs=flex_cfg(
    filter_results=True,
    raise_no_results=False
),
to_markdown_kwargs=flex_cfg(
    remove_code_title=True,
    even_indentation=True
),
to_html_kwargs=flex_cfg(
    resolve_extensions=True,
    make_links=True,
    extensions=[
        'fenced_code',
        'codehilite',
        'meta',
        'admonition',
        'def_list',
        'attr_list',
        'tables',
        'footnotes',
        'md_in_html',
        'toc',
        'abbr',
        'pymdownx.tilde',
        'pymdownx.keys',
        'pymdownx.details',
        'pymdownx.inlinehilite',
        'pymdownx.snippets',
        'pymdownx.superfences',
        'pymdownx.tabbed',
        'pymdownx.progressbar',
        'pymdownx.magiclink',
        'pymdownx.emoji',
        'pymdownx.highlight',
        'pymdownx.tasklist'
    ],
    extension_configs=flex_cfg({

```

```
'<script src="https://cdn.jsdelivr.net/npm/mermaid/dist/mermaid.min.js"></script>',  
<br>'<script>window.mermaidConfig={startOnLoad:!1,theme:"default",flowchart:{htmlLabels:!1},er:{useMaxWidth:!1}},sequence:{useMaxWidth:  
<br>'<script>const uml=async e=>{class t extends HTMLElement{(constructor)(super){let e=this.attachShadow({mode:"open"}),t=document.  
<br>size:16px}).diagram({margin:0,overflow:visible}),e.appendChild(t)}void 0==customElements.get("diagram-div")&&customElements.define("diagram-  
<br>e.childNodes[i];if(c!=""==a.tagName.toLowerCase())for(let d=0;d<a.childNodes.length;d++)let l=a.childNodes[d],o=/^\\s+$/;if("#text"==l.  
<br>{startOnLoad:!1,theme:"default",flowchart:{htmlLabels:!1},er:{useMaxWidth:!1},sequence:{useMaxWidth:!1,nodeFontWeight:"14px",actorFontSize:"1  
<br>mermaid.config?a:mermaidConfig;mermaid.initialize(d);let l=document.querySelectorAll`pre.${e}, diagram-div`,o=document.querySelector`html b  
<br>r.shadowRoot.querySelector`pre.${e}`).r:h=document.createElement("div").h.style.visibility="hidden".h.style.display="display".h.style.padding  
<br>m=await mermaid.render(`diagram.$(n)`).i(s),h),c=m.svg,p=m.bindFunctions,g=document.createElement("div");g.className=e.g.innerHTML=c.p&p(g).  
<br>.div`).y.shadowRoot.appendChild(h),r.parentNode.insertBefore(y,r),s.style.display="none",y.shadowRoot.appendChild(s),s!==r&&r.htmlElement.remove  
<br>()=>{uml("mermaid")}}};</script>
```

```

        model='gpt-4o'
    ),
    llama_index_config=flex_cfg(
        llm='openai',
        llm_configs=flex_cfg(
            openai=flex_cfg(
                model='gpt-4o'
            )
        )
    ),
),
assets=flex_cfg(
    vbt=flex_cfg(
        asset_name=None,
        release_name=None,
        repo_owner='polakowo',
        repo_name='vectorbt.pro',
        token=None,
        token_required=False,
        use_pygithub=None,
        chunk_size=8192,
        minimize_links=False,
        root_metadata_key=None,
        aggregate_fields=False,
        parent_links_only=True,
        metadata_format='markdown',
        clear_metadata=True,
        clear_metadata_kwargs=flex_cfg(),
        dump_metadata_kwargs=flex_cfg(),
        incl_base_attr=True,
        incl_shortcuts=True,
        incl_shortcut_access=True,
        incl_shortcut_call=True,
        incl_instances=True,
        incl_custom=None,
        is_custom_regex=False,
        as_code=False,
        as_regex=True,
        allow_prefix=False,
        allow_suffix=False,
        merge_targets=True,
        chat=flex_cfg(
            system_prompt="You are an assistant with access to the VectorBT PRO (VBT) Python library documentation and Discord history. V
provide clear and accurate answers using only these sources. If metadata with links is present, reference these links to support your answers
VectorBT PRO, which significantly differs from the open-source version. Given the context information and not prior knowledge, answer the que
        )
    ),
    pages=flex_cfg(
        asset_name='pages.json.zip',
        cache_dir='./knowledge/pages/',
        token_required=True,
        append_obj_type=True,
        append_github_link=True,
        use_parent=None,
        use_base_parents=False,
        use_ref_parents=False,
        incl_bases=True,
        incl_ancestors=True,
        incl_base_ancestors=False,
        incl_refs=None,
        incl_descendants=True,
        incl_ancestor_descendants=False,
        incl_ref_descendants=False,
        aggregate=True,
        aggregate_ancestors=False,
        aggregate_refs=False,
        topo_sort=True,
        incl_pages=None,
        excl_pages=None,
        page_find_mode='substring',
        up_aggregate=True,
        up_aggregate_th=0.6666666666666666,
        up_aggregate_pages=True
    ),
    messages=flex_cfg(
        asset_name='messages.json.zip',
        cache_dir='./knowledge/messages/',
        token_required=True
    )
)
)
)

```

logs frozen_cfg Sub-config with settings applied to [Logs](#).

```

frozen_cfg(
    stats=flex_cfg()
)

```

mapped_array frozen_cfg

Sub-config with settings applied to [MappedArray](#).

```
frozen_cfg(  
    stats=flex_cfg(  
        filters=flex_cfg(  
            has_mapping=flex_cfg(  
                filter_func=<function <lambda> at 0x118d0cea0>  
            )  
        ),  
        settings=flex_cfg(  
            incl_all_keys=False  
        )  
    ),  
    plots=flex_cfg()  
)
```

math frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.math_](#).

Note

All math settings are applied only on import.

```
frozen_cfg(  
    use_tol=True,  
    rel_tol=1e-09,  
    abs_tol=1e-12,  
    use_round=True,  
    decimals=12  
)
```

numba frozen_cfg

Sub-config with Numba-related settings.

```
frozen_cfg(  
    disable=False,  
    parallel=None,  
    silence_warnings=False,  
    check_func_type=True,  
    check_func_suffix=False  
)
```

numpy frozen_cfg

Sub-config with NumPy-related settings.

```
frozen_cfg(  
    float_=<class numpy.float64'>,  
    int_=<class numpy.int64'>  
)
```

ohlcv frozen_cfg

Sub-config with settings applied across [vectorbtpro.ohlcv](#).

```
frozen_cfg(  
    ohlc_type='candlestick',  
    feature_map=flex_cfg(),  
    stats=flex_cfg(),  
    plots=flex_cfg()  
)
```

orders frozen_cfg

Sub-config with settings applied to [Orders](#).

```
frozen_cfg(  
    )
```



```
stats=flex_cfg(),
plots=flex_cfg()
)
```

params frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.params](#).

```
frozen_cfg(
    parameterizer=None,
    param_search_kwargs=flex_cfg(),
    skip_single_comb=True,
    template_context=flex_cfg(),
    build_grid=None,
    grid_indices=None,
    random_subset=None,
    random_replace=False,
    random_sort=True,
    max_guesses=None,
    max_misses=None,
    seed=None,
    clean_index_kwargs=flex_cfg(),
    name_tuple_to_str=True,
    selection=None,
    forward_kwargs_as=flex_cfg(),
    mono_min_size=None,
    mono_n_chunks=None,
    mono_chunk_len=None,
    mono_chunk_meta=None,
    mono_merge_func=None,
    mono_merge_kwargs=flex_cfg(),
    mono_reduce=None,
    filter_results=True,
    raise_no_results=True,
    merge_func=None,
    merge_kwargs=flex_cfg(),
    return_meta=False,
    return_param_index=False,
    execute_kwargs=flex_cfg(),
    replace_parameterizer=False,
    merge_to_execute_kwargs=True
)
```

path frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.path_](#).

```
frozen_cfg(
    mkdir=frozen_cfg(
        mkdir=False,
        mode=511,
        parents=True,
        exist_ok=True
    )
)
```

pbar frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.pbar](#).

```
frozen_cfg(
    disable=False,
    disable_desc=False,
    disable_registry=False,
    disable_machinery=False,
    type='tqdm_auto',
    force_open_bar=False,
    reuse=True,
    kwargs=flex_cfg(
        delay=2
    ),
    desc_kwargs=flex_cfg(
        as_postfix=True,
        refresh=False
    ),
    silence_warnings=False
)
```

pfopt frozen_cfg

Sub-config with settings applied across [vectorbtpro.portfolio.pfopt](#).

```
frozen_cfg(
    pypfopt=flex_cfg(
        target='max_sharpe',
        target_is_convex=True,
        weights_sum_to_one=True,
        target_constraints=None,
        target_solver='SLSQP',
        target_initial_guess=None,
        objectives=None,
        constraints=None,
        sector_mapper=None,
        sector_lower=None,
        sector_upper=None,
        discrete_allocation=False,
        allocation_method='lp_portfolio',
        silence_warnings=True,
        ignore_opt_errors=True,
        ignore_errors=False
    ),
    riskfolio=flex_cfg(
        nan_to_zero=True,
        dropna_rows=True,
        dropna_cols=True,
        dropna_any=True,
        factors=None,
        port=None,
        port_cls=None,
        opt_method=None,
        stats_methods=None,
        model=None,
        asset_classes=None,
        constraints_method=None,
        constraints=None,
        views_method=None,
        views=None,
        solvers=None,
        sol_params=None,
        freq=None,
        year_freq=None,
        pre_opt=False,
        pre_opt_kwargs=flex_cfg(),
        pre_opt_as_w=False,
        func_kwargs=flex_cfg(),
        silence_warnings=True,
        return_port=False,
        ignore_errors=False
    ),
    stats=flex_cfg(
        filters=flex_cfg(
            alloc_ranges=flex_cfg(
                filter_func=<function <lambda> at 0x118dcd40>
            )
        )
    ),
    plots=flex_cfg(
        filters=flex_cfg(
            alloc_ranges=flex_cfg(
                filter_func=<function <lambda> at 0x118dcd260>
            )
        )
    )
)
```

pickling 

Sub-config with settings applied to [vectorbtpro.utils.pickling](#).

```
frozen_cfg(
    pickle_classes=None,
    file_format='pickle',
    compression=None,
    extensions=flex_cfg(
        serialization=flex_cfg(
            pickle={
                'p',
                'pk1',
                'pickle'
            },
            config={
                'config',
                'cfg',
                'ini'
            }
        )
    ),
    compression=flex_cfg(
        zip={
            'zip'
        },
        bz2={
            'bz',

```

```

        'bz2',
        'bzip2'
    },
    gzip={
        'gzip',
        'gz'
    },
    lzma={
        'lzma',
        'xz'
    },
    lz4={
        'lz4'
    },
    blosc2={
        'blosc2'
    },
    blosc1={
        'blosc1'
    },
    blosc={
        'blosc'
    }
)
)
)

```

plots_builder frozen_cfg

Sub-config with settings applied to [PlotsBuilderMixin](#).

```

frozen_cfg(
    subplots='all',
    tags='all',
    per_column=False,
    split_columns=True,
    silence_warnings=False,
    template_context=flex_cfg(),
    filters=flex_cfg(
        is_not_grouped=flex_cfg(
            filter_func=<function <lambda> at 0x118dcc9a0>,
            warning_message=Sub(
                template="Subplot '$subplot_name' does not support grouped data",
                context=None,
                strict=None,
                context_merge_kwargs=None,
                eval_id=None
            )
        ),
        has_freq=flex_cfg(
            filter_func=<function <lambda> at 0x118dcca40>,
            warning_message=Sub(
                template="Subplot '$subplot_name' requires frequency to be set",
                context=None,
                strict=None,
                context_merge_kwargs=None,
                eval_id=None
            )
        )
    ),
    settings=flex_cfg(
        use_caching=True,
        hline_shape_kwargs=flex_cfg(
            type='line',
            line=flex_cfg(
                color='gray',
                dash='dash'
            )
        )
    ),
    subplot_settings=flex_cfg(),
    show_titles=True,
    show_legend=None,
    show_column_label=None,
    hide_id_labels=True,
    group_id_labels=True,
    make_subplots_kwargs=flex_cfg(),
    layout_kwargs=flex_cfg()
)

```

plotting frozen_cfg

Sub-config with settings applied to Plotly figures created from [vectorbtpro.utils.figure](#).

```

frozen_cfg(
    use_widgets=True,
    use_resampler=False,
    auto_rangebreaks=False,

```

```

pre_show_func=None,
show_kwargs=flex_cfg(),
use_gl=False,
color_schema=flex_cfg(
    increasing='#26a69a',
    decreasing='#ee534f',
    lightblue='#6ca6cd',
    lightpurple='#6c76cd',
    lightpink='#cd6ca6',
    blue='#1f77b4',
    orange='#ff7f0e',
    green='#2ca02c',
    red='#dc3912',
    purple='#9467bd',
    brown='#8c564b',
    pink='#e377c2',
    gray='#7f7f7f',
    yellow='#bcbd22',
    cyan='#17becf'
),
contrast_color_schema=flex_cfg(
    blue='#4285f4',
    orange='#ffaa00',
    green='#37b13f',
    red='#ea4335',
    gray='#e2e2e2',
    purple='#a661d5',
    pink='#dd59aa'
),
themes=flex_cfg(
    light=frozen_cfg(
        color_schema=flex_cfg(
            blue='#1f77b4',
            orange='#ff7f0e',
            green='#2ca02c',
            red='#dc3912',
            purple='#9467bd',
            brown='#8c564b',
            pink='#e377c2',
            gray='#7f7f7f',
            yellow='#bcbd22',
            cyan='#17becf'
        ),
        path='__name__/templates/light.json'
    ),
    dark=frozen_cfg(
        color_schema=flex_cfg(
            blue='#1f77b4',
            orange='#ff7f0e',
            green='#2ca02c',
            red='#dc3912',
            purple='#9467bd',
            brown='#8c564b',
            pink='#e377c2',
            gray='#7f7f7f',
            yellow='#bcbd22',
            cyan='#17becf'
        ),
        path='__name__/templates/dark.json'
    ),
    seaborn=frozen_cfg(
        color_schema=flex_cfg(
            blue='rgb(76,114,176)',
            orange='rgb(221,132,82)',
            green='rgb(85,168,104)',
            red='rgb(196,78,82)',
            purple='rgb(129,114,179)',
            brown='rgb(147,120,96)',
            pink='rgb(218,139,195)',
            gray='rgb(140,140,140)',
            yellow='rgb(204,185,116)',
            cyan='rgb(100,181,205)'
        ),
        path='__name__/templates/seaborn.json'
    )
),
default_theme='light',
layout=flex_cfg(
    width=700,
    height=350,
    margin=flex_cfg(
        t=30,
        b=30,
        l=30,
        r=30
    ),
    legend=flex_cfg(
        orientation='h',
        yanchor='bottom',
        y=1.02,
        xanchor='right',
        x=1,
        traceorder='normal'
    ),
    template='vbt_light'
)
)

```

portfolio frozen_cfg

Sub-config with settings applied to **Portfolio**.

```

frozen_cfg(
    data=None,
    open=None,
    high=None,
    low=None,
    close=None,
    bm_close=None,
    val_price='price',
    init_cash=100.0,
    init_position=0.0,
    init_price=np.nan,
    cash_deposits=0.0,
    cash_deposits_as_input=False,
    cash_earnings=0.0,
    cash_dividends=0.0,
    cash_sharing=False,
    ffill_val_price=True,
    update_value=False,
    save_state=False,
    save_value=False,
    save_returns=False,
    skip_empty=True,
    fill_pos_info=True,
    track_value=True,
    row_wise=False,
    seed=None,
    group_by=None,
    broadcast_named_args=None,
    broadcast_kwargs=flex_cfg(
        require_kwargs=flex_cfg(
            requirements='W'
        )
    ),
    template_context=flex_cfg(),
    keep_inout_flex=True,
    from_ago=None,
    sim_start=None,
    sim_end=None,
    call_seq=None,
    attach_call_seq=False,
    max_order_records=None,
    max_log_records=None,
    jitted=None,
    chunked=None,
    staticized=False,
    records=None,
    size=np.inf,
    size_type='amount',
    direction='both',
    price='close',
    fees=0.0,
    fixed_fees=0.0,
    slippage=0.0,
    min_size=np.nan,
    max_size=np.nan,
    size_granularity=np.nan,
    leverage=1.0,
    leverage_mode='lazy',
    reject_prob=0.0,
    price_area_vio_mode='ignore',
    allow_partial=True,
    raise_reject=False,
    log=False,
    from_orders=flex_cfg(),
    from_signals=flex_cfg(
        direction='longonly',
        adjust_func_nb=None,
        adjust_args=(),
        signal_func_nb=None,
        signal_args=None,
        post_signal_func_nb=None,
        post_signal_args=(),
        post_segment_func_nb=None,
        post_segment_args=(),
        order_mode=False,
        accumulate=False,
        upon_long_conflict='ignore',
        upon_short_conflict='ignore',
        upon_dir_conflict='ignore',
        upon_opposite_entry='reversedreduce',
        order_type='market',
        limit_reverse=False,
        limit_delta=np.nan,
        limit_tif=-1,
        limit_expiry=-1,
        limit_order_price='limit',
        upon_adj_limit_conflict='keepignore',
        upon_opp_limit_conflict='cancelexecute',
        use_stops=None,
        stop_ladder='disabled',
    )
)

```

```

sl_stop=np.nan,
tsl_th=np.nan,
tsl_stop=np.nan,
tp_stop=np.nan,
td_stop=-1,
dt_stop=-1,
stop_entry_price='close',
stop_exit_price='stop',
stop_order_type='market',
stop_limit_delta=np.nan,
stop_exit_type='close',
upon_stop_update='override',
upon_adj_stop_conflict='keepexecute',
upon_opp_stop_conflict='keepexecute',
delta_format='percent',
time_delta_format='index'
),
hold_direction='longonly',
close_at_end=False,
from_order_func=flex_cfg(
    segment_mask=True,
    call_pre_segment=False,
    call_post_segment=False,
    pre_sim_func_nb=None,
    pre_sim_args=(),
    post_sim_func_nb=None,
    post_sim_args=(),
    pre_group_func_nb=None,
    pre_group_args=(),
    post_group_func_nb=None,
    post_group_args=(),
    pre_row_func_nb=None,
    pre_row_args=(),
    post_row_func_nb=None,
    post_row_args=(),
    pre_segment_func_nb=None,
    pre_segment_args=(),
    post_segment_func_nb=None,
    post_segment_args=(),
    order_func_nb=None,
    order_args=(),
    flex_order_func_nb=None,
    flex_order_args=(),
    post_order_func_nb=None,
    post_order_args=(),
    row_wise=False
),
from_def_order_func=flex_cfg(
    flexible=False
),
freq=None,
year_freq=None,
use_in_outputs=True,
fillna_close=True,
weights=None,
trades_type='exittrades',
stats=flex_cfg(
    filters=flex_cfg(
        has_year_freq=flex_cfg(
            filter_func=<function <lambda> at 0x118dccb80>,
            warning_message=Sub(
                template="Metric '$metric_name' requires year frequency to be set",
                context=None,
                strict=None,
                context_merge_kwargs=None,
                eval_id=None
            )
        ),
        has_bm_returns=flex_cfg(
            filter_func=<function <lambda> at 0x118dcccfe0>,
            warning_message=Sub(
                template="Metric '$metric_name' requires bm_returns to be set",
                context=None,
                strict=None,
                context_merge_kwargs=None,
                eval_id=None
            )
        ),
        has_cash_deposits=flex_cfg(
            filter_func=<function <lambda> at 0x118dcd080>
        ),
        has_cash_earnings=flex_cfg(
            filter_func=<function <lambda> at 0x118dcd120>
        )
    ),
    settings=flex_cfg(
        use_asset_returns=False,
        incl_open=False
    ),
    template_context=flex_cfg(
        incl_open_tags=RepEval(
            template="['open', 'closed'] if incl_open else ['closed']",
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    )
)
)

```

```

    ),
    plots=flex_cfg(
        subplots=[
            'orders',
            'trade_pnl',
            'cumulative_returns'
        ],
        settings=flex_cfg(
            use_asset_returns=False
        )
    )
)

```

qs_adapter frozen_cfg

Sub-config with settings applied to [QSAdapter](#).

```

frozen_cfg(
    defaults=flex_cfg()
)

```

ranges frozen_cfg

Sub-config with settings applied to [Ranges](#).

```

frozen_cfg(
    stats=flex_cfg(),
    plots=flex_cfg()
)

```

records frozen_cfg

Sub-config with settings applied to [Records](#).

```

frozen_cfg(
    stats=flex_cfg(),
    plots=flex_cfg()
)

```

resampling frozen_cfg

Sub-config with settings applied across [vectorbtpro.base.resampling](#).

```

frozen_cfg(
    silence_warnings=False
)

```

returns frozen_cfg

Sub-config with settings applied to [ReturnsAccessor](#).

```

frozen_cfg(
    inf_to_nan=False,
    nan_to_zero=False,
    year_freq='365 days',
    bm_returns=None,
    defaults=flex_cfg(
        start_value=1.0,
        window=10,
        minp=None,
        ddof=1,
        risk_free=0.0,
        levy_alpha=2.0,
        required_return=0.0,
        cutoff=0.05,
        periods=None
    ),
    stats=flex_cfg(
        filters=flex_cfg(
            has_year_freq=flex_cfg(
                filter_func=<function <lambda> at 0x118dcd60>,
                warning_message=Sub(
                    template='Metric '$metric_name' requires year frequency to be set',
                    context=None,

```

```
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
has_bm_returns=flex_cfg(
    filter_func=<function <lambda> at 0x118dccc00>,
    warning_message=Sub(
        template="Metric '$metric_name' requires bm_returns to be set",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
settings=flex_cfg(
    check_is_not_grouped=True
),
plots=flex_cfg()
)
```

search frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.search](#).

```
frozen_cfg(
    traversal='DFS',
    excl_types=(
        <class 'list'>,
        <class 'set'>,
        <class 'frozenset'>
    ),
    incl_types=None,
    max_len=None,
    max_depth=None
)
```

settings SettingsConfig

Global settings config.

Combines all sub-configs defined in this module.

signals frozen_cfg

Sub-config with settings applied to [SignalsAccessor](#).

```
frozen_cfg(
    stats=flex_cfg(
        filters=flex_cfg(
            silent_has_target=flex_cfg(
                filter_func=<function <lambda> at 0x118dccc00>
            )
        ),
        settings=flex_cfg(
            target=None,
            target_name='Target',
            relation='onemany'
        )
    ),
    plots=flex_cfg()
)
```

splitter frozen_cfg

Sub-config with settings applied to [Splitter](#).

```
frozen_cfg(
    stats=flex_cfg(
        settings=flex_cfg(
            normalize=True
        ),
        filters=flex_cfg(
            has_multiple_sets=flex_cfg(
                filter_func=<function <lambda> at 0x118dccc00>
            ),
            normalize=flex_cfg(
                filter_func=<function <lambda> at 0x118dccc20>
            )
        )
    )
)
```



```

    )
),
plots=flex_cfg()
)

```

stats_builder frozen_cfg

Sub-config with settings applied to [StatsBuilderMixin](#).

```

frozen_cfg(
    metrics='all',
    tags='all',
    per_column=False,
    split_columns=True,
    dropna=False,
    silence_warnings=False,
    template_context=flex_cfg(),
    filters=flex_cfg(
        is_not_grouped=flex_cfg(
            filter_func=<function <lambda> at 0x118dcc860>,
            warning_message=Sub(
                template="Metric '$metric_name' does not support grouped data",
                context=None,
                strict=None,
                context_merge_kwargs=None,
                eval_id=None
            )
        ),
        has_freq=flex_cfg(
            filter_func=<function <lambda> at 0x118dcc900>,
            warning_message=Sub(
                template="Metric '$metric_name' requires frequency to be set",
                context=None,
                strict=None,
                context_merge_kwargs=None,
                eval_id=None
            )
        )
    ),
    settings=flex_cfg(
        to_timedelta=None,
        use_caching=True
    ),
    metric_settings=flex_cfg()
)

```

telegram frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.telegram](#).

```

frozen_cfg(
    bot=flex_cfg(
        token=None,
        use_context=True,
        persistence=True,
        defaults=flex_cfg(),
        drop_pending_updates=True
    ),
    giphy=flex_cfg(
        api_key=None,
        weirdness=5
    )
)

```

python-telegram-bot

Sub-config with settings applied to [python-telegram-bot](#).

Set `persistence` to string to use as `filename` in `telegram.ext.PicklePersistence`. For defaults, see `telegram.ext.Defaults`. Other settings will be distributed across `telegram.ext.Updater` and `telegram.ext.updater.Updater.start_polling`.

GIPHY

Sub-config with settings applied to [GIPHY Translate Endpoint](#).

template frozen_cfg

Sub-config with settings applied across [vectorbtpro.utils.template](#).

```

frozen_cfg(
    strict=True,

```

```

        search_kwargs=flex_cfg(),
        context=flex_cfg()
    )

```

trades frozen_cfg

Sub-config with settings applied to [Trades](#).

```

frozen_cfg(
    stats=flex_cfg(
        settings=flex_cfg(
            incl_open=False
        ),
        template_context=flex_cfg(
            incl_open_tags=RepEval(
                template="['open', 'closed'] if incl_open else ['closed']",
                context=None,
                strict=None,
                context_merge_kwargs=None,
                eval_id=None
            )
        )
    ),
    plots=flex_cfg()
)

```

wrapping frozen_cfg

Sub-config with settings applied across [vectorbtpro.base.wrapping](#).

```

frozen_cfg(
    column_only_select=False,
    range_only_select=False,
    group_select=True,
    freq='auto',
    silence_warnings=False,
    zero_to_none=True,
    min_precision=None,
    max_precision=None,
    prec_float_only=True,
    prec_check_bounds=True,
    prec_strict=True
)

```

When enabling `max_precision` and running your code for the first time, make sure to enable `prec_check_bounds`. After that, you can safely disable it to slightly increase performance.

SettingsConfig class

```

SettingsConfig(
    *args,
    **kwargs
)

```

Extends [Config](#) for global settings.

Superclasses

- [Base](#)
- [Comparable](#)
- [Config](#)
- [Pickleable](#)
- [Prettified](#)
- `builtins.dict`
- [pdict](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)

- [Base.find_messages](#)
- [Config.clear](#)
- [Config.copy](#)
- [Config.equals](#)
- [Config.get_option](#)
- [Config.load_update](#)
- [Config.make_checkpoint](#)
- [Config.merge_with](#)
- [Config.options_](#)
- [Config.pop](#)
- [Config.popitem](#)
- [Config.prettify](#)
- [Config.rec_state](#)
- [Config.reset](#)
- [Config.set_option](#)
- [Config.to_dict](#)
- [Config.update](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

register_template method

```
SettingsConfig.register_template(
    theme
)
```

Register template of a theme.

register_templates method

```
SettingsConfig.register_templates()
```

Register templates of all themes.

reset_theme method

```
SettingsConfig.reset_theme()
```

Reset to default theme.

set_theme method

```
SettingsConfig.set_theme(
    theme
)
```

Set default theme.

substitute_sub_config_docs method

```
SettingsConfig.substitute_sub_config_docs(  
    __pdoc__,  
    prettify_kwargs=None  
)
```

Substitute templates in sub-config docs.

flex_cfg class

```
flex_cfg(  
    *args,  
    **kwargs  
)
```

Class representing a flexible sub-config.

Superclasses

- [Base](#)
- [Comparable](#)
- [Config](#)
- [Pickleable](#)
- [Prettified](#)
- `builtins.dict`
- [pdict](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Config.clear](#)
- [Config.copy](#)
- [Config.equals](#)
- [Config.get_option](#)
- [Config.load_update](#)
- [Config.make_checkpoint](#)
- [Config.merge_with](#)
- [Config.options_](#)
- [Config.pop](#)
- [Config.popitem](#)
- [Config.prettify](#)
- [Config.rec_state](#)
- [Config.reset](#)
- [Config.set_option](#)
- [Config.to_dict](#)
- [Config.update](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)

- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

frozen_cfg class 

```
frozen_cfg(  
    *args,  
    **kwargs  
)
```



Class representing a frozen sub-config.

Superclasses

- [Base](#)
- [Comparable](#)
- [Config](#)
- [Pickleable](#)
- [Prettified](#)
- `builtins.dict`
- [pdict](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Config.clear](#)
- [Config.copy](#)
- [Config.equals](#)
- [Config.get_option](#)
- [Config.load_update](#)
- [Config.make_checkpoint](#)
- [Config.merge_with](#)
- [Config.options_](#)
- [Config.pop](#)
- [Config.popitem](#)
- [Config.prettify](#)
- [Config.rec_state](#)
- [Config.reset](#)
- [Config.set_option](#)
- [Config.to_dict](#)
- [Config.update](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)

- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [Prettified.pprint](#)

API generic splitting

base splitting

base module

Base class for splitting.

FixRange class

```
FixRange(
    *args,
    **kwargs
)
```

Class that represents a fixed range.

Superclasses

- Base
- DefineMixin
- Hashable

Inherited members

- Base.chat
- Base.find_api
- Base.find_assets
- Base.find_docs
- Base.find_examples
- Base.find_messages
- DefineMixin.asdict
- DefineMixin.assert_field_not_missing
- DefineMixin.fields
- DefineMixin.fields_dict
- DefineMixin.get_field
- DefineMixin.hash
- DefineMixin.hash_key
- DefineMixin.is_field_missing
- DefineMixin.is_field_optional
- DefineMixin.is_field_required
- DefineMixin.merge_over
- DefineMixin.merge_with
- DefineMixin.replace
- DefineMixin.resolve
- DefineMixin.resolve_field
- Hashable.get_hash

range_ field

Range.

RelRange class

```
RelRange(
    *args,
    **kwargs
)
```

Class that represents a relative range.

Superclasses

- [Base](#)
- [DefineMixin](#)
- [Hashable](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [DefineMixin.asdict](#)
- [DefineMixin.assert_field_not_missing](#)
- [DefineMixin.fields](#)
- [DefineMixin.fields_dict](#)
- [DefineMixin.get_field](#)
- [DefineMixin.hash](#)
- [DefineMixin.hash_key](#)
- [DefineMixin.is_field_missing](#)
- [DefineMixin.is_field_optional](#)
- [DefineMixin.is_field_required](#)
- [DefineMixin.merge_over](#)
- [DefineMixin.merge_with](#)
- [DefineMixin.replace](#)
- [DefineMixin.resolve](#)
- [DefineMixin.resolve_field](#)
- [Hashable.get_hash](#)

is_gap field ⓘ

Whether the range acts as a gap.

length field ⓘ

Length.

Floating values between 0 and 1 are considered relative.

Can be negative.

length_space field ⓘ

Length space.

Supported are

- 'all': All space
- 'free': Remaining space after the offset
- 'free_or_prev': Remaining space after the offset or the start/end of the previous range, depending what comes first in the direction of [RelRange.length](#)

Applied only when [RelRange.length](#) is a relative number.

offset field ⓘ

Offset.

Floating values between 0 and 1 are considered relative.

Can be negative.

offset_anchor field

Offset anchor.

Supported are

- 'start': Start of the range
 - 'end': End of the range
 - 'prev_start': Start of the previous range
 - 'prev_end': End of the previous range
-

offset_space field

Offset space.

Supported are

- 'all': All space
- 'free': Remaining space after the offset anchor
- 'prev': Length of the previous range

Applied only when [RelRange.offset](#) is a relative number.

out_of_bounds field

Check if start and stop are within bounds.

Supported are

- 'keep': Keep out-of-bounds values
 - 'ignore': Ignore if out-of-bounds
 - 'warn': Emit a warning if out-of-bounds
 - 'raise': Raise an error if out-of-bounds
-

to_slice method

```
RelRange.to_slice(  
    total_len,  
    prev_start=0,  
    prev_end=0,  
    index=None,  
    freq=None  
)
```

Convert the relative range into a slice.

Splitter class

```
Splitter(  
    wrapper,  
    index,  
    splits_arr,  
    **kwargs  
)
```

Base class for splitting.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)

- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [StatsBuilderMixin](#)
- [Wrapping](#)

Inherited members

- [Analyzable.cls_dir](#)
- [Analyzable.column_only_select](#)
- [Analyzable.config](#)
- [Analyzable.group_select](#)
- [Analyzable.iloc](#)
- [Analyzable.indexing_kwargs](#)
- [Analyzable.loc](#)
- [Analyzable.range_only_select](#)
- [Analyzable.rec_state](#)
- [Analyzable.self_aliases](#)
- [Analyzable.wrapper](#)
- [Analyzable.xloc](#)
- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)
- [AttrResolverMixin.resolve_attr](#)
- [AttrResolverMixin.resolve_shortcut_attr](#)
- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)
- [Configured.replace](#)
- [Configured.resolve_merge_kwargs](#)
- [Configured.update_config](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)

- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [IndexApplier.add_levels](#)
- [IndexApplier.drop_duplicate_levels](#)
- [IndexApplier.drop_levels](#)
- [IndexApplier.drop_redundant_levels](#)
- [IndexApplier.rename_levels](#)
- [IndexApplier.select_levels](#)
- [IndexingBase.indexing_setter_func](#)
- [PandasIndexer.xs](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)
- [PlotsBuilderMixin.build_subplots_doc](#)
- [PlotsBuilderMixin.override_subplots_doc](#)
- [PlotsBuilderMixin.plots](#)
- [PlotsBuilderMixin.resolve_plots_setting](#)
- [Prettified.pprint](#)
- [StatsBuilderMixin.build_metrics_doc](#)
- [StatsBuilderMixin.override_metrics_doc](#)
- [StatsBuilderMixin.resolve_stats_setting](#)
- [StatsBuilderMixin.stats](#)
- [Wrapping.apply_to_index](#)
- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping.regroup](#)
- [Wrapping.resample](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

apply method 

```

Splitter.apply(
    apply_func,
    *apply_args,
    split=None,
    set_=None,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    remap_to_obj=True,
    obj_index=None,

```

```

obj_freq=None,
range_format='slice_or_any',
point_wise=False,
attach_bounds=False,
right_inclusive=False,
template_context=None,
silence_warnings=False,
index_combine_kwargs=None,
freq=None,
iteration='split_wise',
execute_kwargs=None,
filter_results=True,
raise_no_results=True,
merge_func=None,
merge_kwargs=None,
merge_all=True,
wrap_results=True,
eval_id=None,
**apply_kwargs
)

```

Apply a function on each range.

Uses [Splitter.select_indices](#) to get the indices for selected splits and sets. Arguments `split_group_by` and `set_group_by` can be used to group splits and sets respectively. Ranges belonging to the same split and set group will be merged.

For each index pair, in a lazily manner, resolves the source range using [Splitter.select_range](#) and [Splitter.get_ready_range](#). Then, takes each argument from `args` and `kwargs` wrapped with [Takeable](#), remaps the range into each object's index using [Splitter.get_ready_obj_range](#), and takes the slice from that object using [Splitter.take_range](#). The original object will be substituted by this slice. At the end, substitutes any templates in the prepared `args` and `kwargs` and saves the function and arguments for execution.

For substitution, the following information is available:

- `split/set_group_indices`: Indices corresponding to the selected row/column groups
- `split/set_indices`: Indices corresponding to the selected rows/columns
- `n_splits/sets`: Number of the selected rows/columns
- `split/set_labels`: Labels corresponding to the selected row/column groups
- `split/set_idx`: Index of the selected row/column
- `split/set_label`: Label of the selected row/column
- `range_`: Selected range ready for indexing (see [Splitter.get_ready_range](#))
- `range_meta`: Various information on the selected range
- `obj_range_meta`: Various information on the range taken from each takeable argument. Positional arguments are denoted by position, keyword arguments are denoted by keys.
- `args`: Positional arguments with ranges already selected
- `kwargs`: Keyword arguments with ranges already selected
- `bounds`: A tuple of either integer or index bounds. Can be source or target depending on `attach_bounds`.
- `template_context`: Passed template context

Since each range is processed lazily (that is, upon request), there are multiple iteration modes controlled by the argument `iteration`:

- 'split_major': Flatten all ranges in split-major order and iterate over them
- 'set_major': Flatten all ranges in set-major order and iterate over them
- 'split_wise': Iterate over splits, while ranges in each split are processed sequentially
- 'set_wise': Iterate over sets, while ranges in each set are processed sequentially

The execution is done using `execute` with `execute_kwargs`. Once all results have been obtained, attempts to merge them using `merge_func` with `merge_kwargs` (all templates in it will be substituted as well), which can also be a string or a tuple of strings resolved using [resolve_merge_func](#). If `wrap_results` is enabled, packs the results into a Pandas object. If `apply_func` returns something complex, the resulting Pandas object will be of object data type. If `apply_func` returns a tuple (detected by the first returned result), a Pandas object is built for each element of that tuple.

If `merge_all` is True, will merge all results in a flattened manner irrespective of the iteration mode. Otherwise, will merge by split/set.

If [NoResult](#) is returned, will skip the current iteration and remove it from the final index.

Usage

- Get the return of each data range:

```

>>> from vectorbtpro import *

>>> data = vbt.YFData.pull(
...     "BTC-USD",
...     start="2020-01-01 UTC",
...     end="2021-01-01 UTC"
... )
>>> splitter = vbt.Splitter.from_n_rolling(data.wrapper.index, 5)

>>> def apply_func(data):
...     return data.close.iloc[-1] - data.close.iloc[0]

```

```
>>> splitter.apply(apply_func, vbt.Takeable(data))
split
0    -1636.467285
1     3706.568359
2    2944.720703
3    -118.113281
4    17098.916016
dtype: float64
```

- The same but by indexing manually:

```
>>> def apply_func(range_, data):
...     data = data.iloc[range_]
...     return data.close.iloc[-1] - data.close.iloc[0]

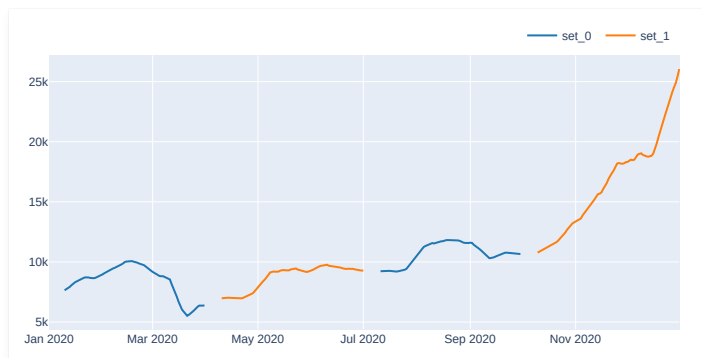
>>> splitter.apply(apply_func, vbt.Rep("range_"), data)
split
0    -1636.467285
1     3706.568359
2    2944.720703
3    -118.113281
4    17098.916016
dtype: float64
```

- Divide into two windows, each consisting of 50% train and 50% test, compute SMA for each range, and row-stack the outputs of each set upon merging:

```
>>> splitter = vbt.Splitter.from_n_rolling(data.wrapper.index, 2, split=0.5)

>>> def apply_func(data):
...     return data.run("SMA", 10).real

>>> splitter.apply(
...     apply_func,
...     vbt.Takeable(data),
...     merge_func="row_stack"
... ).unstack("set").vbt.drop_levels("split", axis=0).vbt.plot().show()
```



bounds class property

[Splitter.get_bounds](#) with default arguments.

bounds_arr class property

[Splitter.get_bounds_arr](#) with default arguments.

break_up_splits method

```
Splitter.break_up_splits(
    new_split,
    sort=False,
    template_context=None,
    wrapper_kwargs=None,
    init_kwargs=None,
    **split_range_kwargs
)
```

Split each split into multiple splits.

If there are multiple sets, make sure to merge them into one beforehand.

Arguments `new_split` and `**split_range_kwargs` are passed to [Splitter.split_range](#).

column_stack class method

```
Splitter.column_stack(
    *objs,
    wrapper_kwargs=None,
    **kwargs
)
```

Stack multiple **Splitter** instances along columns.

Uses **ArrayWrapper.column_stack** to stack the wrappers.

coverage class property

Splitter.get_coverage with default arguments.

duration class property

Splitter.get_duration with default arguments.

from_expanding class method

```
Splitter.from_expanding(
    index,
    min_length,
    offset,
    split=None,
    split_range_kwargs=None,
    range_bounds_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)
```

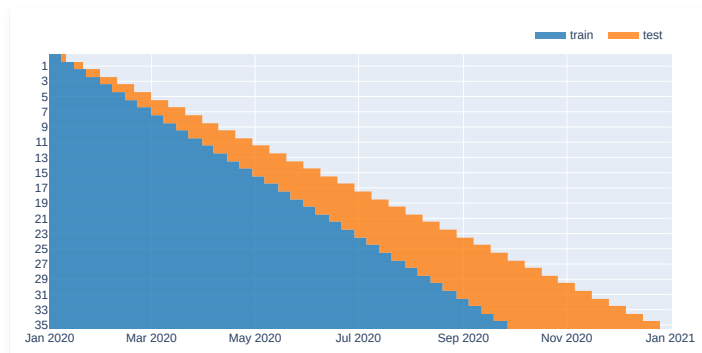
Create a **Splitter** instance from an expanding range.

Argument **min_length** is the minimum length of the expanding range. Provide it as a float between 0 and 1 to make it relative to the length of the index. Argument **offset** is an offset after the right bound of the previous range from which the next range should start. It can also be a float relative to the index length. For other arguments, see **Splitter.from_rolling**.

Usage

- Roll an expanding range with a length of 10 and an offset of 10, and split it into 3/4:

```
>>> from vectorbtpro import *
>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_expanding(
...     index,
...     10,
...     10,
...     split=3/4,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()
```

**from_grouper** class method

```
Splitter.from_grouper(
```

```

    index,
    by,
    groupby_kwargs=None,
    grouper_kwargs=None,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    split_labels=None,
    freq=None,
    **kwargs
)

```

Create a [Splitter](#) instance from a grouper.

See [BaseIDXAccessor.get_grouper](#).

Uses [Splitter.from_splits](#) to prepare the splits array and labels, and to build the instance.

Usage

- Map each month into a range:

```

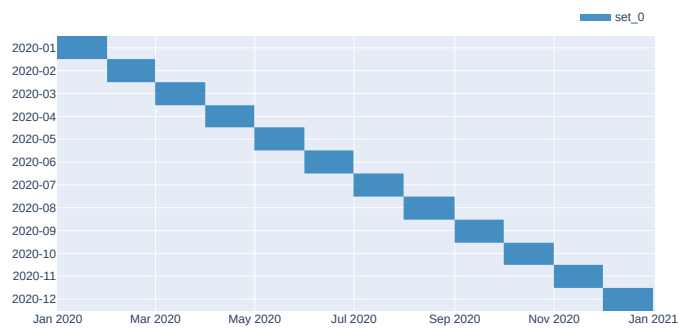
>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> def is_month_end(index, split):
...     last_range = split[-1]
...     return index[last_range][-1].is_month_end

>>> splitter = vbt.Splitter.from_grouper(
...     index,
...     "M",
...     split_check_template=vbt.RepFunc(is_month_end)
... )
>>> splitter.plot().show()

```



from_n_expanding class method

```

Splitter.from_n_expanding(
    index,
    n,
    min_length=None,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)

```

Create a [Splitter](#) instance from a number of expanding ranges.

Picks `n` evenly-spaced, expanding ranges. Argument `min_length` defines the minimum length for each range. For other arguments, see [Splitter.from_rolling](#).

Usage

- Roll 10 expanding ranges with a minimum length of 100, while reserving 50 elements for test:

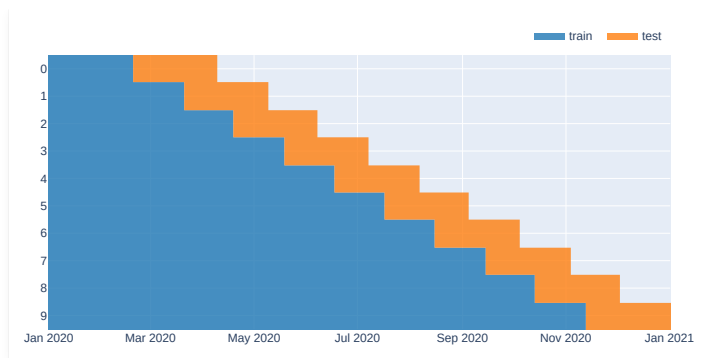
```

>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> splitter = vbt.Splitter.from_n_expanding(
...     index,
...     10,
...     min_length=100,
...     split=-50,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```



from_n_random class method

```

Splitter.from_n_random(
    index,
    n,
    min_length,
    max_length=None,
    min_start=None,
    max_end=None,
    length_choice_func=None,
    start_choice_func=None,
    length_p_func=None,
    start_p_func=None,
    seed=None,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)

```

Create a **Splitter** instance from a number of random ranges.

Randomly picks the length of a range between `min_length` and `max_length` (including) using `length_choice_func`, which receives an array of possible values and selects one. It defaults to `numpy.random.Generator.choice`. Optional function `length_p_func` takes the same as `length_choice_func` and must return either None or probabilities.

Randomly picks the start position of a range starting at `min_start` and ending at `max_end` (excluding) minus the chosen length using `start_choice_func`, which receives an array of possible values and selects one. It defaults to `numpy.random.Generator.choice`. Optional function `start_p_func` takes the same as `start_choice_func` and must return either None or probabilities.

Note

Each function must take two arguments: the iteration index and the array with possible values.

For other arguments, see **Splitter.from_rolling**.

Usage

- Generate 20 random ranges with a length from [40, 100], and split each into 3/4:

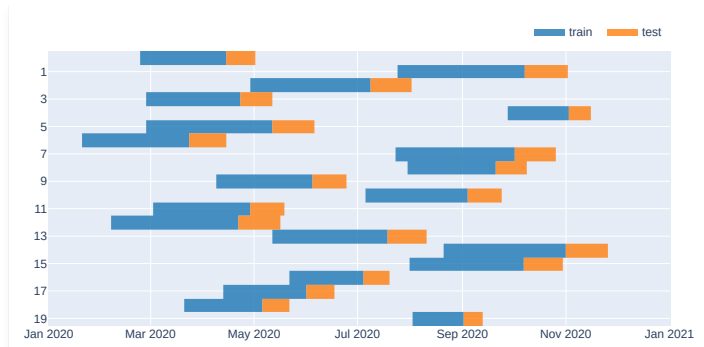
```

>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> splitter = vbt.Splitter.from_n_random(
...     index,
...     20,
...     min_length=40,
...     max_length=100,
...     split=3/4,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```

from_n_rolling class method

```

Splitter.from_n_rolling(
    index,
    n,
    length=None,
    optimize_anchor_set=1,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)

```

Create a **Splitter** instance from a number of rolling ranges of the same length.

If `length` is `None`, splits the index evenly into `n` non-overlapping ranges using **Splitter.from_rolling**. Otherwise, picks `n` evenly-spaced, potentially overlapping ranges of a fixed length. For other arguments, see **Splitter.from_rolling**.

If `length` is "optimize", searches for a length to cover the most of the index. Use `optimize_anchor_set` to provide the index of a set that should become non-overlapping.

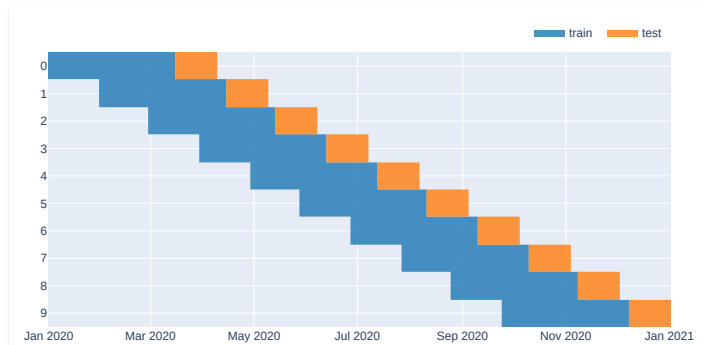
Usage

- Roll 10 ranges with 100 elements, and split it into 3/4:

```

>>> from vectorbtpro import *
>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_n_rolling(
...     index,
...     10,
...     length=100,
...     split=3/4,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```



from_purged class method

```

Splitter.from_purged(
    index,
    purged_splitter,
    pred_times=None,
    eval_times=None,
    split_labels=None,
    set_labels=None,
)

```

```

    **kwargs
)

```

Create a [Splitter](#) instance from a purged splitter.

The splitter must be an instance of [BasePurgedCV](#).

Uses [Splitter.from_splits](#) to prepare the splits array and labels, and to build the instance.

from_purged_kfold class method

```

Splitter.from_purged_kfold(
    index,
    n_folds=10,
    n_test_folds=2,
    purge_td=0,
    embargo_td=0,
    pred_times=None,
    eval_times=None,
    **kwargs
)

```

Create a [Splitter](#) instance from [PurgedKfoldCV](#).

Keyword arguments are passed to [Splitter.from_purged](#).

from_purged_walkforward class method

```

Splitter.from_purged_walkforward(
    index,
    n_folds=10,
    n_test_folds=1,
    min_train_folds=2,
    max_train_folds=None,
    split_by_time=False,
    purge_td=0,
    pred_times=None,
    eval_times=None,
    **kwargs
)

```

Create a [Splitter](#) instance from [PurgedWalkForwardCV](#).

Keyword arguments are passed to [Splitter.from_purged](#).

from_ranges class method

```

Splitter.from_ranges(
    index,
    split=None,
    split_range_kwargs=None,
    template_context=None,
    **kwargs
)

```

Create a [Splitter](#) instance from ranges.

Uses [get_index_ranges](#) to generate start and end indices. Passes only related keyword arguments found in `kwargs`.

Other keyword arguments will be passed to [Splitter.from_splits](#). For details on `split` and `split_range_kwargs`, see [Splitter.from_rolling](#).

Usage

- Translate each quarter into a range:

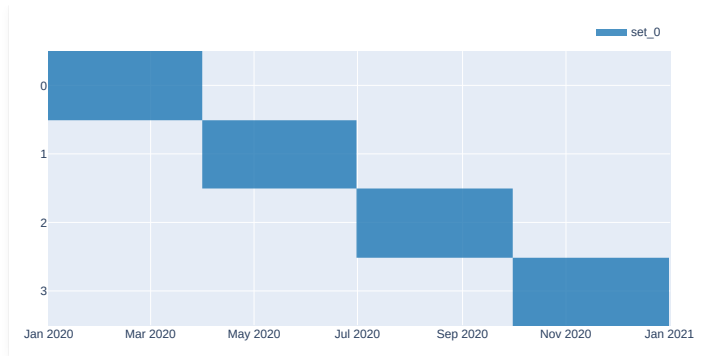
```

>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

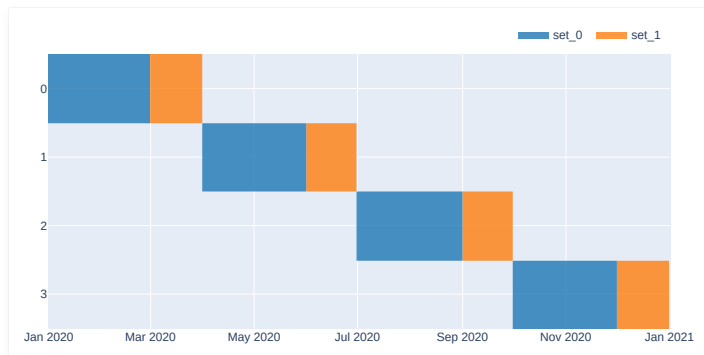
>>> splitter = vbt.Splitter.from_ranges(index, every="QS")
>>> splitter.plot().show()

```



- In addition to the above, reserve the last month for testing purposes:

```
>>> splitter = vbt.Splitter.from_ranges(
...     index,
...     every="QS",
...     split=(1.0, lambda index: index.month == index.month[-1]),
...     split_range_kwargs=dict(backwards=True)
... )
>>> splitter.plot().show()
```



from_rolling class method

```
Splitter.from_rolling(
    index,
    length,
    offset=0,
    offset_anchor='prev_end',
    offset_anchor_set=0,
    offset_space='prev',
    backwards=False,
    split=None,
    split_range_kwargs=None,
    range_bounds_kwargs=None,
    template_context=None,
    freq=None,
    **kwargs
)
```

Create a **Splitter** instance from a rolling range of a fixed length.

Uses **Splitter.from_splits** to prepare the splits array and labels, and to build the instance.

Args

index : index_like

Index.

length : int, float, or timedelta_like

See **RelRange.length**.

offset : int, float, or timedelta_like

See **RelRange.offset**.

offset_anchor : str

See **RelRange.offset_anchor**.

`offset_anchor_set` : int

Offset anchor set.

Selects the set from the previous range to be used as an offset anchor. If None, the whole previous split is considered as a single range. By default, it's the first set.

`offset_space` : str

See [RelRange.offset_space](#).

`backwards` : bool or str

Whether to roll backwards.

If 'sorted', will roll backwards and sort the resulting splits by the start index.

`split` : any

Ranges to split the range into.

If None, will produce the entire range as a single range. Otherwise, will use [Splitter.split_range](#) to split the range into multiple ranges.

`split_range_kwargs` : dict

Keyword arguments passed to [Splitter.split_range](#).

`range_bounds_kwargs` : dict

Keyword arguments passed to [Splitter.get_range_bounds](#).

`template_context` : dict

Context used to substitute templates in ranges.

`freq` : any

Index frequency in case it cannot be parsed from `index`.

If None, will be parsed using [BaseIDXAccessor.get_freq](#).

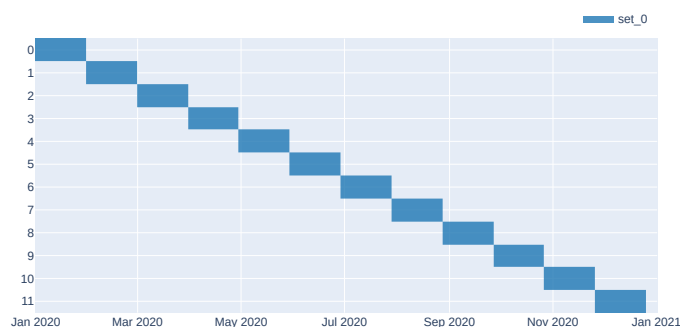
****kwargs**

Keyword arguments passed to the constructor of [Splitter](#).

Usage

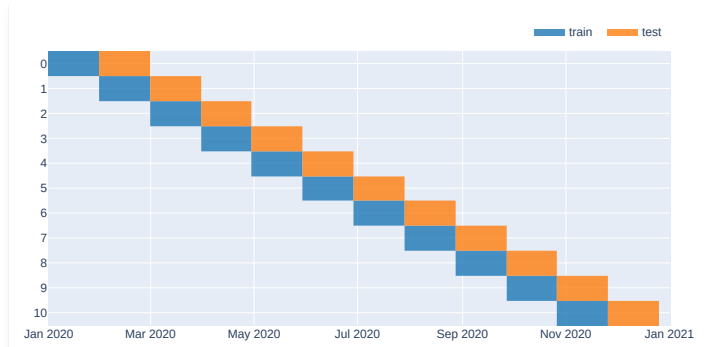
- Divide a range into a set of non-overlapping ranges:

```
>>> from vectorbtpro import *
>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_rolling(index, 30)
>>> splitter.plot().show()
```



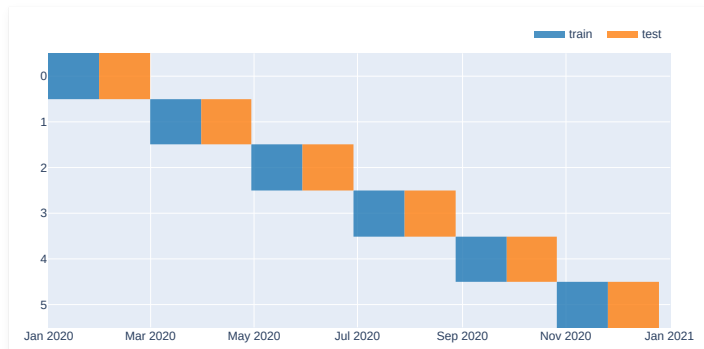
- Divide a range into ranges, each split into 1/2:

```
>>> splitter = vbt.Splitter.from_rolling(
...     index,
...     60,
...     split=1/2,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()
```



- Make the ranges above non-overlapping by using the right bound of the last set as an offset anchor:

```
>>> splitter = vbt.Splitter.from_rolling(
...     index,
...     60,
...     offset_anchor_set=-1,
...     split=1/2,
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()
```



from_single class method

```
Splitter.from_single(
    index,
    split,
    split_range_kwargs=None,
    template_context=None,
    **kwargs
)
```

Create a [Splitter](#) instance from a single split.

from_sklearn class method

```
Splitter.from_sklearn(
    index,
    skl_splitter,
    groups=None,
    split_labels=None,
    set_labels=None,
    **kwargs
)
```

Create a [Splitter](#) instance from a scikit-learn's splitter.

The splitter must be an instance of `sklearn.model_selection.BaseCrossValidator`.

Uses [Splitter.from_splits](#) to prepare the splits array and labels, and to build the instance.

from_split_func class method

```
Splitter.from_split_func(
    index,
    split_func,
```

```

split_args=None,
split_kwargs=None,
fix_ranges=True,
split=None,
split_range_kwargs=None,
range_bounds_kwargs=None,
template_context=None,
freq=None,
**kwargs
)

```

Create a [Splitter](#) instance from a custom split function.

In a while-loop, substitutes templates in `split_args` and `split_kwargs` and passes them to `split_func`, which should return either a split (see `new_split` in [Splitter.split_range](#), also supports a single range if it's not an iterable) or `None` to abrupt the while-loop. If `fix_ranges` is `True`, the returned split is then converted into a fixed split using [Splitter.split_range](#) and the bounds of its sets are measured using [Splitter.get_range_bounds](#).

Each template substitution has the following information:

- `split_idx`: Current split index, starting at 0
- `splits`: Nested list of splits appended up to this point
- `bounds`: Nested list of bounds appended up to this point
- `prev_start`: Left bound of the previous split
- `prev_end`: Right bound of the previous split
- Arguments and keyword arguments passed to [Splitter.from_split_func](#)

Usage

- Rolling window of 30 elements, 20 for train and 10 for test:

```

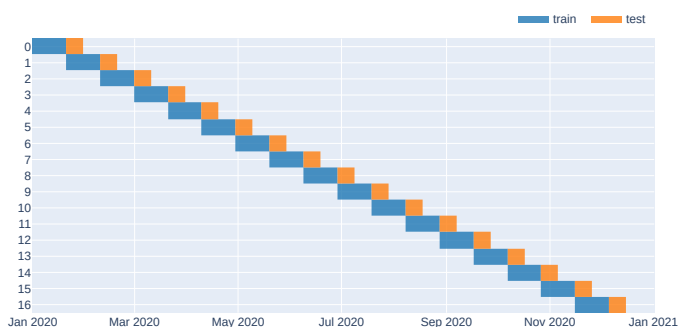
>>> from vectorbtpro import *

>>> index = pd.date_range("2020", "2021", freq="D")

>>> def split_func(splits, bounds, index):
...     if len(splits) == 0:
...         new_split = (slice(0, 20), slice(20, 30))
...     else:
...         # Previous split, first set, right bound
...         prev_end = bounds[-1][0][1]
...         new_split = (
...             slice(prev_end, prev_end + 20),
...             slice(prev_end + 20, prev_end + 30)
...         )
...     if new_split[-1].stop > len(index):
...         return None
...     return new_split

>>> splitter = vbt.Splitter.from_split_func(
...     index,
...     split_func,
...     split_args=(
...         vbt.Rep("splits"),
...         vbt.Rep("bounds"),
...         vbt.Rep("index"),
...     ),
...     set_labels=["train", "test"]
... )
>>> splitter.plot().show()

```



from_splits class method

```

Splitter.from_splits(
    index,
    splits,
    squeeze=False,
    fix_ranges=True,
)

```

```

wrap_with_fixrange=False,
split_range_kwargs=None,
split_check_template=None,
template_context=None,
split_labels=None,
set_labels=None,
wrapper_kwargs=None,
**kwargs
)

```

Create a **Splitter** instance from an iterable of splits.

Argument `splits` supports both absolute and relative ranges. To transform relative ranges into the absolute format, enable `fix_ranges`. Arguments `split_range_kwargs` are then passed to **Splitter.split_range**.

Enable `wrap_with_fixrange` to wrap any fixed range with **FixRange**. If the range is an array, it will be wrapped regardless of this argument to avoid building a 3d array.

Pass a template via `split_check_template` to discard splits that do not fulfill certain criteria. The current split will be available as `split`. Should return a boolean (`False` to discard).

Labels for splits and sets can be provided via `split_labels` and `set_labels` respectively. Both arguments can be provided as templates. The split array will be available as `splits`.

get_bounds method

```

Splitter.get_bounds(
    index_bounds=False,
    right_inclusive=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)

```

Boolean Series/DataFrame where index are bounds and columns are splits stacked together.

Keyword arguments `**kwargs` are passed to **Splitter.get_bounds_arr**.

get_bounds_arr method

```

Splitter.get_bounds_arr(
    index_bounds=False,
    right_inclusive=False,
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **range_bounds_kwargs
)

```

Three-dimensional integer array with bounds.

First axis represents splits. Second axis represents sets. Third axis represents bounds.

Each range is getting selected using **Splitter.select_range** and then measured using **Splitter.get_range_bounds**. Keyword arguments `**kwargs` are passed to **Splitter.get_range_bounds**.

get_coverage method

```

Splitter.get_coverage(
    overlapping=False,
    normalize=True,
    split_group_by=None,
    set_group_by=None,
    **kwargs
)

```

Get the coverage of the entire mask.

If `overlapping` is True, returns the number of overlapping True values. If `normalize` is True, returns the number of True values relative to the length of the index. If `overlapping` and `normalize` are True, returns the number of overlapping True values relative to the total number of True values.

Keyword arguments `**kwargs` are passed to **Splitter.get_mask_arr**.

get_duration method

```
Splitter.get_duration(
    **kwargs
)
```

Get duration.

get_iter_set_mask_arrs method

```
Splitter.get_iter_set_mask_arrs(
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **kwargs
)
```

Generator of two-dimensional boolean arrays, one per set.

First axis represents splits. Second axis represents index.

Keyword arguments `**kwargs` are passed to [Splitter.get_range_mask](#).

get_iter_set_masks method

```
Splitter.get_iter_set_masks(
    split_group_by=None,
    set_group_by=None,
    **kwargs
)
```

Generator of boolean DataFrames, one per set.

Keyword arguments `**kwargs` are passed to [Splitter.get_iter_set_mask_arrs](#).

get_iter_split_mask_arrs method

```
Splitter.get_iter_split_mask_arrs(
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **kwargs
)
```

Generator of two-dimensional boolean arrays, one per split.

First axis represents sets. Second axis represents index.

Keyword arguments `**kwargs` are passed to [Splitter.get_range_mask](#).

get_iter_split_masks method

```
Splitter.get_iter_split_masks(
    split_group_by=None,
    set_group_by=None,
    **kwargs
)
```

Generator of boolean DataFrames, one per split.

Keyword arguments `**kwargs` are passed to [Splitter.get_iter_split_mask_arrs](#).

get_mask method

```
Splitter.get_mask(
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)
```

Boolean Series/DataFrame where index is [Splitter.index](#) and columns are splits stacked together.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

Warning

Boolean arrays for a big number of splits may take a considerable amount of memory.

get_mask_arr method

```
Splitter.get_mask_arr(
    split_group_by=None,
    set_group_by=None,
    template_context=None,
    **kwargs
)
```

Three-dimensional boolean array with splits.

First axis represents splits. Second axis represents sets. Third axis represents index.

Keyword arguments `**kwargs` are passed to [Splitter.get_iter_split_mask_arrs](#).

get_n_sets method

```
Splitter.get_n_sets(
    set_group_by=None
)
```

Get number of sets while considering the grouper.

get_n_splits method

```
Splitter.get_n_splits(
    split_group_by=None
)
```

Get number of splits while considering the grouper.

get_obj_index class method

```
Splitter.get_obj_index(
    obj
)
```

Get index from an object.

get_overlap_matrix method

```
Splitter.get_overlap_matrix(
    by='split',
    normalize=True,
    split_group_by=None,
    set_group_by=None,
    jitted=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)
```

Get the overlap between each pair of ranges.

The argument `by` can be one of 'split', 'set', and 'range'.

If `normalize` is True, returns the number of True values in each overlap relative to the total number of True values in both ranges.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_range_bounds class method

```
Splitter.get_range_bounds(
    range_,
    index_bounds=False,
    right_inclusive=False,
    check_constant=True,
)
```

```

        template_context=None,
        index=None,
        freq=None
    )

```

Get the left (inclusive) and right (exclusive) bound of a range.



Note

Even when mapped to the index, the right bound is always exclusive.

get_range_coverage method

```

Splitter.get_range_coverage(
    normalize=True,
    relative=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    index_combine_kwargs=None,
    **kwargs
)

```

Get the coverage of each range mask.

If `normalize` is True, returns the number of True values in each range relative to the length of the index. If `normalize` and `relative` are True, returns the number of True values in each range relative to the total number of True values in its split.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_range_mask class method

```

Splitter.get_range_mask(
    range_,
    template_context=None,
    index=None
)

```

Get the mask of a range.

get_ready_obj_range class method

```

Splitter.get_ready_obj_range(
    obj,
    range_,
    remap_to_obj=True,
    obj_index=None,
    obj_freq=None,
    template_context=None,
    jitted=None,
    silence_warnings=False,
    index=None,
    freq=None,
    return_obj_meta=False,
    **ready_range_kwargs
)

```

Get a range that is ready to be mapped into an array-like object.

If the object is Pandas-like and `obj_index` is not None, searches for an index in the object using [Splitter.get_obj_index](#). Once found, uses [Splitter.remap_range](#) to get the range that maps to the object index. Finally, uses [Splitter.get_ready_range](#) to convert the range into the one that can be used directly in indexing.

get_ready_range class method

```

Splitter.get_ready_range(
    range_,
    allow_relative=False,
    allow_zero_len=False,
    range_format='slice_or_any',
    template_context=None,
    index=None,
    return_meta=False
)

```

Get a range that can be directly used in array indexing.

Such a range is either an integer or datetime-like slice (right bound is always exclusive!), a one-dimensional NumPy array with integer indices or datetime-like objects, or a one-dimensional NumPy mask of the same length as the index.

Argument `range_format` accepts the following options:

- 'any': Return any format
- 'indices': Return indices
- 'mask': Return mask of the same length as index
- 'slice': Return slice
- 'slice_or_indices': If slice fails, return indices
- 'slice_or_mask': If slice fails, return mask
- 'slice_or_any': If slice fails, return any format

get_set_coverage method

```
Splitter.get_set_coverage(
    overlapping=False,
    normalize=True,
    relative=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_set=True,
    **kwargs
)
```

Get the coverage of each set mask.

If `overlapping` is True, returns the number of overlapping True values between splits in each set. If `normalize` is True, returns the number of True values in each set relative to the length of the index. If `normalize` and `relative` are True, returns the number of True values in each set relative to the total number of True values across all sets.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_set_grouper method

```
Splitter.get_set_grouper(
    set_group_by=None
)
```

Get set grouper.

get_set_labels method

```
Splitter.get_set_labels(
    set_group_by=None
)
```

Get set labels while considering the grouper.

get_split_coverage method

```
Splitter.get_split_coverage(
    overlapping=False,
    normalize=True,
    relative=False,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    **kwargs
)
```

Get the coverage of each split mask.

If `overlapping` is True, returns the number of overlapping True values between sets in each split. If `normalize` is True, returns the number of True values in each split relative to the length of the index. If `normalize` and `relative` are True, returns the number of True values in each split relative to the total number of True values across all splits.

Keyword arguments `**kwargs` are passed to [Splitter.get_mask_arr](#).

get_split_grouper method

```
Splitter.get_split_grouper(  
    split_group_by=None  
)
```

Get split grouper.

get_split_labels method

```
Splitter.get_split_labels(  
    split_group_by=None  
)
```

Get split labels while considering the grouper.

guess_method class method

```
Splitter.guess_method(  
    **kwargs  
)
```

Guess the factory method based on keyword arguments.

Returns None if cannot guess.

index class property

Index.

index_bounds class property

[Splitter.get_bounds](#) with `index_bounds=True`.

index_duration class property

[Splitter.get_duration](#) with `index_bounds=True`.

indexing_func method

```
Splitter.indexing_func(  
    *args,  
    splitter_meta=None,  
    **kwargs  
)
```

Perform indexing on [Splitter](#).

indexing_func_meta method

```
Splitter.indexing_func_meta(  
    *args,  
    wrapper_meta=None,  
    **kwargs  
)
```

Perform indexing on [Splitter](#) and return metadata.

is_range_relative class method

```
Splitter.is_range_relative(  
    range_  
)
```

Return whether a range is relative.

iter_set_mask_arrs class property

[Splitter.get_iter_set_mask_arrs](#) with default arguments.

`iter_set_masks` class property 

[Splitter.get_iter_set_masks](#) with default arguments.

`iter_split_mask_arrs` class property 

[Splitter.get_iter_split_mask_arrs](#) with default arguments.

`iter_split_masks` class property 

[Splitter.get_iter_split_masks](#) with default arguments.

`map_bounds_to_index` class method 

```
Splitter.map_bounds_to_index(
    start,
    stop,
    right_inclusive=False,
    index=None,
    freq=None
)
```

Map bounds to index.

`mask` class property 

[Splitter.get_mask](#) with default arguments.

`mask_arr` class property 

[Splitter.get_mask_arr](#) with default arguments.

`merge_sets` method 

```
Splitter.merge_sets(
    columns=None,
    new_set_label=None,
    insert_at_last=False,
    wrapper_kwargs=None,
    init_kwargs=None,
    **merge_split_kwargs
)
```

Merge multiple sets (columns) into a set (column).

Arguments `**merge_split_kwargs` are passed to [Splitter.merge_split](#).

If columns are not provided, merges all columns. If provided and `insert_at_last` is True, a new column is inserted at the position of the last column.

Use `new_set_label` to specify the label of the new set. To provide final labels, define `columns` in `wrapper_kwargs`.

`merge_split` class method 

```
Splitter.merge_split(
    split,
    range_format=None,
    wrap_with_template=False,
    wrap_with_fixrange=False,
    wrap_with_hslice=False,
    template_context=None,
    index=None
)
```

Merge a split of multiple fixed ranges into a fixed range.

Creates one mask and sets True for each range. If all input ranges are masks, returns that mask. If all input ranges are slices, returns a slice if possible. Otherwise, returns integer indices.

For `range_format`, see [Splitter.get_ready_range](#). Enable `wrap_with_template` to wrap the resulting range with a template of the type [Rep](#).

metrics property 

Metrics supported by [Splitter](#).

```
HybridConfig(
    start=dict(
        title='Index Start',
        calc_func=<function Splitter.<lambda> at 0x14fd10540>,
        agg_func=None,
        tags=[
            'splitter',
            'index'
        ]
    ),
    end=dict(
        title='Index End',
        calc_func=<function Splitter.<lambda> at 0x14fd105e0>,
        agg_func=None,
        tags=[
            'splitter',
            'index'
        ]
    ),
    period=dict(
        title='Index Length',
        calc_func=<function Splitter.<lambda> at 0x14fd10680>,
        agg_func=None,
        tags=[
            'splitter',
            'index'
        ]
    ),
    split_count=dict(
        title='Splits',
        calc_func='n_splits',
        agg_func=None,
        tags=[
            'splitter',
            'splits'
        ]
    ),
    set_count=dict(
        title='Sets',
        calc_func='n_sets',
        agg_func=None,
        tags=[
            'splitter',
            'splits'
        ]
    ),
    coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10720>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        calc_func='coverage',
        overlapping=False,
        post_calc_func=<function Splitter.<lambda> at 0x14fd107c0>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    set_coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10860>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        check_has_multiple_sets=True,
        calc_func='set_coverage',
        overlapping=False,
        relative=False,
        post_calc_func=<function Splitter.<lambda> at 0x14fd10900>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    set_mean_rel_coverage=dict(
        title='Mean Rel Coverage [%]',
        check_has_multiple_sets=True,
        check_normalize=True,

```

```

        calc_func='range_coverage',
        relative=True,
        post_calc_func=<function Splitter.<lambda> at 0x14fd109a0>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    overlap_coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10a40>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        calc_func='coverage',
        overlapping=True,
        post_calc_func=<function Splitter.<lambda> at 0x14fd10ae0>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    ),
    set_overlap_coverage=dict(
        title=RepFunc(
            template=<function Splitter.<lambda> at 0x14fd10b80>,
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        check_has_multiple_sets=True,
        calc_func='set_coverage',
        overlapping=True,
        post_calc_func=<function Splitter.<lambda> at 0x14fd10c20>,
        agg_func=None,
        tags=[
            'splitter',
            'splits',
            'coverage'
        ]
    )
)

```

Returns `Splitter._metrics`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change metrics, you can either change the config in-place, override this property, or overwrite the instance variable `Splitter._metrics`.

n_sets class property ⓘ

Number of sets.

n_splits class property ⓘ

Number of splits.

parse_and_inject_takeables class method ⓘ

```

Splitter.parse_and_inject_takeables(
    flat_ann_args,
    eval_id=None
)

```

Parse **Takeable** instances from function annotations and inject them into flattened annotated arguments.

plot method ⓘ

```

Splitter.plot(
    split_group_by=None,
    set_group_by=None,
    mask_kwargs=None,
    trace_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    **layout_kwargs
)

```

Plot splits as rows and sets as colors.

Args`split_group_by` : anySplit groups. See [BaseIDXAccessor.get_grouper](#).`set_group_by` : anySet groups. See [BaseIDXAccessor.get_grouper](#).`mask_kwargs` : dictKeyword arguments passed to [Splitter.get_iter_set_masks](#).`trace_kwargs` : dictKeyword arguments passed to `plotly.graph_objects.Heatmap`.

Can be a sequence, one per set.

`add_trace_kwargs` : dictKeyword arguments passed to `add_trace`.`fig` : Figure or FigureWidget

Figure to add traces to.

`**layout_kwargs`

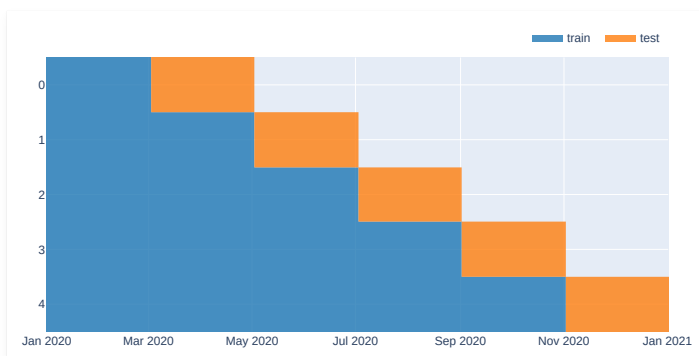
Keyword arguments for layout.

Usage

- Plot a scikit-learn splitter:

```
>>> from vectorbtpro import *
>>> from sklearn.model_selection import TimeSeriesSplit

>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_sklearn(index, TimeSeriesSplit())
>>> splitter.plot().show()
```

**plot_coverage** method

```
Splitter.plot_coverage(
    stacked=True,
    split_group_by=None,
    set_group_by=None,
    mask_kwargs=None,
    trace_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    **layout_kwargs
)
```

Plot index as rows and sets as lines.

Args`stacked` : bool

Whether to plot as an area plot.

`split_group_by` : any

Split groups. See [BaseIDXAccessor.get_grouper](#).

`set_group_by` : any

Set groups. See [BaseIDXAccessor.get_grouper](#).

`mask_kwargs` : dict

Keyword arguments passed to [Splitter.get_iter_set_masks](#).

`trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter`.

Can be a sequence, one per set.

`add_trace_kwargs` : dict

Keyword arguments passed to `add_trace`.

`fig` : Figure or FigureWidget

Figure to add traces to.

`**layout_kwargs`

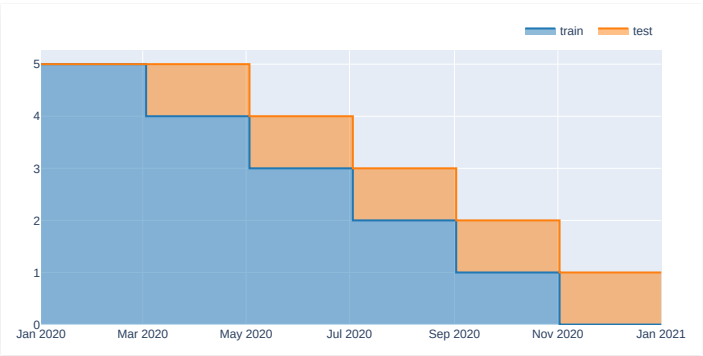
Keyword arguments for layout.

Usage

- Area plot:

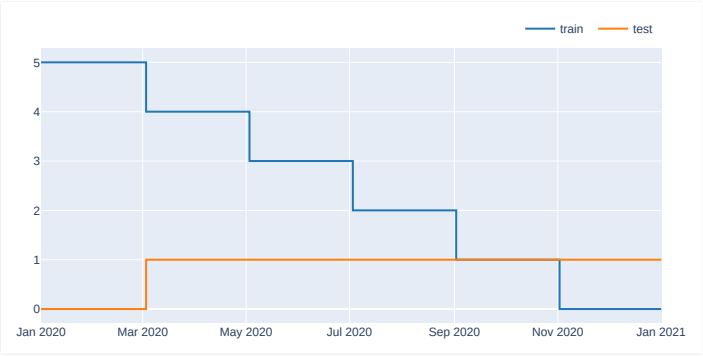
```
>>> from vectorbtpro import *
>>> from sklearn.model_selection import TimeSeriesSplit

>>> index = pd.date_range("2020", "2021", freq="D")
>>> splitter = vbt.Splitter.from_sklearn(index, TimeSeriesSplit())
>>> splitter.plot_coverage().show()
```



- Line plot:

```
>>> splitter.plot_coverage(stacked=False).show()
```



`plots_defaults` class property

Defaults for [PlotsBuilderMixin.plots](#).

Merges [PlotsBuilderMixin.plots_defaults](#) and `plots` from [splitter](#).

range_coverage class property 

[Splitter.get_range_coverage](#) with default arguments.

range_overlap_matrix class property 

[Splitter.get_overlap_matrix](#) with `by="range"`.

remap_range class method 

```
Splitter.remap_range(
    range_,
    target_index,
    target_freq=None,
    template_context=None,
    jitted=None,
    silence_warnings=False,
    index=None,
    freq=None
)
```

Remap a range to a target index.

If `index` and `target_index` are the same, returns the range. Otherwise, uses [Resampler.resample_source_mask](#) to resample the range into the target index. In such a case, `freq` and `target_freq` must be provided.

resolve_column_stack_kwargs class method 

```
Splitter.resolve_column_stack_kwargs(
    *objs,
    reindex_kwargs=None,
    **kwargs
)
```

Resolve keyword arguments for initializing [Splitter](#) after stacking along columns.

resolve_row_stack_kwargs class method 

```
Splitter.resolve_row_stack_kwargs(
    *objs,
    **kwargs
)
```

Resolve keyword arguments for initializing [Splitter](#) after stacking along rows.

row_stack class method 

```
Splitter.row_stack(
    *objs,
    wrapper_kwargs=None,
    **kwargs
)
```

Stack multiple [Splitter](#) instances along rows.

Uses [ArrayWrapper.row_stack](#) to stack the wrappers.

select_indices method 

```
Splitter.select_indices(
    split=None,
    set=None,
    split_group_by=None,
    set_group_by=None
)
```

Get indices corresponding to selected splits and sets.

Arguments `split` and `set_` can be either integers and labels. Also, multiple values are accepted; in such a case, the corresponding ranges are merged. If split/set labels are of an integer data type, treats the provided values as labels rather than indices, unless the split/set index is not of an integer data type or the values are wrapped with [PosSel](#).

If `split_group_by` and/or `set_group_by` are provided, their groupers get created using [BaseIDXAccessor.get_grouper](#) and arguments `split` and `set_` become relative to the groups.

If `split` / `set_` is not provided, selects all indices.

Returns four arrays: split group indices, set group indices, split indices, and set indices.

select_range method

```
Splitter.select_range(
    merge_split_kwargs=None,
    **select_indices_kwargs
)
```

Select a range.

Passes `**select_indices_kwargs` to [Splitter.select_indices](#) to get the indices for selected splits and sets. If multiple ranges correspond to those indices, merges them using [Splitter.merge_split](#).

set_coverage class property

[Splitter.get_set_coverage](#) with default arguments.

set_labels class property

Set labels.

set_overlap_matrix class property

[Splitter.get_overlap_matrix](#) with `by="set"`.

shuffle_splits method

```
Splitter.shuffle_splits(
    size=None,
    replace=False,
    p=None,
    seed=None,
    wrapper_kwargs=None,
    **init_kwargs
)
```

Shuffle splits.

split_and_apply class method

```
Splitter.split_and_apply(
    index,
    apply_func,
    *apply_args,
    splitter=None,
    splitter_kwargs=None,
    apply_kwargs=None,
    template_context=None,
    **var_kwargs
)
```

Split an index and apply a function.

Argument `splitter` can be an actual [Splitter](#) instance, the name of a factory method (such as "from_n_rolling"), or the factory method itself. If `splitter` is None, the right method will be guessed based on the supplied arguments using [Splitter.guess_method](#).

Keyword arguments `splitter_kwargs` are passed to the factory method. Keyword arguments `apply_kwargs` are passed to [Splitter.apply](#). If variable keyword arguments are provided, they will be used as `apply_kwargs` if a splitter instance has been built, otherwise, arguments will be distributed based on the signatures of the factory method and [Splitter.apply](#).

split_and_take class method

```

Splitter.split_and_take(
    index,
    obj,
    splitter=None,
    splitter_kwargs=None,
    take_kwargs=None,
    template_context=None,
    **var_kwargs
)

```

Split an index and take from an object.

Argument `splitter` can be an actual [Splitter](#) instance, the name of a factory method (such as "from_n_rolling"), or the factory method itself. If `splitter` is None, the right method will be guessed based on the supplied arguments using [Splitter.guess_method](#).

Keyword arguments `splitter_kwargs` are passed to the factory method. Keyword arguments `take_kwargs` are passed to [Splitter.take](#). If variable keyword arguments are provided, they will be used as `take_kwargs` if a splitter instance has been built, otherwise, arguments will be distributed based on the signatures of the factory method and [Splitter.take](#).

split_coverage class property

[Splitter.get_split_coverage](#) with default arguments.

split_labels class property

Split labels.

split_overlap_matrix class property

[Splitter.get_overlap_matrix](#) with `by="split"`.

split_range class method

```

Splitter.split_range(
    range_,
    new_split,
    backwards=False,
    allow_zero_len=False,
    range_format=None,
    wrap_with_template=False,
    wrap_with_fixrange=False,
    template_context=None,
    index=None,
    freq=None
)

```

Split a fixed range into a split of multiple fixed ranges.

Range must be either a template, a callable, a tuple (start and stop), a slice, a sequence of indices, or a mask. This range will then be re-mapped into the index.

Each sub-range in `new_split` can be either a fixed or relative range, that is, an instance of [RelRange](#) or a number that will be used as a length to create an [RelRange](#). Each sub-range will then be re-mapped into the main range. Argument `new_split` can also be provided as an integer or a float indicating the length; in such a case the second part (or the first one depending on `backwards`) will stretch. If `new_split` is a string, the following options are supported:

- 'by_gap': Split `range_` by gap using [split_range_by_gap_nb](#).

New ranges are returned relative to the index and in the same order as passed.

For `range_format`, see [Splitter.get_ready_range](#). Enable `wrap_with_template` to wrap the resulting ranges with a template of the type [Rep](#).

split_set method

```

Splitter.split_set(
    new_split,
    column=None,
    new_set_labels=None,
    wrapper_kwargs=None,
    init_kwargs=None,
    **split_range_kwargs
)

```

Split a set (column) into multiple sets (columns).

Arguments `new_split` and `**split_range_kwargs` are passed to [Splitter.split_range](#).

Column must be provided if there are two or more sets.

Use `new_set_labels` to specify the labels of the new sets; it must have the same length as there are new ranges in the new split. To provide final labels, define `columns` in `wrapper_kwargs`.

splits class property 

[Splitter.splits_arr](#) as a DataFrame.

splits_arr class property 

Two-dimensional, object-dtype DataFrame with splits.

First axis represents splits. Second axis represents sets. Elements represent ranges. Range must be either a slice, a sequence of indices, a mask, or a callable that returns such.

stats_defaults class property 

Defaults for [StatsBuilderMixin.stats](#).

Merges [StatsBuilderMixin.stats_defaults](#) and `stats` from [splitter](#).

subplots property 

Subplots supported by [Splitter](#).

```
HybridConfig(
    plot=dict(
        title='Splits',
        yaxis_kwargs=dict(
            title='Split'
        ),
        plot_func='plot',
        tags='splitter'
    ),
    plot_coverage=dict(
        title='Coverage',
        yaxis_kwargs=dict(
            title='Count'
        ),
        plot_func='plot_coverage',
        tags='splitter'
    )
)
```

Returns `Splitter._subplots`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change subplots, you can either change the config in-place, override this property, or overwrite the instance variable `Splitter._subplots`.

take method 

```
Splitter.take(
    obj,
    split=None,
    set=None,
    split_group_by=None,
    set_group_by=None,
    squeeze_one_split=True,
    squeeze_one_set=True,
    into=None,
    remap_to_obj=True,
    obj_index=None,
    obj_freq=None,
    range_format='slice_or_any',
    point_wise=False,
    attach_bounds=False,
    right_inclusive=False,
    template_context=None,
    silence_warnings=False,
    index_combine_kwargs=None,
    stack_axis=1,
    stack_kwargs=None,
    freq=None
)
```

Take all ranges from an array-like object and optionally column-stack them.

Uses `Splitter.select_indices` to get the indices for selected splits and sets. Arguments `split_group_by` and `set_group_by` can be used to group splits and sets respectively. Ranges belonging to the same split and set group will be merged.

For each index pair, resolves the source range using `Splitter.select_range` and `Splitter.get_ready_range`. Then, remaps this range into the object index using `Splitter.get_ready_obj_range` and takes the slice from the object using `Splitter.take_range`. If the object is a custom template, substitutes its instead of calling `Splitter.take_range`. Finally, uses `column_stack_merge` (`stack_axis=1`) or `row_stack_merge` (`stack_axis=0`) with `stack_kwargs` to merge the taken slices.

If `attach_bounds` is enabled, measures the bounds of each range and makes it an additional level in the final index hierarchy. The argument supports the following options:

- True, 'index', 'source', or 'source_index': Attach source (index) bounds
- 'target' or 'target_index': Attach target (index) bounds
- False: Do not attach

Argument `into` supports the following options:

- None: Series of range slices
- 'stacked': Stack all slices into a single object
- 'stacked_by_split': Stack set slices in each split and return a Series of objects
- 'stacked_by_set': Stack split slices in each set and return a Series of objects
- 'split_major_meta': Generator with ranges processed lazily in split-major order. Returns meta with indices and labels, and the generator.
- 'set_major_meta': Generator with ranges processed lazily in set-major order. Returns meta with indices and labels, and the generator.

Prepend any stacked option with "from_start_" (also "reset_") or "from_end_" to reset the index from start and from end respectively.

Usage

- Roll a window and stack it along columns by keeping the index:

```
>>> from vectorbtpro import *

>>> data = vbt.YFData.pull(
...     "BTC-USD",
...     start="2020-01-01 UTC",
...     end="2021-01-01 UTC"
... )
>>> splitter = vbt.Splitter.from_n_rolling(
...     data.wrapper.index,
...     3,
...     length=5
... )
>>> splitter.take(data.close, into="stacked")
split
Date
2020-01-01 00:00:00+00:00    7200.174316    NaN    NaN
2020-01-02 00:00:00+00:00    6985.470215    NaN    NaN
2020-01-03 00:00:00+00:00    7344.884277    NaN    NaN
2020-01-04 00:00:00+00:00    7410.656738    NaN    NaN
2020-01-05 00:00:00+00:00    7411.317383    NaN    NaN
2020-06-29 00:00:00+00:00         NaN    9190.854492    NaN
2020-06-30 00:00:00+00:00         NaN    9137.993164    NaN
2020-07-01 00:00:00+00:00         NaN    9228.325195    NaN
2020-07-02 00:00:00+00:00         NaN    9123.410156    NaN
2020-07-03 00:00:00+00:00         NaN    9087.303711    NaN
2020-12-27 00:00:00+00:00         NaN         NaN    26272.294922
2020-12-28 00:00:00+00:00         NaN         NaN    27084.808594
2020-12-29 00:00:00+00:00         NaN         NaN    27362.437500
2020-12-30 00:00:00+00:00         NaN         NaN    28840.953125
2020-12-31 00:00:00+00:00         NaN         NaN    29001.720703
```

- Disgard the index and attach index bounds to the column hierarchy:

```
>>> splitter.take(
...     data.close,
...     into="reset_stacked",
...     attach_bounds="index"
... )
split
start 2020-01-01 00:00:00+00:00 2020-06-29 00:00:00+00:00
end 2020-01-06 00:00:00+00:00 2020-07-04 00:00:00+00:00
0
7200.174316    9190.854492
1
6985.470215    9137.993164
2
7344.884277    9228.325195
3
7410.656738    9123.410156
4
7411.317383    9087.303711

split
2
start 2020-12-27 00:00:00+00:00
end 2021-01-01 00:00:00+00:00
0
26272.294922
1
27084.808594
2
27362.437500
3
28840.953125
4
29001.720703
```

take_range class method

```

Splitter.take_range(
    obj,
    ready_range,
    point_wise=False
)

```

Take a ready range from an array-like object.

Set `point_wise` to True to select one range point at a time and return a tuple.

take_range_from_takeable class method

```

Splitter.take_range_from_takeable(
    takeable,
    range_,
    remap_to_obj=True,
    obj_index=None,
    obj_freq=None,
    point_wise=False,
    template_context=None,
    return_obj_meta=False,
    return_meta=False,
    **ready_obj_range_kwargs
)

```

Take a range from a takeable object.

to_fixed method

```

Splitter.to_fixed(
    split_range_kwargs=None,
    **kwargs
)

```

Convert relative ranges into fixed ones and return a new [Splitter](#) instance.

Keyword arguments `split_range_kwargs` are passed to [Splitter.split_range](#).

to_grouped method

```

Splitter.to_grouped(
    split=None,
    set_=None,
    split_group_by=None,
    set_group_by=None,
    merge_split_kwargs=None,
    **kwargs
)

```

Merge all ranges within the same group and return a new [Splitter](#) instance.

Takeable class

```

Takeable(
    *args,
    **kwargs
)

```

Class that represents an object from which a range can be taken.

Superclasses

- [Annotatable](#)
- [Base](#)
- [DefineMixin](#)
- [Evaluable](#)
- [Hashable](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)

- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [DefineMixin.asdict](#)
- [DefineMixin.assert_field_not_missing](#)
- [DefineMixin.fields](#)
- [DefineMixin.fields_dict](#)
- [DefineMixin.get_field](#)
- [DefineMixin.hash](#)
- [DefineMixin.hash_key](#)
- [DefineMixin.is_field_missing](#)
- [DefineMixin.is_field_optional](#)
- [DefineMixin.is_field_required](#)
- [DefineMixin.merge_over](#)
- [DefineMixin.merge_with](#)
- [DefineMixin.replace](#)
- [DefineMixin.resolve](#)
- [DefineMixin.resolve_field](#)
- [Evaluable.meets_eval_id](#)
- [Hashable.get_hash](#)

eval_id field 

One or more identifiers at which to evaluate this instance.

freq field 

Frequency of [Takeable.index](#).

index field 

Index of the object.

If not present, will be accessed using [Splitter.get_obj_index](#).

obj field 

Takeable object.

point_wise field 

Whether to select one range point at a time and return a tuple.

remap_to_obj field 

Whether to remap [Splitter.index](#) to the index of [Takeable.obj](#).

Otherwise, will assume that the object has the same index.

ZeroLengthError class 

```
ZeroLengthError(  
    *args,  
    **kwargs  
)
```



Thrown whenever a range has a length of zero.

Superclasses

- `builtins.BaseException`
- `builtins.Exception`
- `builtins.ValueError`

API generic

builders stats_builder

stats_builder module

Mixin for building statistics out of performance metrics.

MetaStatsBuilderMixin class

```
MetaStatsBuilderMixin(
    *args,
    **kwargs
)
```

Meta class that exposes a read-only class property [StatsBuilderMixin.metrics](#).

Superclasses

- builtins.type

Subclasses

- [MetaAnalyzable](#)

metrics class property

Metrics supported by [StatsBuilderMixin.stats](#).

StatsBuilderMixin class

```
StatsBuilderMixin()
```

Mixin that implements [StatsBuilderMixin.stats](#).

Required to be a subclass of [Wrapping](#).

Superclasses

- [Base](#)

Inherited members

- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)

Subclasses

- [Analyzable](#)

build_metrics_doc class method

```
StatsBuilderMixin.build_metrics_doc(
    source_cls=None
)
```

Build metrics documentation.

metrics property

Metrics supported by [StatsBuilderMixin](#).

```

HybridConfig(
  start_index=dict(
    title='Start Index',
    calc_func=<function StatsBuilderMixin.<lambda> at 0x12318cea0>,
    agg_func=None,
    tags='wrapper'
  ),
  end_index=dict(
    title='End Index',
    calc_func=<function StatsBuilderMixin.<lambda> at 0x12318cf40>,
    agg_func=None,
    tags='wrapper'
  ),
  total_duration=dict(
    title='Total Duration',
    calc_func=<function StatsBuilderMixin.<lambda> at 0x12318cfe0>,
    apply_to_timedelta=True,
    agg_func=None,
    tags='wrapper'
  )
)

```

Returns `StatsBuilderMixin._metrics`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change metrics, you can either change the config in-place, override this property, or overwrite the instance variable `StatsBuilderMixin._metrics`.

override_metrics_doc class method

```

StatsBuilderMixin.override_metrics_doc(
    __pdoc__,
    source_cls=None
)

```

Call this method on each subclass that overrides `StatsBuilderMixin.metrics`.

resolve_stats_setting method

```

StatsBuilderMixin.resolve_stats_setting(
    value,
    key,
    merge=False
)

```

Resolve a setting for `StatsBuilderMixin.stats`.

stats method

```

StatsBuilderMixin.stats(
    metrics=None,
    tags=None,
    column=None,
    group_by=None,
    per_column=None,
    split_columns=None,
    agg_func=<function mean>,
    dropna=None,
    silence_warnings=None,
    template_context=None,
    settings=None,
    filters=None,
    metric_settings=None
)

```

Compute various metrics on this object.

Args

metrics : str, tuple, iterable, or dict

Metrics to calculate.

Each element can be either:

- Metric name (see keys in `StatsBuilderMixin.metrics`)
- Tuple of a metric name and a settings dict as in `StatsBuilderMixin.metrics`
- Tuple of a metric name and a template of instance `CustomTemplate`
- Tuple of a metric name and a list of settings dicts to be expanded into multiple metrics

The settings dict can contain the following keys:

- **title**: Title of the metric. Defaults to the name.

- `tags`: Single or multiple tags to associate this metric with. If any of these tags is in `tags`, keeps this metric.
- `check_{filter}` and `inv_check_{filter}`: Whether to check this metric against a filter defined in `filters`. True (or False for inverse) means to keep this metric.
- `calc_func` (required): Calculation function for custom metrics. Must return either a scalar for one column/group, pd.Series for multiple columns/groups, or a dict of such for multiple sub-metrics.
- `resolve_calc_func`: whether to resolve `calc_func`. If the function can be accessed by traversing attributes of this object, you can specify the path to this function as a string (see [deep_getattr](#) for the path format). If `calc_func` is a function, arguments from merged metric settings are matched with arguments in the signature (see below). If `resolve_calc_func` is False, `calc_func` must accept (resolved) self and dictionary of merged metric settings. Defaults to True.
- `use_shortcuts`: Whether to use shortcut properties whenever possible when resolving `calc_func`. Defaults to True.
- `post_calc_func`: Function to post-process the result of `calc_func`. Must accept (resolved) self, output of `calc_func`, and dictionary of merged metric settings, and return whatever is acceptable to be returned by `calc_func`. Defaults to None.
- `fill_wrap_kwargs`: Whether to fill `wrap_kwargs` with `to_timedelta` and `silence_warnings`. Defaults to False.
- `apply_to_timedelta`: Whether to apply [ArrayWrapper.arr_to_timedelta](#) on the result. To disable this globally, pass `to_timedelta=False` in `settings`. Defaults to False.
- `pass_{arg}`: Whether to pass any argument from the settings (see below). Defaults to True if this argument was found in the function's signature. Set to False to not pass. If argument to be passed was not found, `pass_{arg}` is removed.
- `resolve_path_{arg}`: Whether to resolve an argument that is meant to be an attribute of this object and is the first part of the path of `calc_func`. Passes only optional arguments. Defaults to True. See [AttrResolverMixin.resolve_attr](#).
- `resolve_{arg}`: Whether to resolve an argument that is meant to be an attribute of this object and is present in the function's signature. Defaults to False. See [AttrResolverMixin.resolve_attr](#).
- `use_shortcuts_{arg}`: Whether to use shortcut properties whenever possible when resolving an argument. Defaults to True.
- `select_col_{arg}`: Whether to select the column from an argument that is meant to be an attribute of this object. Defaults to False.
- `template_context`: Mapping to replace templates in metric settings. Used across all settings.
- Any other keyword argument that overrides the settings or is passed directly to `calc_func`.

If `resolve_calc_func` is True, the calculation function may "request" any of the following arguments by accepting them or if `pass_{arg}` was found in the settings dict:

- Each of [AttrResolverMixin.self_aliases](#): original object (ungrouped, with no column selected)
- `group_by`: won't be passed if it was used in resolving the first attribute of `calc_func` specified as a path, use `pass_group_by=True` to pass anyway
- `column`
- `metric_name`
- `agg_func`
- `silence_warnings`
- `to_timedelta`: replaced by True if None and frequency is set
- Any argument from `settings`
- Any attribute of this object if it meant to be resolved (see [AttrResolverMixin.resolve_attr](#))

Pass `metrics='all'` to calculate all supported metrics.

`tags`: str or iterable

Tags to select.

See [match_tags](#).

`column`: str

Name of the column/group.

Hint

There are two ways to select a column: `obj['a'].stats()` and `obj.stats(column='a')`. They both accomplish the same thing but in different ways: `obj['a'].stats()` computes statistics of the column 'a' only, while `obj.stats(column='a')` computes statistics of all columns first and only then selects the column 'a'. The first method is preferred when you have a lot of data or caching is disabled. The second method is preferred when most attributes have already been cached.

`group_by`: any

Group or ungroup columns. See [Grouper](#).

`per_column`: bool

Whether to compute per column and then stack along columns.

`split_columns`: bool

Whether to split this instance into multiple columns when `per_column` is True.

Otherwise, iterates over columns and passes `column` to the whole instance.

`agg_func` : callable

Aggregation function to aggregate statistics across all columns. By default, takes the mean of all columns. If None, returns all columns as a DataFrame.

Must take `pd.Series` and return a const.

Takes effect if `column` was specified or this object contains only one column of data.

If `agg_func` has been overridden by a metric:

- Takes effect if global `agg_func` is not None
- Raises a warning if it's None but the result of calculation has multiple values

`dropna` : bool

Whether to hide metrics that are all NaN.

`silence_warnings` : bool

Whether to silence all warnings.

`template_context` : mapping

Context used to substitute templates.

Gets merged over `template_context` from `stats_builder` and `StatsBuilderMixin.stats_defaults`.

Applied on `settings` and then on each metric settings.

`filters` : dict

Filters to apply.

Each item consists of the filter name and settings dict.

The settings dict can contain the following keys:

- `filter_func` : Filter function that must accept resolved self and merged settings for a metric, and return either True or False.
- `warning_message` : Warning message to be shown when skipping a metric. Can be a template that will be substituted using merged metric settings as context. Defaults to None.
- `inv_warning_message` : Same as `warning_message` but for inverse checks.

Gets merged over `filters` from `stats_builder` and `StatsBuilderMixin.stats_defaults`.

`settings` : dict

Global settings and resolution arguments.

Extends/overrides `settings` from `stats_builder` and `StatsBuilderMixin.stats_defaults`. Gets extended/overridden by metric settings.

`metric_settings` : dict

Keyword arguments for each metric.

Extends/overrides all global and metric settings.

For template logic, see [vectorbtpro.utils.template](#).

For defaults, see `stats_builder` and `StatsBuilderMixin.stats_defaults`.

Hint

There are two types of arguments: optional (or resolution) and mandatory arguments. Optional arguments are only passed if they are found in the function's signature. Mandatory arguments are passed regardless of this. Optional arguments can only be defined using `settings` (that is, globally), while mandatory arguments can be defined both using default metric settings and `{metric_name}_kwargs`. Overriding optional arguments using default metric settings or `{metric_name}_kwargs` won't turn them into mandatory. For this, pass `pass_(arg)=True`.

Hint

Make sure to resolve and then to re-use as many object attributes as possible to utilize built-in caching (even if global caching is disabled).

`stats_defaults` class property 

Defaults for `StatsBuilderMixin.stats`.

API portfolio

records

trades

trades module

Base class for working with trade records.

Trade records capture information on trades.

In vectorbt, a trade is a sequence of orders that starts with an opening order and optionally ends with a closing order. Every pair of opposite orders can be represented by a trade. Each trade has a PnL info attached to quickly assess its performance. An interesting effect of this representation is the ability to aggregate trades: if two or more trades are happening one after another in time, they can be aggregated into a bigger trade. This way, for example, single-order trades can be aggregated into positions; but also multiple positions can be aggregated into a single blob that reflects the performance of the entire symbol.

Warning

All classes return both closed AND open trades/positions, which may skew your performance results. To only consider closed trades/positions, you should explicitly query the `status_closed` attribute.

Trade types

There are three main types of trades.

Entry trades

An entry trade is created from each order that opens or adds to a position.

For example, if we have a single large buy order and 100 smaller sell orders, we will see a single trade with the entry information copied from the buy order and the exit information being a size-weighted average over the exit information of all sell orders. On the other hand, if we have 100 smaller buy orders and a single sell order, we will see 100 trades, each with the entry information copied from the buy order and the exit information being a size-based fraction of the exit information of the sell order.

Use [EntryTrades.from_orders](#) to build entry trades from orders. Also available as [Portfolio.entry_trades](#).

Exit trades

An exit trade is created from each order that closes or removes from a position.

Use [ExitTrades.from_orders](#) to build exit trades from orders. Also available as [Portfolio.exit_trades](#).

Positions

A position is created from a sequence of entry or exit trades.

Use [Positions.from_trades](#) to build positions from entry or exit trades. Also available as [Portfolio.positions](#).

Example

- Increasing position:

```
>>> from vectorbtpro import *

>>> # Entry trades
>>> pf_kwargs = dict(
...     close=pd.Series([1., 2., 3., 4., 5.]),
...     size=pd.Series([1., 1., 1., 1., -4.]),
...     fixed_fees=1.
... )
>>> entry_trades = vbt.Portfolio.from_orders(**pf_kwargs).entry_trades
>>> entry_trades.readable
```

Entry Trade Id	Column	Size	Entry Order Id	Entry Index	Avg Entry Price	\
0	0	1.0	0	0	1.0	
1	1	0	1	1	2.0	
2	2	0	2	2	3.0	
3	3	0	3	3	4.0	

Entry Fees	Exit Order Id	Exit Index	Avg Exit Price	Exit Fees	PnL	\
0	1.0	4	4	5.0	0.25	2.75
1	1.0	4	4	5.0	0.25	1.75
2	1.0	4	4	5.0	0.25	0.75
3	1.0	4	4	5.0	0.25	-0.25

Return	Direction	Status	Position Id
0	2.7500	Long Closed	0
1	0.8750	Long Closed	0
2	0.2500	Long Closed	0
3	-0.0625	Long Closed	0

```
>>> # Exit trades
>>> exit_trades = vbt.Portfolio.from_orders(**pf_kwargs).exit_trades
>>> exit_trades.readable
Exit Trade Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0              0      0  4.0              0          0          2.5

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          4.0          4          4          5.0      1.0  5.0

Return Direction  Status  Position Id
0          0.5      Long  Closed          0

>>> # Positions
>>> positions = vbt.Portfolio.from_orders(**pf_kwargs).positions
>>> positions.readable
Position Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0              0      0  4.0              0          0          2.5

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          4.0          4          4          5.0      1.0  5.0

Return Direction  Status
0          0.5      Long  Closed

>>> entry_trades.pnl.sum() == exit_trades.pnl.sum() == positions.pnl.sum()
True
```

- Decreasing position:

```
>>> # Entry trades
>>> pf_kwargs = dict(
...     close=pd.Series([1., 2., 3., 4., 5.]),
...     size=pd.Series([4., -1., -1., -1., -1.]),
...     fixed_fees=1.
... )
>>> entry_trades = vbt.Portfolio.from_orders(**pf_kwargs).entry_trades
>>> entry_trades.readable
Entry Trade Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0              0      0  4.0              0          0          1.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          1.0          4          4          3.5      4.0  5.0

Return Direction  Status  Position Id
0          1.25      Long  Closed          0

>>> # Exit trades
>>> exit_trades = vbt.Portfolio.from_orders(**pf_kwargs).exit_trades
>>> exit_trades.readable
Exit Trade Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0              0      0  1.0              0          0          1.0
1              1      0  1.0              0          0          1.0
2              2      0  1.0              0          0          1.0
3              3      0  1.0              0          0          1.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          0.25          1          1          2.0      1.0 -0.25
1          0.25          2          2          3.0      1.0  0.75
2          0.25          3          3          4.0      1.0  1.75
3          0.25          4          4          5.0      1.0  2.75

Return Direction  Status  Position Id
0          -0.25      Long  Closed          0
1           0.75      Long  Closed          0
2           1.75      Long  Closed          0
3           2.75      Long  Closed          0

>>> # Positions
>>> positions = vbt.Portfolio.from_orders(**pf_kwargs).positions
>>> positions.readable
Position Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0              0      0  4.0              0          0          1.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          1.0          4          4          3.5      4.0  5.0

Return Direction  Status
0          1.25      Long  Closed

>>> entry_trades.pnl.sum() == exit_trades.pnl.sum() == positions.pnl.sum()
True
```

- Multiple reversing positions:

```
>>> # Entry trades
>>> pf_kwargs = dict(
...     close=pd.Series([1., 2., 3., 4., 5.]),
...     size=pd.Series([1., -2., 2., -2., 1.]),
...     fixed_fees=1.
... )
>>> entry_trades = vbt.Portfolio.from_orders(**pf_kwargs).entry_trades
>>> entry_trades.readable
Entry Trade Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0              0      0  1.0              0          0          1.0
```

```

1          1      0  1.0          1          1          2.0
2          2      0  1.0          2          2          3.0
3          3      0  1.0          3          3          4.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          1.0          1          1          2.0          0.5 -0.5
1          0.5          2          2          3.0          0.5 -2.0
2          0.5          3          3          4.0          0.5  0.0
3          0.5          4          4          5.0          1.0 -2.5

Return Direction  Status  Position Id
0 -0.500          Long  Closed          0
1 -1.000          Short Closed          1
2  0.000          Long  Closed          2
3 -0.625          Short Closed          3

>>> # Exit trades
>>> exit_trades = vbt.Portfolio.from_orders(**pf_kwargs).exit_trades
>>> exit_trades.readable
Exit Trade Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0          0      0  1.0          0          0          1.0
1          1      0  1.0          1          1          2.0
2          2      0  1.0          2          2          3.0
3          3      0  1.0          3          3          4.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          1.0          1          1          2.0          0.5 -0.5
1          0.5          2          2          3.0          0.5 -2.0
2          0.5          3          3          4.0          0.5  0.0
3          0.5          4          4          5.0          1.0 -2.5

Return Direction  Status  Position Id
0 -0.500          Long  Closed          0
1 -1.000          Short Closed          1
2  0.000          Long  Closed          2
3 -0.625          Short Closed          3

>>> # Positions
>>> positions = vbt.Portfolio.from_orders(**pf_kwargs).positions
>>> positions.readable
Position Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0          0      0  1.0          0          0          1.0
1          1      0  1.0          1          1          2.0
2          2      0  1.0          2          2          3.0
3          3      0  1.0          3          3          4.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          1.0          1          1          2.0          0.5 -0.5
1          0.5          2          2          3.0          0.5 -2.0
2          0.5          3          3          4.0          0.5  0.0
3          0.5          4          4          5.0          1.0 -2.5

Return Direction  Status  Position Id
0 -0.500          Long  Closed          0
1 -1.000          Short Closed          1
2  0.000          Long  Closed          2
3 -0.625          Short Closed          3

>>> entry_trades.pnl.sum() == exit_trades.pnl.sum() == positions.pnl.sum()
True

```

- Open position:

```

>>> # Entry trades
>>> pf_kwargs = dict(
...     close=pd.Series([1., 2., 3., 4., 5.]),
...     size=pd.Series([1., 0., 0., 0., 0.]),
...     fixed_fees=1.
... )
>>> entry_trades = vbt.Portfolio.from_orders(**pf_kwargs).entry_trades
>>> entry_trades.readable
Entry Trade Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0          0      0  1.0          0          0          1.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          1.0          -1          4          5.0          0.0  3.0

Return Direction  Status  Position Id
0          3.0          Long  Open          0

>>> # Exit trades
>>> exit_trades = vbt.Portfolio.from_orders(**pf_kwargs).exit_trades
>>> exit_trades.readable
Exit Trade Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \
0          0      0  1.0          0          0          1.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL  \
0          1.0          -1          4          5.0          0.0  3.0

Return Direction  Status  Position Id
0          3.0          Long  Open          0

>>> # Positions
>>> positions = vbt.Portfolio.from_orders(**pf_kwargs).positions
>>> positions.readable
Position Id  Column  Size  Entry Order Id  Entry Index  Avg Entry Price  \

```



```

0      0      0  1.0      0      0      1.0

Entry Fees  Exit Order Id  Exit Index  Avg Exit Price  Exit Fees  PnL \
0      1.0      -1      4      5.0      0.0  3.0

Return Direction Status
0      3.0      Long  Open

>>> entry_trades.pnl.sum() == exit_trades.pnl.sum() == positions.pnl.sum()
True

```

Get trade count, trade PnL, and winning trade PnL:

```

>>> price = pd.Series([1., 2., 3., 4., 3., 2., 1.])
>>> size = pd.Series([1., -0.5, -0.5, 2., -0.5, -0.5, -0.5])
>>> trades = vbt.Portfolio.from_orders(price, size).trades

>>> trades.count()
6

>>> trades.pnl.sum()
-3.0

>>> trades.winning.count()
2

>>> trades.winning.pnl.sum()
1.5

```

Get count and PnL of trades with duration of more than 2 days:

```

>>> mask = (trades.records['exit_idx'] - trades.records['entry_idx']) > 2
>>> trades_filtered = trades.apply_mask(mask)
>>> trades_filtered.count()
2

>>> trades_filtered.pnl.sum()
-3.0

```

Stats

Hint

See [StatsBuilderMixin.stats](#) and [Trades.metrics](#).

```

>>> price = vbt.RandomData.pull(
...     ['a', 'b'],
...     start=datetime(2020, 1, 1),
...     end=datetime(2020, 3, 1),
...     seed=vbt.symbol_dict(a=42, b=43)
... ).get()

```

100%

```

>>> size = pd.DataFrame({
...     'a': np.random.randint(-1, 2, size=len(price.index)),
...     'b': np.random.randint(-1, 2, size=len(price.index)),
... }, index=price.index, columns=price.columns)
>>> pf = vbt.Portfolio.from_orders(price, size, fees=0.01, init_cash="auto")

>>> pf.trades['a'].stats()
Start                2019-12-31 23:00:00+00:00
End                  2020-02-29 23:00:00+00:00
Period                61 days 00:00:00
First Trade Start    2019-12-31 23:00:00+00:00
Last Trade End       2020-02-29 23:00:00+00:00
Coverage              60 days 00:00:00
Overlap Coverage     49 days 00:00:00
Total Records         19.0
Total Long Trades      2.0
Total Short Trades    17.0
Total Closed Trades   18.0
Total Open Trades      1.0
Open Trade PnL        16.063
Win Rate [%]          61.111111
Max Win Streak         11.0
Max Loss Streak         7.0
Best Trade [%]         3.526377
Worst Trade [%]        -6.543679
Avg Winning Trade [%]   2.225861
Avg Losing Trade [%]    -3.601313
Avg Winning Trade Duration 32 days 19:38:10.909090909
Avg Losing Trade Duration  5 days 00:00:00
Profit Factor           1.022425
Expectancy             0.028157
SQN                     0.039174
Name: agg_stats, dtype: object

```

Positions share almost identical metrics with trades:

```
>>> pf.positions['a'].stats()
Start                2019-12-31 23:00:00+00:00
End                  2020-02-29 23:00:00+00:00
Period               61 days 00:00:00
First Trade Start    2019-12-31 23:00:00+00:00
Last Trade End       2020-02-29 23:00:00+00:00
Coverage             60 days 00:00:00
Overlap Coverage     0 days 00:00:00
Total Records        5.0
Total Long Trades     2.0
Total Short Trades    3.0
Total Closed Trades   4.0
Total Open Trades     1.0
Open Trade PnL        38.356823
Win Rate [%]          0.0
Max Win Streak        0.0
Max Loss Streak       4.0
Best Trade [%]        -1.529613
Worst Trade [%]       -6.543679
Avg Winning Trade [%] NaN
Avg Losing Trade [%]   -3.786739
Avg Winning Trade Duration NaT
Avg Losing Trade Duration 4 days 00:00:00
Profit Factor          0.0
Expectancy            -5.446748
SQN                   -1.794214
Name: agg_stats, dtype: object
```

To also include open trades/positions when calculating metrics such as win rate, pass `incl_open=True`:

```
>>> pf.trades['a'].stats(settings=dict(incl_open=True))
Start                2019-12-31 23:00:00+00:00
End                  2020-02-29 23:00:00+00:00
Period               61 days 00:00:00
First Trade Start    2019-12-31 23:00:00+00:00
Last Trade End       2020-02-29 23:00:00+00:00
Coverage             60 days 00:00:00
Overlap Coverage     49 days 00:00:00
Total Records        19.0
Total Long Trades     2.0
Total Short Trades    17.0
Total Closed Trades   18.0
Total Open Trades     1.0
Open Trade PnL        16.063
Win Rate [%]          61.111111
Max Win Streak        12.0
Max Loss Streak       7.0
Best Trade [%]        3.526377
Worst Trade [%]       -6.543679
Avg Winning Trade [%] 2.238896
Avg Losing Trade [%]   -3.601313
Avg Winning Trade Duration 33 days 18:00:00
Avg Losing Trade Duration 5 days 00:00:00
Profit Factor          1.733143
Expectancy             0.872096
SQN                    0.804714
Name: agg_stats, dtype: object
```

StatsBuilderMixin.stats also supports (re-)grouping:

```
>>> pf.trades.stats(group_by=True)
Start                2019-12-31 23:00:00+00:00
End                  2020-02-29 23:00:00+00:00
Period               61 days 00:00:00
First Trade Start    2019-12-31 23:00:00+00:00
Last Trade End       2020-02-29 23:00:00+00:00
Coverage             61 days 00:00:00
Overlap Coverage     61 days 00:00:00
Total Records        37
Total Long Trades     5
Total Short Trades    32
Total Closed Trades   35
Total Open Trades     2
Open Trade PnL        1.336259
Win Rate [%]          37.142857
Max Win Streak        11
Max Loss Streak       10
Best Trade [%]        3.526377
Worst Trade [%]       -8.710238
Avg Winning Trade [%] 1.907799
Avg Losing Trade [%]   -3.259135
Avg Winning Trade Duration 28 days 14:46:09.230769231
Avg Losing Trade Duration 14 days 00:00:00
Profit Factor          0.340493
Expectancy            -1.292596
SQN                   -2.509223
Name: group, dtype: object
```

Plots

Hint

See `PlotsBuilderMixin.plots` and `Trades.subplots`.

`Trades` class has two subplots based on `Trades.plot` and `Trades.plot_pnl`:

```
>>> pf.trades['a'].plots().show()
```



entry_trades_field_config ReadOnlyConfig

Field config for `EntryTrades`.

```
ReadOnlyConfig(
    settings=dict(
        id=dict(
            title='Entry Trade Id'
        ),
        idx=dict(
            name='entry_idx'
        )
    )
)
```

exit_trades_field_config ReadOnlyConfig

Field config for `ExitTrades`.

```
ReadOnlyConfig(
    settings=dict(
        id=dict(
            title='Exit Trade Id'
        )
    )
)
```

positions_field_config ReadOnlyConfig

Field config for `Positions`.

```
ReadOnlyConfig(
    settings=dict(
        id=dict(
            title='Position Id'
        ),
        parent_id=dict(
```

```

        title='Parent Id',
        ignore=True
    )
)

```

trades_attach_field_config ReadOnlyConfig

Config of fields to be attached to [Trades](#).

```

ReadOnlyConfig(
    return=dict(
        attach='returns'
    ),
    direction=dict(
        attach_filters=True
    ),
    status=dict(
        attach_filters=True,
        on_conflict='ignore'
    )
)

```

trades_field_config ReadOnlyConfig

Field config for [Trades](#).

```

ReadOnlyConfig(
    dtype=np.dtype([
        ('id', 'int64'),
        ('col', 'int64'),
        ('size', 'float64'),
        ('entry_order_id', 'int64'),
        ('entry_idx', 'int64'),
        ('entry_price', 'float64'),
        ('entry_fees', 'float64'),
        ('exit_order_id', 'int64'),
        ('exit_idx', 'int64'),
        ('exit_price', 'float64'),
        ('exit_fees', 'float64'),
        ('pnl', 'float64'),
        ('return', 'float64'),
        ('direction', 'int64'),
        ('status', 'int64'),
        ('parent_id', 'int64')
    ]),
    settings=dict(
        id=dict(
            title='Trade Id'
        ),
        idx=dict(
            name='exit_idx'
        ),
        start_idx=dict(
            name='entry_idx'
        ),
        end_idx=dict(
            name='exit_idx'
        ),
        size=dict(
            title='Size'
        ),
        entry_order_id=dict(
            title='Entry Order Id',
            mapping='ids'
        ),
        entry_idx=dict(
            title='Entry Index',
            mapping='index'
        ),
        entry_price=dict(
            title='Avg Entry Price'
        ),
        entry_fees=dict(
            title='Entry Fees'
        ),
        exit_order_id=dict(
            title='Exit Order Id',
            mapping='ids'
        ),
        exit_idx=dict(
            title='Exit Index',
            mapping='index'
        ),
        exit_price=dict(
            title='Avg Exit Price'
        ),
        exit_fees=dict(

```

```

        title='Exit Fees'
    ),
    pnl=dict(
        title='PnL'
    ),
    return=dict(
        title='Return',
        hovertemplate='$Title: %{customdata[$index]:,%)'
    ),
    direction=dict(
        title='Direction',
        mapping=TradeDirectionT(
            Long=0,
            Short=1
        )
    ),
    status=dict(
        title='Status',
        mapping=TradeStatusT(
            Open=0,
            Closed=1
        )
    ),
    parent_id=dict(
        title='Position Id',
        mapping='ids'
    )
)
)

```

trades_shortcut_config ReadOnlyConfig

Config of shortcut properties to be attached to [Trades](#).

```

ReadOnlyConfig(
    ranges=dict(),
    long_view=dict(),
    short_view=dict(),
    winning=dict(),
    losing=dict(),
    winning_streak=dict(
        obj_type='mapped_array'
    ),
    losing_streak=dict(
        obj_type='mapped_array'
    ),
    win_rate=dict(
        obj_type='red_array'
    ),
    profit_factor=dict(
        obj_type='red_array',
        method_kwargs=dict(
            use_returns=False
        )
    ),
    rel_profit_factor=dict(
        obj_type='red_array',
        method_name='get_profit_factor',
        method_kwargs=dict(
            use_returns=True,
            wrap_kwargs=dict(
                name_or_index='rel_profit_factor'
            )
        )
    ),
    expectancy=dict(
        obj_type='red_array',
        method_kwargs=dict(
            use_returns=False
        )
    ),
    rel_expectancy=dict(
        obj_type='red_array',
        method_name='get_expectancy',
        method_kwargs=dict(
            use_returns=True,
            wrap_kwargs=dict(
                name_or_index='rel_expectancy'
            )
        )
    ),
    sqn=dict(
        obj_type='red_array',
        method_kwargs=dict(
            use_returns=False
        )
    ),
    rel_sqn=dict(
        obj_type='red_array',
        method_name='get_sqn',
        method_kwargs=dict(
            use_returns=True,
            wrap_kwargs=dict(

```

```

        name_or_index='rel_sqn'
    )
    )
),
best_price=dict(
    obj_type='mapped_array'
),
worst_price=dict(
    obj_type='mapped_array'
),
best_price_idx=dict(
    obj_type='mapped_array'
),
worst_price_idx=dict(
    obj_type='mapped_array'
),
expanding_best_price=dict(
    obj_type='array'
),
expanding_worst_price=dict(
    obj_type='array'
),
mfe=dict(
    obj_type='mapped_array'
),
mfe_returns=dict(
    obj_type='mapped_array',
    method_name='get_mfe',
    method_kwargs=dict(
        use_returns=True
    )
),
mae=dict(
    obj_type='mapped_array'
),
mae_returns=dict(
    obj_type='mapped_array',
    method_name='get_mae',
    method_kwargs=dict(
        use_returns=True
    )
),
expanding_mfe=dict(
    obj_type='array'
),
expanding_mfe_returns=dict(
    obj_type='array',
    method_name='get_expanding_mfe',
    method_kwargs=dict(
        use_returns=True
    )
),
expanding_mae=dict(
    obj_type='array'
),
expanding_mae_returns=dict(
    obj_type='array',
    method_name='get_expanding_mae',
    method_kwargs=dict(
        use_returns=True
    )
),
edge_ratio=dict(
    obj_type='red_array'
),
running_edge_ratio=dict(
    obj_type='array'
)
)
)

```

EntryTrades class

```

EntryTrades(
    wrapper,
    records_arr,
    open=None,
    high=None,
    low=None,
    close=None,
    **kwargs
)

```

Extends **Trades** for working with entry trade records.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)

- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [PriceRecords](#)
- [Ranges](#)
- [Records](#)
- [RecordsWithFields](#)
- [StatsBuilderMixin](#)
- [Trades](#)
- [Wrapping](#)

Inherited members

- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)
- [AttrResolverMixin.resolve_attr](#)
- [AttrResolverMixin.resolve_shortcut_attr](#)
- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)
- [Configured.resolve_merge_kwargs](#)
- [Configured.update_config](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)
- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [IndexApplier.add_levels](#)

- IndexApplier.drop_duplicate_levels
- IndexApplier.drop_levels
- IndexApplier.drop_redundant_levels
- IndexApplier.rename_levels
- IndexApplier.select_levels
- IndexingBase.indexing_setter_func
- PandasIndexer.xs
- Pickleable.decode_config
- Pickleable.decode_config_node
- Pickleable.dumps
- Pickleable.encode_config
- Pickleable.encode_config_node
- Pickleable.file_exists
- Pickleable.getsize
- Pickleable.load
- Pickleable.loads
- Pickleable.modify_state
- Pickleable.resolve_file_path
- Pickleable.save
- PlotsBuilderMixin.build_subplots_doc
- PlotsBuilderMixin.override_subplots_doc
- PlotsBuilderMixin.plots
- PlotsBuilderMixin.resolve_plots_setting
- Prettified.pprint
- PriceRecords.field_config
- PriceRecords.from_records
- PriceRecords.get_bar_close
- PriceRecords.get_bar_close_time
- PriceRecords.get_bar_high
- PriceRecords.get_bar_low
- PriceRecords.get_bar_open
- PriceRecords.get_bar_open_time
- PriceRecords.indexing_func
- PriceRecords.indexing_func_meta
- PriceRecords.resample
- PriceRecords.resolve_column_stack_kwargs
- PriceRecords.resolve_row_stack_kwargs
- Ranges.crop
- Ranges.filter_max_duration
- Ranges.filter_min_duration
- Ranges.from_array
- Ranges.from_delta
- Ranges.get_avg_duration
- Ranges.get_coverage
- Ranges.get_duration
- Ranges.get_first_idx
- Ranges.get_first_pd_mask
- Ranges.get_invalid
- Ranges.get_last_idx
- Ranges.get_last_pd_mask
- Ranges.get_max_duration
- Ranges.get_projections
- Ranges.get_ranges_pd_mask
- Ranges.get_real_duration

- [Ranges.get_valid](#)
- [Ranges.plot_projections](#)
- [Ranges.plot_shapes](#)
- [Ranges.with_delta](#)
- [Records.apply](#)
- [Records.apply_mask](#)
- [Records.build_field_config_doc](#)
- [Records.column_stack](#)
- [Records.column_stack_records_arrs](#)
- [Records.count](#)
- [Records.coverage_map](#)
- [Records.first_n](#)
- [Records.get_apply_mapping_arr](#)
- [Records.get_apply_mapping_str_arr](#)
- [Records.get_column_stack_record_indices](#)
- [Records.get_field_arr](#)
- [Records.get_field_mapping](#)
- [Records.get_field_name](#)
- [Records.get_field_setting](#)
- [Records.get_field_title](#)
- [Records.get_map_field](#)
- [Records.get_map_field_to_columns](#)
- [Records.get_map_field_to_index](#)
- [Records.get_pd_mask](#)
- [Records.get_row_stack_record_indices](#)
- [Records.has_conflicts](#)
- [Records.is_sorted](#)
- [Records.last_n](#)
- [Records.map](#)
- [Records.map_array](#)
- [Records.map_field](#)
- [Records.override_field_config_doc](#)
- [Records.prepare_customdata](#)
- [Records.random_n](#)
- [Records.replace](#)
- [Records.resample_meta](#)
- [Records.resample_records_arr](#)
- [Records.row_stack](#)
- [Records.row_stack_records_arrs](#)
- [Records.select_cols](#)
- [Records.sort](#)
- [Records.to_readable](#)
- [StatsBuilderMixin.build_metrics_doc](#)
- [StatsBuilderMixin.override_metrics_doc](#)
- [StatsBuilderMixin.resolve_stats_setting](#)
- [StatsBuilderMixin.stats](#)
- [Trades.avg_duration](#)
- [Trades.bar_close](#)
- [Trades.bar_close_time](#)
- [Trades.bar_high](#)
- [Trades.bar_low](#)
- [Trades.bar_open](#)
- [Trades.bar_open_time](#)
- [Trades.best_price](#)

- Trades.best_price_idx
- Trades.close
- Trades.cls_dir
- Trades.col
- Trades.col_arr
- Trades.col_mapper
- Trades.column_only_select
- Trades.config
- Trades.coverage
- Trades.direction
- Trades.direction_long
- Trades.direction_short
- Trades.duration
- Trades.edge_ratio
- Trades.end_idx
- Trades.entry_fees
- Trades.entry_idx
- Trades.entry_order_id
- Trades.entry_price
- Trades.exit_fees
- Trades.exit_idx
- Trades.exit_order_id
- Trades.exit_price
- Trades.expanding_best_price
- Trades.expanding_mae
- Trades.expanding_mae_returns
- Trades.expanding_mfe
- Trades.expanding_mfe_returns
- Trades.expanding_worst_price
- Trades.expectancy
- Trades.field_names
- Trades.first_idx
- Trades.first_pd_mask
- Trades.get_best_price
- Trades.get_best_price_idx
- Trades.get_edge_ratio
- Trades.get_expanding_best_price
- Trades.get_expanding_mae
- Trades.get_expanding_mfe
- Trades.get_expanding_worst_price
- Trades.get_expectancy
- Trades.get_long_view
- Trades.get_losing
- Trades.get_losing_streak
- Trades.get_mae
- Trades.get_mfe
- Trades.get_profit_factor
- Trades.get_ranges
- Trades.get_running_edge_ratio
- Trades.get_short_view
- Trades.get_sqn
- Trades.get_win_rate
- Trades.get_winning
- Trades.get_winning_streak

- `Trades.get_worst_price`
- `Trades.get_worst_price_idx`
- `Trades.group_select`
- `Trades.high`
- `Trades.id`
- `Trades.id_arr`
- `Trades.idx_arr`
- `Trades.iloc`
- `Trades.indexing_kwargs`
- `Trades.invalid`
- `Trades.last_idx`
- `Trades.last_pd_mask`
- `Trades.loc`
- `Trades.long_view`
- `Trades.losing`
- `Trades.losing_streak`
- `Trades.low`
- `Trades.mae`
- `Trades.mae_returns`
- `Trades.max_duration`
- `Trades.mfe`
- `Trades.mfe_returns`
- `Trades.open`
- `Trades.overlap_coverage`
- `Trades.parent_id`
- `Trades.pd_mask`
- `Trades.plot`
- `Trades.plot_against_pnl`
- `Trades.plot_expanding`
- `Trades.plot_expanding_mae`
- `Trades.plot_expanding_mae_returns`
- `Trades.plot_expanding_mfe`
- `Trades.plot_expanding_mfe_returns`
- `Trades.plot_mae`
- `Trades.plot_mae_returns`
- `Trades.plot_mfe`
- `Trades.plot_mfe_returns`
- `Trades.plot_pnl`
- `Trades.plot_returns`
- `Trades.plot_running_edge_ratio`
- `Trades.plots_defaults`
- `Trades.pnl`
- `Trades.profit_factor`
- `Trades.projections`
- `Trades.range_only_select`
- `Trades.ranges`
- `Trades.ranges_pd_mask`
- `Trades.readable`
- `Trades.real_duration`
- `Trades.rec_state`
- `Trades.reccarray`
- `Trades.records`
- `Trades.records_arr`
- `Trades.records_readable`

- [Trades.rel_expectancy](#)
- [Trades.rel_profit_factor](#)
- [Trades.rel_sqn](#)
- [Trades.returns](#)
- [Trades.running_edge_ratio](#)
- [Trades.self_aliases](#)
- [Trades.short_view](#)
- [Trades.size](#)
- [Trades.sqn](#)
- [Trades.start_idx](#)
- [Trades.stats_defaults](#)
- [Trades.status](#)
- [Trades.status_closed](#)
- [Trades.status_open](#)
- [Trades.valid](#)
- [Trades.values](#)
- [Trades.win_rate](#)
- [Trades.winning](#)
- [Trades.winning_streak](#)
- [Trades.worst_price](#)
- [Trades.worst_price_idx](#)
- [Trades.wrapper](#)
- [Trades.xloc](#)
- [Wrapping.apply_to_index](#)
- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping regroup](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

from_orders class method 

```
EntryTrades.from_orders(  
    orders,  
    open=None,  
    high=None,  
    low=None,  
    close=None,  
    init_position=0.0,  
    init_price=nan,  
    sim_start=None,  
    sim_end=None,  
    jitted=None,  
    chunked=None,  
    **kwargs  
)
```

Build **EntryTrades** from **Orders**.

plot_signals method 

```
EntryTrades.plot_signals(  
    column=None,  
    plot_ohlc=True,  
    plot_close=True,  
    ohlc_type=None,  
    ohlc_trace_kwargs=None,  
    close_trace_kwargs=None,  
)
```

```

    long_entry_trace_kwargs=None,
    short_entry_trace_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    **layout_kwargs
)

```

Plot entry trade signals.

Args

column : str

Name of the column to plot.

plot_ohlc : bool

Whether to plot OHLC.

plot_close : bool

Whether to plot close.

ohlc_type

Either 'OHLC', 'Candlestick' or Plotly trace.

Pass None to use the default.

ohlc_trace_kwargs : dict

Keyword arguments passed to `ohlc_type`.

close_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for `EntryTrades.close`.

long_entry_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Long Entry" markers.

short_entry_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Short Entry" markers.

add_trace_kwargs : dict

Keyword arguments passed to `add_trace`.

fig : Figure or FigureWidget

Figure to add traces to.

****layout_kwargs**

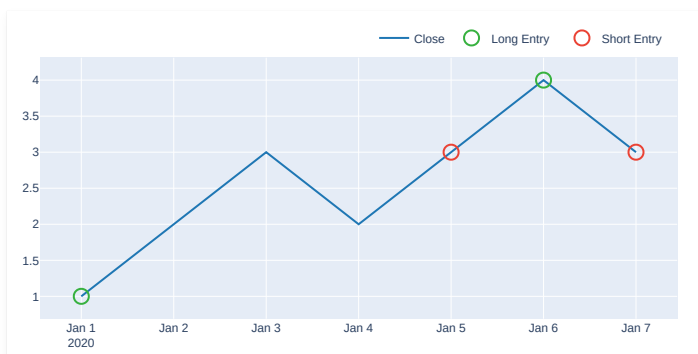
Keyword arguments for layout.

Usage

```

>>> index = pd.date_range("2020", periods=7)
>>> price = pd.Series([1, 2, 3, 2, 3, 4, 3], index=index)
>>> orders = pd.Series([1, 0, -1, 0, -1, 2, -2], index=index)
>>> pf = vbt.Portfolio.from_orders(price, orders)
>>> pf.entry_trades.plot_signals().show()

```



ExitTrades class

```
ExitTrades(
    wrapper,
    records_arr,
    open=None,
    high=None,
    low=None,
    close=None,
    **kwargs
)
```

Extends [Trades](#) for working with exit trade records.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Iterable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [PriceRecords](#)
- [Ranges](#)
- [Records](#)
- [RecordsWithFields](#)
- [StatsBuilderMixin](#)
- [Trades](#)
- [Wrapping](#)

Inherited members

- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)
- [AttrResolverMixin.resolve_attr](#)
- [AttrResolverMixin.resolve_shortcut_attr](#)
- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)

- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `HasSettings.get_path_setting`
- `HasSettings.get_path_settings`
- `HasSettings.get_setting`
- `HasSettings.get_settings`
- `HasSettings.has_path_setting`
- `HasSettings.has_path_settings`
- `HasSettings.has_setting`
- `HasSettings.has_settings`
- `HasSettings.reset_settings`
- `HasSettings.resolve_setting`
- `HasSettings.resolve_settings_paths`
- `HasSettings.set_settings`
- `IndexApplier.add_levels`
- `IndexApplier.drop_duplicate_levels`
- `IndexApplier.drop_levels`
- `IndexApplier.drop_redundant_levels`
- `IndexApplier.rename_levels`
- `IndexApplier.select_levels`
- `IndexingBase.indexing_setter_func`
- `PandasIndexer.xs`
- `Pickleable.decode_config`
- `Pickleable.decode_config_node`
- `Pickleable.dumps`
- `Pickleable.encode_config`
- `Pickleable.encode_config_node`
- `Pickleable.file_exists`
- `Pickleable.getsize`
- `Pickleable.load`
- `Pickleable.loads`
- `Pickleable.modify_state`
- `Pickleable.resolve_file_path`
- `Pickleable.save`
- `PlotsBuilderMixin.build_subplots_doc`
- `PlotsBuilderMixin.override_subplots_doc`
- `PlotsBuilderMixin.plots`
- `PlotsBuilderMixin.resolve_plots_setting`
- `Prettified.pprint`
- `PriceRecords.field_config`
- `PriceRecords.from_records`
- `PriceRecords.get_bar_close`
- `PriceRecords.get_bar_close_time`
- `PriceRecords.get_bar_high`
- `PriceRecords.get_bar_low`
- `PriceRecords.get_bar_open`
- `PriceRecords.get_bar_open_time`
- `PriceRecords.indexing_func`
- `PriceRecords.indexing_func_meta`
- `PriceRecords.resample`
- `PriceRecords.resolve_column_stack_kwargs`
- `PriceRecords.resolve_row_stack_kwargs`
- `Ranges.crop`
- `Ranges.filter_max_duration`

- `Ranges.filter_min_duration`
- `Ranges.from_array`
- `Ranges.from_delta`
- `Ranges.get_avg_duration`
- `Ranges.get_coverage`
- `Ranges.get_duration`
- `Ranges.get_first_idx`
- `Ranges.get_first_pd_mask`
- `Ranges.get_invalid`
- `Ranges.get_last_idx`
- `Ranges.get_last_pd_mask`
- `Ranges.get_max_duration`
- `Ranges.get_projections`
- `Ranges.get_ranges_pd_mask`
- `Ranges.get_real_duration`
- `Ranges.get_valid`
- `Ranges.plot_projections`
- `Ranges.plot_shapes`
- `Ranges.with_delta`
- `Records.apply`
- `Records.apply_mask`
- `Records.build_field_config_doc`
- `Records.column_stack`
- `Records.column_stack_records_arrs`
- `Records.count`
- `Records.coverage_map`
- `Records.first_n`
- `Records.get_apply_mapping_arr`
- `Records.get_apply_mapping_str_arr`
- `Records.get_column_stack_record_indices`
- `Records.get_field_arr`
- `Records.get_field_mapping`
- `Records.get_field_name`
- `Records.get_field_setting`
- `Records.get_field_title`
- `Records.get_map_field`
- `Records.get_map_field_to_columns`
- `Records.get_map_field_to_index`
- `Records.get_pd_mask`
- `Records.get_row_stack_record_indices`
- `Records.has_conflicts`
- `Records.is_sorted`
- `Records.last_n`
- `Records.map`
- `Records.map_array`
- `Records.map_field`
- `Records.override_field_config_doc`
- `Records.prepare_customdata`
- `Records.random_n`
- `Records.replace`
- `Records.resample_meta`
- `Records.resample_records_arr`
- `Records.row_stack`
- `Records.row_stack_records_arrs`

- [Records.select_cols](#)
- [Records.sort](#)
- [Records.to_readable](#)
- [StatsBuilderMixin.build_metrics_doc](#)
- [StatsBuilderMixin.override_metrics_doc](#)
- [StatsBuilderMixin.resolve_stats_setting](#)
- [StatsBuilderMixin.stats](#)
- [Trades.avg_duration](#)
- [Trades.bar_close](#)
- [Trades.bar_close_time](#)
- [Trades.bar_high](#)
- [Trades.bar_low](#)
- [Trades.bar_open](#)
- [Trades.bar_open_time](#)
- [Trades.best_price](#)
- [Trades.best_price_idx](#)
- [Trades.close](#)
- [Trades.cls_dir](#)
- [Trades.col](#)
- [Trades.col_arr](#)
- [Trades.col_mapper](#)
- [Trades.column_only_select](#)
- [Trades.config](#)
- [Trades.coverage](#)
- [Trades.direction](#)
- [Trades.direction_long](#)
- [Trades.direction_short](#)
- [Trades.duration](#)
- [Trades.edge_ratio](#)
- [Trades.end_idx](#)
- [Trades.entry_fees](#)
- [Trades.entry_idx](#)
- [Trades.entry_order_id](#)
- [Trades.entry_price](#)
- [Trades.exit_fees](#)
- [Trades.exit_idx](#)
- [Trades.exit_order_id](#)
- [Trades.exit_price](#)
- [Trades.expanding_best_price](#)
- [Trades.expanding_mae](#)
- [Trades.expanding_mae_returns](#)
- [Trades.expanding_mfe](#)
- [Trades.expanding_mfe_returns](#)
- [Trades.expanding_worst_price](#)
- [Trades.expectancy](#)
- [Trades.field_names](#)
- [Trades.first_idx](#)
- [Trades.first_pd_mask](#)
- [Trades.get_best_price](#)
- [Trades.get_best_price_idx](#)
- [Trades.get_edge_ratio](#)
- [Trades.get_expanding_best_price](#)
- [Trades.get_expanding_mae](#)
- [Trades.get_expanding_mfe](#)

- `Trades.get_expanding_worst_price`
- `Trades.get_expectancy`
- `Trades.get_long_view`
- `Trades.get_losing`
- `Trades.get_losing_streak`
- `Trades.get_mae`
- `Trades.get_mfe`
- `Trades.get_profit_factor`
- `Trades.get_ranges`
- `Trades.get_running_edge_ratio`
- `Trades.get_short_view`
- `Trades.get_sqn`
- `Trades.get_win_rate`
- `Trades.get_winning`
- `Trades.get_winning_streak`
- `Trades.get_worst_price`
- `Trades.get_worst_price_idx`
- `Trades.group_select`
- `Trades.high`
- `Trades.id`
- `Trades.id_arr`
- `Trades.idx_arr`
- `Trades.iloc`
- `Trades.indexing_kwargs`
- `Trades.invalid`
- `Trades.last_idx`
- `Trades.last_pd_mask`
- `Trades.loc`
- `Trades.long_view`
- `Trades.losing`
- `Trades.losing_streak`
- `Trades.low`
- `Trades.mae`
- `Trades.mae_returns`
- `Trades.max_duration`
- `Trades.mfe`
- `Trades.mfe_returns`
- `Trades.open`
- `Trades.overlap_coverage`
- `Trades.parent_id`
- `Trades.pd_mask`
- `Trades.plot`
- `Trades.plot_against_pnl`
- `Trades.plot_expanding`
- `Trades.plot_expanding_mae`
- `Trades.plot_expanding_mae_returns`
- `Trades.plot_expanding_mfe`
- `Trades.plot_expanding_mfe_returns`
- `Trades.plot_mae`
- `Trades.plot_mae_returns`
- `Trades.plot_mfe`
- `Trades.plot_mfe_returns`
- `Trades.plot_pnl`
- `Trades.plot_returns`

- [Trades.plot_running_edge_ratio](#)
- [Trades.plots_defaults](#)
- [Trades.pnl](#)
- [Trades.profit_factor](#)
- [Trades.projections](#)
- [Trades.range_only_select](#)
- [Trades.ranges](#)
- [Trades.ranges_pd_mask](#)
- [Trades.readable](#)
- [Trades.real_duration](#)
- [Trades.rec_state](#)
- [Trades.reccarray](#)
- [Trades.records](#)
- [Trades.records_arr](#)
- [Trades.records_readable](#)
- [Trades.rel_expectancy](#)
- [Trades.rel_profit_factor](#)
- [Trades.rel_sqn](#)
- [Trades.returns](#)
- [Trades.running_edge_ratio](#)
- [Trades.self_aliases](#)
- [Trades.short_view](#)
- [Trades.size](#)
- [Trades.sqn](#)
- [Trades.start_idx](#)
- [Trades.stats_defaults](#)
- [Trades.status](#)
- [Trades.status_closed](#)
- [Trades.status_open](#)
- [Trades.valid](#)
- [Trades.values](#)
- [Trades.win_rate](#)
- [Trades.winning](#)
- [Trades.winning_streak](#)
- [Trades.worst_price](#)
- [Trades.worst_price_idx](#)
- [Trades.wrapper](#)
- [Trades.xloc](#)
- [Wrapping.apply_to_index](#)
- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping.regroup](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

`from_orders` class method 

```
ExitTrades.from_orders(  
    orders,  
    open=None,
```

```

        high=None,
        low=None,
        close=None,
        init_position=0.0,
        init_price=nan,
        sim_start=None,
        sim_end=None,
        jitted=None,
        chunked=None,
        **kwargs
    )

```

Build [ExitTrades](#) from [Orders](#).

plot_signals method

```

ExitTrades.plot_signals(
    column=None,
    plot_ohlc=True,
    plot_close=True,
    ohlc_type=None,
    ohlc_trace_kwargs=None,
    close_trace_kwargs=None,
    long_exit_trace_kwargs=None,
    short_exit_trace_kwargs=None,
    add_trace_kwargs=None,
    fig=None,
    **layout_kwargs
)

```

Plot exit trade signals.

Args

column : str

Name of the column to plot.

plot_ohlc : bool

Whether to plot OHLC.

plot_close : bool

Whether to plot close.

ohlc_type

Either 'OHLC', 'Candlestick' or Plotly trace.

Pass None to use the default.

ohlc_trace_kwargs : dict

Keyword arguments passed to `ohlc_type`.

close_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for [ExitTrades.close](#).

long_exit_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Long Exit" markers.

short_exit_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Short Exit" markers.

add_trace_kwargs : dict

Keyword arguments passed to `add_trace`.

fig : Figure or FigureWidget

Figure to add traces to.

****layout_kwargs**

Keyword arguments for layout.

Usage

```
>>> index = pd.date_range("2020", periods=7)
>>> price = pd.Series([1, 2, 3, 2, 3, 4, 3], index=index)
>>> orders = pd.Series([1, 0, -1, 0, -1, 2, -2], index=index)
>>> pf = vbt.Portfolio.from_orders(price, orders)
>>> pf.exit_trades.plot_signals().show()
```



Positions class

```
Positions(
    wrapper,
    records_arr,
    open=None,
    high=None,
    low=None,
    close=None,
    **kwargs
)
```

Extends [Trades](#) for working with position records.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [PriceRecords](#)
- [Ranges](#)
- [Records](#)
- [RecordsWithFields](#)
- [StatsBuilderMixin](#)
- [Trades](#)
- [Wrapping](#)

Inherited members

- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)

- `AttrResolverMixin.resolve_attr`
- `AttrResolverMixin.resolve_shortcut_attr`
- `Base.chat`
- `Base.find_api`
- `Base.find_assets`
- `Base.find_docs`
- `Base.find_examples`
- `Base.find_messages`
- `Cacheable.get_ca_setup`
- `Chainable.chain`
- `Chainable.pipe`
- `Configured.copy`
- `Configured.equals`
- `Configured.get_writeable_attrs`
- `Configured.prettify`
- `Configured.resolve_merge_kwargs`
- `Configured.update_config`
- `HasSettings.get_path_setting`
- `HasSettings.get_path_settings`
- `HasSettings.get_setting`
- `HasSettings.get_settings`
- `HasSettings.has_path_setting`
- `HasSettings.has_path_settings`
- `HasSettings.has_setting`
- `HasSettings.has_settings`
- `HasSettings.reset_settings`
- `HasSettings.resolve_setting`
- `HasSettings.resolve_settings_paths`
- `HasSettings.set_settings`
- `IndexApplier.add_levels`
- `IndexApplier.drop_duplicate_levels`
- `IndexApplier.drop_levels`
- `IndexApplier.drop_redundant_levels`
- `IndexApplier.rename_levels`
- `IndexApplier.select_levels`
- `IndexingBase.indexing_setter_func`
- `PandasIndexer.xs`
- `Pickleable.decode_config`
- `Pickleable.decode_config_node`
- `Pickleable.dumps`
- `Pickleable.encode_config`
- `Pickleable.encode_config_node`
- `Pickleable.file_exists`
- `Pickleable.getsize`
- `Pickleable.load`
- `Pickleable.loads`
- `Pickleable.modify_state`
- `Pickleable.resolve_file_path`
- `Pickleable.save`
- `PlotsBuilderMixin.build_subplots_doc`
- `PlotsBuilderMixin.override_subplots_doc`
- `PlotsBuilderMixin.plots`
- `PlotsBuilderMixin.resolve_plots_setting`
- `Prettified.pprint`

- `PriceRecords.field_config`
- `PriceRecords.from_records`
- `PriceRecords.get_bar_close`
- `PriceRecords.get_bar_close_time`
- `PriceRecords.get_bar_high`
- `PriceRecords.get_bar_low`
- `PriceRecords.get_bar_open`
- `PriceRecords.get_bar_open_time`
- `PriceRecords.indexing_func`
- `PriceRecords.indexing_func_meta`
- `PriceRecords.resample`
- `PriceRecords.resolve_column_stack_kwargs`
- `PriceRecords.resolve_row_stack_kwargs`
- `Ranges.crop`
- `Ranges.filter_max_duration`
- `Ranges.filter_min_duration`
- `Ranges.from_array`
- `Ranges.from_delta`
- `Ranges.get_avg_duration`
- `Ranges.get_coverage`
- `Ranges.get_duration`
- `Ranges.get_first_idx`
- `Ranges.get_first_pd_mask`
- `Ranges.get_invalid`
- `Ranges.get_last_idx`
- `Ranges.get_last_pd_mask`
- `Ranges.get_max_duration`
- `Ranges.get_projections`
- `Ranges.get_ranges_pd_mask`
- `Ranges.get_real_duration`
- `Ranges.get_valid`
- `Ranges.plot_projections`
- `Ranges.plot_shapes`
- `Ranges.with_delta`
- `Records.apply`
- `Records.apply_mask`
- `Records.build_field_config_doc`
- `Records.column_stack`
- `Records.column_stack_records_arrs`
- `Records.count`
- `Records.coverage_map`
- `Records.first_n`
- `Records.get_apply_mapping_arr`
- `Records.get_apply_mapping_str_arr`
- `Records.get_column_stack_record_indices`
- `Records.get_field_arr`
- `Records.get_field_mapping`
- `Records.get_field_name`
- `Records.get_field_setting`
- `Records.get_field_title`
- `Records.get_map_field`
- `Records.get_map_field_to_columns`
- `Records.get_map_field_to_index`
- `Records.get_pd_mask`

- `Records.get_row_stack_record_indices`
- `Records.has_conflicts`
- `Records.is_sorted`
- `Records.last_n`
- `Records.map`
- `Records.map_array`
- `Records.map_field`
- `Records.override_field_config_doc`
- `Records.prepare_customdata`
- `Records.random_n`
- `Records.replace`
- `Records.resample_meta`
- `Records.resample_records_arr`
- `Records.row_stack`
- `Records.row_stack_records_arrs`
- `Records.select_cols`
- `Records.sort`
- `Records.to_readable`
- `StatsBuilderMixin.build_metrics_doc`
- `StatsBuilderMixin.override_metrics_doc`
- `StatsBuilderMixin.resolve_stats_setting`
- `StatsBuilderMixin.stats`
- `Trades.avg_duration`
- `Trades.bar_close`
- `Trades.bar_close_time`
- `Trades.bar_high`
- `Trades.bar_low`
- `Trades.bar_open`
- `Trades.bar_open_time`
- `Trades.best_price`
- `Trades.best_price_idx`
- `Trades.close`
- `Trades.cls_dir`
- `Trades.col`
- `Trades.col_arr`
- `Trades.col_mapper`
- `Trades.column_only_select`
- `Trades.config`
- `Trades.coverage`
- `Trades.direction`
- `Trades.direction_long`
- `Trades.direction_short`
- `Trades.duration`
- `Trades.edge_ratio`
- `Trades.end_idx`
- `Trades.entry_fees`
- `Trades.entry_idx`
- `Trades.entry_order_id`
- `Trades.entry_price`
- `Trades.exit_fees`
- `Trades.exit_idx`
- `Trades.exit_order_id`
- `Trades.exit_price`
- `Trades.expanding_best_price`

- Trades.expanding_mae
- Trades.expanding_mae_returns
- Trades.expanding_mfe
- Trades.expanding_mfe_returns
- Trades.expanding_worst_price
- Trades.expectancy
- Trades.field_names
- Trades.first_idx
- Trades.first_pd_mask
- Trades.get_best_price
- Trades.get_best_price_idx
- Trades.get_edge_ratio
- Trades.get_expanding_best_price
- Trades.get_expanding_mae
- Trades.get_expanding_mfe
- Trades.get_expanding_worst_price
- Trades.get_expectancy
- Trades.get_long_view
- Trades.get_losing
- Trades.get_losing_streak
- Trades.get_mae
- Trades.get_mfe
- Trades.get_profit_factor
- Trades.get_ranges
- Trades.get_running_edge_ratio
- Trades.get_short_view
- Trades.get_sqn
- Trades.get_win_rate
- Trades.get_winning
- Trades.get_winning_streak
- Trades.get_worst_price
- Trades.get_worst_price_idx
- Trades.group_select
- Trades.high
- Trades.id
- Trades.id_arr
- Trades.idx_arr
- Trades.iloc
- Trades.indexing_kwargs
- Trades.invalid
- Trades.last_idx
- Trades.last_pd_mask
- Trades.loc
- Trades.long_view
- Trades.losing
- Trades.losing_streak
- Trades.low
- Trades.mae
- Trades.mae_returns
- Trades.max_duration
- Trades.mfe
- Trades.mfe_returns
- Trades.open
- Trades.overlap_coverage

- `Trades.parent_id`
- `Trades.pd_mask`
- `Trades.plot`
- `Trades.plot_against_pnl`
- `Trades.plot_expanding`
- `Trades.plot_expanding_mae`
- `Trades.plot_expanding_mae_returns`
- `Trades.plot_expanding_mfe`
- `Trades.plot_expanding_mfe_returns`
- `Trades.plot_mae`
- `Trades.plot_mae_returns`
- `Trades.plot_mfe`
- `Trades.plot_mfe_returns`
- `Trades.plot_pnl`
- `Trades.plot_returns`
- `Trades.plot_running_edge_ratio`
- `Trades.plots_defaults`
- `Trades.pnl`
- `Trades.profit_factor`
- `Trades.projections`
- `Trades.range_only_select`
- `Trades.ranges`
- `Trades.ranges_pd_mask`
- `Trades.readable`
- `Trades.real_duration`
- `Trades.rec_state`
- `Trades.reccarray`
- `Trades.records`
- `Trades.records_arr`
- `Trades.records_readable`
- `Trades.rel_expectancy`
- `Trades.rel_profit_factor`
- `Trades.rel_sqn`
- `Trades.returns`
- `Trades.running_edge_ratio`
- `Trades.self_aliases`
- `Trades.short_view`
- `Trades.size`
- `Trades.sqn`
- `Trades.start_idx`
- `Trades.stats_defaults`
- `Trades.status`
- `Trades.status_closed`
- `Trades.status_open`
- `Trades.valid`
- `Trades.values`
- `Trades.win_rate`
- `Trades.winning`
- `Trades.winning_streak`
- `Trades.worst_price`
- `Trades.worst_price_idx`
- `Trades.wrapper`
- `Trades.xloc`
- `Wrapping.apply_to_index`

- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping.regroup](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

from_trades class method 

```
Positions.from_trades(
    trades,
    open=None,
    high=None,
    low=None,
    close=None,
    jitted=None,
    chunked=None,
    **kwargs
)
```

Build [Positions](#) from [Trades](#).

Trades class 

```
Trades(
    wrapper,
    records_arr,
    open=None,
    high=None,
    low=None,
    close=None,
    **kwargs
)
```

Extends [Ranges](#) for working with trade-like records, such as entry trades, exit trades, and positions.

Superclasses

- [Analyzable](#)
- [AttrResolverMixin](#)
- [Base](#)
- [Cacheable](#)
- [Chainable](#)
- [Comparable](#)
- [Configured](#)
- [ExtPandasIndexer](#)
- [HasSettings](#)
- [IndexApplier](#)
- [IndexingBase](#)
- [Itemable](#)
- [PandasIndexer](#)
- [Paramable](#)
- [Pickleable](#)
- [PlotsBuilderMixin](#)
- [Prettified](#)
- [PriceRecords](#)
- [Ranges](#)
- [Records](#)
- [RecordsWithFields](#)
- [StatsBuilderMixin](#)

- [Wrapping](#)

Inherited members

- [AttrResolverMixin.deep_getattr](#)
- [AttrResolverMixin.post_resolve_attr](#)
- [AttrResolverMixin.pre_resolve_attr](#)
- [AttrResolverMixin.resolve_attr](#)
- [AttrResolverMixin.resolve_shortcut_attr](#)
- [Base.chat](#)
- [Base.find_api](#)
- [Base.find_assets](#)
- [Base.find_docs](#)
- [Base.find_examples](#)
- [Base.find_messages](#)
- [Cacheable.get_ca_setup](#)
- [Chainable.chain](#)
- [Chainable.pipe](#)
- [Configured.copy](#)
- [Configured.equals](#)
- [Configured.get_writeable_attrs](#)
- [Configured.prettify](#)
- [Configured.resolve_merge_kwargs](#)
- [Configured.update_config](#)
- [HasSettings.get_path_setting](#)
- [HasSettings.get_path_settings](#)
- [HasSettings.get_setting](#)
- [HasSettings.get_settings](#)
- [HasSettings.has_path_setting](#)
- [HasSettings.has_path_settings](#)
- [HasSettings.has_setting](#)
- [HasSettings.has_settings](#)
- [HasSettings.reset_settings](#)
- [HasSettings.resolve_setting](#)
- [HasSettings.resolve_settings_paths](#)
- [HasSettings.set_settings](#)
- [IndexApplier.add_levels](#)
- [IndexApplier.drop_duplicate_levels](#)
- [IndexApplier.drop_levels](#)
- [IndexApplier.drop_redundant_levels](#)
- [IndexApplier.rename_levels](#)
- [IndexApplier.select_levels](#)
- [IndexingBase.indexing_setter_func](#)
- [PandasIndexer.xs](#)
- [Pickleable.decode_config](#)
- [Pickleable.decode_config_node](#)
- [Pickleable.dumps](#)
- [Pickleable.encode_config](#)
- [Pickleable.encode_config_node](#)
- [Pickleable.file_exists](#)
- [Pickleable.getsize](#)
- [Pickleable.load](#)
- [Pickleable.loads](#)
- [Pickleable.modify_state](#)
- [Pickleable.resolve_file_path](#)
- [Pickleable.save](#)

- `PlotsBuilderMixin.build_subplots_doc`
- `PlotsBuilderMixin.override_subplots_doc`
- `PlotsBuilderMixin.plots`
- `PlotsBuilderMixin.resolve_plots_setting`
- `Prettified.pprint`
- `PriceRecords.from_records`
- `PriceRecords.get_bar_close`
- `PriceRecords.get_bar_close_time`
- `PriceRecords.get_bar_high`
- `PriceRecords.get_bar_low`
- `PriceRecords.get_bar_open`
- `PriceRecords.get_bar_open_time`
- `PriceRecords.indexing_func`
- `PriceRecords.indexing_func_meta`
- `PriceRecords.resample`
- `PriceRecords.resolve_column_stack_kwargs`
- `PriceRecords.resolve_row_stack_kwargs`
- `Ranges.avg_duration`
- `Ranges.bar_close`
- `Ranges.bar_close_time`
- `Ranges.bar_high`
- `Ranges.bar_low`
- `Ranges.bar_open`
- `Ranges.bar_open_time`
- `Ranges.close`
- `Ranges.cls_dir`
- `Ranges.col`
- `Ranges.col_arr`
- `Ranges.col_mapper`
- `Ranges.column_only_select`
- `Ranges.config`
- `Ranges.coverage`
- `Ranges.crop`
- `Ranges.duration`
- `Ranges.end_idx`
- `Ranges.field_names`
- `Ranges.filter_max_duration`
- `Ranges.filter_min_duration`
- `Ranges.first_idx`
- `Ranges.first_pd_mask`
- `Ranges.from_array`
- `Ranges.from_delta`
- `Ranges.get_avg_duration`
- `Ranges.get_coverage`
- `Ranges.get_duration`
- `Ranges.get_first_idx`
- `Ranges.get_first_pd_mask`
- `Ranges.get_invalid`
- `Ranges.get_last_idx`
- `Ranges.get_last_pd_mask`
- `Ranges.get_max_duration`
- `Ranges.get_projections`
- `Ranges.get_ranges_pd_mask`
- `Ranges.get_real_duration`

- [Ranges.get_valid](#)
- [Ranges.group_select](#)
- [Ranges.high](#)
- [Ranges.id](#)
- [Ranges.id_arr](#)
- [Ranges.idx_arr](#)
- [Ranges.iloc](#)
- [Ranges.indexing_kwargs](#)
- [Ranges.invalid](#)
- [Ranges.last_idx](#)
- [Ranges.last_pd_mask](#)
- [Ranges.loc](#)
- [Ranges.low](#)
- [Ranges.max_duration](#)
- [Ranges.open](#)
- [Ranges.overlap_coverage](#)
- [Ranges.pd_mask](#)
- [Ranges.plot_projections](#)
- [Ranges.plot_shapes](#)
- [Ranges.projections](#)
- [Ranges.range_only_select](#)
- [Ranges.ranges_pd_mask](#)
- [Ranges.readable](#)
- [Ranges.real_duration](#)
- [Ranges.rec_state](#)
- [Ranges.reccarray](#)
- [Ranges.records](#)
- [Ranges.records_arr](#)
- [Ranges.records_readable](#)
- [Ranges.self_aliases](#)
- [Ranges.start_idx](#)
- [Ranges.status](#)
- [Ranges.status_closed](#)
- [Ranges.status_open](#)
- [Ranges.valid](#)
- [Ranges.values](#)
- [Ranges.with_delta](#)
- [Ranges.wrapper](#)
- [Ranges.xloc](#)
- [Records.apply](#)
- [Records.apply_mask](#)
- [Records.build_field_config_doc](#)
- [Records.column_stack](#)
- [Records.column_stack_records_arrs](#)
- [Records.count](#)
- [Records.coverage_map](#)
- [Records.first_n](#)
- [Records.get_apply_mapping_arr](#)
- [Records.get_apply_mapping_str_arr](#)
- [Records.get_column_stack_record_indices](#)
- [Records.get_field_arr](#)
- [Records.get_field_mapping](#)
- [Records.get_field_name](#)
- [Records.get_field_setting](#)

- [Records.get_field_title](#)
- [Records.get_map_field](#)
- [Records.get_map_field_to_columns](#)
- [Records.get_map_field_to_index](#)
- [Records.get_pd_mask](#)
- [Records.get_row_stack_record_indices](#)
- [Records.has_conflicts](#)
- [Records.is_sorted](#)
- [Records.last_n](#)
- [Records.map](#)
- [Records.map_array](#)
- [Records.map_field](#)
- [Records.override_field_config_doc](#)
- [Records.prepare_customdata](#)
- [Records.random_n](#)
- [Records.replace](#)
- [Records.resample_meta](#)
- [Records.resample_records_arr](#)
- [Records.row_stack](#)
- [Records.row_stack_records_arrs](#)
- [Records.select_cols](#)
- [Records.sort](#)
- [Records.to_readable](#)
- [StatsBuilderMixin.build_metrics_doc](#)
- [StatsBuilderMixin.override_metrics_doc](#)
- [StatsBuilderMixin.resolve_stats_setting](#)
- [StatsBuilderMixin.stats](#)
- [Wrapping.apply_to_index](#)
- [Wrapping.as_param](#)
- [Wrapping.get_item_keys](#)
- [Wrapping.items](#)
- [Wrapping.regroup](#)
- [Wrapping.resolve_self](#)
- [Wrapping.resolve_stack_kwargs](#)
- [Wrapping.select_col](#)
- [Wrapping.select_col_from_obj](#)
- [Wrapping.split](#)
- [Wrapping.split_apply](#)

Subclasses

- [EntryTrades](#)
- [ExitTrades](#)
- [Positions](#)

`best_price` cacheable_property 

[Trades.get_best_price](#) with default arguments.

`best_price_idx` cacheable_property 

[Trades.get_best_price_idx](#) with default arguments.

`direction` cached_property 

Mapped array of the field `direction`.

direction_long	cached_property	
Records filtered by <code>direction == 0</code> .		
direction_short	cached_property	
Records filtered by <code>direction == 1</code> .		
edge_ratio	cached_property	
Trades.get_edge_ratio with default arguments.		
entry_fees	cached_property	
Mapped array of the field <code>entry_fees</code> .		
entry_idx	cached_property	
Mapped array of the field <code>entry_idx</code> .		
entry_order_id	cached_property	
Mapped array of the field <code>entry_order_id</code> .		
entry_price	cached_property	
Mapped array of the field <code>entry_price</code> .		
exit_fees	cached_property	
Mapped array of the field <code>exit_fees</code> .		
exit_idx	cached_property	
Mapped array of the field <code>exit_idx</code> .		
exit_order_id	cached_property	
Mapped array of the field <code>exit_order_id</code> .		
exit_price	cached_property	
Mapped array of the field <code>exit_price</code> .		
expanding_best_price	cacheable_property	
Trades.get_expanding_best_price with default arguments.		
expanding_mae	cacheable_property	
Trades.get_expanding_mae with default arguments.		
expanding_mae_returns	cacheable_property	
Trades.get_expanding_mae with arguments <code>{'use_returns': True}</code> .		

expanding_mfe cacheable_property

[Trades.get_expanding_mfe](#) with default arguments.

expanding_mfe_returns cacheable_property

[Trades.get_expanding_mfe](#) with arguments `{'use_returns': True}`.

expanding_worst_price cacheable_property

[Trades.get_expanding_worst_price](#) with default arguments.

expectancy cached_property

[Trades.get_expectancy](#) with arguments `{'use_returns': False}`.

field_config property

Field config of [Trades](#).

```
HybridConfig(
  dtype=np.dtype([
    ('id', 'int64'),
    ('col', 'int64'),
    ('size', 'float64'),
    ('entry_order_id', 'int64'),
    ('entry_idx', 'int64'),
    ('entry_price', 'float64'),
    ('entry_fees', 'float64'),
    ('exit_order_id', 'int64'),
    ('exit_idx', 'int64'),
    ('exit_price', 'float64'),
    ('exit_fees', 'float64'),
    ('pnl', 'float64'),
    ('return', 'float64'),
    ('direction', 'int64'),
    ('status', 'int64'),
    ('parent_id', 'int64')
  ]),
  settings=dict(
    id=dict(
      name='id',
      title='Trade Id',
      mapping='ids'
    ),
    col=dict(
      name='col',
      title='Column',
      mapping='columns',
      as_customdata=False
    ),
    idx=dict(
      name='exit_idx',
      title='Index',
      mapping='index'
    ),
    start_idx=dict(
      title='Start Index',
      mapping='index',
      name='entry_idx'
    ),
    end_idx=dict(
      title='End Index',
      mapping='index',
      name='exit_idx'
    ),
    status=dict(
      title='Status',
      mapping=TradeStatusT(
        Open=0,
        Closed=1
      )
    ),
    size=dict(
      title='Size'
    ),
    entry_order_id=dict(
      title='Entry Order Id',
      mapping='ids'
    ),
    entry_idx=dict(
      title='Entry Index',
      mapping='index'
    ),
  ),
```

```

        entry_price=dict(
            title='Avg Entry Price'
        ),
        entry_fees=dict(
            title='Entry Fees'
        ),
        exit_order_id=dict(
            title='Exit Order Id',
            mapping='ids'
        ),
        exit_idx=dict(
            title='Exit Index',
            mapping='index'
        ),
        exit_price=dict(
            title='Avg Exit Price'
        ),
        exit_fees=dict(
            title='Exit Fees'
        ),
        pnl=dict(
            title='PnL'
        ),
        return=dict(
            title='Return',
            hovertemplate='$title: %{customdata[$index]:,}'
        ),
        direction=dict(
            title='Direction',
            mapping=TradeDirectionT(
                Long=0,
                Short=1
            )
        ),
        parent_id=dict(
            title='Position Id',
            mapping='ids'
        )
    )
)

```

Returns `Trades._field_config`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change fields, you can either change the config in-place, override this property, or overwrite the instance variable `Trades._field_config`.

get_best_price method

```

Trades.get_best_price(
    entry_price_open=False,
    exit_price_close=False,
    max_duration=None,
    **kwargs
)

```

Get best price.

See [best_price_nb](#).

get_best_price_idx method

```

Trades.get_best_price_idx(
    entry_price_open=False,
    exit_price_close=False,
    max_duration=None,
    relative=True,
    **kwargs
)

```

Get (relative) index of best price.

See [best_price_idx_nb](#).

get_edge_ratio method

```

Trades.get_edge_ratio(
    volatility=None,
    entry_price_open=False,
    exit_price_close=False,
    max_duration=None,
    group_by=None,
    jitted=None,
    chunked=None,
    wrap_kwargs=None
)

```

Get edge ratio.

See [edge_ratio_nb](#).

If `volatility` is None, calculates the 14-period ATR if both high and low are provided, otherwise the 14-period rolling standard deviation.

get_expanding_best_price method

```
Trades.get_expanding_best_price(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    jitted=None,  
    clean_index_kwargs=None,  
    wrap_kwargs=None  
)
```

Get expanding best price.

See [expanding_best_price_nb](#).

get_expanding_mae method

```
Trades.get_expanding_mae(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    use_returns=False,  
    jitted=None,  
    chunked=None,  
    **kwargs  
)
```

Get expanding MAE.

See [expanding_mae_nb](#).

get_expanding_mfe method

```
Trades.get_expanding_mfe(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    use_returns=False,  
    jitted=None,  
    chunked=None,  
    **kwargs  
)
```

Get expanding MFE.

See [expanding_mfe_nb](#).

get_expanding_worst_price method

```
Trades.get_expanding_worst_price(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    jitted=None,  
    clean_index_kwargs=None,  
    wrap_kwargs=None  
)
```

Get expanding worst price.

See [expanding_worst_price_nb](#).

get_expectancy method

```
Trades.get_expectancy(  
    use_returns=False,  
    group_by=None,  
    jitted=None,  
    chunked=None,  
    wrap_kwargs=None,  
    **kwargs  
)
```

```
)
```

Get average profitability.

get_long_view method

```
Trades.get_long_view(  
    **kwargs  
)
```

Get long view.

get_losing method

```
Trades.get_losing(  
    **kwargs  
)
```

Get losing trades.

get_losing_streak method

```
Trades.get_losing_streak(  
    **kwargs  
)
```

Get losing streak at each trade in the current column.

See [trade_losing_streak_nb](#).

get_mae method

```
Trades.get_mae(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    use_returns=False,  
    jitted=None,  
    chunked=None,  
    **kwargs  
)
```

Get MAE.

See [mae_nb](#).

get_mfe method

```
Trades.get_mfe(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    use_returns=False,  
    jitted=None,  
    chunked=None,  
    **kwargs  
)
```

Get MFE.

See [mfe_nb](#).

get_profit_factor method

```
Trades.get_profit_factor(  
    use_returns=False,  
    group_by=None,  
    jitted=None,  
    chunked=None,  
    wrap_kwargs=None,  
    **kwargs  
)
```

Get profit factor.

get_ranges method

```
Trades.get_ranges(  
    **kwargs  
)
```

Get records of type [Ranges](#).

get_running_edge_ratio method

```
Trades.get_running_edge_ratio(  
    volatility=None,  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    incl_shorter=False,  
    group_by=None,  
    jitted=None,  
    wrap_kwargs=None  
)
```

Get running edge ratio.

See [running_edge_ratio_nb](#).

If `volatility` is None, calculates the 14-period ATR if both high and low are provided, otherwise the 14-period rolling standard deviation.

get_short_view method

```
Trades.get_short_view(  
    **kwargs  
)
```

Get short view.

get_sqn method

```
Trades.get_sqn(  
    ddof=1,  
    use_returns=False,  
    group_by=None,  
    jitted=None,  
    chunked=None,  
    wrap_kwargs=None,  
    **kwargs  
)
```

Get System Quality Number (SQN).

get_win_rate method

```
Trades.get_win_rate(  
    group_by=None,  
    jitted=None,  
    chunked=None,  
    wrap_kwargs=None,  
    **kwargs  
)
```

Get rate of winning trades.

get_winning method

```
Trades.get_winning(  
    **kwargs  
)
```

Get winning trades.

get_winning_streak method

```
Trades.get_winning_streak(  
    **kwargs  
)
```

Get winning streak at each trade in the current column.

See [trade_winning_streak_nb](#).

get_worst_price method

```
Trades.get_worst_price(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    **kwargs  
)
```

Get worst price.

See [worst_price_nb](#).

get_worst_price_idx method

```
Trades.get_worst_price_idx(  
    entry_price_open=False,  
    exit_price_close=False,  
    max_duration=None,  
    relative=True,  
    **kwargs  
)
```

Get (relative) index of worst price.

See [worst_price_idx_nb](#).

long_view cached_property

[Trades.get_long_view](#) with default arguments.

losing cached_property

[Trades.get_losing](#) with default arguments.

losing_streak cacheable_property

[Trades.get_losing_streak](#) with default arguments.

mae cacheable_property

[Trades.get_mae](#) with default arguments.

mae_returns cacheable_property

[Trades.get_mae](#) with arguments `{'use_returns': True}`.

metrics property

Metrics supported by [Trades](#).

```
HybridConfig(  
    start_index=dict(  
        title='Start Index',  
        calc_func=<function Trades.<lambda> at 0x16a927ba0>,  
        agg_func=None,  
        tags='wrapper'  
    ),  
    end_index=dict(  

```

```

        title='End Index',
        calc_func=<function Trades.<lambda> at 0x16a927c40>,
        agg_func=None,
        tags='wrapper'
    ),
    total_duration=dict(
        title='Total Duration',
        calc_func=<function Trades.<lambda> at 0x16a927ce0>,
        apply_to_timedelta=True,
        agg_func=None,
        tags='wrapper'
    ),
    first_trade_start=dict(
        title='First Trade Start',
        calc_func='entry_idx.nth',
        n=0,
        wrap_kwargs=dict(
            to_index=True
        ),
        tags=[
            'trades',
            'index'
        ]
    ),
    last_trade_end=dict(
        title='Last Trade End',
        calc_func='exit_idx.nth',
        n=-1,
        wrap_kwargs=dict(
            to_index=True
        ),
        tags=[
            'trades',
            'index'
        ]
    ),
    coverage=dict(
        title='Coverage',
        calc_func='coverage',
        overlapping=False,
        normalize=False,
        apply_to_timedelta=True,
        tags=[
            'ranges',
            'coverage'
        ]
    ),
    overlap_coverage=dict(
        title='Overlap Coverage',
        calc_func='coverage',
        overlapping=True,
        normalize=False,
        apply_to_timedelta=True,
        tags=[
            'ranges',
            'coverage'
        ]
    ),
    total_records=dict(
        title='Total Records',
        calc_func='count',
        tags='records'
    ),
    total_long_trades=dict(
        title='Total Long Trades',
        calc_func='direction_long.count',
        tags=[
            'trades',
            'long'
        ]
    ),
    total_short_trades=dict(
        title='Total Short Trades',
        calc_func='direction_short.count',
        tags=[
            'trades',
            'short'
        ]
    ),
    total_closed_trades=dict(
        title='Total Closed Trades',
        calc_func='status_closed.count',
        tags=[
            'trades',
            'closed'
        ]
    ),
    total_open_trades=dict(
        title='Total Open Trades',
        calc_func='status_open.count',
        tags=[
            'trades',
            'open'
        ]
    ),
    open_trade_pnl=dict(
        title='Open Trade PnL',
        calc_func='status_open.pnl.sum',

```

```

        tags=[
            'trades',
            'open'
        ]
    ),
    win_rate=dict(
        title='Win Rate [%]',
        calc_func='status_closed.get_win_rate',
        post_calc_func=<function Trades.<lambda> at 0x16a927d80>,
        tags=RepEval(
            template=['trades', *incl_open_tags],
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    winning_streak=dict(
        title='Max Win Streak',
        calc_func=RepEval(
            template='winning_streak.max' if incl_open else 'status_closed.winning_streak.max',
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        wrap_kwargs=dict(
            dtype=Int64Dtype()
        ),
    ),
    tags=RepEval(
        template=['trades', *incl_open_tags, 'streak'],
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
    losing_streak=dict(
        title='Max Loss Streak',
        calc_func=RepEval(
            template='losing_streak.max' if incl_open else 'status_closed.losing_streak.max',
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        wrap_kwargs=dict(
            dtype=Int64Dtype()
        ),
    ),
    tags=RepEval(
        template=['trades', *incl_open_tags, 'streak'],
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
    best_trade=dict(
        title='Best Trade [%]',
        calc_func=RepEval(
            template='returns.max' if incl_open else 'status_closed.returns.max',
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        post_calc_func=<function Trades.<lambda> at 0x16a927e20>,
        tags=RepEval(
            template=['trades', *incl_open_tags],
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    worst_trade=dict(
        title='Worst Trade [%]',
        calc_func=RepEval(
            template='returns.min' if incl_open else 'status_closed.returns.min',
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        ),
        post_calc_func=<function Trades.<lambda> at 0x16a927ec0>,
        tags=RepEval(
            template=['trades', *incl_open_tags],
            context=None,
            strict=None,
            context_merge_kwargs=None,
            eval_id=None
        )
    ),
    avg_winning_trade=dict(
        title='Avg Winning Trade [%]',
        calc_func=RepEval(
            template='winning.returns.mean' if incl_open else 'status_closed.winning.returns.mean',
            context=None,

```



```

        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    post_calc_func=<function Trades.<lambda> at 0x16a927f60>,
    tags=RepEval(
        template="['trades', *incl_open_tags, 'winning']",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
avg_losing_trade=dict(
    title='Avg Losing Trade [%]',
    calc_func=RepEval(
        template="'losing.returns.mean' if incl_open else 'status_closed.losing.returns.mean'",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    post_calc_func=<function Trades.<lambda> at 0x16a984040>,
    tags=RepEval(
        template="['trades', *incl_open_tags, 'losing']",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
avg_winning_trade_duration=dict(
    title='Avg Winning Trade Duration',
    calc_func=RepEval(
        template="'winning.avg_duration' if incl_open else 'status_closed.winning.get_avg_duration'",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    fill_wrap_kwargs=True,
    tags=RepEval(
        template="['trades', *incl_open_tags, 'winning', 'duration']",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
avg_losing_trade_duration=dict(
    title='Avg Losing Trade Duration',
    calc_func=RepEval(
        template="'losing.avg_duration' if incl_open else 'status_closed.losing.get_avg_duration'",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    fill_wrap_kwargs=True,
    tags=RepEval(
        template="['trades', *incl_open_tags, 'losing', 'duration']",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
profit_factor=dict(
    title='Profit Factor',
    calc_func=RepEval(
        template="'profit_factor' if incl_open else 'status_closed.get_profit_factor'",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    tags=RepEval(
        template="['trades', *incl_open_tags]",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
expectancy=dict(
    title='Expectancy',
    calc_func=RepEval(
        template="'expectancy' if incl_open else 'status_closed.get_expectancy'",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    tags=RepEval(
        template="['trades', *incl_open_tags]",
        context=None,
        strict=None,
        context_merge_kwargs=None,

```

```

        eval_id=None
    )
),
sqn=dict(
    title='SQN',
    calc_func=RepEval(
        template="'sqn' if incl_open else 'status_closed.get_sqn'",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    tags=RepEval(
        template="['trades', *incl_open_tags]",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
),
edge_ratio=dict(
    title='Edge Ratio',
    calc_func=RepEval(
        template="'edge_ratio' if incl_open else 'status_closed.get_edge_ratio'",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    ),
    tags=RepEval(
        template="['trades', *incl_open_tags]",
        context=None,
        strict=None,
        context_merge_kwargs=None,
        eval_id=None
    )
)
)
)

```

Returns `Trades._metrics`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change metrics, you can either change the config in-place, override this property, or overwrite the instance variable `Trades._metrics`.

mfe cacheable_property

`Trades.get_mfe` with default arguments.

mfe_returns cacheable_property

`Trades.get_mfe` with arguments `{'use_returns': True}`.

parent_id cached_property

Mapped array of the field `parent_id`.

plot method

```

Trades.plot(
    column=None,
    plot_ohlc=True,
    plot_close=True,
    plot_markers=True,
    plot_zones=True,
    plot_by_type=True,
    ohlc_type=None,
    ohlc_trace_kwargs=None,
    close_trace_kwargs=None,
    entry_trace_kwargs=None,
    exit_trace_kwargs=None,
    exit_profit_trace_kwargs=None,
    exit_loss_trace_kwargs=None,
    active_trace_kwargs=None,
    profit_shape_kwargs=None,
    loss_shape_kwargs=None,
    add_trace_kwargs=None,
    xref='x',
    yref='y',
    fig=None,
    **layout_kwargs
)

```

Plot trades.

Args

`column` : `str`

Name of the column to plot.

`plot_ohlc` : `bool`

Whether to plot OHLC.

`plot_close` : `bool`

Whether to plot close.

`plot_markers` : `bool`

Whether to plot markers.

`plot_zones` : `bool`

Whether to plot zones.

`plot_by_type` : `bool`

Whether to plot exit trades by type.

Otherwise, the appearance will be controlled using `exit_trace_kwargs`.

`ohlc_type`

Either 'OHLC', 'Candlestick' or Plotly trace.

Pass None to use the default.

`ohlc_trace_kwargs` : `dict`

Keyword arguments passed to `ohlc_type`.

`close_trace_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Scatter` for **Trades.close**.

`entry_trace_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Entry" markers.

`exit_trace_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Exit" markers.

`exit_profit_trace_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Exit - Profit" markers.

`exit_loss_trace_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Exit - Loss" markers.

`active_trace_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Active" markers.

`profit_shape_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Figure.add_shape` for profit zones.

`loss_shape_kwargs` : `dict`

Keyword arguments passed to `plotly.graph_objects.Figure.add_shape` for loss zones.

`add_trace_kwargs` : `dict`

Keyword arguments passed to `add_trace`.

`xref` : `str`

X coordinate axis.

`yref` : `str`

Y coordinate axis.

fig : Figure or FigureWidget

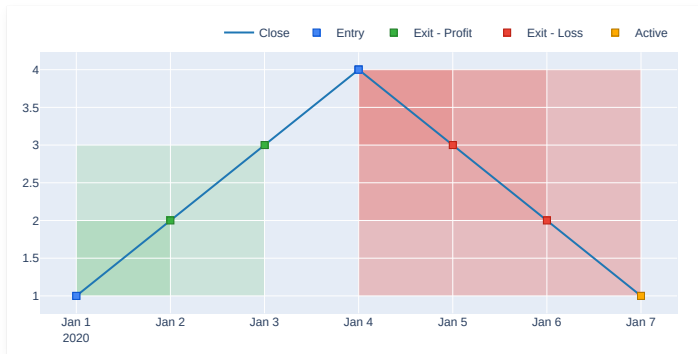
Figure to add traces to.

****layout_kwargs**

Keyword arguments for layout.

Usage

```
>>> index = pd.date_range("2020", periods=7)
>>> price = pd.Series([1., 2., 3., 4., 3., 2., 1.], index=index)
>>> size = pd.Series([1., -0.5, -0.5, 2., -0.5, -0.5, -0.5], index=index)
>>> pf = vbt.Portfolio.from_orders(price, size)
>>> pf.trades.plot().show()
```



plot_against_pnl method

```
Trades.plot_against_pnl(
    field,
    field_label=None,
    column=None,
    group_by=False,
    pct_scale=False,
    field_pct_scale=False,
    closed_trace_kwargs=None,
    closed_profit_trace_kwargs=None,
    closed_loss_trace_kwargs=None,
    open_trace_kwargs=None,
    hline_shape_kwargs=None,
    vline_shape_kwargs=None,
    add_trace_kwargs=None,
    xref='x',
    yref='y',
    fig=None,
    **layout_kwargs
)
```

Plot a field against PnL or returns.

Args

field : str, MappedArray, or array_like

Field to be plotted.

Can be also provided as a mapped array or 1-dim array.

field_label : str

Label of the field.

column : str

Name of the column to plot.

group_by : any

Group columns. See [Grouper](#).

pct_scale : bool

Whether to set x-axis to [Trades.returns](#), otherwise to [Trades.pnl](#).

field_pct_scale : bool

Whether to make y-axis a percentage scale.

`closed_trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Closed" markers.

`closed_profit_trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Closed - Profit" markers.

`closed_loss_trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Closed - Loss" markers.

`open_trace_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Open" markers.

`hline_shape_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Figure.add_shape` for horizontal zeroline.

`vline_shape_kwargs` : dict

Keyword arguments passed to `plotly.graph_objects.Figure.add_shape` for vertical zeroline.

`add_trace_kwargs` : dict

Keyword arguments passed to `add_trace`.

`xref` : str

X coordinate axis.

`yref` : str

Y coordinate axis.

`fig` : Figure or FigureWidget

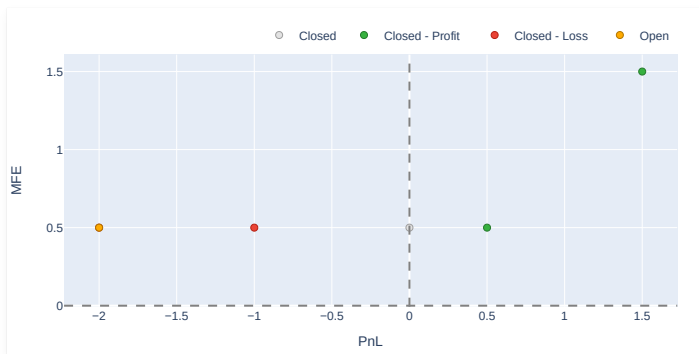
Figure to add traces to.

`**layout_kwargs`

Keyword arguments for layout.

Usage

```
>>> index = pd.date_range("2020", periods=10)
>>> price = pd.Series([1., 2., 3., 4., 5., 6., 5., 3., 2., 1.], index=index)
>>> orders = pd.Series([1., -0.5, 0., -0.5, 2., 0., -0.5, -0.5, 0., -0.5], index=index)
>>> pf = vbt.Portfolio.from_orders(price, orders)
>>> trades = pf.trades
>>> trades.plot_against_pnl("MFE").show()
```



`plot_expanding` method

```
Trades.plot_expanding(
    field,
    field_label=None,
    column=None,
    group_by=False,
    plot_bands=False,
```

```

        colorize='last',
        field_pct_scale=False,
        add_trace_kwargs=None,
        fig=None,
        **kwargs
    )

```

Plot projections of an expanding field.

Args

field : str or array_like

Field to be plotted.

Can be also provided as a 2-dim array.

field_label : str

Label of the field.

column : str

Name of the column to plot. Optional.

group_by : any

Group columns. See [Grouper](#).

plot_bands : bool

See [GenericDFAccessor.plot_projections](#).

colorize : bool, str or callable

See [GenericDFAccessor.plot_projections](#).

field_pct_scale : bool

Whether to make y-axis a percentage scale.

add_trace_kwargs : dict

Keyword arguments passed to `add_trace`.

fig : Figure or FigureWidget

Figure to add traces to.

****kwargs**

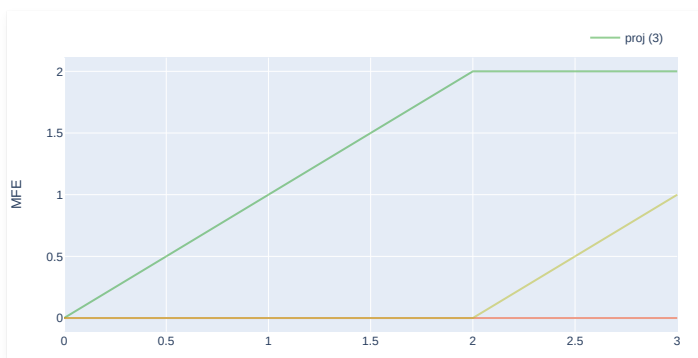
Keyword arguments passed to [GenericDFAccessor.plot_projections](#).

Usage

```

>>> index = pd.date_range("2020", periods=10)
>>> price = pd.Series([1., 2., 3., 2., 4., 5., 6., 5., 6., 7.], index=index)
>>> orders = pd.Series([1., 0., 0., -2., 0., 0., 2., 0., 0., -1.], index=index)
>>> pf = vbt.Portfolio.from_orders(price, orders)
>>> pf.trades.plot_expanding("MFE").show()

```



`plot_expanding_mae` [method](#)

```
Trades.plot_expanding_mae(  
    *args,  
    **kwargs  
)
```

[Trades.plot_expanding](#) for [Trades.expanding_mae](#).

plot_expanding_mae_returns method

```
Trades.plot_expanding_mae_returns(  
    *args,  
    **kwargs  
)
```

[Trades.plot_expanding](#) for [Trades.expanding_mae_returns](#).

plot_expanding_mfe method

```
Trades.plot_expanding_mfe(  
    *args,  
    **kwargs  
)
```

[Trades.plot_expanding](#) for [Trades.expanding_mfe](#).

plot_expanding_mfe_returns method

```
Trades.plot_expanding_mfe_returns(  
    *args,  
    **kwargs  
)
```

[Trades.plot_expanding](#) for [Trades.expanding_mfe_returns](#).

plot_mae method

```
Trades.plot_mae(  
    *args,  
    **kwargs  
)
```

[Trades.plot_against_pnl](#) for [Trades.mae](#).

plot_mae_returns method

```
Trades.plot_mae_returns(  
    *args,  
    **kwargs  
)
```

[Trades.plot_against_pnl](#) for [Trades.mae_returns](#).

plot_mfe method

```
Trades.plot_mfe(  
    *args,  
    **kwargs  
)
```

[Trades.plot_against_pnl](#) for [Trades.mfe](#).

plot_mfe_returns method

```
Trades.plot_mfe_returns(  
    *args,  
    **kwargs  
)
```

[Trades.plot_against_pnl](#) for [Trades.mfe_returns](#).

plot_pnl method

```

Trades.plot_pnl(
    column=None,
    group_by=False,
    pct_scale=False,
    marker_size_range=(7, 14),
    opacity_range=(0.75, 0.9),
    closed_trace_kwargs=None,
    closed_profit_trace_kwargs=None,
    closed_loss_trace_kwargs=None,
    open_trace_kwargs=None,
    hline_shape_kwargs=None,
    add_trace_kwargs=None,
    xref='x',
    yref='y',
    fig=None,
    **layout_kwargs
)

```

Plot trade PnL or returns.

Args

column : str

Name of the column to plot.

group_by : any

Group columns. See [Grouper](#).

pct_scale : bool

Whether to set y-axis to [Trades.returns](#), otherwise to [Trades.pnl](#).

marker_size_range : tuple

Range of marker size.

opacity_range : tuple

Range of marker opacity.

closed_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Closed" markers.

closed_profit_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Closed - Profit" markers.

closed_loss_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Closed - Loss" markers.

open_trace_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Scatter` for "Open" markers.

hline_shape_kwargs : dict

Keyword arguments passed to `plotly.graph_objects.Figure.add_shape` for zeroline.

add_trace_kwargs : dict

Keyword arguments passed to `add_trace`.

xref : str

X coordinate axis.

yref : str

Y coordinate axis.

fig : Figure or FigureWidget

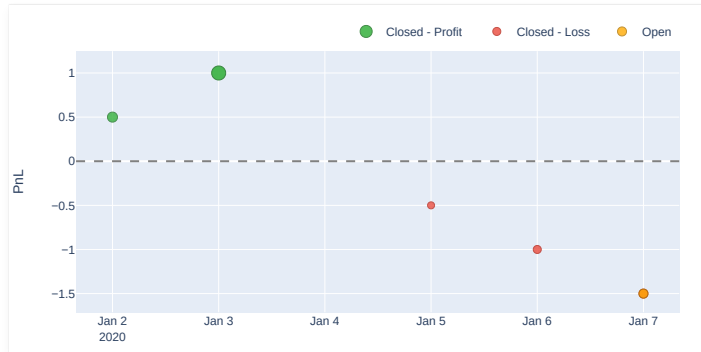
Figure to add traces to.

****layout_kwargs**

Keyword arguments for layout.

Usage

```
>>> index = pd.date_range("2020", periods=7)
>>> price = pd.Series([1., 2., 3., 4., 3., 2., 1.], index=index)
>>> orders = pd.Series([1., -0.5, -0.5, 2., -0.5, -0.5, -0.5], index=index)
>>> pf = vbt.Portfolio.from_orders(price, orders)
>>> pf.trades.plot_pnl().show()
```



plot_returns method

```
Trades.plot_returns(
    *args,
    **kwargs
)
```

[Trades.plot_pnl](#) for [Trades.returns](#).

plot_running_edge_ratio method

```
Trades.plot_running_edge_ratio(
    column=None,
    volatility=None,
    entry_price_open=False,
    exit_price_close=False,
    max_duration=None,
    incl_shorter=False,
    group_by=None,
    jitted=None,
    xref='x',
    yref='y',
    hline_shape_kwargs=None,
    **kwargs
)
```

Plot one column/group of edge ratio.

****kwargs** are passed to [GenericAccessor.plot_against](#).

plots_defaults class property

Defaults for [PlotsBuilderMixin.plots](#).

Merges [Ranges.plots_defaults](#) and [plots](#) from [trades](#).

pnl cached_property

Mapped array of the field `pnl`.

profit_factor cached_property

[Trades.get_profit_factor](#) with arguments `{'use_returns': False}`.

ranges cached_property

[Trades.get_ranges](#) with default arguments.

`rel_expectancy` cached_property 

[Trades.get_expectancy](#) with arguments `{'use_returns': True, 'wrap_kwargs': {'name_or_index': 'rel_expectancy'}}`.

`rel_profit_factor` cached_property 

[Trades.get_profit_factor](#) with arguments `{'use_returns': True, 'wrap_kwargs': {'name_or_index': 'rel_profit_factor'}}`.

`rel_sqn` cached_property 

[Trades.get_sqn](#) with arguments `{'use_returns': True, 'wrap_kwargs': {'name_or_index': 'rel_sqn'}}`.

`returns` cached_property 

Mapped array of the field `return`.

`running_edge_ratio` cacheable_property 

[Trades.get_running_edge_ratio](#) with default arguments.

`short_view` cached_property 

[Trades.get_short_view](#) with default arguments.

`size` cached_property 

Mapped array of the field `size`.

`sqn` cached_property 

[Trades.get_sqn](#) with arguments `{'use_returns': False}`.

`stats_defaults` class property 

Defaults for [StatsBuilderMixin.stats](#).

Merges [Ranges.stats_defaults](#) and `stats` from [trades](#).

`subplots` property 

Subplots supported by [Trades](#).

```
HybridConfig(
    plot=dict(
        title='Trades',
        yaxis_kwargs=dict(
            title='Price'
        ),
        check_is_not_grouped=True,
        plot_func='plot',
        tags='trades'
    ),
    plot_pnl=dict(
        title='Trade PnL',
        yaxis_kwargs=dict(
            title='Trade PnL'
        ),
        check_is_not_grouped=True,
        plot_func='plot_pnl',
        tags='trades'
    )
)
```

Returns `Trades._subplots`, which gets (hybrid-) copied upon creation of each instance. Thus, changing this config won't affect the class.

To change subplots, you can either change the config in-place, override this property, or overwrite the instance variable `Trades._subplots`.

`win_rate` cached_property 

[Trades.get_win_rate](#) with default arguments.

`winning` cached_property 

[Trades.get_winning](#) with default arguments.

`winning_streak` cacheable_property 

[Trades.get_winning_streak](#) with default arguments.

`worst_price` cacheable_property 

[Trades.get_worst_price](#) with default arguments.

`worst_price_idx` cacheable_property 

[Trades.get_worst_price_idx](#) with default arguments.